

Go: разработка приложений в микросервисной архитектуре с нуля



Вы узнаете, как:

- разрабатывать микросервисы на языке Go
- выстраивать синхронное и асинхронное взаимодействие между микросервисами
- выполнять распределенные транзакции
- организовать взаимодействие между микросервисами
- использовать паттерны проектирования
- разворачивать микросервисы в облаке
- использовать и настраивать Docker, Docker-Compose
- настраивать Kubernetes в удаленной среде



Юлия Попова

Go: **разработка приложений** **в микросервисной** **архитектуре** с нуля

Санкт-Петербург
«БХВ-Петербург»

2026

УДК 004.43
ББК 32.973.26-018.1
П58

Попова Ю. Ю.

П58 Go: разработка приложений в микросервисной архитектуре с нуля. — СПб.: БХВ-Петербург, 2026. — 320 с.: ил. — (С нуля)

ISBN 978-5-9775-2120-8

Базовая книга по построению микросервисной архитектуры с практическими примерами на языке Go. Также рассмотрена работа с оркестратором Kubernetes и контейнерами Docker в среде Docker Compose. Разобраны основные принципы и техники разработки распределенных систем, в частности показано, как написать и развернуть четыре микросервиса, управлять СУБД, настроить брокер сообщений Kafka, внедрить кеш Redis. Объяснены паттерны проектирования. Особое внимание уделено распределенным транзакциям и разворачиванию микросервисов на удаленном сервере. Показано, как обеспечить расширяемость и отказоустойчивость приложений, поддерживая высокую скорость загрузки страниц и приложений.

Электронный архив на сайте издательства содержит дополнительные материалы к книге.

Для начинающих веб-разработчиков

УДК 004.43
ББК 32.973.26-018.1

Группа подготовки издания:

Руководитель проекта	<i>Олег Сивченко</i>
Зав. редакцией	<i>Людмила Гауль</i>
Редактор	<i>Григорий Добин</i>
Компьютерная верстка	<i>Ольги Сергиенко</i>
Дизайн обложки	<i>Зои Канторович</i>

Подписано в печать 30.12.25.
Формат 70×100^{1/8}. Печать офсетная. Усл. печ. л. 25,8.
Тираж 1000 экз. Заказ № 16492.
"БХВ-Петербург", 191036, Санкт-Петербург, Гончарная ул., 20.
Отпечатано с готового оригинал-макета
ООО "Принт-М", 142300, М.О., г. Чехов, ул. Полиграфистов, д. 1

ISBN 978-5-9775-2120-8

© ООО "БХВ", 2026
© Оформление. ООО "БХВ-Петербург", 2026

Оглавление

Введение	7
Для кого эта книга?	7
Обзор Golang	7
Сравнение монолитной и микросервисной архитектуры	9
Список литературы и источников	11
 Глава 1. Разработка первого микросервиса (User).....	13
Настройка локального окружения.....	13
Редактор кода	13
Go	14
GVM.....	17
Установка Protobuf	18
Git.....	19
Codestyle	21
Docker и Docker-Compose	22
Тестирование API	24
Создание структуры проекта	27
Подключение необходимых библиотек	34
Переменные окружения	34
Логирование	36
ORM	40
Swagger	43
Проектирование базы данных PostgreSQL	47
Что такое база данных и какие они бывают?	47
Нормализация данных.....	48
Первая нормальная форма (1НФ).....	48
Вторая нормальная форма (2НФ).....	50
Третья нормальная форма (3НФ).....	51
Разработка бизнес-логики и маршрутизации для модуля User.....	52
Тестирование микросервиса	79
Список литературы и источников	86
 Глава 2. Разработка микросервиса авторизации и аутентификации (Auth)	87
Теоретический обзор способов авторизации и аутентификации.....	87
Аутентификация, идентификация и авторизация.....	87

Аутентификация по паролю	88
Аутентификация по сертификатам	89
Аутентификация по одноразовым паролям	90
Аутентификация по ключам доступа	91
Аутентификация по токенам	91
Базовые меры предосторожности от возможных уязвимостей	93
Переполнение буфера	94
Состояние гонки	94
Атаки проверки ввода	94
Атаки аутентификации	95
Атаки авторизации	95
Атаки на стороне клиента	96
Разработка модуля Auth	96
Список литературы и источников	110
Глава 3. Способы взаимодействия между микросервисами	111
HTTP-протокол	111
Модель OSI	111
Физический уровень	112
Канальный уровень	112
Сетевой уровень	113
Транспортный уровень	113
Сеансовый уровень	113
Уровень представления	114
Прикладной уровень	114
Устройство HTTP-протокола	114
Структура HTTP-запроса	115
Структура HTTP-ответа	116
gRPC	116
RabbitMQ	120
Apache Kafka	123
Redis	125
Разработка сервиса Gateway	126
Список литературы и источников	156
Глава 4. Разработка модуля Transaction	157
Проектирование базы данных	157
Частная форма третьей нормальной формы: нормальная форма Бойса — Кодда (НФБК)	157
Четвертая нормальная форма	159
Пятая нормальная форма	159
Доменно-ключевая нормальная форма	160
Шестая нормальная форма	160
Понятия миграций и транзакций в контексте базы данных PostgreSQL	161
Миграции	161
Индексы	163
Транзакции	166
ACID	166
Параллельные транзакции	167
Уровни изоляции транзакций в SQL	168

Разработка модуля Transaction	169
Интеграция Transaction и Account	187
Проблема распределенных транзакций	227
Двухфазная фиксация	228
Saga	229
Реализация паттерна Saga	231
Список литературы и источников	265
Глава 5. Развертывание микросервисов	267
Обертывание микросервисов в docker-контейнеры	294
Масштабирование при помощи оркестратора Kubernetes	306
Заключение	311
Список литературы и источников	311
Приложение. Описание файлового архива	313
Предметный указатель	315

Введение

Цель этой книги — показать на практике, как разрабатываются микросерверные приложения на Golang. Здесь всё начинается с одного небольшого микросервиса, выполняющего определенный набор задач, а заканчивается уже группой из микросервисов, применением множества технологий и доставкой готового приложения на виртуальные серверы.

Для кого эта книга?

Основное внимание в ней уделяется именно практическим навыкам написания микросервисов. Она пригодится тем, кто достаточно глубоко владеет знаниями о Golang. Упор делается именно на конкретный стек технологий: Golang, PostgreSQL, Kafka, Redis, Docker, Docker-Compose, Kubernetes и некоторые другие. Желательно, чтобы у вас был опыт написания приложений на Golang, т. к. глубоко в сам язык мы погружаться не станем. Главный акцент сделан на микросервисах, проблемах, с которыми вы можете столкнуться при их разработке, и путях их решения.

Обзор Golang

Golang — это компилируемый язык программирования, созданный в Google и используемый в первую очередь для серверной разработки [1].

В современной бэкенд-разработке Golang встречается всё чаще, многие компании тратят значительные ресурсы для того, чтобы перейти на его использование. Ведь это язык, который сразу проектировался для высокопроизводительных распределенных систем и работы с сетевыми сервисами. Его часто выбирают как альтернативу Java, Python или Node.js, когда нужна простота кода в сочетании с высокой скоростью выполнения.

Go изначально создавался для разработки крупных сервисов внутри Google, поэтому в нем «из коробки» заложены решения для типичных проблем: конкуренция при работе с потоками, масштабируемость, сборка мусора и строгая типизация. Всё это

сделало его очень привлекательным для индустрии — сейчас на Go пишут инфраструктурные инструменты, веб-серверы, системы потоковой обработки данных, микросервисы и CLI-утилиты.

Важнейшей особенностью Go является его модель параллелизма. В то время как в Node.js, например, основная ставка сделана на асинхронность и событийный цикл, в Go ключевое понятие — goroutines и каналы.

- ❑ Goroutines (горутины) — это легкие потоки выполнения, которые создаются буквально в одну строчку кода. Они гораздо дешевле системных потоков и могут запускаться десятками тысяч одновременно.
- ❑ Каналы используются для обмена данными между горутинами, что позволяет строить безопасные многопоточные программы без классических ловушек вроде «race condition».

Go также известен своей скоростью компиляции. Код компилируется в машинный, а значит, приложения на Go обычно работают быстрее, чем те же самые решения, написанные на интерпретируемых языках (JS, Python). При этом процесс сборки чаще всего занимает считанные секунды, что сильно упрощает цикл разработки.

Еще одна особенность языка — минимализм. У Go небольшой стандартный синтаксис, нет перегрузки функций, наследования классов или сложных абстракций. Зато есть мощная стандартная библиотека: работа с сетью, HTTP, файловой системой и процессами доступна без дополнительных зависимостей. Такая простота позволяет легче поддерживать код, а новым разработчикам — быстрее обучаться языку.

При этом вокруг Go также сформировалась богатая экосистема. Центрального пакетного менеджера вроде `npm` у него не было изначально, но сейчас используется модульная система с файлом `go.mod`. Она фиксирует зависимости и их версии, что дает предсказуемые сборки и защиту от конфликтов библиотек. Появился и единый репозиторий открытых библиотек — `pkg.go.dev`, где можно искать и переиспользовать готовые решения.

Разумеется, у Go есть и свои недостатки:

- ❑ язык намеренно ограничен в плане синтаксиса — многие разработчики считают его «слишком простым» и скучным;
- ❑ нет привычного объектно-ориентированного подхода — всё строится через композицию;
- ❑ параметризованные типы появились только недавно, и их функциональность сильно уступает Java или C#.

Но при этом строгость и простота Go как раз и являются причиной его популярности — писать приходится чуть дольше, но результат выходит надежным и понятным.

Что касается областей применения — Go активно используется для создания микросервисов, API-сервисов, высоконагруженных веб-серверов, брокеров сообщений

и распределенных систем. Многие облачные инструменты сейчас написаны на Go: Docker, Kubernetes, Prometheus и другие. То есть, по сути, Go стал еще и языком современной облачной инфраструктуры.

Таким образом, если цель — написать быстрый, устойчивый, хорошо масштабируемый сервис, то Go становится одним из лучших выборов. Он сочетает в себе простоту синтаксиса, мощную многопоточность и строгую приверженность минимализму. А благодаря постоянно растущему сообществу и множеству библиотек, разрабатывать на нем становится всё удобнее.

Сравнение монолитной и микросервисной архитектуры

Изначально, когда Всемирная паутина еще лишь начинала развиваться, создавали только монолиты. В основном это было связано с тем, что создание монолитов — наиболее простой способ, а задачи и разработка тогда были менее сложными по своей функциональности. Однако с развитием технологий и расширением спектра задач разработчики начали сталкиваться с ограничениями такого подхода.

Главные минусы монолитов заключались в том, что их код становилось сложно поддерживать: когда у вас команда разработчиков из 20 человек, каждое изменение в проекте будет приводить к конфликтам версий, — большому количеству людей очень тяжело работать над одной кодовой базой. Обновлять версии инструментов и внедрять новые технологии также становилось проблематичным — если вам необходимо перевести сразу весь огромный проект на другую версию, вы неизбежно будете при этом сталкиваться с конфликтами, а новые API библиотек могут ломать работающую функциональность. Если же не стараться поддерживать актуальность, будет очень сложно найти разработчиков, — мало кто хочет работать с легаси (устаревшим) кодом, поскольку это не дает развиваться специалистам.

Второй момент — безопасность: новые версии библиотек часто содержат важные изменения и исправления багов. То есть чем быстрее устареет ваш код, тем больше вероятность нахождения в нем уязвимостей.

Из-за стремительно разрастающейся кодовой базы сложность понимания того, что в проекте происходит, неуклонно растет. Вновь пришедшим в проект специалистам становится невероятно сложно погрузиться в него и разобраться что к чему. Соответственно разработка станет сильно замедляться: начиная от проблем с IDE (редактором кода), которому будет тяжело работать с большим проектом, и заканчивая сборками приложений, на скорость которых также влияет кодовая база.

Доставка приложения до конечного потребителя тоже начнет вызывать проблемы, т. к. будет происходить в лучшем случае один раз в две недели, а то и в месяц, хотя в реальности мир слишком ускорился, чтобы считать это хорошим темпом. Сейчас всё больше набирает популярность подход SaaS (Software-as-a-Service), т. е. непрерывное развертывание. Это позволяет доставлять изменения в продакшен (произ-

водственную сферу) несколько раз в сутки и желательно в рабочее время. При этом правильно настроенный цикл позволяет делать это максимально незаметно для пользователей.

Плюс к сказанному, монолит может порождать больше багов при тестировании релиза. И это погружает нас в цикл: релиз → тестирование → исправление багов → тестирование. И так, пока не выяснится, что критичных багов больше нет и можно «раскатывать» приложение на пользователей.

Отдельным пунктом при работе с монолитом выделяют надежность. Потому что, если у вас отказал монолит, у вас отказала вся система целиком. Нельзя просто взять и отключить часть функциональности до восстановления, что позволительно в микросервисах.

Вся эта аргументация выглядит так, будто микросервисы как раз и решают отмеченные проблемы, а что смотреть в сторону монолитов не имеет смысла. Но и такой подход не вполне верен, — у микросервисов тоже есть ряд существенных недостатков.

Первая задача, которая встает перед разработчиками микросервисов, — правильно разбить приложение на модули. Если же где-то на этом этапе просчитаться, то получится нечто, взявшее недостатки и от монолитов, и от микросервисов, потому что по факту это будет распределенный монолит — набор сильно связанных друг с другом сервисов, которые способны работать только в совокупности и не могут существовать раздельно.

Существует также важная проблема со сложностью распределенных систем. Очень часто возникают ситуации, когда необходимо гарантировать доставку данных до сервиса и получить ответ, — особенно это касается темы, связанной с финансовым оборотом. Каждый сервис при этом имеет собственную базу данных, и сохранять согласованность между данными приходится с помощью отдельных механизмов — двухфазных фиксаций, или саг (об этом будет подробно рассказано в *главе 4*).

Доставка приложения до конечного пользователя хотя и ускоряется с помощью монолитов, однако предполагает использование большого количества сложных инструментов, таких как Docker, Kubernetes, CI/CD и ArgoCD, отдельных сервисов Vault для хранения секретных ключей и т. д. Мы коснемся темы развертывания в последней главе и рассмотрим, как это можно сделать с помощью Docker, Docker-Compose, а также познакомимся поближе с CI/CD и Kubernetes на практике.

Таким образом, выбор между монолитом и микросервисами, как и выбор фреймворка, сводится к вопросу о цели проекта. Если это долгоиграющая история на несколько лет с поддержкой кода, исправлением багов и доработкой новой функциональности, большими планами и командой, то стоит сразу закладывать возможность микросервисной архитектуры. В другом случае, когда надо простое мини-сайт приложение, которое достаточно опубликовать один раз и больше ничего в нем не менять, — вам хватит и монолита, а микросервисы только приведут к лишнему усложнению системы, которая того абсолютно не требует.

ФАЙЛОВЫЙ АРХИВ

Файловый архив с финальным проектом выложен на сервер издательства «БХВ» по адресу: <https://zip.bhv.ru/9785977521208.zip>. Ссылка доступна и со страницы книги на сайте <https://bhv.ru/> (см. приложение).

Список литературы и источников

1. <https://habr.com/ru/companies/kryptonite/articles/798703/>.
2. Крис Ричардсон. Микросервисы. Паттерны разработки и рефакторинга (<https://goo.su/ja8FC7>).

ГЛАВА 1



Разработка первого микросервиса (User)

Настройка локального окружения

Редактор кода

Перед тем как приступить к разработке любого приложения на любом языке программирования, необходимо установить и настроить все необходимые для этого инструменты. И в первую очередь выбрать удобный редактор кода. Есть несколько самых распространенных: Visual Studio Code, Sublime Text, Vim, Atom, Brackets, Notepad++.

Notepad++ и SublimeText — простенькие редакторы, которые необходимо настраивать самостоятельно — т. е. ставить нужные плагины, поскольку с их настройками, так сказать, «из коробки», работать будет сложно, да и во всем прочем они значительно уступают остальным. Отдельно отмечу Cursor — редактор кода с интегрированной поддержкой искусственного интеллекта, созданный на основе открытого исходного кода Visual Studio Code, первый релиз которого состоялся в 2023 году, а первая стабильная версия вышла только в 2025-м. Это мощный инструмент для работы опытных программистов. Visual Studio Code, Atom и Brackets — бесплатные редакторы с мощной функциональностью, такой как автодополнение, поддержка контроля версий, отладка. Наибольшую популярность из них приобрел Visual Studio Code за счет меньшего энергопотребления и нагрузки на компьютер. Рассматривать каждый из редакторов кода — слишком объемная задача, тем более что не она является нашей целью в этой книге, поэтому без лишней лирики остановим свой выбор на доступном и оптимальном варианте — Visual Studio Code.

Первый запуск редактора кода показан на рис. 1.1.

Установим в нем расширения для Go. В строке поиска расширений введите: `go`, найдите и установите этот пакет от автора **Go Team Google**. Пакет включает расширение IntelliSense, которое предоставляет подсказки возможных вариаций завершения кода во время его написания, а также подсвечивает ошибки в синтаксисе и неверное использование переменных или операторов — удобное свойство средства диагностики, обеспечивающее наглядное отображение ошибок сборки по мере ввода кода или при его сохранении. Это основной рекомендуемый плагин для

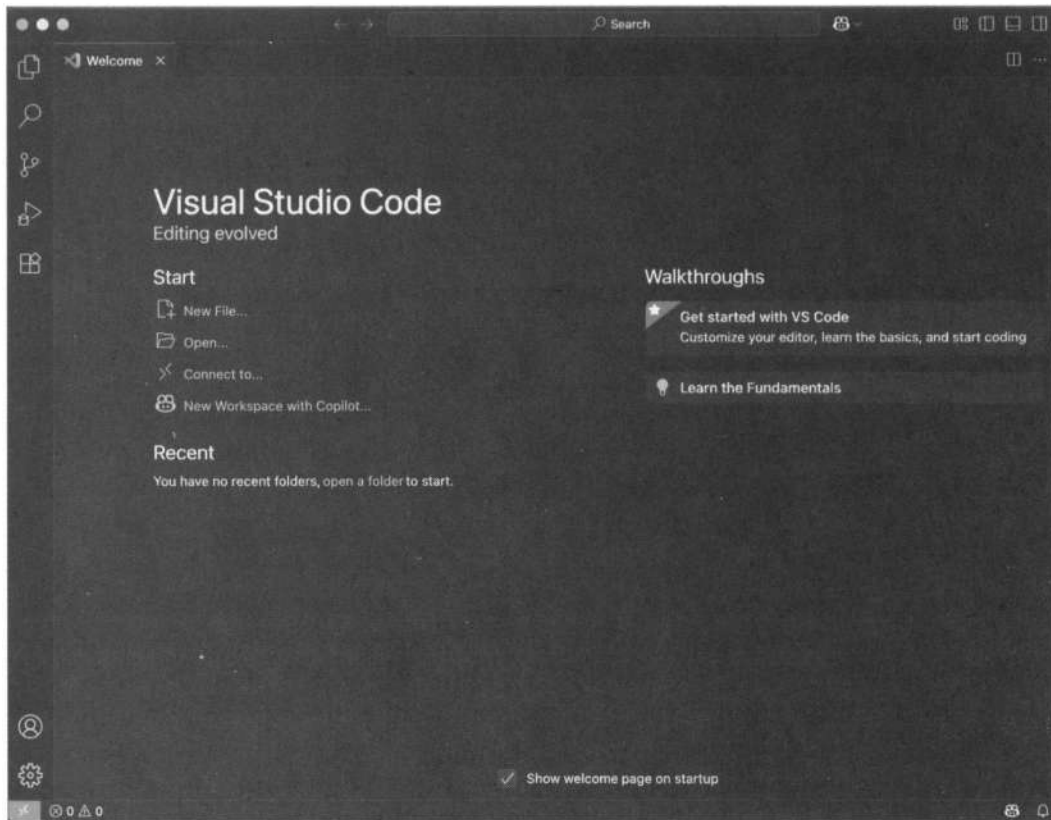


Рис. 1.1. Первый запуск Visual Studio Code

работы с Golang в Visual Studio Code, покрывающий большинство потребностей при разработке.

Go

Чтобы установить непосредственно сам Golang, есть несколько способов. Если вы работаете в среде Windows, macOS или Linux, лучше всего это сделать с помощью установщика. Все версии Go для установки представлены на официальном сайте Golang: <https://go.dev/dl/>.

Проверить, что всё установилось верно, можно с помощью команды в терминале:

```
go version
```

В ответ должна быть показана версия установленного компилятора Go:

```
go version go1.24.5 darwin/arm64
```

Существует несколько пакетных менеджеров для Go: Go Modules, \$GOPATH, glide, dep, govendor, godep. Однако с выходом версии Go 1.11, стандартным инструментом управления зависимостями стал Go Modules, а начиная с версии 1.16, он используется по умолчанию. Этот менеджер позволяет управлять зависимостями

через файлы `go.mod` и `go.sum` прямо в проекте, поэтому все остальные инструменты на текущий момент считаются устаревшими и больше не поддерживаются.

Первым основным механизмом для работы с зависимостями и структурой рабочего пространства была `$GOPATH` — одна конкретная специальная папка, в которую устанавливались все исходные файлы, зависимости, бинарники и прочее.

`$GOPATH` — это переменная среды, определяющая корень дерева для всего Go-кода на компьютере. В каталоге `src` должны были находиться ваши собственные проекты и все сторонние библиотеки, установленные через `go get` или сторонними утилитами (`dep`, `glide`, `govendor`). Скомпилированные объектные файлы, которые использовались для ускорения сборки, сохранялись в каталоге `pkg`, а итоговые бинарные файлы после сборки кода — в каталоге `bin`. Все зависимости устанавливались в один каталог, а их версии практически не фиксировались, т. к. из репозитория скачивалась всегда самая актуальная. В результате часто возникали конфликты, — ведь использовать разные версии одного и того же пакета было невозможно, а не всегда бывает так, что все проекты используют одну и ту же последнюю версию.

Импорты тогда работали только по полному пути внутри `$GOPATH/src`, потому что компилятор искал пакеты именно там, а любые исходники вне этой структуры не работали.

Это накладывало определенные ограничения: проекты нужно было хранить только в определенном месте, переключение между разными версиями и зависимостями осуществлялось лишь вручную (переключением ветки внутри самой зависимости), обновление какого-либо пакета могло сломать все ваши проекты, нельзя было безболезненно поделить свой код, использовавшим сторонние пакеты.

Такая ситуация побудила сообщество к созданию сторонних инструментов, способных решить указанные проблемы.

Первым популярным инструментом для решения всех обозначенных проблем стал инструмент от Google — `godep`. Он сохранял список всех зависимостей и их версий в файле `Godeps/Godeps.json` и клонировал копии пакетов в каталог `Godeps/_workspace` — для старых версий Go или в каталог `vendor` — для более новых. Через команду:

```
godep restore
```

можно было установить все версии библиотек, упомянутых в файле `Godeps/Godeps.json`, меняя состояние пакетов в каталоге `$GOPATH`.

Под влиянием пакетных менеджеров `bundler` (для Ruby), `npm` для (JavaScript) и `Composer` (для PHP) был создан `glide`. Поэтому работает он по тому же принципу, что и все эти инструменты. Создается файл `glide.yaml`, содержащий все правила для извлечения зависимостей. Основываясь на этих правилах, сканируется каталог `vendor` — чтобы сгенерировать файл `glide.lock`, в котором будут записаны все зависимости, в том числе транзитивные. Для удобного управления предусмотрен набор команд: `glide init` — для настройки нового проекта, `glide install` — чтобы установить все версии, указанные в `glide.lock`, `glide update` — для восстановления версий зависимостей с помощью сканирования и правил.

Менеджер `govendor` отличается по идеологии от `glide`: в нем нет четкого разделения на основной манифест (`glide.yaml`) и точного дерева зависимостей (`glide.lock`) — вместо этого информация о зависимостях собирается в файл `vendor.json`, но основной контроль в первую очередь осуществляется через каталог `vendor`. При этом неосторожным использованием разных версий пакета можно легко создать конфликт, если их API слишком различается. С вложенными зависимостями `govendor` требует больше ручного контроля, он не следит за переиспользованием пакетов и допускает дублирование. Таким образом, в итоге он оказался ближе к будущему стандарту `Go Modules`.

Начиная с версии `Go 1.16`, менеджер `Go Modules` стал частью экосистемы языка, в результате чего все рассмотренные здесь инструменты были признаны архивными и перестали поддерживаться. С этого момента работа с зависимостями стала единой для всех проектов. Основные причины его создания:

- ❑ контроль версий — для каждой зависимости явно фиксируется конкретная версия, теперь никаких случайных обновлений;
- ❑ воспроизводимость сборки — код собирается всегда одинаково на любом компьютере и не зависит от обновлений в сторонних пакетах;
- ❑ размещать проекты больше не нужно исключительно в `GOPATH` в соответствии с предопределенной структурой;
- ❑ безопасность — контрольные суммы всех зависимостей фиксируются в файле `go.sum` для защиты от подмен и атак на цепочку поставок.

Тут так же, как и у предшественников, есть файл с указанием названия модуля, который разрабатывается, совместимой версии `Go` и всех зависимостей с их версиями.

Выглядит он следующим образом:

```
module github.com/username/myproject // название модуля
```

```
go 1.24.5 // совместимая версия Go
```

```
require (  
    github.com/pkg/errors v0.9.1 // установленный пакет errors с версией v0.9.1  
    github.com/rs/zerolog v1.33.0  
)
```

Файл `go.sum`, в котором `Go Modules` хранит контрольные суммы всех загружаемых зависимостей, включая их вложенные зависимости и точные версии, генерируется автоматически. Для обеспечения безопасности архива с исходным кодом пакета используются хеш-контрольные суммы. В случае, если кто-то решит подменить зависимость — например, в публичном репозитории поместит пакет с таким же именем и другим содержимым, — это будет выявлено при сравнении новых и старых хешей. Еще один вид атаки, от которой защищает такой подход, — `Supply Chain Attack` (атака на цепочку поставок). Если вдруг злоумышленник завладеет доступом к репозиторию стороннего пакета и внесет туда вредоносный код, не меняя при этом версию пакета, тогда получится, что при следующей сборке проекта

Go скачает архив зависимости, подсчитает хеш, сравнит его с тем, что указан в `go.sum`, и, если он не совпадет, сборка будет прервана. Так сразу можно будет определить, что пакет изменился, и разобраться в ситуации подробнее.

Генерируется файл `go.sum` при первом запуске команд `go get`, `go build`, `go test` для новой зависимости/версии. В этот момент он обновляет или добавляет хеши, если их еще нет, а если хеши есть, то без указания явной команды они не обновятся. Этот механизм похож на схемы, реализуемые с помощью рассмотренных ранее файлов `glide.lock` и `godeps.json`, только тут контрольные суммы являются обязательной частью системы.

Пример `go.sum`:

```
github.com/pkg/errors v0.9.1 h1:FEBLx1zS214owpjy7qsBeixbURkuhQAwrK5UwLGTwt4=
github.com/pkg/errors v0.9.1/go.mod h1:bwawHFNVL2hUplrhADufv3IMtndRdf1r5NINE10=
```

Первая строка тут — это хеш самого исходника пакета, вторая — хеш его файла `go.mod`. Версия хеш-суммы обозначается тут как `h1`: — это указывает на то, что контрольная сумма вычислена по алгоритму SHA-256, а результат закодирован в `base64`. Такое указание введено с тем, чтобы у команды разработчиков Go была возможность в будущем ввести другие типы или версии расчета хешей.

GVM

Пакет GVM (Go Version Manager) не является обязательным для установки, но крайне желателен. Этот инструмент позволяет управлять установленными версиями Go. В основном это актуально, когда ведется работа сразу над несколькими проектами и в них могут использоваться разные версии Go. Если говорить о том, как вообще происходит повышение версии в уже давно написанном проекте, то для этого отводится отдельная ветка, где версию и обновляют. Важный момент заключается в том, что нельзя сразу «прыгать» на несколько версий вперед, — необходимо повышать версию поэтапно. Это связано с тем, что переход к другой версии может потребовать обновления зависимостей и библиотек, используемых в проекте, — они могут быть несовместимы с новой версией Go. Поэтому после каждого этапа повышения версии следует проводить полноценное тестирование всей функциональности проекта.

Для того чтобы установить `gvm` на Linux или macOS, нужно выполнить ряд шагов:

1. Открыть терминал и выполнить команду для скачивания скрипта установки:

```
bash < <(curl -s -S -L
https://raw.githubusercontent.com/moovweb/gvm/master/binscripts/gvm-installer)
```

2. Закрыть и заново открыть терминал либо выполнить команду обновления текущей сессии терминала:

```
source ~/.bashrc
```

3. Проверить, установлен ли `gvm`, с помощью команды:

```
gvm list
```

Если в терминале будет показан список установленных версий go, значит, всё установлено правильно.

Установить нужную версию Go можно с помощью команды:

```
gvm install 20.0.0
```

После установки версия Go будет выбрана автоматически. Проверить, что версия Go установлена правильно, можно командой:

```
go version
```

Переключаться между уже установленными версиями Go позволит команда:

```
gvm use go <нужная версия>
```

Установка Protobuf

Protobuf (Protocol Buffers) — это механизм сериализации структурированных данных, подробнее мы познакомимся с ним, когда будем разбираться с протоколом gRPC в главе 3, а сейчас займемся установкой Protobuf Compiler. Скачайте его¹, выбрав удобный вариант, и распакуйте. После установки перезапустите терминал и убедитесь, что всё прошло успешно, проверив установленную версию Protocol Buffers с помощью команды:

```
protoc --version
```

Вам также понадобится плагин protoc-gen-go — чтобы генерировать Go-код из proto-файлов. Это позволит упростить взаимодействие между программами на Go и другими сервисами через протокол gRPC или обычные Protocol Buffers (буферы протоколов). А для поддержки gRPC дополнительно потребуется плагин protoc-gen-go-grpc. Указанные плагины существуют для большинства популярных языков программирования: C++, Java, Kotlin, PHP, Python, Go, C#, JavaScript и других. Отдельно отмечу, что инструмент Protobuf может использоваться не только для взаимодействия через gRPC, но и через HTTP.

Установить плагины можно с помощью Go:

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
```

Теперь, чтобы сгенерировать Go-код из proto-файла, достаточно выполнить команду:

```
protoc -go_out=. -go-grpc_out=. yourfile.proto
```

Здесь:

- `--go_out=.` — создает стандартные структуры Protocol Buffers на Go;
- `--go-grpc_out=.` — генерирует код для gRPC на Go.

В результате мы получим из proto-файла два файла Go: с описанием структур сообщений и с gRPC-сервисом. Благодаря этому можно не отвлекаться на написание

¹ См. <https://protobuf.dev/installation/>.

самого сервиса и сериализации/десериализации данных вручную, поскольку этот процесс здесь полностью автоматизирован.

Git

Git — это система контроля версий, которая позволяет управлять историей изменений в проекте. Для установки Git нужно перейти на ее официальный сайт¹ и скачать установщик для своей операционной системы, а после установки проверить, что Git установлена корректно, выполнив в терминале команду:

```
git --version
```

Рассмотрим базовые концепции Git и то, как они могут применяться в процессе разработки приложений на Go.

❑ Создание репозитория.

Первое, с чего начинается работа над любым проектом, — создание репозитория. Он содержит историю развития проекта и является хранилищем всех его версий. Для создания репозитория необходимо выполнить команду `git init` — она и создаст пустой репозиторий в текущем каталоге. Выглядит он как созданная папка `.git`, в которой содержатся все необходимые объекты для работы Git.

❑ Добавление файлов.

Для отслеживания изменений в файлах необходимо их проиндексировать. Это можно сделать командой `git add`. Вы можете добавлять по одному файлу или сразу несколько файлов за раз. Так, команда `git add` добавляет в индекс сразу все файлы из текущего каталога.

❑ Коммиты.

Сохранение изменений в репозитории Git называется *коммитом* (commit). Команда `git commit` создает новый коммит, содержащий все изменения, добавленные в индекс. При создании коммита необходимо указать комментарий — как правило, это краткое описание того, что было сделано. Существует рекомендованный набор правил для написания комментариев в коммите — *conventional commits*. Он нужен в первую очередь для стандартизации формата сообщений, чтобы другим разработчикам было проще понять, что было изменено или исправлено. На основе этих комментариев также делают скрипты для автоматической генерации информации о выпущенных версиях и изменениях, которые были внесены.

❑ Ветки.

Ветки в Git позволяют вести параллельную работу над проектом и разделять историю коммитов на отдельные линии разработки. Команда `git branch` создает новую ветку, а команда `git checkout` позволяет переключаться между ветками. Это полезно, когда ведется длительная разработка над новой функциональностью

¹ См. <https://git-scm.com/>.

в отдельной ветке, а вам срочно нужно «починить баг» в основной версии, — тогда вы можете сохранить изменения в своей ветке и безболезненно переключиться на главную (master) версию.

❑ Слияние (merge).

Эта функциональность представляет собой объединение изменений из одной ветки в другую. Реализуется она командой `git merge`. При слиянии Git объединяет изменения из обеих веток, сохраняя при этом историю коммитов. Слияние может не завершиться автоматически, если в одни и те же файлы были внесены изменения в разных ветках, — тогда необходимо «вручную» разрешить эти конфликты, т. е. выбрать, какие изменения в файле должны быть в итоге внесены. Сейчас большинство редакторов кода наглядно подсвечивают конфликтные места и предлагают варианты решения.

Для удобной работы с Git рекомендуется использовать графический интерфейс или терминал. Одна из важных задач Git — отслеживание изменений в коде, а также контроль версий. Благодаря этому не так страшно допустить ошибку, потому что всегда можно вернуться на предыдущую версию. Также Git помогает в совместной работе над проектом, позволяя нескольким разработчикам работать с одним кодом. Важно понимать, что Git — это не только инструмент для контроля версий, но и способ организации рабочего процесса в команде. Процесс и правила для организации работы с репозиторием и управлением ветвлением для Git называются *git flow*. В первую очередь рекомендуется использовать *git flow* для сложных и больших проектов, которые имеют длительный жизненный цикл и множество разработчиков. В этой модели определены следующие ветки:

- ❑ Master (main) — ветка с готовым к релизу кодом;
- ❑ Develop — ветка, на которой ведется разработка следующей версии продукта. Веток Develop может быть несколько — для каждого разработчика или команды, а также для выкатки кода на develop-стенд с целью проверки работоспособности и интеграций, поскольку бывают ситуации, когда невозможно проверить всё локально;
- ❑ Feature — ветка, создаваемая для разработки новой функциональности. Ветки Feature должны отделяться от ветки Develop и сливаться обратно с ней после завершения работы;
- ❑ Release — релизная ветка, создаваемая для подготовки к выпуску новой версии продукта. В этой ветке производится исправление багов и подготовка документации. Release-ветка создается из ветки Develop и вливается в Master и Develop после ее завершения;
- ❑ Hotfix — ветка, создаваемая для решения критических багов в готовом продукте, обнаруженных после его релиза. Ветки Hotfix создаются из ветки Master и сливаются обратно в Master и Develop.

Как правило, в проектах реализуется *git flow* в той или иной редакции, причем команды часто адаптируют эти правила под себя. Например, всё чаще находит применение практика, когда есть еще ветки Test и Preprod, которые отвечают за

релизы на тестовых стендах. В больших проектах мало иметь для продакшена один стенд, поскольку для полноценного тестирования необходимо где-то еще разворачивать код, чтобы тестировщики могли проверить его работоспособность и обнаружить возможные баги. Поэтому к приведенному списку добавляются еще несколько веток, отвечающих за выкатку кода на соответствующие стенды:

- ❑ **Test** — ветка, на которой проводится тестирование тестировщиками, она соответствует тестовому стенду;
- ❑ **Preprod** — ветка с полноценной функциональностью, максимально приближенная к продакшену. Чаще всего используется для промежуточных показов заказчику и другим заинтересованным лицам.

Отдельно стоит отметить, что для Git есть три типа файлов: отслеживаемые, неотслеживаемые и игнорируемые. Файл считается *отслеживаемым*, если он проиндексирован или зафиксирован в коммите. *Неотслеживаемый* файл — это файл, который не проиндексирован. Так, при создании нового файла по умолчанию он считается не проиндексированным, но как только мы его проиндексируем, он станет считаться отслеживаемым. *Игнорируемый* файл — это файл, явным образом помеченный для Git как файл, который необходимо игнорировать. Это, как правило, установленные зависимости, артефакты сборки, скрытые системные файлы (.DS_Store), файлы, относящиеся к настройкам вашей IDE, и любые произвольные файлы, которые вы посчитаете нужным не распространять вместе с проектом. Игнорируемые файлы отмечаются в специальном файле .gitignore, который располагается в корневом каталоге проекта. Чтобы добавить игнорируемые файлы, необходимо вручную в этом файле их прописать — специальной команды для этого в Git нет.

Игнорируемые файлы необходимы, потому что не стоит загружать в удаленный репозиторий Git абсолютно всё из проекта, поскольку, во-первых, в таком случае репозиторий станет очень «тяжелым», а во-вторых, постоянно понадобится решать конфликты при слиянии, если вы работаете еще с кем-то.

Codestyle

Codestyle — это соглашения и правила форматирования кода, которые определяются для улучшения читаемости и поддержки кода. Обычно в подавляющем большинстве языков нет выработанного сообществом или авторами языка единого способа написания кода, поэтому в каждом проекте все команды договариваются между собой вести запись кода, как им удобно. В Go ситуация же обратная относительно остальных — стиль написания кода достаточно специфичный и строгий, это считается частью культуры языка.

Нужно это всё для того, чтобы в результате было неважно, кто именно работал над проектом, — код не должен различаться в зависимости от предпочтений его автора.

Для автоформатирования кода в Go существует стандартный инструмент `gofmt`. Применяют его в любой продуктовой разработке — он обеспечивает единое форматирование отступов, расстановку фигурных скобок и пробелов. Это позволяет

легко читать и понимать код в любом проекте и не вызывает лишних споров о том, как «правильно» писать.

Docker и Docker-Compose

Docker — это инструмент для контейнеризации приложений. Он позволяет упаковывать приложения и их зависимости в изолированные контейнеры, которые можно переносить на любую платформу, и в любой среде, независимо от хост-системы и окружения приложение будет работать одинаково.

Одной из главных причин использования Docker при локальной разработке является отсутствие необходимости ставить все сервисы, базы данных и прочие компоненты инфраструктуры приложения непосредственно на сам компьютер. Когда устанавливаешь на компьютер базу данных определенной версии, которая используется в одном вашем приложении, довольно затруднительно работать с другой версией базы данных для другого приложения. Docker решает эту проблему через контейнеризацию.

И если Docker — это инструмент для управления отдельными контейнерами (сервисами), из которых состоит приложение, то Docker-compose используется для управления одновременно несколькими контейнерами, входящими в состав проекта. Можно сказать, что Docker-Compose предоставляет те же возможности, что и Docker, только позволяет еще и работать с более сложными приложениями.

Для примера можно представить, что мы уже развернули наше готовое приложение, состоящее из трех сервисов, на какой-то машине. Если мы не станем использовать Docker-Compose, то на каждой отдельно взятой машине нам придется настраивать приложение заново, а с его использованием это можно будет сделать несколькими командами.

Всё, что нужно при разработке проекта, — это прописать в конфигурационный файл `docker-compose.yml` все необходимые зависимости и запустить всю необходимую инфраструктуру одной командой:

```
docker-compose up
```

Для того чтобы установить Docker-Compose, необходимо скачать установщик¹ и выполнить всё по инструкции.

Оборачивание самого приложения в Docker-Compose мы рассмотрим позже, когда будут готовы все микросервисы, а пока подготовим для работы базу данных, используя Docker-Compose. Где размещать файлы Docker-Compose, принципиального значения не имеет: можно в проекте создать отдельный каталог, а можно расположить всё отдельно от проекта. Для удобства и чтобы не путаться, создадим папку `docker` в корневом каталоге проекта и создадим там файл `docker-compose.yml` (листинг 1.1).

¹ См. <https://docs.docker.com/get-docker/>.

Листинг 1.1. Файл `docker/docker-compose.yml`

```
services:
  postgres:
    image: postgres:latest
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: account
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - app-network

volumes:
  postgres_data:

networks:
  app-network:
    driver: bridge
```

Файл `docker-compose` должен начинаться с тега версии. В настоящий момент актуальной считается версия 3.9. Затем мы описываем сервисы, учитывая, что один сервис = один контейнер. Сервисом считается база данных, сервер, клиент и остальное. Нам сейчас нужна только СУБД (система управления базами данных) PostgreSQL — поэтому сервис так и назовем: `postgres`. Далее указываем, что шаблоном для создания контейнера является образ `postgres:latest`, — `tag latest` означает, что нужно ссылаться на самый актуальный образ. Задаем название для контейнера (оно может быть произвольным) и пробрасываем порты, т. к. контейнеры будут запущены в изоляции от операционной системы (все порты между ОС и контейнерами окажутся закрытыми). Для этого указываем первым порт в локальной сети, а вторым — порт внутри контейнера (для `postgresql` порт 5432 считается стандартным). `Volumes` означает путь, где будут храниться артефакты для контейнера на хост-машине. Последняя опция нужна для непрерывного рестарта, если вдруг наша база по какой-то причине упадет. Есть для Docker-Compose и другие настройки, но на текущем этапе этого набора достаточно.

На вкладке **Logs** (логи) панели Docker Desktop (рис. 1.2) можно увидеть, что происходит с контейнером, на вкладке **Inspect** — посмотреть, какие переменные окружения установлены, их значения, а также многое другое.



Рис. 1.2. База данных account-db в Docker Desktop: логи

Для того чтобы запустить контейнер, нужно перейти в каталог, где находится ваш файл `docker-compose.yml`, и выполнить команду:

```
docker-compose up
```

В результате вы сможете подключаться к PostgreSQL через указанный порт, а если вдруг база данных в какой-то момент окажется больше не нужна, то достаточно просто удалить контейнер. Либо, если понадобится запустить другую версию для PostgreSQL, можно будет создать новый контейнер, указав другую версию базы данных и обозначив другой порт, чтобы не было конфликтов.

Тестирование API

API (Application Programming Interface) — это программный интерфейс приложений. Принцип работы API проще всего представить на классическом примере «клиент-сервер», а описать такое взаимодействие — воспользовавшись аналогией с ручками от ящика, где в роли ящика выступает сервер. Благодаря ручкам клиент может получать доступ к содержимому ящиков — т. е. к функциям сервера. Проще говоря, API — это интерфейс, благодаря которому одна система может взаимодействовать с другой.

Взаимодействовать с API в роли клиента можно через специальные инструменты. Самые распространенные из них это: cURL (служба командной строки), Swagger UI, Insomnia, Postman и другие, менее популярные [2]. Рассмотрим их подробнее.

❑ cURL (client URL) — служебная программа командной строки, позволяющая взаимодействовать с серверами по множеству различных протоколов с синтаксисом URL.

Выпущена она в 1996 году, и тогда это был единственный вариант тестировать API. Использование cURL может быть актуально, если размер оперативной памяти на вашей машине совсем небольшой. Плюс программы в том, что выполнить запрос можно на любой системе, ничего дополнительно не устанавливая, а вот минусов сильно больше: каждый запрос надо писать или заново, или где-то сохранять (либо каждый раз обращаться к истории запросов в командной строке), неудобно разбирать ответ, если это довольно большой текст в формате JSON, нельзя заранее настроить переменные, чтобы их переиспользовать в разных запросах, надо знать синтаксис для запросов и пр.

- ❑ **Swagger UI** — это набор инструментов, доступных через браузер, в случае, если разработчик добавил документацию в формате OpenAPI (теперь так называется спецификация Swagger).

Как правило, это полностью готовые инструменты с прописанными доступными значениями и endpoint'ами¹. Правда, такой подход может быть воспринят неоднозначно — и как положительный, и как отрицательный: заранее заданные значения могут быть как недостаточными, так и избыточными, а для того чтобы проверить разные сценарии, придется каждый раз заново редактировать запросы. Кроме того, помимо примеров запросов такие сценарии содержат еще и примеры ожидаемых полей в ответе. А из существенных минусов стоит отметить, что с помощью Swagger UI нельзя задавать никакие окружения (test, preprod), нельзя создавать собственные коллекции (наборы) запросов, поскольку нет возможности сохранять свои варианты этих коллекций. Чаще всего Swagger UI используется для сверки контрактов — чтобы параметры были названы, как ожидалось, и чтобы никакое поле не было пропущено.

- ❑ Инструменты **Insomnia** (рис. 1.3) и **Postman** (рис. 1.4) очень похожи по своей функциональности — с их помощью можно создавать окружения и коллекции запросов, импортировать проекты из Swagger, настраивать динамические переменные (если, например, нам нужно генерировать каждый раз уникальный uuidv4).

Интерфейс в обоих случаях достаточно интуитивно понятный, разве что в Postman он может показаться несколько громоздким, но разобраться с ним не так уж и сложно. Postman, кроме всего отмеченного, еще имеет множество встроенных скриптов-рандомайзеров, возможность создавать собственные JavaScript-скрипты и другое. К минусам Postman можно отнести тот момент, что коллекции и окружение к ним импортируются отдельно. Insomnia считается более простым инструментом и менее популярным по сравнению с Postman, но вполне покрывает необходимую функциональность при тестировании API для разработчика.

Резюмируя, отметим, что для наших задач наиболее подходящие инструменты — это Insomnia и Postman, но мы остановимся на Postman, т. к. нам пригодится обеспечиваемая им поддержка gRPC запросов, которой нет в Insomnia.

¹ Endpoint — конечный маршрут для URL. Например, если URL — это **google.com**, то один из его endpoint'ов будет **google.com/search**.

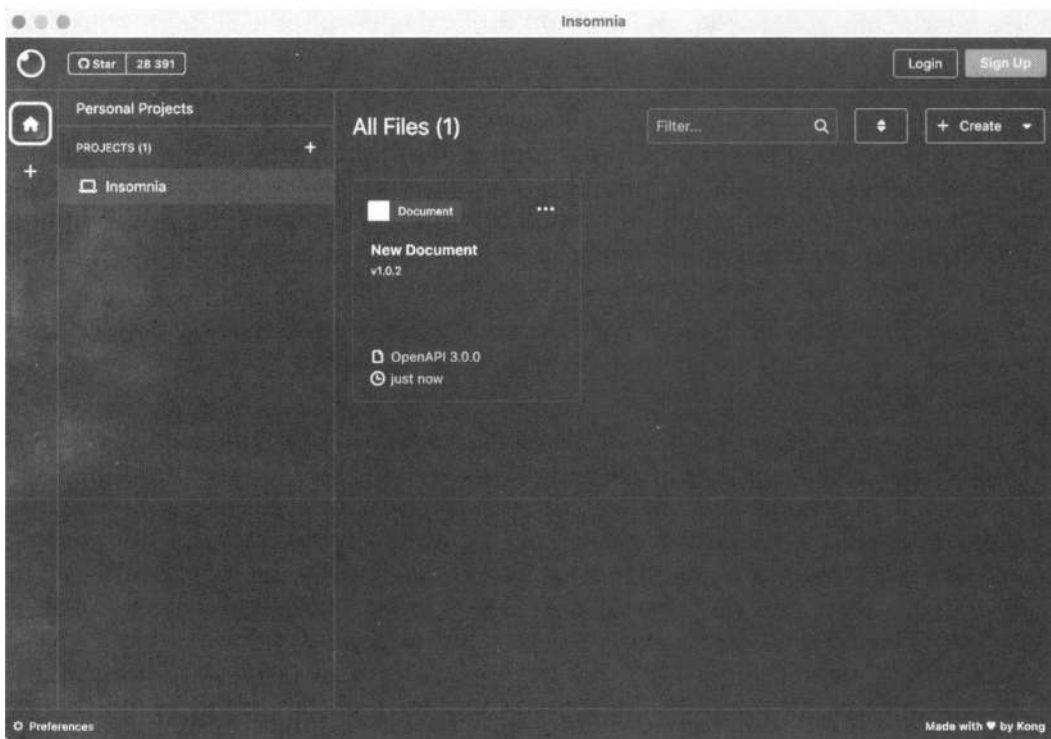


Рис. 1.3. Интерфейс Insomnia

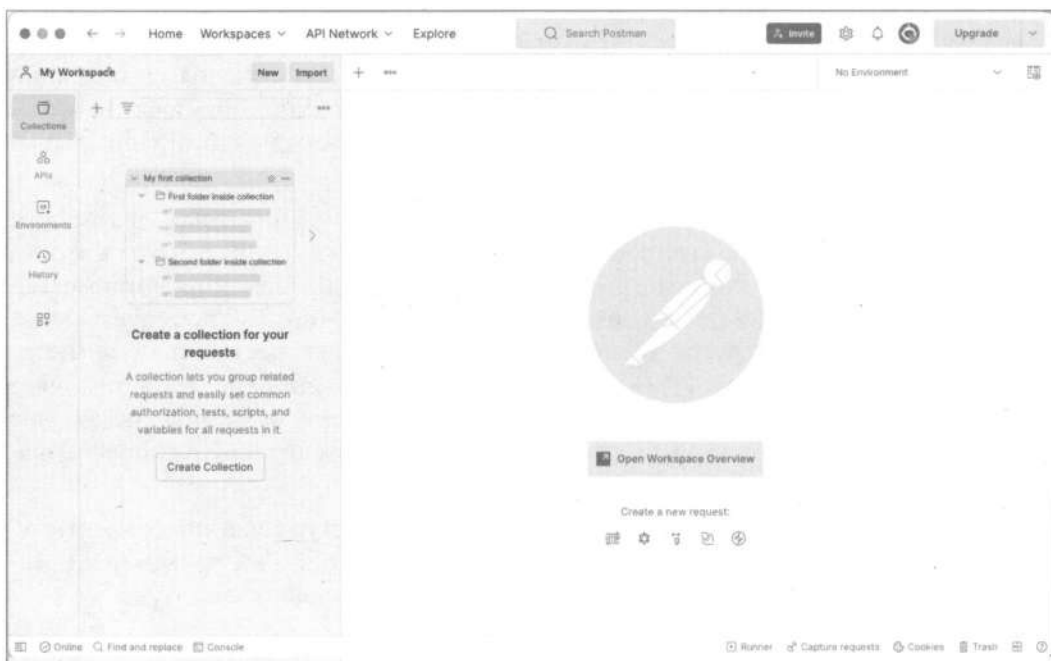


Рис. 1.4. Интерфейс Postman

* * *

Итак, мы рассмотрели и настроили необходимые для начала работы над проектом инструменты.

Создание структуры проекта

Первым шагом при разработке проекта на Go является определение основных компонентов приложения и организация структуры каталогов. Go не предоставляет инструментов автоматической генерации кода на уровне фреймворков, и в целом от концепции фреймворков отличается — в Go считается правильным собирать проект из небольших библиотек, чтобы сохранять полную прозрачность и контроль над архитектурой. Поэтому здесь гораздо чаще можно встретить модульные или чистые архитектурные подходы.

Сначала создадим каталог (пакет) Account — т. к. у нас, помимо основной логики сущности User, могут в том же репозитории иметься и связанные модули. Например, если мы захотим знать адреса пользователей, хорошей практикой считается сделать для этого отдельную сущность Address и построить между ними связь «один ко многим» (в одном городе может быть много пользователей). Подобное разнесение логики позволяет в дальнейшем упростить разработку. При таком подходе, например, если нам понадобится сохранять отдельно адрес, по которому была проведена транзакция, мы также установим связь «один ко многим» (один город — много транзакций). В случае, если мы приняли бы решение хранить города в одной таблице с пользователями, у нас произошло бы дублирование информации: мы бы хранили адреса и для пользователей, и для транзакций. Это всё излишние расходы на ресурсы, потому что, во-первых, данные занимают место, за которое нужно платить, а во-вторых, излишние данные влияют на скорость выполнения запросов к базе данных.

В каталоге Account мы будем размещать основной код для работы с пользователями и связанными сущностями. Создать заготовку структуры можно простой командой:

```
mkdir account & cd account  
go mod init account
```

Теперь приступим к созданию структуры проекта. На этом этапе закладывается фундамент для масштабируемости, удобства сопровождения кода, организации бизнес-логики и инфраструктурных компонентов. Переходя к созданию структуры проекта, рассмотрим, какие элементы и каталоги должны присутствовать в сервисе, чтобы проект был устойчивым к росту и изменениям, а также соответствовал распространенным практикам микросервисной архитектуры.

Оптимальная структура проекта должна решать несколько задач: обеспечивать читаемость кода, облегчать навигацию, отделять бизнес-логику от технических деталей и позволять масштабировать сервис без необходимости масштабных рефакторингов.

Начнем с создания каталога cmd, в котором размещаются точки входа для всех исполняемых файлов проекта. Главный файл запуска приложения — main.go, именно

он всегда отвечает за запуск любого Go-проекта. Как правило, именно тут настраивается конфигурация сервиса и запускается сервер. Размещение файла `main.go` в отдельном каталоге `cmd` позволяет четко разграничить стартовую логику приложения от его основной бизнес-логики и инфраструктурных компонентов.

При создании файла `main.go` пакет по умолчанию будет назван так же, как и каталог, в котором он находится, однако в Go только пакет с именем `main` может быть скомпилирован в самостоятельный исполняемый файл. Любой файл, где объявлен `package main` и есть функция `main`, считается точкой входа и после сборки становится самостоятельной программой.

Для внутренней логики принято использовать каталог `internal`. Весь код, относящийся к бизнес-логике, работе с базой данных, обращению к другим сервисам, помещается именно туда. Согласно соглашению Go всё, что находится в `internal`, не может быть импортировано извне — это гарантирует, что внутренние детали реализации не будут случайно использоваться в других проектах. Чаще всего внутреннее деление на модули состоит из следующих пакетов:

- ❑ `service` (или `app`) — тут размещается вся бизнес-логика приложения, этот слой не зависит и не должен ничего знать об инфраструктуре или внешнем вводе/выводе, он работает только с внутренними моделями и интерфейсами;
- ❑ `model` — содержит все основные структуры данных, используемые в бизнес-логике, часто разделяется на отдельные файлы для разных сущностей;
- ❑ `repository` (или `storage`) — пакет для взаимодействия с внешними системами хранения данных (например, базой данных), тут же находятся и методы, отвечающие за создание, получение, удаление и обновление данных;
- ❑ `api` (`transport`) — название говорит само за себя, тут находится всё, что касается взаимодействия системы с внешним миром: обработчики для HTTP, gRPC, WebSocket-запросов и другие, сюда в первую очередь попадают и проходят валидацию входные данные, здесь вызываются методы бизнес-логики, формируются и отправляются ответы;
- ❑ `migrations` — миграции, которые необходимы для изменения состояния базы данных, также размещаются в отдельном пакете;
- ❑ `config` — всё, что касается конфигурации сервиса: парсинг переменных окружения, загрузка и валидация конфигов;
- ❑ `pkg` — сюда помещается весь код, который допустимо переиспользовать из других сервисов или пакетов.

Помимо основных указанных здесь пакетов, могут быть по необходимости созданы также и другие, в этом нет ограничений.

Важной частью любого Go-приложения является автоматизация рутинных задач: форматирование кода, запуск тестов, сборка, миграции, запуск линтеров, пересборка Docker-образов и деплой. Чтобы не запоминать и не набирать каждый раз вручную эти команды, часто используют такой простой и мощный инструмент автоматизации, как `Makefile` — специальный файл, в котором описываются все не-

обходимые сценарии для автоматизации задач. Изначально он поддерживается большинством UNIX-подобных систем, но для Windows его также можно задействовать, хотя и с некоторыми нюансами: по умолчанию для него нет утилиты `make`, но ее можно установить отдельно, некоторые команды, привычные для UNIX-систем (например, `rm`, `cp`, `ls`), также придется заменить на команды, подходящие для Windows, либо использовать их через утилиты UNIX-оболочки. Есть и другие удобные и кросс-платформенные инструменты для этой задачи, но т. к. в продуктовой разработке гораздо чаще встречается этот инструмент, будем использовать именно его.

Итак, создадим `Makefile` (листинг 1.2) и рассмотрим содержащиеся в нем команды.

Листинг 1.2. Файл `Makefile`

```
# Go application Makefile
APP_NAME ?= account
BINARY_NAME ?= bin/$(APP_NAME)
MAIN_PATH ?= ./cmd
VERSION ?= $(shell git describe --tags --always --dirty 2>/dev/null || echo "dev")
BUILD_TIME ?= $(shell date -u '+%Y-%m-%d_%H:%M:%S')
LD_FLAGS ?= -ldflags "-X main.Version=$(VERSION) -X main.BuildTime=$(BUILD_TIME)"

# Go files to format and lint
GOFILES ?= $(shell find . -name "*.go" -type f -not -path "./vendor/*" -not -path "./.git/*")

# Default target
.DEFAULT_GOAL := build

# Build the application
build:
    @echo "Building $(APP_NAME)..."
    @mkdir -p bin
    go build $(LD_FLAGS) -o $(BINARY_NAME) $(MAIN_PATH)

# Run the application
run: build
    @echo "Running $(APP_NAME)..."
    ./$(BINARY_NAME)

# Run without building (if binary exists)
run-only:
    @echo "Running $(APP_NAME)..."
    ./$(BINARY_NAME)

# Install dependencies
deps:
    @echo "Installing dependencies..."
    go mod download
    go mod tidy
```

```
# Update dependencies
deps-update:
    @echo "Updating dependencies..."
    go get -u ./...
    go mod tidy

# Format code
fmt:
    @echo "Formatting code..."
    gofmt -s -w $(GOFILES)
    gofumpt -l -w -s $(GOFILES)
    goimports -l -w $(GOFILES)

# Run linter
lint:
    @echo "Running linter..."
    golangci-lint run --timeout=10m -v ./...

# Run tests
test:
    @echo "Running tests..."
    go test -v ./...

# Run tests with coverage
test-coverage:
    @echo "Running tests with coverage..."
    go test -v -coverprofile=coverage.out ./...
    go tool cover -html=coverage.out -o coverage.html
    @echo "Coverage report generated: coverage.html"

# Run benchmarks
bench:
    @echo "Running benchmarks..."
    go test -bench=. ./...

# Clean build artifacts
clean:
    @echo "Cleaning build artifacts..."
    rm -rf bin/
    rm -f coverage.out coverage.html

# Install the application
install: build
    @echo "Installing $(APP_NAME)..."
    go install $(LDFLAGS) $(MAIN_PATH)
```

```
# Generate documentation
docs:
    @echo "Generating documentation..."
    godoc -http=:6060

# Show help
help:
    @echo "Available targets:"
    @echo "  build      - Build the application"
    @echo "  run        - Build and run the application"
    @echo "  run-only   - Run the application (without building)"
    @echo "  deps       - Install dependencies"
    @echo "  deps-update - Update dependencies"
    @echo "  fmt        - Format code"
    @echo "  lint       - Run linter"
    @echo "  test       - Run tests"
    @echo "  test-coverage - Run tests with coverage report"
    @echo "  bench      - Run benchmarks"
    @echo "  clean      - Clean build artifacts"
    @echo "  install    - Install the application"
    @echo "  docs       - Generate documentation"
    @echo "  help       - Show this help"

# Development workflow
dev: fmt lint test build

# CI/CD workflow
ci: deps fmt lint test build

.PHONY: build run run-only deps deps-update fmt lint test test-coverage bench clean install docs help dev ci
```

Мы включили сюда команды для запуска и сборки приложения, запуска без сборки, установки и обновления зависимостей, форматирования кода и линтера, прогона тестов, генерации документации и др. Этого набора достаточно для разработки — чтобы не отвлекаться, вспоминая, какая команда запускает ту или иную функциональность, а при необходимости его всегда можно дополнить.

Стоит также учесть конфигурацию приложения, включающую некоторые глобальные значения, которые хранить в самом коде считается небезопасным, — например, пароль от базы данных. Если держать такие данные в коде как обычные переменные, то к ним будут иметь доступ все, кто может просматривать репозитории или у кого случайно в результате какого-либо сбоя или слива окажется исходный код. Вторая проблема такого подхода в том, что редактировать эти данные становится очень сложно: они разбросаны по всему коду, и попробуй найди еще, где нужен пароль. Впрочем, обычно развертыванием приложений на стенде занимаются отдельные специалисты — devops. В идеале им нет необходимости знать, как работает код и что он вообще делает, — достаточно только запустить приложение

на виртуальной машине. Поэтому для таких целей используют *переменные окружения* (environment variables).

Как правило, хранятся эти переменные окружения в специальном файле: `.env`. Для удобной доставки этого файла между разработчиками команды его часто записывают как `.env.example` (листинг 1.3). А сам файл `.env` при этом у каждого свой, со своими значениями, и чтобы случайно он не оказался в репозитории, его принято добавлять в файл `.gitignore`. Смысл такого подхода в том, чтобы измененные переменные окружения в целях безопасности не попали в репозиторий, а также не сломали все приложения другому разработчику, потому что у него могут быть разные пароли от баз данных или базы развернуты на разных портах. К тому же при деплое приложения в разные окружения: продакшен или тестовую среду, параметры приложения обычно различаются — содержат разные пароли и разные адреса сервисов, с которыми интегрируются в рамках этого же окружения.

Листинг 1.3. Файл `.env.example`

```
### BASE
SERVICE_NAME=account
APP_ENV=local
LOG_LEVEL=debug

### HTTP
HTTP_PORT=9000
HTTP_HOST=localhost
```

Мы описали здесь конфигурацию, название сервиса, тип окружения (самые распространенные мы уже рассматривали ранее), уровень логирования и всё, что касается HTTP-соединения: порт, хост.

Подробнее о том, какие переменные окружения для чего используются, можно описать в файле `README.md` (листинг 1.4). Вообще, цель документирования в первую очередь в том, чтобы другим разработчикам было как можно легче разобраться в коде, и в том, что для чего используется.

Листинг 1.4. Файл `README.md`

Конфигурация сервиса Account

Переменная	Окружения	Назначение	По умолчанию
SERVICE_NAME		Название сервиса	Account
APP_ENV		Окружение	local
LOG_LEVEL		Уровень логирования	in
HTTP_PORT		Порт, на котором слушает приложение	9000
HTTP_HOST		Хост, на котором поднято приложение	localhost

Еще одним важным шагом при создании нового проекта является настройка списка игнорируемых файлов — тех, которые не должны попадать в репозиторий. Для

этого в корне проекта создается файл `.gitignore`. Как уже упоминалось ранее, этот файл служит для того, чтобы хранить в репозитории только действительно необходимые артефакты, тем самым предотвращая случайную отправку во внешний репозиторий временных, лишних или чувствительных данных. В `.gitignore` обычно заносят файлы и каталоги, которые не предназначены для общего доступа или не несут ценности для истории версий. В листинге 1.5 приведен файл `.gitignore`, содержащий базовый перечень таких файлов, рекомендуемый для большинства проектов при их инициализации.

Листинг 1.5. Файл `.gitignore`

```
# Binaries for programs and plugins
*.exe
*.exe~
*.dll
*.so
*.dylib

# Test binary, built with `go test -c`
*.test

# Output of the go coverage tool, specifically when used with LiteIDE
*.out

# Go coverage files
coverage.txt
coverage.html
coverage.out

# Dependency directories (remove the comment below to include it)
# vendor/

# Go workspace file
go.work

# Environment variables
.env
.env.local
.env.development
.env.test
.env.production

# IDE files
.vscode/
.idea/
*.swp
*.sw0
*~
```

```
# OS generated files
.DS_Store
.DS_Store?
._*
.Spotlight-V100
.Trashes
ehthumbs.db
Thumbs.db

# Logs
*.log
*.log.*

# Build artifacts
build/
dist/
bin/

# Temporary files
tmp/
temp/

# Go specific
*.prof
*.pprof
```

Здесь учтены файлы сборки проекта, логов проекта (туда обычно записываются возникающие ошибки и не только), файлы DS_Store (принадлежность операционной системы macOS), файлы, сгенерированные тестами и показывающие, например, насколько приложение покрыто тестами (coverage), файлы, связанные с вашей IDE (каталог .ide), некоторые локальные настройки. В основном этого набора достаточно, но при необходимости его можно отредактировать и добавить то, чего не хотелось бы видеть в репозитории.

Подключение необходимых библиотек

Переменные окружения

В Go доступ к переменным окружения осуществляется с помощью стандартной библиотеки `os`. Используя функции вроде `os.Getenv` и `os.LookupEnv`, можно получать значения переменных окружения, установленных для текущего процесса. Это позволяет удобно работать с конфигурацией приложения, извлекая необходимые параметры из среды выполнения.

Для преобразования переменных среды в структуру, с которой уже можно будет напрямую удобно работать, используем простую библиотеку, в которой нет лишних зависимостей, и для ее установки выполним команду:

```
go get github.com/caarlos0/env/v10
```

Структура конфигурации сервиса при этом будет выглядеть следующим образом:

```
type Config struct {
    ServiceName string `env:"SERVICE_NAME" required:"true"`
    AppEnv      string `env:"APP_ENV" required:"true" default:"development"`
    Host        string `env:"HTTP_HOST" required:"true" default:"localhost"`
    Port        int    `env:"HTTP_PORT" required:"true" default:"9000"`
    LogLevel    string `env:"LOG_LEVEL" required:"true" default:"info"`
}
```

Тег `env` тут используется для связывания полей структуры конфигурации с соответствующими переменными окружения (листинг 1.6). Кроме тега `env`, можно также применять дополнительные теги — указывающие на то, что наличие той или иной переменной обязательно (`required`), и определяющие значение по умолчанию (`default`).

Листинг 1.6. Файл `internal/config/config.go`

```
package config

import (
    "log"

    "github.com/caarlos0/env/v10"
    "github.com/joho/godotenv"
)

// Config содержит конфигурацию приложения
type Config struct {
    ServiceName string `env:"SERVICE_NAME" required:"true" default:"account-service"`
    AppEnv      string `env:"APP_ENV" required:"true" default:"development"`
    Host        string `env:"HTTP_HOST" required:"true" default:"localhost"`
    Port        int    `env:"HTTP_PORT" required:"true" default:"9000"`
    LogLevel    string `env:"LOG_LEVEL" required:"true" default:"info"`
}

// Load загружает конфигурацию из переменных окружения
func Load() (*Config, error) {
    // Загружаем .env файл
    if err := godotenv.Load(); err != nil {
        log.Println("Warning: .env file not found, using system environment variables")
    }

    cfg := &Config{}
    err := env.Parse(cfg)
    if err != nil {
        return nil, err
    }
}
```

```
    return cfg, nil  
}
```

В go для подключения внешних и стандартных библиотек используется ключевое слово `import`. Мы уже обсуждали стиль написания кода в go, так вот, согласно этому стилю, порядок импортируемых пакетов тоже имеет значение: сначала должны идти стандартные пакеты, которые являются частью языка (в нашем случае `log`), затем пустая строка для разделения и потом уже сторонние пакеты. Стандартный пакет `log` предоставляет функции для вывода сообщений, ошибок, информации и т. п. в стандартный вывод или в файл.

Чтобы приложение автоматически использовало переменные окружения из файла `.env`, используем еще один внешний пакет, для чего выполним команду:

```
go get github.com/joho/godotenv
```

Логирование

В микросервисах *логирование* — это важный инструмент для управления и наблюдения за работой системы. Особенно, когда сервисы являются распределенными, общаются между собой по сети и зависят от множества внешних факторов. Логи позволяют следить за внутренним устройством системы, благодаря им можно понять, как код функционирует, обрабатывает запросы, какие ошибки из-за него возникают, а также как система взаимодействует с другими сервисами. Логирование помогает разработчику восстановить порядок действий пользователя, чтобы отследить, на каком этапе и почему у него возникла ошибка. Иногда одна ошибка может породить за собой целую цепочку сбоев системы.

Кроме отладки и поиска неисправностей, логи играют важную роль в обеспечении безопасности. Записывая ключевые события, например изменения критичных данных, подозрительные запросы или неверные попытки авторизации, мы создаем основу для аудита. Важно при этом не забывать следить за тем, что именно попадает в логи — там не должно оказаться никаких паролей или персональной информации, т. к. это может повлечь за собой утечку чувствительных данных.

В современных системах логирование делают структурированным, что позволяет писать логи в удобном для автоматической обработки событий формате и собрать централизованную систему, где можно будет все логи быстро фильтровать по нужным полям, строить статистику и находить закономерности.

Помимо расследования инцидентов, логирование предоставляет фундамент для мониторинга за системой. Когда у вас становится сервисов всё больше, невозможно полагаться только на ручные проверки состояния системы. Нужно наглядно видеть, как система живет в режиме реального времени, и логи становятся при этом одним из главных источников такой информации.

В момент, когда возникает подозрительно много ошибок конкретного типа, — например, если ошибки 504 превысили допустимую для нашей системы норму, это можно считать сигналом о том, что у нас что-то идет не так. Обработку таких сиг-

налов можно автоматизировать, настроив алерты. Как только в потоке логов появляется подозрительный паттерн, мы получаем уведомление, — и у нас есть шанс исправить проблему еще до того, как пользователь ее заметит.

Для мониторинга логов и оповещения о событиях существуют разные инструменты: Prometheus, Loki и др. Так, мы можем, например, считать количество определенных событий за минуту, отслеживать время отклика в разных методах или мониторить время ответа API от других систем. Лог при этом фактически становится метрикой, а метрика — основа для графиков и предупреждений. Таким образом, логи позволяют не только расследовать инциденты, когда они уже и произошли, но и вовремя реагировать на них, а в некоторых случаях — даже предотвращать их.

В качестве основы для логирования (листинг 1.7) мы примем библиотеку zerolog. Она отличается высокой скоростью работы и при этом остается очень гибкой. Библиотека поддерживает вывод в JSON, что часто применяется для кода в среде продакшен, а также позволяет настроить цветной человекочитаемый вывод, что гораздо удобнее для локальной разработки. Для печати структур в развернутом виде мы воспользуемся пакетом spew, который умеет рекурсивно обходить структуры и формировать аккуратный многострочный вывод.

Листинг 1.7. Файл internal/logger/logger.go

```
package logger

import (
    "fmt"
    "os"
    "regexp"
    "strings"

    "account/internal/config"

    "github.com/davecgh/go-spew/spew"
    "github.com/fatih/color"
    "github.com/rs/zerolog"
)

func New(cfg *config.Config) zerolog.Logger {
    writer := zerolog.ConsoleWriter{
        Out:      os.Stdout,
        TimeFormat: "2006-01-02 15:04:05",
        FormatLevel: func(i interface{}) string {
            if i == nil {
                return ""
            }
            level := strings.ToUpper(fmt.Sprintf("%s", i))
            switch level {
            case "INFO":
                return color.New(color.FgGreen).Sprint("INF")
            }
        },
    }
    log := zerolog.New(writer).With().Timestamp().Logger()
    log.Info().Msg("Logger initialized")
    return log
}
```

```

    case "WARN":
        return color.New(color.FgYellow).Sprint("WRN")
    case "ERROR":
        return color.New(color.FgRed).Sprint("ERR")
    case "DEBUG":
        return color.New(color.FgBlue).Sprint("DBG")
    default:
        return level
    }
},
)

log := zerolog.New(writer).
    With().
    Timestamp().
    Str("source", cfg.ServiceName).
    Str("env", cfg.AppEnv).
    Logger()

// Настройки spew
spew.Config.Indent = "  "
spew.Config.DisablePointerAddresses = true
spew.Config.DisableCapacities = true
spew.Config.SortKeys = true

// Генерируем дамп
dump := spew.Sdump(cfg)

// Регэксп для поиска ключей struct'ов в формате spew: "FieldName:"
keyColor := color.New(color.FgCyan).SprintfFunc()
re := regexp.MustCompile(`(?m)^(?s*)([A-Za-z0-9_]+):`)
coloredDump := re.ReplaceAllStringFunc(dump, func(s string) string {
    matches := re.FindStringSubmatch(s)
    if len(matches) != 3 {
        return s
    }
    return fmt.Sprintf("%s%s:", matches[1], keyColor(matches[2]))
})

log.Info().Msg("Loaded configuration:")
log.Info().Msg(coloredDump)

return log
}

```

Идея здесь заключается в том, чтобы при вызове функции `logger.New()` создавался единственный экземпляр логгера, который уже будет готов к использованию во

всем проекте. Этот логгер выводит сообщения с указанием времени, а уровни логов имеют разные цвета, чтобы можно было быстро отличать информационные сообщения от ошибок и предупреждений. Сразу после инициализации в лог попадает вся конфигурация приложения в структурированном виде, что особенно полезно для диагностики. В коде мы создаем `ConsoleWriter` с нужным форматом времени и определяем, как будут отображаться уровни логов. Например, информационные сообщения окрашиваются в зеленый цвет, предупреждения — в желтый, ошибки — в красный, а отладочные сообщения — в синий. Такой подход позволяет визуально ориентироваться в потоке сообщений, даже если лог достаточно объемный.

Далее мы добавляем базовые поля — имя сервиса и окружение, из которого он запущен. Эти поля будут автоматически добавляться ко всем последующим записям. После этого с помощью `spew` формируем дамп конфигурации, а затем дорабатываем его с помощью регулярных выражений, чтобы выделить ключи структуры цветом. Результат выводится в лог в несколько строк, сохраняя при этом всю структуру объекта конфигурации.

В файле `main.go` использование этого логгера выглядит максимально просто. Мы загружаем конфигурацию, передаем ее в `logger.New()`, а возвращенный экземпляр сохраняем в структуре приложения. После этого логгер становится доступен во всех методах и модулях, которые работают с этой структурой. Такой подход исключает необходимость повторной инициализации и делает логирование единообразным по всему проекту.

При запуске программы мы получаем аккуратный вывод: сначала появляется строка с сообщением, что конфигурация загружена, затем — многострочный цветной дамп всех параметров, а следом — сообщения о ходе запуска. Таким образом, еще до выполнения основной логики приложения мы уже видим всю важную информацию об окружении.

```
~/work/book/account > make run 17:53:25
Cleaning build artifacts...
rm -rf bin/
rm -f coverage.out coverage.html
Building account...
go build -ldflags "-X main.Version=dev -X main.BuildTime=2025-08-09_14:53:28" -o
bin/account ./cmd
Running account...
./bin/account
2025-08-09 17:53:28 INF Loaded configuration: source=account
2025-08-09 17:53:28 INF (*config.Config){
  ServiceName: (string) (len=7) "account",
  AppEnv: (string) (len=5) "local",
  Host: (string) (len=9) "localhost",
  Port: (int) 9000,
  LogLevel: (string) (len=5) "debug"
})
source=account
2025-08-09 17:53:28 INF service starting up source=account
~/work/book/account > 17:53:28
```

Рис. 1.5. Запущенный сервис

Этот подход позволяет централизовать настройку логирования и избавляет от необходимости думать о форматировании в каждом отдельном модуле, а благодаря передаче логгера в качестве зависимости мы избегаем глобальных переменных и сохраняем модульность кода.

Чтобы продемонстрировать работу приложения на текущем этапе — выполним команду `make run` (рис. 1.5).

Запущенный проект сразу же завершится, потому что он еще не работает как сервис, т. е. не слушает никакой порт и не имеет обработчиков.

ORM

Для удобной работы с запросами к базам данных часто используются различные ORM-пакеты (Object Relational Mapping, объектно-реляционное отображение). Разбирая эту аббревиатуру по словам, поясним, что Object отвечает за часть, относящуюся в приложении к предметной области/модели, Relational — за часть отношений между таблицами и СУБД (системой управления базами данных), а Mapping — это представление из таблиц в наши описанные модели. То есть ORM — это инструмент для сопоставления сущностей в приложении с таблицами базы данных. ORM позволяет упростить процесс разработки за счет автоматизации преобразования объекта в таблицу и наоборот. Это нужно для того, чтобы убрать однообразную работу с запросами и преобразованием данных. Когда проект развивается, база данных увеличивается, как и количество таблиц, связи между ними усложняются, и простые операции по созданию или изменению сущностей начинают повторяться множество раз, ORM превращают это в лаконичные вызовы методов, позволяя сосредоточиться на логике приложения, а не на механике работы базы данных.

Но бывает и так, что разработчики предпочитают не использовать никакие ORM в своих проектах, особенно когда нужен максимальный контроль над работой с базой. ORM хотя и упрощает такую работу, но все равно взаимодействует с данными через SQL-запросы, которые самостоятельно генерирует, и результат таких сгенерированных скриптов не всегда может быть оптимальным. Это также может сказываться на производительности, и если на простых задачах типа «получить объект по идентификатору» падение производительности будет почти незаметно, то на больших объемах ORM может сильно проигрывать. Это связано с тем, что между кодом и базой появляется дополнительная прослойка, которая должна интерпретировать команды, генерировать SQL и выполнить маппинг результатов. На небольших проектах это не критично, но на высоконагруженных сервисах задержка может быть ощутимой.

А ситуация, когда вам все равно приходится писать сложные нестандартные SQL-запросы вручную, полностью ломает иллюзию автоматизированного контроля за взаимодействием с базой данных.

При работе с базой данных в Go всегда стоит выбор между удобством и контролем. ORM позволяет описывать данные в виде структур и манипулировать ими методами, что делает код чище, компактнее и прозрачнее для чтения. Это ценится, когда

важна скорость разработки и возможность быстро менять модель данных. Чистый SQL, в свою очередь, дает полный контроль. Разработчик может сам принять решение, как использовать индексы и оптимизировать каждый запрос под конкретную задачу. Писать SQL напрямую сложнее и дольше, зато это дает предсказуемый результат.

На практике в Go редко выбирают что-то одно. Чаще всего практикуется смешанный подход: ORM применяют для стандартных и простых операций, а чистый SQL — для сложных запросов и оптимизации производительности.

Самый известный и используемый ORM-пакет в Go — это GORM. Он появился, когда Go только набирал популярность, и за счет своих возможностей быстро стал неофициальным стандартом. GORM поддерживает автоматические миграции, работу со связями («один к одному», «один ко многим», «многие ко многим»), мягкое удаление записей и многое другое. С другой стороны, эта гибкость достигается не самым маленьким размером пакета и не всегда быстрой работой, особенно если не использовать никаких оптимизаций.

Для установки GORM в проект необходимо выполнить в терминале команду:

```
go get gorm.io/gorm gorm.io/driver/postgres
```

Она устанавливает пакет `gorm.io/gorm`, предоставляющий ORM-функциональность для Go и используемый для настройки подключения к базе данных, пакет `gorm.io/gorm gorm.io/driver/postgres` — драйвер для работы непосредственно с PostgreSQL (напрямую он использовать не будет, но эта зависимость также необходима)

Сразу настроим в проекте подключение к базе данных, добавив необходимые переменные окружения в файл `.env` (листинг 1.8).

Листинг 1.8. Файл `.env`

```
### BASE
SERVICE_NAME=account
NODE_ENV=local

### HTTP
HTTP_PORT=9000
HTTP_HOST=localhost
HTTP_PREFIX=/api/account
HTTP_VERSION=1

### DATABASE
DB_DSN=postgres://postgres:postgres@localhost:5432/account?sslmode=disable
```

Тут мы добавили для работы с базой данных `DB_DSN` — адрес, по которому следует подключаться к базе данных (он иногда встречается, когда его формируют из отдельно заданных полей).

Для работы с базой данных мы воспользуемся паттерном «репозиторий». Это поможет отделить бизнес-логику от деталей хранения данных, что позволит изменять

или оптимизировать слой доступа к базе без необходимости переписывать остальную часть приложения.

Создадим для этого отдельный каталог `internal/repository`, в котором будет храниться весь код, связанный с доступом к базе данных. Такой подход упрощает поддержку проекта и делает его структуру интуитивно понятной для всех. Основным файлом этого слоя станет `repository.go` — своеобразная точка входа в слой данных, тут будут объявлены методы для чтения, удаления, создания или обновления записей (листинг 1.9).

Листинг 1.9. Файл `internal/repository/repository.go`

```
package repository

import (
    "github.com/rs/zerolog"
    "gorm.io/gorm"
)

type Repository struct {
    db      *gorm.DB
    logger  *zerolog.Logger
}

func NewRepository(db *gorm.DB, logger *zerolog.Logger) *Repository {
    return &Repository{
        db:      db,
        logger:  logger,
    }
}
```

Само же подключение находится в точке входа в приложение — `cmd/main.go`, т. к. слой репозитория не должен отвечать за установление соединения с базой данных, а только работать с готовым объектом `*gorm.DB`. Теперь файл `main.go` будет выглядеть следующим образом (листинг 1.10).

Листинг 1.10. Файл `cmd/main.go`

```
package main

import (
    "log"

    "account/internal/config"
    "account/internal/logger"
    "account/internal/repository"

    "gorm.io/driver/postgres"
    "gorm.io/gorm"
)
```

```
func main() {  
    // Загружаем конфигурацию  
    cfg, err := config.Load()  
    if err != nil {  
        log.Fatalf("Failed to load config: %v", err)  
    }  
  
    // Инициализируем общий логгер с названием сервиса  
    logger := logger.New(cfg)  
  
    // Подключаемся к базе данных  
    db, err := gorm.Open(postgres.Open(cfg.DbDsn), &gorm.Config{})  
    if err != nil {  
        logger.Error().Msgf("Failed to connect to database: %v", err)  
        return  
    }  
    logger.Info().Msg("database connected")  
  
    // Инициализируем репозиторий  
    repo := repository.NewRepository(db, &logger)  
    _ = repo  
  
    // Пример использования общего логгера  
    logger.Info().Msg("service starting up")  
}
```

Swagger

Ранее мы уже немного касались темы инструмента Swagger — в рамках описания подходов к тестированию API, но на самом деле основное его назначение — в документировании API. Если обратиться к определению этого инструмента, то мы узнаем, что это язык описания интерфейса для RESTful API.

RESTful API — это популярный архитектурный подход для создания API (аббревиатура REST расшифровывается как Representation State Transfer, что буквально означает «передача состояния представления»). Термин REST был введен Роем Филдингом (Roy Fielding) — одним из создателей протокола HTTP. Принципы REST он сформулировал для того, чтобы наилучшим образом использовать протокол HTTP [3]. Мы подробнее рассмотрим и сам протокол, и этот архитектурный подход в *разд. 3.1*.

Итак, документирование API несет в себе важную задачу: не пересказывать всем заинтересованным лицам, что поменялось в вашей системе, а предоставить документацию, на которую можно будет ориентироваться как на источник правды. Документировать код можно несколькими способами: самостоятельно описывать каждый предоставляемый метод либо настроить генерацию документации на основе вашего кода. Минус автоматической генерации документации на основании кода

заключается в том, что это увеличивает код: необходимо добавлять аннотацию для документации, и вся информация для нее по факту будет содержаться в коде, однако это очень легко интегрируется и не даст забыть обновить документацию после каких-либо изменений. Написание документации отдельно от кода требует больше времени и знания синтаксиса Swagger Specification.

Устанавливается Swagger следующими командами:

```
go get -u github.com/swaggo/swag/cmd/swag
go get -u github.com/swaggo/http-swagger
go get -u github.com/swaggo/gin-swagger
go get -u github.com/swaggo/files
```

Первый пакет `swag` — это CLI-утилита для генерации документации, а остальные нужны, чтобы отдавать сгенерированные файлы через HTTP-сервер и отображать их в Swagger UI. Далее нам нужно инициализировать базовую структуру Swagger. В Go документация описывается с помощью комментариев в формате `etel`, которые размещаются над обработчиками запросов и в главном файле приложения. Например, общую информацию о сервисе можно добавить в файл `cmd/main.go`.

Настраивается инструмент очень просто — в нашем случае даже нет необходимости выносить его функциональность в отдельный модуль. Однако это может потребоваться, если нужно будет как-либо усложнить документацию, — например, добавить возможность скачивания в формате JSON. Часто довольно трудно определить столь тонкую грань: когда имеет смысл выносить отдельно какую-либо функциональность, а когда лучше сделать всё, что называется, «на месте».

Сначала нужно запустить наш проект как сервер, у которого будет API для Swagger. Для этого мы воспользуемся HTTP-фреймворком `Gin`¹, специально созданным с упором на высокую производительность и простоту разработки REST API. Его важная особенность состоит в том, что он работает на базе собственной реализации роутера, использующей `Radix Tree` для маршрутизации запросов. Это обеспечивает быстрое распределение нужного обработчика даже при большом количестве маршрутов, что критично для микросервисов с высокой нагрузкой. Также он не перегружен лишними абстракциями и предоставляет минималистичный и расширяемый инструментарий для создания приложений.

Установим `Gin`, выполнив команду:

```
go get github.com/gin-gonic/gin
```

и подключим его в файл `cmd/main.go`, а также сразу добавим для примера API `/ping` (листинг 1.11).

Листинг 1.11. Файл `cmd/main.go`

```
package main

import (
    "log"
```

¹ См. github.com/gin-gonic/gin.

```
_ "account/docs"
"account/internal/config"
"account/internal/logger"
"account/internal/repository"

"github.com/gin-gonic/gin"
swaggerFiles "github.com/swaggo/files"
ginSwagger "github.com/swaggo/gin-swagger"
"gorm.io/driver/postgres"
"gorm.io/gorm"
}

// @title Account Service
// @version 1.0
// @description Account Service
// @host localhost:8080
// @BasePath /
func main() {
    // Загружаем конфигурацию
    cfg, err := config.Load()
    if err != nil {
        log.Fatalf("Failed to load config: %v", err)
    }

    // Инициализируем общий логгер с названием сервиса
    logger := logger.New(cfg)

    // Подключаемся к базе данных
    db, err := gorm.Open(postgres.Open(cfg.DbDsn), &gorm.Config{})
    if err != nil {
        logger.Error().Msgf("Failed to connect to database: %v", err)
        return
    }
    logger.Info().Msg("database connected")

    // Инициализируем репозиторий
    repo := repository.NewRepository(db, &logger)
    _ = repo

    router := gin.Default()
    err = router.SetTrustedProxies([]string{"127.0.0.1"})
    if err != nil {
        logger.Error().Msgf("Failed to set trusted proxies: %v", err)
        return
    }
    router.GET("/swagger/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))
    router.GET("/ping", PingExample)
    router.Run(":8080")
}
```

```
// Пример использования общего логгера
logger.Info().Msg("service starting up")
}

// PingExample godoc
// @Summary      Проверка доступности сервиса
// @Description  Возвращает pong
// @Tags         health
// @Success      200 (string) string "pong"
// @Router       /ping [get]
func PingExample(c *gin.Context) {
    c.String(200, "pong")
}
```

Теперь чтобы увидеть Swagger-документацию в браузере, надо выполнить следующие действия:

1. Устанавливаем CLI:

```
go install github.com/swaggo/swag/cmd/swag@latest
```

2. Проверяем, есть ли в PATH каталог, куда Go устанавливает бинарники:

```
export PATH=$PATH:(go env GOPATH)/bin
```

3. Перезапускаем терминал или выполняем одну из команд (в зависимости от терминала):

```
source ~/.bashrc
# или
source ~/.zshrc
```



Рис. 1.6. Сгенерированная документация Swagger

4. Теперь убеждаемся, что `swag` работает верно:

```
swag -version
```

Если после всех этих действий выводится версия `swag`, значит, всё сделано правильно и можно запускать генерацию документации:

```
swag init -g cmd/main.go
```

Остается только запустить проект и перейти в браузере по адресу `localhost:9000/swagger/index.html` (рис. 1.6).

Проектирование базы данных PostgreSQL

Что такое база данных и какие они бывают?

Есть два основных способа хранить и обрабатывать данные: базы данных или файлы. Файлы — документы, фото, картинки — для обработки данных обычно используются редко, как правило, они задействуются в основном для их хранения и чтения и загружаются извне. Сервер их содержимое, как правило, обрабатывает и сохраняет, а по запросу отдает, но никак его не меняет. Такое отношение к файлам устоялось вследствие того, что чтение их для поиска информации или их редактирование — очень трудоемкий процесс, потому что они никак не оптимизированы.

Базы данных предназначены для хранения и манипулирования большими объемами данных, связанных чаще всего между собой. Базы данных состоят из таблиц, в которых содержится информация. При создании базы данных стоит позаботиться о том, какие таблицы нужны и какие связи между ними будут.

За структуру базы данных и взаимодействие между составляющими ее различными элементами отвечает СУБД — система управления базами данных. СУБД представляет собой специальную программу, имеющую основной своей функцией создание, изменение и ведение базы данных некоторой совокупностью пользователей через прикладные программы. База данных без СУБД — это просто набор данных, с которым ничего нельзя сделать, и именно СУБД позволяет управлять данными. Основные функции СУБД:

- ☐ создание баз данных, изменение, удаление и объединение их по определенным признакам;
- ☐ хранение данных, в том числе больших массивов, в структурированном виде и нужном формате;
- ☐ защита данных от взлома и нежелательных изменений при помощи распределенного доступа — когда разным группам пользователей доступны разный объем и сегменты данных;
- ☐ выгрузка и сортировка данных по заданным фильтрам при помощи SQL-запросов;
- ☐ поддержка целостности баз данных, резервное копирование и восстановление после сбоев.

Есть разные виды баз данных: реляционные, нереляционные, «ключ-значение», графовые, колоночные [4].

- ❑ *Реляционные* базы данных можно представить в виде табличек Excel, где строки — это записи, а в столбцах содержатся данные, относящиеся к этим записям. При этом каждый столбец не существует сам по себе — данные в нем каким-либо образом соотносятся с данными из соседних столбцов. Например, для записи, касающейся одного пользователя, в столбцах может содержаться информация о его фамилии, имени, отчестве, дате рождения и телефоне.
- ❑ *Нереляционные* базы данных являются документоориентированными. Это означает, что каждая запись представляет собой объект, который к тому же может быть сколько угодно вложенным. Говоря о всё той же таблице с данными о пользователях, теперь мы можем его Ф.И.О. хранить как объект, а внутри него уже иметь отдельно фамилию, имя и отчество. Это, конечно, очень утрированный пример, и делать так нет никакой необходимости, но суть в том, что объект должен представлять собой некую совокупность данных.
- ❑ С базами данных типа «ключ-значение» мы познакомимся позже, когда будем рассматривать межсерверное взаимодействие.
- ❑ *Графовые* базы данных, как можно понять из их названия, построены на графах и чаще всего применяются для задач, связанных с системой оценок и рекомендательными алгоритмами. Впрочем, в последнее время графовые базы данных всё больше набирают популярность также и в проектах, где приходится работать с большими объемами данных.
- ❑ *Колоночные* базы данных нужны для очень больших данных и сложных запросов для них.

Нормализация данных

Указания для правильного проектирования реляционных баз данных изложены в реляционной модели данных. Они собраны в шесть групп, которые называются *нормальными формами*. Первая нормальная форма предоставляет самый низкий уровень нормализации баз данных, а шестая — высший.

Нормальная форма — это рекомендация по проектированию баз данных. Не обязательно придерживаться всех шести нормальных форм, однако рекомендуется учитывать эти правила хотя бы в некоторой степени, поскольку такой подход имеет ряд существенных преимуществ с точки зрения эффективности и удобства обращения с базой данных. Тем не менее редко когда следуют всем шести нормальным формам, — часто ограничиваются второй или третьей нормальной формой, а те, что выше, учитываются значительно реже [5].

Первая нормальная форма (1НФ)

Согласно первой нормальной форме таблица базы данных — это представление сущности. Например: пользователи, заказы, товары, билеты и т. д. Каждая запись представляет один экземпляр сущности. Например, в таблице пользователей каждая запись представляет одного пользователя.

Первичный ключ

- ❑ **Правило 1:** каждая таблица должна иметь первичный ключ, состоящий из наименьшего возможного количества полей.

Первичный ключ может быть совокупностью полей — например, содержать фамилию и имя человека, но вряд ли можно ожидать, что этот набор всегда будет уникальным, — лучше было бы выбрать для первичного ключа серию и номер паспорта, т. к. этот набор гарантированно обеспечит уникальность. Если же для первичного ключа нет очевидного кандидата, то используют суррогатный первичный ключ в виде числового автоинкрементного поля или в формате UUID (Universally Unique Identifier), который генерирует практически всегда уникальные значения [6].

- ❑ **Правило 2:** поля не имеют дубликатов в каждой записи, и каждое поле содержит только одно значение.

То есть все атрибуты в таблице должны быть простыми, все сохраняемые данные на пересечении столбцов и строк — содержать лишь скалярные значения. Не должно быть дублирования строк.

Рассмотрим, например, таблицу «Телефоны» (табл. 1.1).

Таблица 1.1. Телефоны

Фирма	Модели
Samsung	Galaxy S23
Apple	iPhone 14, iPhone 13, iPhone 12

Нарушение нормализации 1НФ происходит здесь в моделях Apple, поскольку в одной ячейке содержится список из трех элементов: iPhone 14, iPhone 13, iPhone 12 — т. е. он не является атомарным. Преобразуем таблицу «Телефоны» к 1НФ с учетом правила 2 (табл. 1.2).

Таблица 1.2. Телефоны 1НФ

Фирма	Модели
Samsung	Galaxy S23
Apple	iPhone 14
Apple	iPhone 13
Apple	iPhone 12

- ❑ **Правило 3:** порядок записей не должен иметь значения.

Возможно, вам понадобится знать, кто из пользователей зарегистрировался раньше, но тогда лучше использовать поля даты и времени регистрации, а не учитывать порядок следования записей. Порядок записей может меняться, когда

пользователи будут удалять или изменять свои учетные записи. Поэтому полагаться на порядок записей в таблице не стоит.

Вторая нормальная форма (2НФ)

Содержащиеся в таблице отошения соответствуют 2НФ, если они соответствуют 1НФ и каждый столбец неприводимо зависит от первичного ключа.

Неприводимость означает, что в составе потенциального первичного ключа отсутствует меньшее подмножество столбцов, от которых можно также вывести имеющуюся функциональную зависимость.

Первичный ключ — это особое поле, в котором сохраняется уникальный идентификатор записи. Он нужен для того, чтобы организовывать связь между записями, которые хранятся в нескольких таблицах.

Обратимся, например, к таблице «Телефоны — Цена» (табл. 1.3).

Таблица находится в 1НФ, но не соответствует 2НФ. Дело в том, что цена телефонов зависит и от модели, и от производителя, а размер скидки — только от производителя. Поэтому функциональная зависимость от первичного ключа является в этой таблице неполной.

Таблица 1.3. Телефоны — Цена

Модель	Фирма	Цена	Скидка
Galaxy S23	Samsung	80 000	10%
iPhone 14	Apple	120 000	5%
iPhone 13	Apple	100 000	5%
iPhone 12	Apple	90 000	5%

Исправляется такое несоответствие путем декомпозиции таблицы на два отношения, в которых неключевые атрибуты зависят от первичного ключа [8]: таблицу «Модель — Цена» (табл. 1.4) и таблицу «Фирма — Скидка» (табл. 1.5).

Таблица 1.4. Модель — Цена

Модель	Фирма	Цена
Galaxy S23	Samsung	80 000
iPhone 14	Apple	120 000
iPhone 13	Apple	100 000
iPhone 12	Apple	90 000

Таблица 1.5. Фирма — Скидка

Фирма	Скидка
Samsung	10%
Apple	5%

Третья нормальная форма (3НФ)

Содержащиеся в таблице отношения соответствуют 3НФ, если они соответствуют 2НФ и любой столбец, который не является ключом, зависит лишь от первичного ключа.

Таблица «Модель — Телефон» (табл. 1.6) находится в 2НФ, но не в 3НФ. Здесь атрибут «Модель» является первичным ключом, но личных телефонов у автомобилей нет, а номера телефонов зависят исключительно от магазинов.

Таблица 1.6. Модель — Телефон

Модель	Магазин	Телефон
BMW	Риал-авто	87-33-98
Audi	Риал-авто	87-33-98
Nissan	Некст-Авто	94-54-12

Таким образом, в представленном в табл. «Модель — Телефон» отношении существуют следующие функциональные зависимости [7]:

- ☐ Модель → Магазин;
- ☐ Магазин → Телефон;
- ☐ Модель → Телефон.

Зависимость Модель → Телефон является транзитивной, следовательно, отношение не находится в 3НФ.

Исправить ситуацию помогает разделение исходного отношения на два отношения, находящихся в 3НФ: таблицу «Магазин — Телефон» (табл. 1.7) и таблицу «Модель — Магазин» (табл. 1.8).

Таблица 1.7. Магазин — Телефон

Магазин	Телефон
Риал-авто	87-33-98
Некст-Авто	94-54-12

Таблица 1.8. Модель — Магазин

Модель	Магазин
BMW	Риал-авто
Audi	Риал-авто
Nissan	Некст-Авто

Разработка бизнес-логики и маршрутизации для модуля User

Когда приходит время проектировать бизнес-логику и маршрутизацию, важным моментом становится формализация контракта для взаимодействия. На первых этапах разработки можно ограничиться шаблоном DTO (Data Transfer Object) — структурами данных, описывающими входящие и исходящие параметры внутри одного приложения. Однако в микросервисной архитектуре на Go выбирают в большинстве случаев Protocol Buffers (Protobuf). Этот язык описания структур данных еще подробнее будет рассматриваться в *главе 3*.

Для Go Protobuf особенно удобен в связке с gRPC, но даже если нужно работать через HTTP-API, он остается ценным инструментом, потому что позволяет поддерживать четко описанные структуры и легко их расширять. В отличие от JSON, Protobuf обеспечивает более компактную сериализацию и строгую типизацию, что критично при высоких нагрузках и в системах, где важна предсказуемость формата. Ошибки или несогласованность в контрактах приводят к ошибкам взаимодействия между сервисами, когда один сервис ожидает одно, а другой сервис отправляет ему другое, — из-за этого обычно стараются выносить контракты в отдельный репозиторий.

В нашем проекте мы заведем отдельный репозиторий `contracts`, в котором разместим контракты по их названию, — т. е. для сервиса `account` контракты будут находиться в каталоге `contracts/account` (листинг 1.12).

Листинг 1.12. Файл `contracts/account/account_service.proto`

```
syntax = "proto3";

package account;
option go_package = "github.com/yuliaapopova/contracts/account/go;account";

import "google/api/annotations.proto";
import "google/protobuf/empty.proto";
import "account_model.proto";
import "pagination/pagination.proto";

service Account {
  rpc CreateUser(CreateUserRequest) returns (google.protobuf.Empty) {
    option (google.api.http) = {
      post: "account/api/v1/create_user",
      body: "*"
    };
  }

  rpc GetUser(GetUserRequest) returns (GetUserResponse) {
    option (google.api.http) = {
      get: "account/api/v1/users/{user_id}"
    };
  }
}
```

```
rpc GetUsers(GetUsersRequest) returns (GetUsersResponse) {
    option (google.api.http) = {
        get: "account/api/v1/users"
    };
}

rpc DeleteUser(DeleteUserRequest) returns (google.protobuf.Empty) {
    option (google.api.http) = {
        delete: "account/api/v1/users/{user_id}"
    };
}

rpc UpdateUser(UpdateUserRequest) returns (google.protobuf.Empty) {
    option (google.api.http) = {
        put: "account/api/v1/users/{user_id}"
        body: "*"
    };
}

}

message CreateUserRequest {
    CreateUser user = 1;
}

message GetUserRequest {
    uint64 user_id = 1;
}

message GetUserResponse {
    User user = 1;
}

message GetUsersRequest {
    pagination.Pagination pagination = 1;
}

message GetUsersResponse {
    repeated User users = 1;
    pagination.Pagination pagination = 2;
}

message DeleteUserRequest {
    uint64 user_id = 1;
}

message UpdateUserRequest {
    uint64 user_id = 1;
    User user = 2;
}
```

Тут мы сразу разделили код сервиса с входящими и исходящими параметрами, и модели, с которыми предстоит работать. Это делается с тем, чтобы если в другом сервисе нам потребуется переиспользовать сущность `user`, не понадобилось бы тащить в импортах файл `account_service.proto`, — ведь если файлы будут импортировать друг друга, это создаст циклические импорты. Дело в том, что компилятор `protoc` не умеет разрешать проблему цикла: он загружает один импорт, видит, что в нем используется второй импорт, загружает его, а в том уже есть первый, — он пытается загрузить и его, и получается бесконечный цикл, в итоге компилятор выдаст ошибку. `Proto`-файлы сами по себе модули, и если два модуля не могут существовать один без другого, значит, модульность нарушена. Даже если предположить, что `protoc` научился поддерживать циклические импорты, мы не сможем скомпилироваться тогда уже на уровне языка (листинг 1.13).

Листинг 1.13. Файл `contracts/account/account_model.proto`

```
syntax = "proto3";

package account;
option go_package = "github.com/yuliaapopova/contracts/account/go;account";

import "google/protobuf/timestamp.proto";

message CreateUser {
    string login = 1;
    string phone = 2;
    string first_name = 3;
    string last_name = 4;
    string middle_name = 5;
    string email = 6;
    uint32 age = 7;
}

message User {
    uint64 id = 1;
    string login = 2;
    string phone = 3;
    string first_name = 4;
    string last_name = 5;
    string middle_name = 6;
    string email = 7;
    uint32 age = 8;
    google.protobuf.Timestamp created_at = 9;
    google.protobuf.Timestamp updated_at = 10;
}
```

И сразу еще отдельно вынесем пагинацию для запросов — это как раз та сущность, которая регулярно используется в разных сервисах (листинг 1.14).

Листинг 1.14. Файл contracts/pagination/pagination.proto

```
syntax = "proto3";

package pagination;

option go_package = "github.com/yuliaapopova/contracts/pagination/go;pagination";

message Pagination {
    uint32 offset = 1;
    uint32 limit = 2;
}
```

Теперь самое интересное: нужно сделать так, чтобы из proto-файлов (.proto) генерировались go-файлы (.go) — для дальнейшего использования в сервисах. Важно учесть при этом особенности генерации — если мы установим локально protoc на каждом компьютере, то могут возникнуть проблемы: у разных людей могут различаться версии компилятора, использоваться разные плагины для Go или других языков, отсутствовать сторонние proto-файлы вроде google/api/annotations.proto. В какой-то момент один разработчик фиксирует свои изменения, и у него всё стабильно работает, а у другого — нет.

Чтобы этого избежать, принято использовать два стандартных инструмента: Dockerfile и Makefile. Их задача — превратить процесс сборки в предсказуемый и полностью автоматизированный.

Dockerfile служит тут своеобразным слоем изоляции. В нем можно описать необходимое идеальное окружение для генерации кода. Туда устанавливается сам компилятор protoc (конкретной версии), туда же скачиваются официальные googleapis proto-файлы, которые часто используются для REST-аннотаций и других расширений. В этот же контейнер можно поставить и плагины Go — protoc-gen-go и protoc-gen-go-grpc.

Это всё нужно для того, чтобы не было важно, на какой операционной системе работает человек, какая у него установлена версия Go и установлен ли protoc глобально. В момент запуска генерации всегда откроется наш подготовленный контейнер, где есть всё, что нужно, и именно в тех версиях, которые требуются. Такой контейнер — универсальный инструмент, его можно передать любому члену команды, и результат всегда будет одинаковым (листинг 1.15).

Листинг 1.15. Файл contracts/Dockerfile

```
FROM golang:1.24

# Устанавливаем protoc
ARG PROTOC_VERSION=27.1
RUN apt-get update && apt-get install -y unzip curl git && rm -rf /var/lib/apt/lists/* && \
    curl -sSL \
    https://github.com/protocolbuffers/protobuf/releases/download/v${PROTOC_VERSION}/ \
    protoc-${PROTOC_VERSION}-linux-x86_64.zip -o /tmp/protoc.zip && \
    unzip /tmp/protoc.zip -d /usr/local && rm /tmp/protoc.zip
```



```
# Качаем googleapis внутрь образа
RUN git clone --depth=1 https://github.com/googleapis/googleapis
    /usr/local/include/googleapis

# Устанавливаем gRPC плагины для protoc
RUN go install google.golang.org/protobuf/cmd/protoc-gen-go@latest && \
    go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

ENV PATH="$PATH:/go/bin"
WORKDIR /app
```

Dockerfile задает среду, но работать с ним напрямую неудобно. Каждый раз писать длинную команду `docker run`, добавлять параметры, монтировать тома — слишком утомительно. Именно поэтому вторым необходимым нам инструментом является Makefile (листинг 1.16).

Листинг 1.16. Файл `contracts/Makefile`

```
PROTO_ROOT=.
DOCKER_IMAGE=proto-builder

.PHONY: docker-build gen clean

docker-build:
    docker build -t $(DOCKER_IMAGE) .

gen: docker-build
    docker run --rm -v $(abspath $(PROTO_ROOT)): /app $(DOCKER_IMAGE) \
    bash -c '\
        set -e; \
        for dir in account pagination; do \
            echo ">> Processing $$dir"; \
            mkdir -p /app/$$dir/go; \
            cd /app/$$dir; \
            for file in *.proto; do \
                echo "    Generating $$dir/$$file"; \
                protoc \
                    -I . \
                    -I /app \
                    -I /usr/local/include/googleapis \
                    --go_out=go \
                    --go_opt=paths=source_relative \
                    --go-grpc_out=go \
                    --go-grpc_opt=paths=source_relative \
                    $$file; \
            done; \
        done \
```

```
clean:
```

```
find account pagination -type d -name go -exec rm -rf {}
```

В результате такой связки получается инфраструктура, дающая всегда стабильный результат в разработке.

Теперь остается только сгенерировать код на Go для сервиса, опубликовать все на github для удобства, установить версию пакета через тег и после этого использовать в самом сервисе `account` с основной логикой:

```
git tag v1.0.0
```

```
git push origin v1.0.0
```

Версия в нашем случае — `v1.0.0`, и если понадобится внести изменения в контракты и доставить их в сервис, нужно будет также поднять версию через тег, т. е. увеличить ее. Следующей версией будет — `v1.1.0`, если это добавление новой функциональности, которая не ломает предыдущую версию. Руководствоваться тем, какую цифру в версии следует увеличить, нужно в соответствии с семантическим версионированием [1].

Вернемся к разработке логики сервиса `account`: заменим наш роутер `gin` на маршрутизацию, описанную в контрактах через `Protocol Buffers`, и подключим модуль с контрактами:

```
go get github.com/yuliaapopova/contracts
```

Добавим в `main.go` запуск gRPC-сервера и избавимся от HTTP-сервера (листинг 1.17).

Листинг 1.17. Файл `cmd/main.go`

```
package main
```

```
import (  
    "account/internal/config"  
    "account/internal/logger"  
    "account/internal/repository"  
    "fmt"  
    "log"  
    "net"  
  
    "google.golang.org/grpc"  
    "google.golang.org/grpc/health"  
    "google.golang.org/grpc/health/grpc_health_v1"  
    "google.golang.org/grpc/reflection"  
    "gorm.io/driver/postgres"  
    "gorm.io/gorm"  
)
```

```
// @title Account Service
```

```
// @version 1.0
```

```
// @description Account Service
// @host localhost:8080
// @BasePath /
func main() {
    // Загружаем конфигурацию
    cfg, err := config.Load()
    if err != nil {
        log.Fatalf("Failed to load config: %v", err)
    }

    // Инициализируем общий логгер с названием сервиса
    logger := logger.New(cfg)

    // Подключаемся к базе данных
    db, err := gorm.Open(postgres.Open(cfg.DbDsn), &gorm.Config{})
    if err != nil {
        logger.Error().Msgf("Failed to connect to database: %v", err)
        return
    }
    logger.Info().Msg("database connected")

    // Инициализируем репозиторий
    repo := repository.NewRepository(db, &logger)
    _ = repo

    // Запускаем gRPC-сервер согласно контрактам
    listenAddr := fmt.Sprintf("%s:%d", cfg.Host, cfg.Port)
    lis, err := net.Listen("tcp", listenAddr)
    if err != nil {
        logger.Error().Msgf("Failed to listen on %s: %v", listenAddr, err)
        return
    }

    grpcServer := grpc.NewServer()

    // Регистрация health-сервиса и reflection для лебара
    healthSrv := health.NewServer()
    grpc_health_v1.RegisterHealthServer(grpcServer, healthSrv)
    reflection.Register(grpcServer)

    logger.Info().Msgf("gRPC server listening on %s", listenAddr)
    if err := grpcServer.Serve(lis); err != nil {
        logger.Error().Msgf("Failed to serve gRPC: %v", err)
        return
    }

    logger.Info().Msg("service starting up")
}
```

Теперь, когда инфраструктурная часть приложения готова: выделен отдельный пакет с контрактами, настроена загрузка конфигурации и логирование, организовано подключение к базе данных и автоматизирована часть необходимых процессов, самое время перейти к построению логики сервиса.

Мы уже выделили главную сущность пользователя для транспортного слоя (данные, которые будут передаваться по сети), теперь есть большой соблазн переиспользовать эту же структуру и во всем сервисе, — ведь странно же писать точно такие же новые модели. Но всё не так просто. Дело в том, что слои должны быть максимально независимыми.

Представим ситуацию, когда по каким-либо причинам будет принято решение отказаться от gRPC в пользу REST или асинхронного взаимодействия. Если домен знает только о себе и о своей модели, нам понадобится изменить только один слой — адаптер, который переводит модели для сетевого взаимодействия в доменные сущности, а вот если бы весь сервис был построен вокруг proto-генерированного кода, такой подход был бы невозможен без огромного рефакторинга. Это и есть привязка бизнес-кода к инфраструктурным деталям — антипаттерн при проектировании микросервисов.

Кроме того, реальные системы редко живут в изоляции, получают данные лишь из одного источника и общаются только по одному каналу связи, — чаще всего один сервис взаимодействует с другой системой, например, через gRPC, со второй — через Kafka, а еще с кем-то — посредством JSON. Форматы везде различаются, и он будет вынужден знать про всю эту разнородность. Гораздо удобнее создать чистую внутреннюю модель, не зависящую ни от кого, и выполнять маппинг из входных данных во внутреннюю универсальную модель.

Другой важный момент, который всегда возникает со временем, — протоколы обмена данными должны быть обратно совместимыми, поэтому нельзя просто удалить поле из proto-контракта, если оно больше не используется. Правильно объявить его зарезервированным, но оно навсегда останется частью внешней схемы. Внутри же эти ограничения ни к чему, там модель живет своей жизнью: лишние атрибуты можно удалять без оглядки на клиентов, поля можно переименовывать так, как удобно разработчикам. Если этими ограничениями будет обложен сам домен, то это лишает гибкости, и развитие предметной модели начнет тормозить под давлением транспортного слоя.

Но самая главная причина, почему так делать не стоит, кроется в том, что модели разных слоев должны меняться изолированно друг от друга. Стоит изменить один раз proto-файл, как следом потребуются реализовать целый каскад исправлений везде, где он используется. Система приобретает такую хрупкость, что становится опасно вносить в модель даже небольшое изменение. Если же каждый слой владеет своей моделью, изменения локализируются: меняется контракт, обновляется маппер, и этого достаточно. Это и есть изоляция, ради которой и задумывалась архитектурная чистота.

Поэтому следующим шагом будет написание внутренней модели для бизнес-слоя и слоя репозитория (листинг 1.18), а также мапперов для них.

Листинг 1.18. Файл `internal/model/user.go`

```
package model

import "time"

type User struct {
    ID          uint64
    Login       string
    Email       string
    Phone       string
    FirstName   string
    LastName    string
    MiddleName  string
    Age         uint32
    CreatedAt   time.Time
    UpdatedAt   time.Time
}

type CreateUser struct {
    Login       string
    Email       string
    Phone       string
    FirstName   string
    LastName    string
    MiddleName  string
    Age         uint32
}

type UpdateUser struct {
    Email       string
    Phone       string
    FirstName   string
    LastName    string
    MiddleName  string
    Age         uint32
}
```

Создадим — чтобы не создавать путаницы в коде — сразу несколько структур для работы с пользователями (листинг 1.19). Для понимания того, почему так лучше, представим, что у нас есть только одна сущность пользователя. Когда мы приступим к разработке сценария обновления пользователя, быстро выяснится, что в рамках этой задачи обновлять пароль, например, было бы ошибкой, а логин и вовсе не должен обновляться никогда. При создании пользователя возникает нюанс с паролем — мы его в системе, по сути, не храним в том же виде, в котором он поступает, а только на его основе генерируем хеш пароля и работаем уже с ним. Если просто передавать далее в поле пароля хеш пароля, это создаст лишнюю когнитивную на-

грузку на других разработчиков: сначала им нужно будет разобраться, где находится пароль, а где хеш пароля, и не забывать об этой особенности.

Листинг 1.19. internal/mapper/mapper.go

```
package mapper

import (
    accountpb "github.com/yuliaapopova/contracts/account/go"
    "google.golang.org/protobuf/types/known/timestamppb"

    "account/internal/model"
)

func PbToUser(userpb *accountpb.User) model.User {
    return model.User{
        ID:          userpb.Id,
        Login:       userpb.Login,
        Email:       userpb.Email,
        Phone:       userpb.Phone,
        FirstName:   userpb.FirstName,
        LastName:    userpb.LastName,
        MiddleName:  userpb.MiddleName,
        Age:         userpb.Age,
        CreatedAt:   userpb.CreatedAt.AsTime(),
        UpdatedAt:   userpb.UpdatedAt.AsTime(),
    }
}

func UserToPb(user model.User) *accountpb.User {
    return &accountpb.User{
        Id:          user.ID,
        Login:       user.Login,
        Email:       user.Email,
        Phone:       user.Phone,
        FirstName:   user.FirstName,
        LastName:    user.LastName,
        MiddleName:  user.MiddleName,
        Age:         user.Age,
        CreatedAt:   timestamppb.New(user.CreatedAt),
        UpdatedAt:   timestamppb.New(user.UpdatedAt),
    }
}

func UsersToPbs(users []model.User) []*accountpb.User {
    pbs := make([]*accountpb.User, len(users))
    for i, user := range users {
        pbs[i] = UserToPb(user)
    }
}
```

```

    return pbs
}

func PbsToUsers(pbs []*accountpb.User) []model.User {
    users := make([]model.User, len(pbs))
    for i, pb := range pbs {
        users[i] = PbToUser(pb)
    }
    return users
}

func PbToUserCreate(accountpbUser *accountpb.CreateUser) model.CreateUser {
    return model.CreateUser{
        Login:      accountpbUser.Login,
        Email:      accountpbUser.Email,
        Password:   accountpbUser.Password,
        Phone:      accountpbUser.Phone,
        FirstName:  accountpbUser.FirstName,
        LastName:   accountpbUser.LastName,
        MiddleName: accountpbUser.MiddleName,
        Age:       accountpbUser.Age,
    }
}

func PbToUserUpdate(userpb *accountpb.UpdateUser) model.UpdateUser {
    return model.UpdateUser{
        Email:      userpb.Email,
        Phone:      userpb.Phone,
        FirstName:  userpb.FirstName,
        LastName:   userpb.LastName,
        MiddleName: userpb.MiddleName,
        Age:       userpb.Age,
    }
}
}

```

Этим мы выполнили преобразование из моделей транспортного слоя в модели для бизнес-логики. Далее нам нужны модель для слоя репозитория (листинг 1.20) и его маппер (листинг 1.21).

Листинг 1.20. Файл `internal/repository/model/model.go`

```

package model

import "time"

type User struct {
    ID          uint64    `gorm:"column:id"`
    Login       string    `gorm:"column:login"`

```

```
Email      string    `gorm:"column:email"`
Phone       string    `gorm:"column:phone"`
FirstName   string    `gorm:"column:first_name"`
LastName    string    `gorm:"column:last_name"`
MiddleName  string    `gorm:"column:middle_name"`
Age         uint32     `gorm:"column:age"`
CreatedAt   time.Time  `gorm:"column:created_at"`
UpdatedAt   time.Time  `gorm:"column:updated_at"`
}

func (User) TableName() string {
    return "users"
}
```

Листинг 1.21. Файл `internal/repository/mapper/mapper.go`

```
package mapper

import (
    "account/internal/model"
    repomodel "account/internal/repository/model"
)

func UserToRepoUser(user model.User) repomodel.User {
    return repomodel.User{
        ID:          user.ID,
        Login:       user.Login,
        Email:       user.Email,
        Phone:       user.Phone,
        FirstName:   user.FirstName,
        LastName:    user.LastName,
        MiddleName:  user.MiddleName,
        Age:         user.Age,
        CreatedAt:   user.CreatedAt,
        UpdatedAt:   user.UpdatedAt,
    }
}

func RepoUserToUser(user repomodel.User) model.User {
    return model.User{
        ID:          user.ID,
        Login:       user.Login,
        Email:       user.Email,
        Phone:       user.Phone,
        FirstName:   user.FirstName,
        LastName:    user.LastName,
        MiddleName:  user.MiddleName,
        Age:         user.Age,
    }
}
```



```

        CreatedAt:    user.CreatedAt,
        UpdatedAt:    user.UpdatedAt,
    }
}

func RepoUsersToUsers(users []repoModel.User) []model.User {
    res := make([]model.User, len(users))
    for i, user := range users {
        res[i] = RepoUserToUser(user)
    }
    return res
}

func UpdateUserToRepoUser(user model.UpdateUser) repoModel.User {
    return repoModel.User{
        Email:    user.Email,
        Phone:    user.Phone,
        FirstName: user.FirstName,
        LastName: user.LastName,
        MiddleName: user.MiddleName,
        Age:      user.Age,
    }
}

```

Преобразование моделей между собой — процесс простой и не требующий дополнительных пояснений. Теперь мы можем перейти непосредственно к логике репозитория и сервиса (листинг 1.22).

Листинг 1.22. Файл `internal/repository/repository.go`

```

package repository

import (
    "account/internal/model"
    "account/internal/repository/mapper"
    repoModel "account/internal/repository/model"
    "context"
    "errors"
    "fmt"

    "github.com/rs/zerolog"
    "gorm.io/gorm"
    "gorm.io/gorm/clause"
)

type Repository struct {
    db      *gorm.DB
    logger  *zerolog.Logger
}

```

```

func NewRepository(db *gorm.DB, logger *zerolog.Logger) *Repository {
    return &Repository{
        db:      db,
        logger:  logger,
    }
}

func (r *Repository) CreateUser(ctx context.Context, user model.User) error {
    userRepo := mapper.UserToRepoUser(user)
    res := r.db.WithContext(ctx).
        Clauses(clause.OnConflict{UpdateAll: true}).
        Create(&userRepo)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to save user")
        return fmt.Errorf("failed to save user: %w", res.Error)
    }
    return nil
}

func (r *Repository) GetUser(ctx context.Context, userID uint64) (model.User, error) {
    var userRepo model.User
    res := r.db.WithContext(ctx).
        Model(&userRepo).
        Where("id = ?", userID).
        First(&user)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return model.User{}, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user id")
    }
    return mapper.RepoUserToUser(user), nil
}

func (r *Repository) GetUsers(ctx context.Context, limit int, offset int) ([]model.User, error) {
    var users []model.User
    res := r.db.WithContext(ctx).
        Model(&userRepo).
        Offset(offset).
        Limit(limit).
        Find(&users)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return nil, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user id")
    }
}

```

```

    return mapper.RepoUsersToUsers(users), nil
}

func (r *Repository) DeleteUser(ctx context.Context, userID uint64) error {
    res := r.db.WithContext(ctx).
        Where("id = ?", userID).
        Delete(&repomodel.User{})
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to delete user")
        return fmt.Errorf("failed to delete user")
    }
    return nil
}

func (r *Repository) UpdateUser(ctx context.Context, userID uint64, user model.UpdateUser) error {
    res := r.db.WithContext(ctx).Where("id = ?", userID).Updates(user)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to update user")
        return fmt.Errorf("failed to update user")
    }
    return nil
}

```

Метод `GetUsers` реализован таким образом, чтобы возвращать записи с учетом пагинации. *Пагинация* — это разбиение данных по страницам, и для полноценной ее работы еще необходимо реализовать `offset` и `limit`: `limit` ограничивает количество возвращаемых записей, а `offset` указывает, сколько записей от начала должно быть пропущено. Таким образом, если бы у нас был бы еще фронтенд на эту функциональность, переход на следующую страницу с записями был бы реализован путем передачи: `limit:20, offset:20`, — если у нас предусмотрено на странице отображение 20 записей. Для перехода на вторую страницу соответственно были бы переданы значения: `limit:20, offset:40`.

Еще из интересного тут обработка ошибок, и логирование перед тем, как вернуть результат. Нужно всегда стремиться к тому, чтобы в одном запросе лог с ошибкой был только один, — так впоследствии будет проще разобраться по логам с тем, что именно привело к ошибке, поэтому, если мы залогировали на уровне репозитория, то в сервисном слое этого делать уже не нужно.

Для того чтобы модель для работы с базой данных была применима к ней, необходимо создать и запустить миграцию. Для этого сначала нужно установить пакет `goose`:

```
go install github.com/pressly/goose/v3/cmd/goose@latest
```

Убедимся, что всё установилось верно и пакет готов к использованию, как обычно, — проверив версию:

```
goose --version
```

Остается только создать папку для миграций и сгенерировать заготовку для самой миграции:

```
mkdir internal/migrations
goose -dir internal/migrations create initdb
```

При генерации мы явно должны указывать под флагом `-dir` каталог, где будет создана миграция, далее следует команда `create` и название самой миграции. Посмотрим на получившийся файл (листинг 1.23).

Листинг 1.23. Файл 20250825102441_initdb.go

```
package migrations

import (
    "context"
    "database/sql"
    "github.com/pressly/goose/v3"
)

func init() {
    goose.AddMigrationContext(upInitdb, downInitdb)
}

func upInitdb(ctx context.Context, tx *sql.Tx) error {
    // This code is executed when the migration is applied.
    return nil
}

func downInitdb(ctx context.Context, tx *sql.Tx) error {
    // This code is executed when the migration is rolled back.
    return nil
}
```

При генерации миграциям присваивается название в формате: временная метка + «введенное название». Это сделано для того, чтобы миграции между собой не конфликтовали из-за названий, — вероятность того, что в одну и ту же секунду будут сгенерированы две миграции с одинаковым названием, ничтожно мала.

Код миграции выглядит пустым, но на самом деле в этом и смысл: разработчик самостоятельно должен вписать туда SQL или код, определяющий изменения в базе.

Это всё работает следующим образом:

- регистрация миграций происходит в функции `init`, именно оттуда вызываются все миграции, в которых определяются две функции: `upInitdb` и `downInitdb` (обратите внимание, что функции называются по правилу `up` + «название миграции» и `down` + «название миграции»). Они связаны с конкретной миграцией, и благодаря этому `goose` понимает, что именно они должны быть вызваны при применении или откате миграции;

- ❑ функция `upInitdb` — сюда нужно написать инструкции для создания новых таблиц, индексов и других объектов. В качестве аргумента приходит `*sql.Tx` — транзакция базы данных. `goose` гарантирует, что миграция выполняется внутри транзакции, и если что-то пойдет не так, изменения будут отменены;
- ❑ функция `downInitdb` — это «обратный путь». Если понадобится отменить изменения, сделанные в `upInitdb`, сюда нужно будет поместить SQL для удаления таблиц или отмены других структурных изменений.

Добавим создание таблицы `users` и чуть-чуть изменим сгенерированную заготовку (листинг 1.24).

Листинг 1.24. Файл `20250825102441_initdb.go`

```
package migrations

import (
    "context"
    "database/sql"

    "github.com/pressly/goose/v3"
)

func init() {
    goose.AddNamedMigrationContext("20250825102441_initdb.go", upCreateUsersTable,
                                                                    downCreateUsersTable)
}

func upCreateUsersTable(ctx context.Context, tx *sql.Tx) error {
    _, err := tx.Exec(`
        CREATE TABLE IF NOT EXISTS users (
            id BIGSERIAL PRIMARY KEY,
            login TEXT NOT NULL UNIQUE,
            email TEXT NOT NULL UNIQUE,
            phone TEXT,
            first_name TEXT,
            last_name TEXT,
            middle_name TEXT,
            age INT,
            created_at TIMESTAMP NOT NULL DEFAULT NOW(),
            updated_at TIMESTAMP NOT NULL DEFAULT NOW()
        );
    `)
    return err
}

func downCreateUsersTable(ctx context.Context, tx *sql.Tx) error {
    _, err := tx.Exec(`DROP TABLE IF EXISTS users;`)
```

```
    return err
}
```

Тут следует изменить метод `goose.AddMigrationContext` который вызывается при регистрации миграции, на `goose.AddNamedMigrationContext`. Дело в том, что изначально при генерации заготовки этот метод указывается как базовый, — он принимает функции `up` и `down`, но не дает имени, — уникальность миграции определяется только порядком регистрируемых функций и порядковым номером файла. А метод `goose.AddNamedMigrationContext` — это расширенный вариант, где миграция регистрируется не только с функциями, но и с заданным именем. Поскольку в дальнейшем новые миграции будут создаваться в новых файлах, чтобы легко можно было проследить логику их создания, логично передавать имена в соответствии с названиями миграций, — так станет четко понятно, в каком месте произошла ошибка, если что...

К этому моменту у приложения есть фундамент в виде работы с базой данных посредством миграций и запросов. Но сама по себе эта часть еще бесполезна — нужно теперь реализовать сервисный слой, отвечающий за то, как данные должны взаимодействовать между собой. На текущем этапе логика в целом будет простой (листинг 1.25).

Листинг 1.25. Файл `internal/service/service.go`

```
package service

import (
    "account/internal/model"
    "context"
    "fmt"
    "time"

    "github.com/rs/zerolog"
    "golang.org/x/crypto/bcrypt"
)

type AccountService struct {
    repo    Repository
    logger  *zerolog.Logger
}

func New(repo Repository, logger *zerolog.Logger) *AccountService {
    return &AccountService{repo: repo, logger: logger}
}

type Repository interface {
    CreateUser(context.Context, model.User) error
    GetUser(context.Context, uint64) (model.User, error)
    GetUsers(context.Context, int, int) ([]model.User, error)
```

```
DeleteUser(context.Context, uint64) error
UpdateUser(context.Context, uint64, model.UpdateUser) error
}

func (s *AccountService) CreateUser(ctx context.Context, newUser model.CreateUser) error {
    user := model.User{
        Login:      newUser.Login,
        Email:      newUser.Email,
        Phone:      newUser.Phone,
        FirstName:  newUser.FirstName,
        LastName:   newUser.LastName,
        MiddleName: newUser.MiddleName,
        Age:        newUser.Age,
        CreatedAt:  time.Now(),
        UpdatedAt:  time.Now(),
    }

    return s.repo.CreateUser(ctx, user)
}

func (s *AccountService) GetUsers(ctx context.Context, limit int, offset int) ([]model.User,
error) {
    return s.repo.GetUsers(ctx, limit, offset)
}

func (s *AccountService) GetUser(ctx context.Context, userID uint64) (model.User, error) {
    return s.repo.GetUser(ctx, userID)
}

func (s *AccountService) DeleteUser(ctx context.Context, userID uint64) error {
    return s.repo.DeleteUser(ctx, userID)
}

func (s *AccountService) UpdateUser(ctx context.Context, userID uint64,
user model.UpdateUser) error {
    return s.repo.UpdateUser(ctx, userID, user)
}
```

Тут мы используем интерфейс репозитория и не ссылаемся на пакет, в котором он реализован, — именно так достигается изоляция слоев между собой. Мы описываем в интерфейсе методы, которые должны быть реализованы, и когда будем создавать сервисный слой, при вызове `New` должны будем передать `repository`, удовлетворяющий описанному интерфейсу. Еще одно неочевидное преимущество такого подхода — тестируемость. Чтобы покрыть код `unit`-тестами, достаточно будет создать заглушку для слоя репозитория и «скормить» ее сервису, — при этом тестировать можно будет без реальной базы.

Определив слои миграций, репозитория и бизнес-логики, мы выходим на следующую нерешенную пока задачу: каким образом запросы из внешнего мира будут обращаться к этой логике? Это означает, что нужен еще один слой, готовый принять запрос, разобрать его, превратить во внутренние сущности и передать на обработку сервису. В архитектуре это называется *серверный*, или *транспортный*, слой. Серверный слой служит своеобразным передатчиком между двумя средами. Снаружи он говорит на языке протоколов, а внутри же обращается к бизнес-логике, для которой важны не форматы сетевых сообщений, а сами факты и правила, по которым живет система.

Принципиально важно, что серверный уровень (листинг 1.26) не содержит бизнес-правил. Он не решает, как именно зарегистрировать пользователя или какие условия наложить на его пароль. Это остается задачей сервисного слоя.

Листинг 1.26. `internal/server/server.go`

```
package server

import (
    "account/internal/mapper"
    "account/internal/model"
    "context"

    accountpb "github.com/yuliaapopova/contracts/account/go"
    "google.golang.org/protobuf/types/known/emptypb"

    "github.com/rs/zerolog"
)

type Server struct {
    accountpb.UnimplementedAccountServer

    accountService AccountService
    logger          *zerolog.Logger
}

func New(accountService AccountService, logger *zerolog.Logger) *Server {
    return &Server{accountService: accountService, logger: logger}
}

type AccountService interface {
    CreateUser(context.Context, model.CreateUser) error
    GetUser(ctx context.Context, userID uint64) (model.User, error)
    GetUsers(ctx context.Context, limit int, offset int) ([]model.User, error)
    DeleteUser(ctx context.Context, userID uint64) error
    UpdateUser(ctx context.Context, userID uint64, user model.UpdateUser) error
}
```



```
func (s *Server) CreateUser(ctx context.Context, req *accountpb.CreateUserRequest)
(*emptypb.Empty, error) {
    user := mapper.PbToUserCreate(req.GetUser())
    if err := s.accountService.CreateUser(ctx, user); err != nil {
        return nil, err
    }
    return &emptypb.Empty(), nil
}

func (s *Server) GetUser(ctx context.Context, req *accountpb.GetUserRequest)
(*accountpb.GetUserResponse, error) {
    res, err := s.accountService.GetUser(ctx, req.GetUserId())
    if err != nil {
        return nil, err
    }
    return &accountpb.GetUserResponse{
        User: mapper.UserToPb(res),
    }, nil
}

func (s *Server) GetUsers(ctx context.Context, req *accountpb.GetUsersRequest)
(*accountpb.GetUsersResponse, error) {
    res, err := s.accountService.GetUsers(ctx, int(req.GetPagination().GetLimit()),
                                                int(req.GetPagination().GetOffset()))
    if err != nil {
        return nil, err
    }
    return &accountpb.GetUsersResponse{
        Users: mapper.UsersToPbs(res),
    }, nil
}

func (s *Server) UpdateUser(ctx context.Context, req *accountpb.UpdateUserRequest)
(*emptypb.Empty, error) {
    user := mapper.PbToUserUpdate(req.User)
    if err := s.accountService.UpdateUser(ctx, uint64(req.GetUserId()), user); err != nil {
        return nil, err
    }
    return &emptypb.Empty(), nil
}

func (s *Server) DeleteUser(ctx context.Context, req *accountpb.DeleteUserRequest)
(*emptypb.Empty, error) {
    if err := s.accountService.DeleteUser(ctx, uint64(req.GetUserId())); err != nil {
        return nil, err
    }
    return &emptypb.Empty(), nil
}
```

Всё, что тут происходит, — это подготовка данных для передачи в сервисный слой, вызов соответствующего метода и преобразование результата в формат ответа, который ожидается.

Теперь у нас есть все необходимые модули, четко разграниченные между собой: хранение данных, сервисная логика и серверный слой, принимающий запросы и отправляющий ответы. Остается только собрать всё это воедино (листинг 1.27).

Листинг 1.27. Файл cmd/main.go

```
package main

import (
    "account/internal/config"
    "account/internal/logger"
    _ "account/internal/migrations"
    "account/internal/repository"
    "account/internal/server"
    "account/internal/service"
    "database/sql"
    "fmt"
    "log"
    "net"

    "github.com/pressly/goose/v3"
    accountpb "github.com/yuliaapopova/contracts/account/go"
    _ "google.golang.org/genproto/googleapis/api/annotations"

    _ "github.com/yuliaapopova/contracts/pagination/go"
    "google.golang.org/grpc"

    "gorm.io/driver/postgres"
    "gorm.io/gorm"

    _ "github.com/lib/pq"
)

func main() {
    // Загружаем конфигурацию
    cfg, err := config.Load()
    if err != nil {
        log.Fatalf("Failed to load config: %v", err)
    }

    // Инициализируем общий логгер с названием сервиса
    logger := logger.New(cfg)

    // Подключаемся к базе данных
    db, err := gorm.Open(postgres.Open(cfg.DbDsn), &gorm.Config{})
```

```
if err != nil {
    logger.Error().Msgf("Failed to connect to database: %v", err)
    return
}
logger.Info().Msg("database connected")

// Настраиваем миграции
migrationConn, err := db.DB()
if migrationConn == nil {
    logger.Fatal().Err(err).Msg("db connection is nil")
}

if err := goose.SetDialect("postgres"); err != nil {
    logger.Fatal().Err(err).Msg("failed to select migrations dialect")
}

dbGoose, err := sql.Open("postgres", cfg.DbDsn)
if err != nil {
    logger.Fatal().Err(err).Msg("failed to create sql connection")
}

// Запуск миграций
if err := goose.Up(dbGoose, "internal/migrations"); err != nil {
    logger.Fatal().Err(err).Msg("failed to run up migrations")
}

// Инициализируем репозиторий
repo := repository.NewRepository(db, &logger)
// Инициализируем сервис
service := service.New(repo, &logger)
// Инициализируем сервер
server := server.New(service, &logger)

// Создание gRPC сервера
s := grpc.NewServer()
accountpb.RegisterAccountServer(s, server)

listenAddr := fmt.Sprintf("%s:%d", cfg.Host, cfg.Port)
lis, err := net.Listen("tcp", listenAddr)
if err != nil {
    logger.Fatal().Msg("failed to listen")
}

logger.Info().Msg("gRPC server listening")
if err = s.Serve(lis); err != nil {
    logger.Err(err).Msg("failed to serve")
    return
}
}
```

Комментариями в коде отмечены основные моменты старта сервера.

На текущем этапе видно, как файл `cmd/main.go` начинает разрастаться: в нем уже сконцентрировано подключение конфигурации, инициализация логирования, настройка подключения к базе данных и запуск gRPC-сервера. Подобная концентрация ответственности затрудняет поддержку и понимание кода, — ведь входная точка приложения должна выполнять в основном роль старта процесса, а не содержать всю логику подготовки окружения.

Чтобы вернуть архитектуре ясность, используют подход, который принято называть *dependency injection* (внедрение зависимости) или сокращенно *DI*. Его суть проста: каждая часть системы должна получать свои зависимости извне, а не создавать их внутри себя. Мы не позволяем сервису напрямую решать, какой репозиторий ему использовать, или обработчику — выбирать конкретный сервис. Всё это решается на уровне «сборки» приложения, вне границ собственно бизнес-логики.

В Go для этого часто создают дополнительный слой — например, в каталоге `internal/app`. Там пишется «компоновщик», который отвечает только за одно — собрать зависимости в правильном порядке и вернуть готовое приложение. В таком варианте `main.go` остается максимально легким: загрузить конфигурацию, передать ее в сборщик `app`, а затем запустить сервер. Всё остальное сосредоточено внутри `internal/app/app.go`.

Сам файл `app.go` (листинг 1.28) можно представить как примитивный DI-контейнер, написанный вручную. Он создает подключение к базе, оборачивает его в репозиторий, затем формирует сервисы, передает их в обработчики и, наконец, прокидывает в сервер. Здесь хорошо видно, как зависимости складываются каскадом. Сервис зависит от интерфейса репозитория, репозиторий — от подключения к базе, и т. д. Нигде внутри слоев мы не создаем новые объекты, а только принимаем уже готовые. Благодаря этому каждый слой можно изолировать и тестировать отдельно.

Листинг 1.28. `internal/app/app.go`

```
package app

import (
    "account/internal/config"
    _ "account/internal/migrations"
    "account/internal/repository"
    "account/internal/server"
    "account/internal/service"
    "context"
    "database/sql"
    "fmt"
    "net"

    "github.com/pressly/goose/v3"
    accountpb "github.com/yuliaapopova/contracts/account/go"
    _ "google.golang.org/genproto/googleapis/api/annotations"
    "google.golang.org/grpc"
```

```

    _ "github.com/lib/pq"
    "github.com/rs/zerolog"
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
}

type App struct {
    cfg      *config.Config
    logger   *zerolog.Logger

    accountRepository *repository.Repository
    accountService    *service.AccountService
    accountServer     *server.Server
    grpcServer        *grpc.Server
}

func New(logger *zerolog.Logger, cfg *config.Config) *App {
    return &App{
        cfg:      cfg,
        logger:   logger,
    }
}

func (a *App) Run(ctx context.Context) error {
    // Инициализируем gRPC сервер
    accountServer, err := a.getAccountServer(ctx)
    if err != nil {
        return fmt.Errorf("failed to get account server: %w", err)
    }

    // Создаем gRPC сервер
    a.grpcServer = getGRPCServer(accountServer)

    listenAddr := fmt.Sprintf("%s:%d", a.cfg.Host, a.cfg.Port)
    lis, err := net.Listen("tcp", listenAddr)
    if err != nil {
        a.logger.Fatal().Err(err).Msg("failed to listen")
        return err
    }

    a.logger.Info().Msg("gRPC server listening")

    serveErrCh := make(chan error, 1)
    go func() {
        serveErrCh <- a.grpcServer.Serve(lis)
    }()
}

```

```
select (
case <-ctx.Done():
    a.grpcServer.GracefulStop()
    return ctx.Err()
case err := <-serveErrCh:
    if err != nil {
        a.logger.Error().Err(err).Msg("failed to serve")
    }
    return err
}
}

func (a *App) getRepository(ctx context.Context) (*repository.Repository, error) {
    if a.accountRepository == nil {
        if err := a.runMigrations(ctx); err != nil {
            return nil, fmt.Errorf("failed to get repository: %w", err)
        }
        db, err := gorm.Open(postgres.Open(a.cfg.DbDsn), &gorm.Config{})
        if err != nil {
            return nil, fmt.Errorf("gorm init failed: %w", err)
        }
        a.accountRepository = repository.NewRepository(db, a.logger)
    }
    return a.accountRepository, nil
}

func (a *App) runMigrations(ctx context.Context) error {
    if err := goose.SetDialect("postgres"); err != nil {
        return fmt.Errorf("failed to select migrations dialect: %w", err)
    }

    dbGoose, err := sql.Open("postgres", a.cfg.DbDsn)
    if err != nil {
        return fmt.Errorf("failed to create sql connection: %w", err)
    }

    if err := goose.UpContext(ctx, dbGoose, "internal/migrations"); err != nil {
        return fmt.Errorf("failed to run up migrations: %w", err)
    }
    return nil
}

func (a *App) getAccountService(ctx context.Context) (*service.AccountService, error) {
    if a.accountService == nil {
        repo, err := a.getRepository(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get repository: %w", err)
        }
    }
}
```

```

    a.accountService = service.New(repo, a.logger)
}
return a.accountService, nil
}

func (a *App) getAccountServer(ctx context.Context) (*server.Server, error) {
    if a.accountServer == nil {
        service, err := a.getAccountService(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get service: %w", err)
        }
        a.accountServer = server.New(service, a.logger)
    }
    return a.accountServer, nil
}

func getGRPCServer(srv *server.Server) *grpc.Server {
    grpcSrv := grpc.NewServer()
    accountpb.RegisterAccountServer(grpcSrv, srv)
    return grpcSrv
}

// Close корректно завершает работу приложения
func (a *App) Close() error {
    if a.grpcServer != nil {
        a.grpcServer.GracefulStop()
    }
    return nil
}

```

Метод `New` возвращает новый контейнер, сохраняя в нем минимально необходимое: конфигурацию и логгер, всё остальное будет добавляться постепенно. Запуском приложения служит метод `Run` — здесь инициализируется gRPC-сервер, создается прослушивание TCP-порта и запускается цикл обработки запросов. При этом приложение остается отзывчивым к сигналам остановки: используется `context.Context` и канал для ошибок, чтобы корректно завершать работу и делать `GracefulStop` при получении сигнала завершения.

Особое внимание стоит обратить на отложенное создание серверного слоя. Вызов `a.getAccountServer` тащит за собой цепочку: если не создан сервис — он создается, если не создан репозиторий — он тоже поднимется. Эта цепочка отражает нашу архитектуру: сверху вниз от транспорта к бизнес-логике и дальше к данным.

Метод `getRepository` отвечает еще и за подготовку базы данных. Но перед тем, как открыть соединение, он обязательно запускает миграции — отдельный шаг, который гарантирует, что структура базы соответствует текущей версии приложения.

Такой подход позволяет вернуть файлу `main.go` роль настоящей точки входа — он больше не перегружен логикой инициализации (листинг 1.29). Кроме того, сборка

становится прозрачной — вся инфраструктурная часть сосредоточена в одном месте. Файл сборки также легче расширять: можно заменить базу данных на другую, добавить новый сервис или протокол не затрагивая основной код. Фактически `app.go` играет роль каркаса, скрепляющего отдельные кирпичики системы.

Листинг 1.29. Файл `cmd/main.go`

```
package main

import (
    "account/internal/app"
    "account/internal/config"
    "account/internal/logger"
    "context"
)

func main() {
    ctx := context.Background()

    cfg, err := config.Load()
    if err != nil {
        panic(err)
    }

    l := logger.New(cfg)

    application := app.New(&l, cfg)

    if err := application.Run(ctx); err != nil {
        l.Fatal().Err(err).Msg("error")
    }
}
```

В отличие от полноценных DI-контейнеров, которые часто встречаются в больших фреймворках других языков, в Go такой контейнер обычно делают руками, без магии и автогенерации. Это сознательный выбор — код сборки остается предельно явным, а значит, управляемым и прогнозируемым. Каждый программист видит, как устроена композиция приложения, и может легко изменить ее при необходимости.

Таким образом, вынося инфраструктурный код из `main.go` в `internal/app/app.go`, мы не просто очищаем точку входа. Мы вводим четкую дисциплину зависимости: кто от кого зависит и в каком порядке строится приложение. Это создает архитектуру, которая легко тестируется, поддерживается и развивается.

Тестирование микросервиса

Перед тем как приступить к проверке разработанной функциональности, необходимо настроить интерфейс Postman (рис. 1.7) для удобной работы — чтобы в дальнейшем не приходилось выполнять лишние действия.

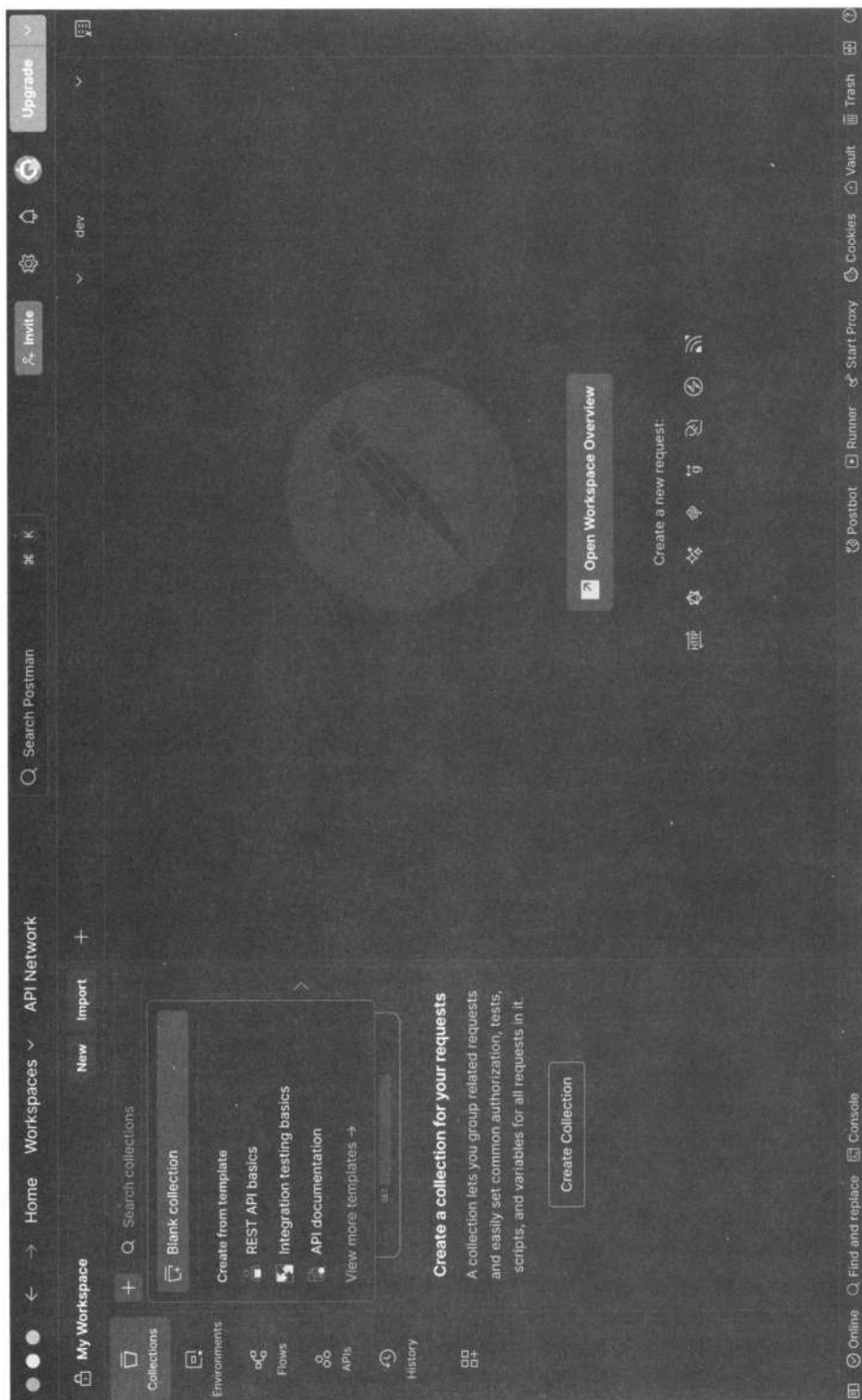


Рис. 1.7. Интерфейс Postman

И первое, с чего стоит начать, — настройка окружения в этом инструменте. Выберите опцию **Create Collection** и создайте коллекцию запросов, назвав ее `microservices` (рис. 1.8) — в ней мы разместим все запросы, которые у нас будут в рамках этого проекта.

Создайте далее набор переменных окружения — для этого на вкладке **No Environments** выберите опцию **Create Environment** (см. рис. 1.8, *справа*). Для начала нам хватит задать URL-адрес для сервиса `account`. Соответственно, в поле **Variable** введем `account`, а в поле **Initial value** этот URL — `localhost:50051` (рис. 1.9). Окружение назовем `dev` — в соответствии с окружением, с которым сейчас работаем (если бы у нас сервис был развернут также на внешних контурах, имело бы смысл задать переменные и для них тоже). Сюда можно добавить и другие переменные, которые редко изменяются. Не забываем нажать **Save**, иначе не увидим изменений.

Напишем теперь запросы для созданных API. Начнем с создания пользователей — пока не создано ни одной записи, остальные роуты смысла не имеют. Выбираем в меню опцию **New**, после чего в открывшемся поле выбираем **gRPC**, а в поле **URL** вводим `{{account}}`. Через обращение `{{}}` мы получаем доступ к переменной окружения с соответствующим названием. В поле **Message** заполняется объект запроса, есть также функция **Use Example Message** — автоматическая генерация запроса со случайно заданными значениями. Это удобно использовать как шаблон, а после заполнять нужными параметрами.

Чтобы выполнить gRPC-запрос, необходимо импортировать в Postman proto-файл сервиса. Для этого надо перейти на вкладку **Service definition** и указать путь к нужному файлу. Так как у нас контракт состоит из нескольких proto-файлов, их тоже нужно указать. Важный момент: для дополнительных proto-файлов следует указывать не путь к самому файлу, а к каталогу, относительно которого используется импорт в самом proto-файле сервиса. Так, у нас в файле `account_service.proto` есть импорт `account_model.proto`, который выглядит так:

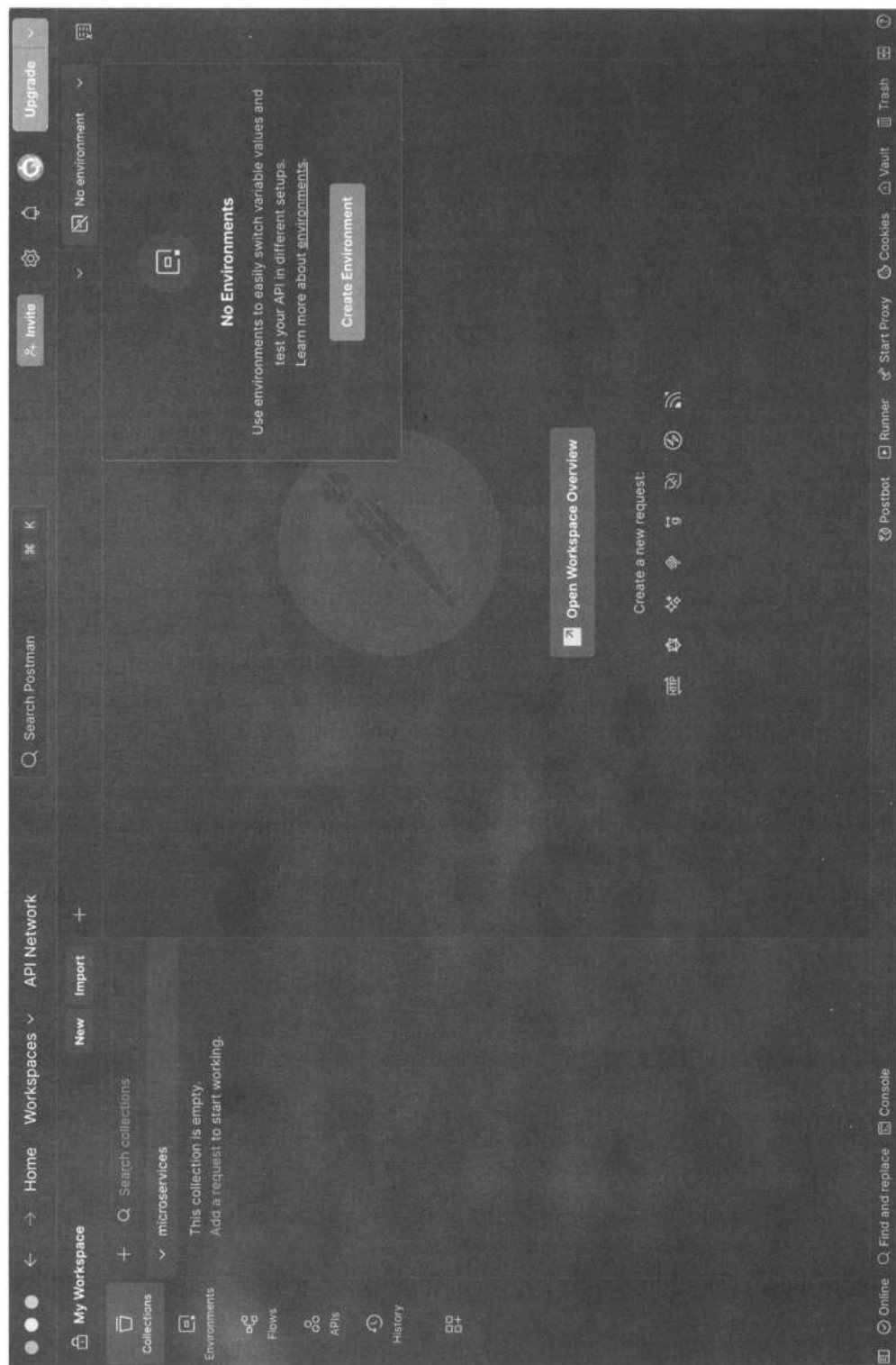
```
import "account/account_model.proto";
```

Соответственно, в Postman нужно указать путь к каталогу `account` (он должен быть внутри каталога, который мы открываем), а не к файлу `account_model.proto`.

И выполняем запрос (рис. 1.10).

В результате мы получили статус запроса **0 — OK**, а это означает, что запись создана, вернулись также данные записи. Создадим по той же схеме еще несколько пользователей, чтобы было с чем работать дальше.

Теперь мы можем выполнить запрос на получение списка пользователей, хоть целиком, хоть по фильтрам. Запрос с фильтром по `limit` и `offset`, например, будет выглядеть так, как показано на рис. 1.11.

Рис. 1.8. Создание коллекции запросов `microservices`

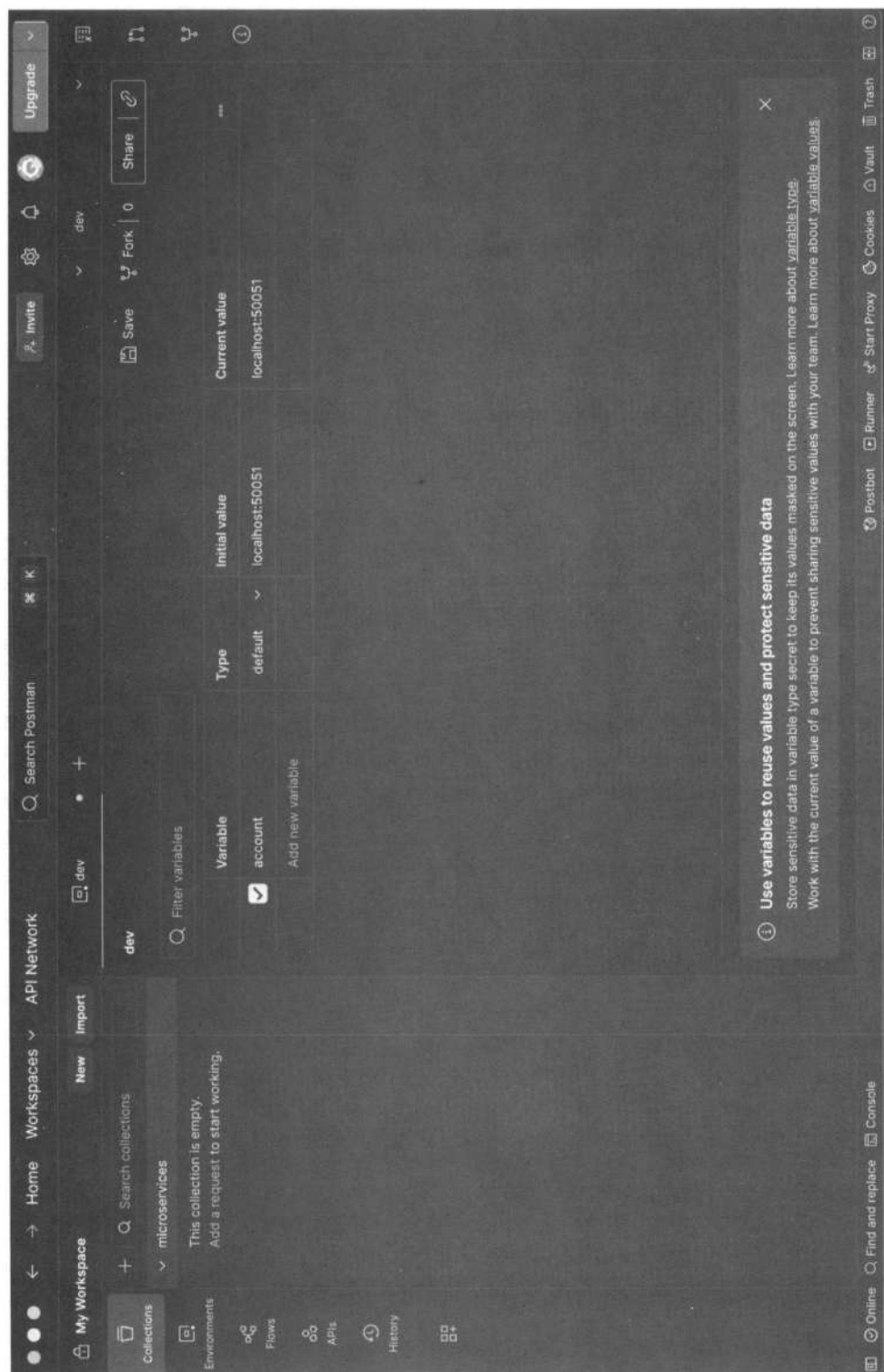


Рис. 1.9. Заполнение переменных окружения в Postman

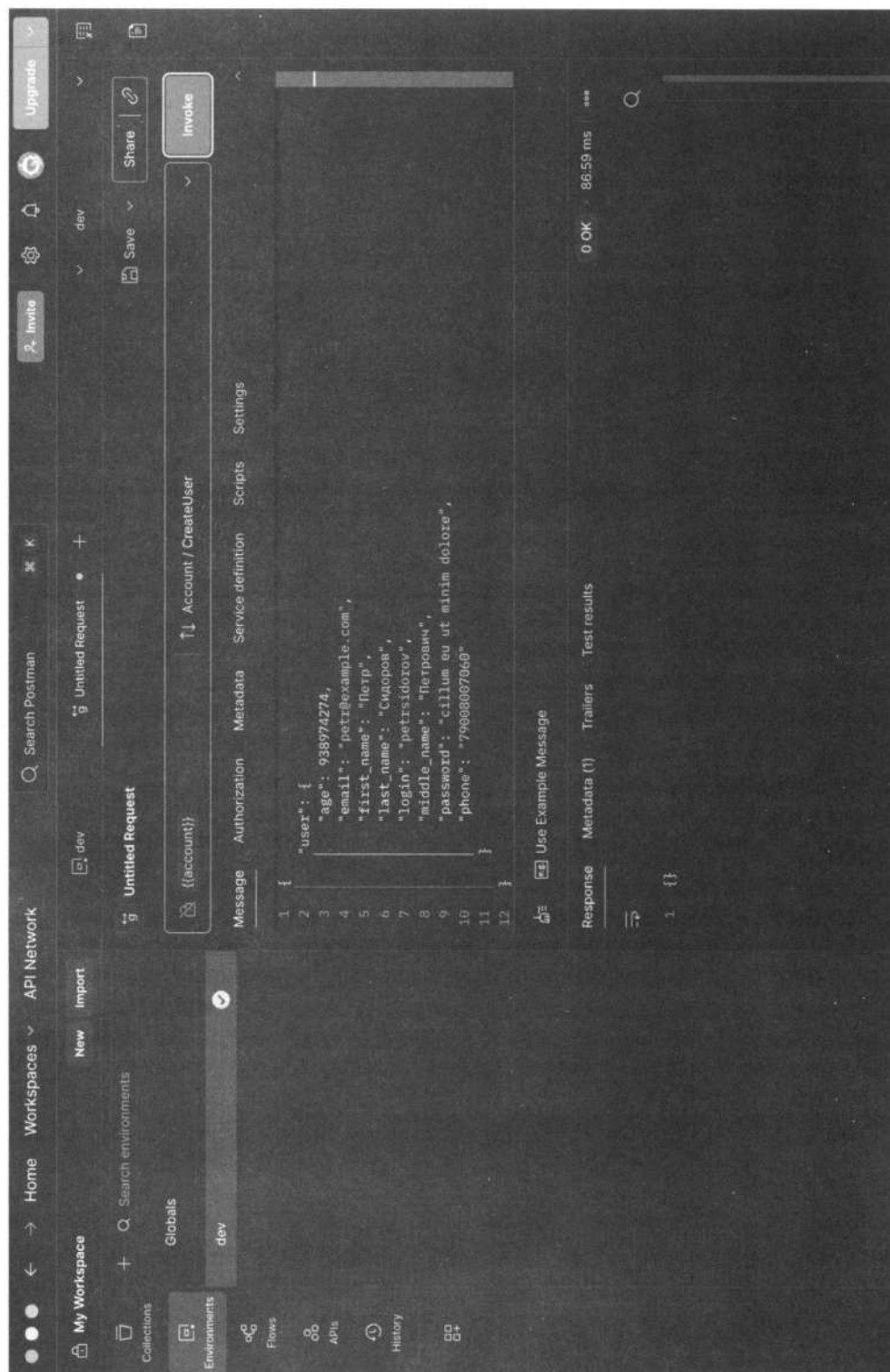


Рис. 1.10. Выполнение запроса на создание записи пользователя

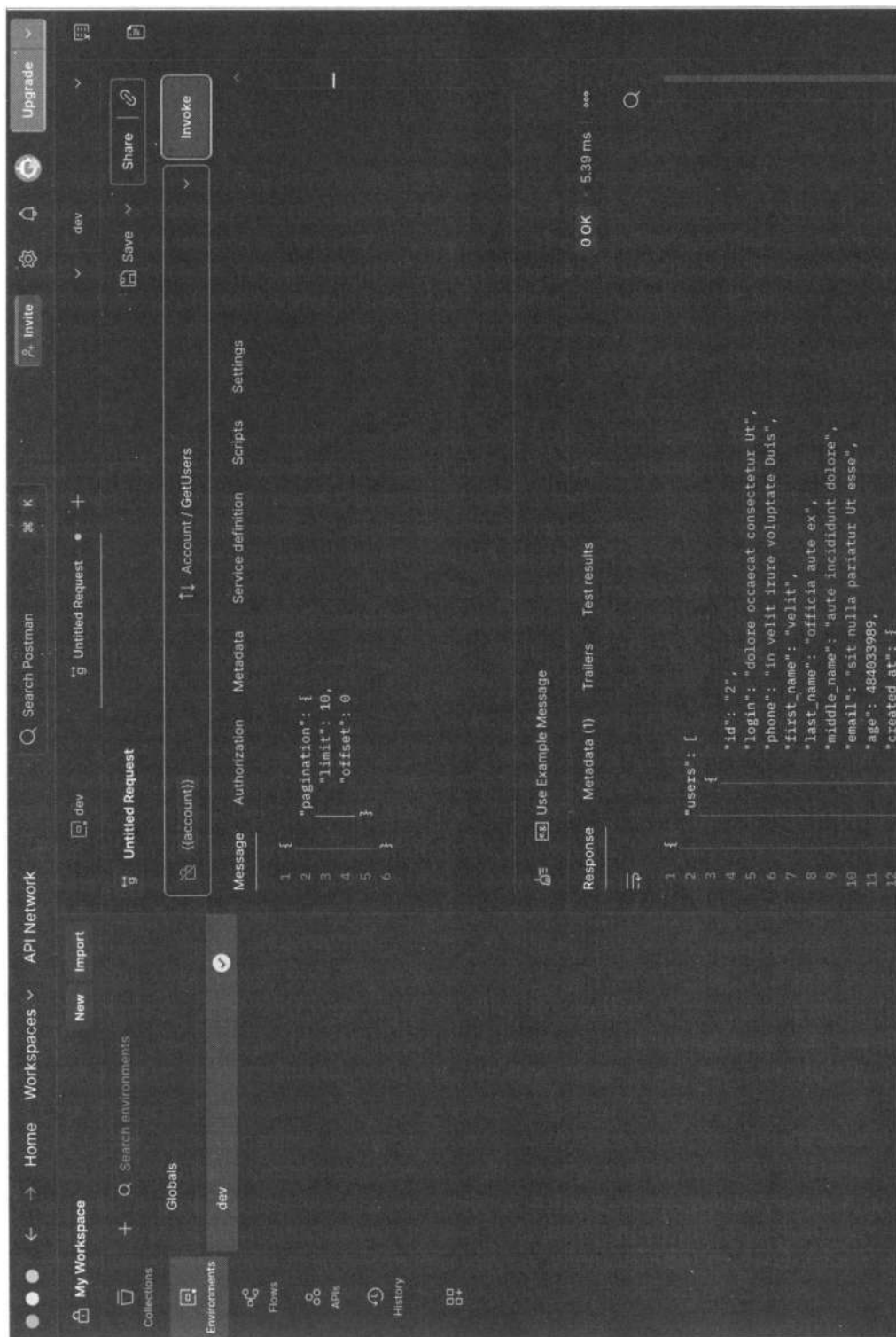


Рис. 1.11. Запрос пользователей с пагинацией

Список литературы и источников

1. <https://semver.org/lang/ru/>.
2. <https://habr.com/ru/companies/simbirsoft/articles/675878/>.
3. <https://habr.com/ru/articles/483202/>.
4. <https://habr.com/ru/articles/579248/>.
5. <https://habr.com/ru/articles/193756/>.
6. <https://habr.com/ru/articles/193756/>.
7. Лекции по дисциплине «Базы данных» БГТУ им. В. Г. Шухова.



Разработка микросервиса авторизации и аутентификации (Auth)

Теоретический обзор способов авторизации и аутентификации

Аутентификация, идентификация и авторизация

При разработке приложений несколько более сложных, чем одностраничные (Single Page Application, SPA), особенно если там еще нужна и микросервисная архитектура, весьма высока вероятность, что придется работать с данными пользователей. Соответственно, необходимо обеспокоиться гарантией того, что никто не сможет получить к чужим данным несанкционированный доступ. Для начала разберемся с терминами.

- ❑ **Идентификация** — предоставление информации о том, кем вы являетесь. Это может быть что угодно: имя, адрес электронной почты, номер телефона, идентификатор учетной записи и т. д. Стоит отметить, что в момент, когда вы заполняете форму анкеты на каком-либо сайте, вы идентифицируете себя в качестве кого-то. При этом, даже если вы не нажали кнопку «сохранить», это все равно будет считаться идентификацией.
- ❑ **Аутентификация** — предоставление доказательств того, что вы на самом деле есть тот, кем идентифицировались. То есть кто-то должен подтвердить, что вы именно тот, за кого себя выдаете. Это может быть осуществлено, если вы предоставите документ, удостоверяющий личность, и контролер удостоверится, что на фотографии в документе действительно вы, а документ при этом не поддельный.
- ❑ **Авторизация** — проверка того, что вам разрешен доступ к запрашиваемому ресурсу. То есть если вы хотите удалить, например, нарушающий правила форума комментарий, системе перед этим необходимо убедиться, что вы как минимум являетесь модератором этого форума и вам разрешен доступ на совершение указанного действия.

Если говорить про способы аутентификации в веб-среде в таких терминах, то традиционно под *идентификацией* понимают получение вашей учетной записи по

имени пользователя (username), а под *аутентификацией* — проверку того, что вы знаете пароль от этой учетной записи. *Авторизацией* же будет считаться проверка вашей роли в системе и решение о предоставлении доступа к запрашиваемому ресурсу.

Рассмотрим подробнее способы аутентификации и авторизации [1].

Аутентификация по паролю

Этот метод основан на том, что при регистрации пользователь задает имя пользователя (логин) и пароль, а при повторном входе для успешной идентификации и аутентификации в системе их предъявляет. Часто в качестве имени пользователя используется адрес электронной почты.

Прежде всего, при аутентификации по логину и паролю используется стандартный HTTP/HTTPS-протокол. Это работает следующим образом:

1. При запросе неавторизованного клиента на доступ к ресурсу сервис отвечает статусом 401 «Unauthorized» и добавляет заголовок «WWW-Authenticate» с указанием схемы и параметров аутентификации.
2. Браузер, получив такой ответ, перенаправляет посетителя на страницу авторизации, чтобы посетитель ввел свои логин (username) и пароль (password).
3. Во всех следующих запросах к этому веб-сайту браузер добавляет заголовок «Authorization», в котором передаются необходимые данные для аутентификации посетителя сервером.
4. Сервер аутентифицирует посетителя по данным из этого заголовка на каждый запрос и принимает решение о том, предоставлять ему или нет доступ к запрашиваемому ресурсу (авторизация).
5. При этом существует несколько схем аутентификации, различающихся по уровню безопасности:
 - Basic — username/password пользователя передаются в заголовке «Authorization» в незашифрованном виде (base64-encoded). Впрочем, при использовании HTTPS-протокола эта схема является относительно безопасной;
 - Digest — challenge/response схема, при которой сервер посылает уникальное значение nonce (чаще всего оно представляет собой временную метку и, возможно, какую-то еще добавочную информацию, что не обязательно), а браузер передает MD5 хеш пароля пользователя, который вычисляется с использованием указанного nonce;
 - схемы NTLM и Negotiate применяются для пользователей домена Windows, что не способствует широкому их распространению.

Стоит отметить, что при использовании HTTP-аутентификации у пользователя нет стандартной возможности выйти из веб-приложения, кроме как закрыть все окна браузера.

Описанный здесь способ аутентификации пользователей применяется для веб-сайтов, однако при разработке систем более широкого применения (с участием мо-

бильной разработки — например, для iOS или Android), наряду с HTTP-аутентификацией, часто используются и другие способы, где данные для аутентификации передаются в других частях запроса. Например, логин и пароль также можно передать в HTTP-запросах:

- ❑ в URL query — при использовании HTTP, а не HTTPS это считается небезопасным, поскольку строки URL могут запоминаться браузерами и веб-серверами. Кроме того, такие данные могут быть перехвачены злоумышленником;
- ❑ в Request body — безопасный вариант, но применяется только для POST-, PUT- и PATCH-запросов, где задействуется тело сообщения;
- ❑ в HTTP header — оптимальный вариант, при этом могут использоваться и стандартный заголовок «Authorization» вместе с Basic-схемой, и другие пользовательские заголовки.

Аутентификация по сертификатам

Идентификационный документ (сертификат), заверенный органом по сертификации (Certificate Authority, CA), содержит характеристики, которые дают возможность опознать его владельца. Роль CA аналогична роли посредника, обеспечивающего подтверждение подлинности сертификатов (по аналогии с органами, выдающими паспорта). Кроме того, криптографически связанный с закрытым ключом сертификат находится в собственности владельца документа и позволяет четко установить, что документ принадлежит ему.

На стороне пользователя сертификат и соответствующий закрытый ключ могут храниться в операционной системе, браузере, файле или на физическом устройстве (например, смарт-карте, USB-токене). Как правило, закрытый ключ дополнительно защищен паролем или PIN-кодом.

В контексте веб-приложений часто используются сертификаты формата X.509. Проверка подлинности с помощью сертификата X.509 происходит при установлении связи с сервером и интегрирована в протокол SSL/TLS. Этот метод также хорошо поддерживается веб-браузерами, позволяя пользователям выбирать и использовать сертификаты, если веб-сайт тоже поддерживает такой способ аутентификации.

При этом сертификат проверяется сервером на основании следующих правил:

- ❑ сертификат должен быть подписан доверенным органом по сертификации (CA);
- ❑ сертификат должен быть действительным на текущую дату (осуществляется проверка срока действия);
- ❑ сертификат не должен быть отозван соответствующим CA (производится проверка списков исключения).

После успешной проверки подлинности сертификата веб-приложение может принять решение, разрешить ли доступ на основе данных из сертификата — таких как имя владельца, эмитент (тот, кто выдал сертификат), серийный номер или отпечаток открытого ключа.

Применение сертификатов для проверки подлинности — более надежный способ, чем использование паролей. Его надежность обеспечивается благодаря цифровой подписи, которая создается при проверке подлинности и подтверждает использование определенного закрытого ключа (невозможность отрицания). Однако сложности с распространением и поддержкой сертификатов делают этот способ не столь доступным для многих пользователей.

Аутентификация по одноразовым паролям

Аутентификация с использованием временных паролей зачастую дополняет обычную аутентификацию паролями, образуя *двухфакторную аутентификацию (2FA)*. В этом методе пользователь обязан подтвердить как обычный пароль (который он знает), так и уникальный временный пароль. Применение двух факторов позволяет повысить безопасность, что особенно актуально для приложений, связанных с финансовыми активами, хотя и не ограничивается только ими.

Помимо использования для обеспечения доступа к ресурсам, одноразовые пароли могут потребоваться для подтверждения личности при совершении значимых операций — таких как денежные переводы, подтверждение покупок, продление подписок или изменение аккаунтных настроек.

В последнее время временные пароли также нашли свое применение при подключении к корпоративным виртуальным частным сетям (VPN), повышая уровень безопасности и обеспечивая защиту конфиденциальной информации.

Для генерации одноразовых паролей есть разные способы, вот наиболее распространенные из них:

□ аппаратные или программные токены.

Стандартный аппаратный токен — это небольшое устройство, обычно представленное в виде пластиковой карты или брелока для ключей. Простейшие аппаратные токены похожи на USB-флешки (их часто называют *донглами*) и содержат сертификат или уникальный идентификатор. У более сложных токенов могут иметься ЖК-дисплеи, клавиатуры для ввода паролей, биометрические считыватели, устройства беспроводной связи и дополнительные функции для повышения безопасности [1].

С технической точки зрения информация для аутентификации может передаваться в различных частях HTTP-запроса (наиболее оптимальный вариант — header). В некоторых случаях для передачи токена могут использоваться заголовки *Bearer*. Чтобы избежать перехвата ключей, соединение с сервером должно быть обязательно защищено протоколом SSL/TLS;

□ случайно генерируемые коды, передаваемые пользователю через SMS или другой канал связи.

В этой ситуации фактор владения — SIM-карта, привязанная к определенному номеру. Сейчас также активно используются способы типа *Google Authenticator* — их принцип работы заключается в том, что генерируется одноразовый код, который действует очень короткий промежуток времени (часто меньше минуты);

- ❑ распечатка или scratch card со списком заранее сформированных одноразовых паролей.

При этом для каждого нового входа в систему требуется ввести новый одноразовый пароль с указанным номером.

В веб-приложениях такой механизм аутентификации часто реализуется посредством расширения forms authentication: после первичной аутентификации по паролю создается сессия пользователя, однако в контексте этой сессии пользователь не имеет доступа к приложению до тех пор, пока не выполнит дополнительную аутентификацию по одноразовому паролю.

Аутентификация по ключам доступа

Этот метод чаще всего используется для подтверждения подлинности устройств, сервисов или других приложений при их обращении к веб-сервисам. В этом случае в роли секрета выступают ключи доступа (access key, API key) — уникальные длинные строки, содержащие произвольный набор символов. Суть состоит в том, что они заменяют комбинацию имени пользователя и пароля.

В большинстве случаев сервер генерирует эти ключи по запросу пользователей, которые впоследствии сохраняют их в клиентских приложениях. При создании ключа можно также ограничить срок его действия и уровень доступа, предоставляемый клиентскому приложению при аутентификации через этот ключ.

С точки зрения технической реализации метода единого протокола нет — ключи могут передаваться в HTTP-запросе разными способами, включая URL-параметры, тело запроса или заголовок HTTP. Как и в случае с аутентификацией по паролю, оптимальным выбором является использование заголовка HTTP. В некоторых случаях для передачи токена в заголовке применяется HTTP-схема Bearer (Authorization: Bearer [token]). Чтобы предотвратить перехват ключей, обязательно должно быть реализовано защищенное SSL/TLS-соединение с сервером.

Кроме того, существуют более сложные схемы аутентификации по ключам для незащищенных соединений. При этом ключ обычно разделяется на две части: публичную и секретную. Публичная часть служит для идентификации клиента, в то время как секретная часть позволяет создать подпись. Подобно схеме цифровой аутентификации (digest authentication) сервер может отправить клиенту уникальное значение nonce или timestamp, а клиент возвращает хеш или HMAC этого значения, вычисленный с использованием секретной части ключа. Такой способ позволяет избежать передачи всего ключа в открытом виде и обеспечивает защиту от атак с повторным воспроизведением (replay attacks).

Аутентификация по токенам

В настоящее время такой метод аутентификации становится всё более распространенным. Он используется, когда один сервис передает ответственность за проверку подлинности другому сервису — например, через почтовый аккаунт или социаль-

ные сети. Этот подход наиболее востребован при построении распределенных систем.

Суть метода заключается в том, что поставщик идентификации (Identity Provider, IP) предоставляет надежные пользовательские данные в виде токена, а сервис-поставщик (Service Provider, SP) использует этот токен для определения личности пользователя, а также для проверки его подлинности и предоставления доступа к ресурсам.

Весь процесс выглядит следующим образом:

1. Клиент аутентифицируется в IP одним из способов, специфичным для него (пароль, ключ доступа, сертификат).
2. Клиент просит IP предоставить ему токен для конкретного SP-приложения. IP генерирует токен и отправляет его клиенту.
3. Клиент аутентифицируется в SP-приложении при помощи этого токена.

Описанный процесс демонстрирует принцип аутентификации активного клиента — клиента, способного выполнять заданную последовательность действий, как, например, приложения для iOS/Android. С другой стороны, браузер является пассивным клиентом, который может лишь отображать страницы по запросу пользователя. В таком случае аутентификация достигается путем автоматического направления браузера между веб-приложениями поставщика идентификации и поставщика услуг.

Токен, в свою очередь, обычно представляет собой структуру данных, содержащую разнообразную информацию — такую как данные о выдаче токена, его срок действия и информацию о пользователе, для которого токен был выдан. Дополнительно токен подписывается для предотвращения несанкционированных изменений и обеспечения подлинности.

При аутентификации с помощью токена SP-приложение должно выполнить следующие проверки:

- ☐ токен должен быть выдан доверенным приложением;
- ☐ токен предназначен для текущего SP-приложению;
- ☐ срок действия токена еще не истек;
- ☐ токен подлинный и не был изменен.

Есть несколько распространенных форматов токенов веб-приложений:

- ☐ Simple Web Token (SWT) — наиболее простой формат, представляющий собой набор произвольных пар имя/значение в формате кодирования HTML form. Стандарт определяет несколько зарезервированных имен: Issuer, Audience, ExpiresOn и HMACSHA256. Токен подписывается с помощью симметричного ключа — таким образом, оба приложения (и IP, и SP) должны иметь этот ключ для возможности создания/проверки токена;
- ☐ JSON Web Token (JWT) — содержит три блока, разделенных точками: заголовок, набор полей (claims) и подпись. Первые два блока представлены в JSON-

формате и дополнительно закодированы в формат base64. Набор полей содержит произвольные пары имя/значение, притом стандарт JWT определяет несколько зарезервированных имен (iss, aud, exp и другие). Подпись может генерироваться при помощи как симметричных алгоритмов шифрования, так и асимметричных. Кроме того, существует отдельный стандарт, описывающий формат зашифрованного JWT-токена;

- ❑ **Security Assertion Markup Language (SAML)** — определяет токены в XML-формате, включающем информацию об эмитенте, о субъекте, необходимые условия для проверки токена и набор дополнительных утверждений (statements) о пользователе. Подпись SAML-токенов осуществляется при помощи асимметричной криптографии. Кроме того, в отличие от предыдущих форматов, SAML-токены содержат механизм для подтверждения владения токеном, что позволяет предотвратить перехват токенов с помощью атак «человек посередине» (Man-in-the-Middle-атаки) при использовании незащищенных соединений.

В настоящее время широко распространено применение аутентификации с использованием JWT. Даже передовые методы аутентификации, такие как OAuth и OpenID Connect, используют в своей основе этот тип токена. Процесс генерации токена, однако, может различаться в зависимости от сервиса, через который происходит аутентификация. Детальное рассмотрение OAuth и OpenID Connect, которые внедряют этот механизм, не является здесь нашей целью — более важно разобраться в механизме генерации, в информации, содержащейся в токене, и в собственно возможности его выдачи.

Отдельно стоит отметить, что в последние несколько лет активно набирает популярность инструмент Keycloak. Этот инструмент базируется на стандарте OpenID Connect и предоставляет средства аутентификации, авторизации, управления пользователями и обеспечения безопасности в общем смысле. Для небольших проектов Keycloak представляет собой мощное готовое решение, готовое к развертыванию и использованию. В случае больших коммерческих проектов, однако, могут возникнуть ограничения, требующие дополнительной настройки этого инструмента, а иногда и доработки его под конкретные потребности. Существуют также определенные сложности, связанные с большой нагрузкой, — сообщается, что при большом количестве активных сессий Keycloak может вести себя непредсказуемым образом. В то же время для слишком простых проектов использование Keycloak может оказаться излишним «переусложнением».

Базовые меры предосторожности от возможных уязвимостей

Разрабатываемое программное обеспечение подвержено множеству распространенных уязвимостей, способных представлять угрозу для его безопасности. Сюда включают переполнение буфера, состояние гонки, атаки на проверку ввода, атаки на аутентификацию и авторизацию, а также криптографические атаки. Однако соблюдение некоторых базовых принципов при разработке нового ПО позволяет

относительно легко избегать этих уязвимостей. Просто следует не применять методики программирования, которые способствуют их появлению.

Естественно, среди базовых рекомендаций присутствуют и банальные, но важные шаги. К примеру, необходимо предотвращать создание пользователями слишком простых паролей. Точно так же конечным пользователям настоятельно рекомендуется избегать использования одного и того же пароля для нескольких аккаунтов или хранения всех паролей в публично доступных местах — таких как обычный блокнот или стикер на рабочем столе.

Переполнение буфера

Этот раздел преимущественно касается оправданной и наивысшей валидации. Всё, что допустимо проверить на начальном этапе запроса, следует проверить, предотвращая таким образом даже попадание этого запроса в бизнес-логику. Если, например, предполагается, что номер телефона должен содержать 11 цифр, следует настроить валидацию таким образом, чтобы при получении меньшего или большего числа цифр ваше приложение сразу же сообщило об ошибке.

Отсутствие валидации для определенных значений может вызвать переполнение буфера. В большей степени это затрагивает языки с жесткой типизацией — такие как Java, C++ и аналогичные. В этих языках вы вручную управляете выделением памяти для переменных. Например, если вы резервируете место для 200 символов, но получаете в запросе 1000 символов, существует риск переполнения буфера, что может вызвать непредсказуемое поведение программы.

Состояние гонки

Состояние гонки выявляется при некорректном конструировании системы, когда более одного запроса одновременно стремятся получить доступ и изменить одни и те же данные. Прямо зависит функционирование системы и от последовательности выполнения потоков. В случае, если поток, который, по идее, должен предшествовать, уступает победу в гонке другому потоку, это вызывает изменение поведения кода, что поднимает завесу недетерминированных ошибок.

Атаки проверки ввода

Такой вид атаки также заслуживает внимания в рамках аспекта валидации. Помимо переполнения буфера, еще одна уязвимость, в которой затаилась опасность из-за недостаточной валидации, заключается в возможности внесения данных, не соответствующих заданному полю. Особое внимание следует обратить на специальные символы — такие как те, что применяются для экранирования информации или форматирования данных.

Через осознанный ввод специальных значений, предназначенных для управления форматом вывода, злоумышленник способен обрести возможность просматривать внутреннюю память приложения. Также возможен вариант записи информации в места, которые не предусмотрены для этого и доступ к которым извне обычно

ограничен. Под угрозой в результате окажется общее функционирование всего приложения.

Атаки аутентификации

В контексте атак на аутентификацию злоумышленники стремятся обрести доступ к ресурсам, не обладая необходимыми учетными данными. Применение надежных методов аутентификации в ваших приложениях способствует предотвращению этих видов атак.

Так, сорвать планы злоумышленников может требование создания сложных паролей. Восьмизначный пароль в нижнем регистре — такой как, например: `qwerty01`, — мощные вычислительные системы способны взломать практически мгновенно. Однако если пароль будет десятизначным и содержащим буквы и цифры в разных регистрах — например: `S7tvZsVmlq`, необходимое время для взлома возрастет до 20 лет и более. Важно также избегать внедрения в приложения жестко закодированных, неизменяемых паролей.

Следует воздержаться от осуществления аутентификации на клиентской стороне (на устройстве пользователя), т. к. это место подвержено наибольшей уязвимости перед атаками. Это аналогично предоставлению злоумышленникам прямого доступа к контрольным пунктам и допущению манипуляций с ними, что существенно ослабляет эффективность мер безопасности.

Если вы опираетесь на локальное приложение или процедуры для завершения этапов аутентификации, а затем отправляете серверу сообщение «все в порядке», ничто не мешает злоумышленнику напрямую повторить это сообщение на сервере, не завершив при этом аутентификацию. Правильной практикой является размещение механизмов аутентификации на серверной стороне — настолько далеко от доступа злоумышленников, насколько это возможно.

Атаки авторизации

Атаки на авторизацию направлены на получение доступа к ресурсам без необходимого разрешения. Аналогично механизмам аутентификации реализация механизмов авторизации на стороне клиента считается неблагоприятной стратегией. Любой процесс, осуществляемый в уязвимом месте, подверженный возможным нападениям или воздействию со стороны пользователей, практически неизбежно становится угрозой безопасности. Более разумным подходом станет реализация авторизации на удаленном сервере или на специализированном оборудовании, особенно если оно переносное, что обеспечит более серьезный контроль.

Крайне важна минимизация необходимых разрешений как для пользователей, так и для программного обеспечения, — иначе вы рискуете стать уязвимыми перед атаками. Кроме того, при каждой попытке пользователя или процесса выполнить действие, требующее привилегий, необходимо проводить проверку на наличие соответствующих разрешений. Если какой-либо пользователь случайно или умышленно получает доступ к ограниченным участкам вашего приложения, следует немедленно принять меры по его ограничению.

Атаки на стороне клиента

Атаки, связанные с клиентской стороной, направлены на слабые звенья в ПО, установленном на клиентских устройствах, или опираются на социальную инженерию для обмана пользователей. В этой категории атак существует множество видов, но сейчас мы ограничимся рассмотрением тех из них, что используют Интернет как среду для проведения атак.

Одной из таких атак является *межсайтовый скриптинг* (XSS). Он реализуется путем внедрения скриптового кода на веб-страницу или в другие медиафайлы — такие как анимации Flash или видео, которые затем выполняются в браузере пользователя. После просмотра такой страницы или медиафайла скрипт запускается, проводя атаку. Например, злоумышленник может добавить вредоносный скрипт в комментарий к блогу, который автоматически выполняется при просмотре страницы.

Другими видами атак на клиентской стороне являются подделка межсайтовых запросов и кликджекинг:

- ❑ в атаке с подделкой межсайтового запроса злоумышленник размещает ссылку на странице так, чтобы посетитель автоматически перешел по ней. Эта ссылка инициирует действие на другой странице или в приложении, где посетитель уже аутентифицирован. Примером может быть добавление товаров в корзину на Ozon или перевод денег между счетами;
- ❑ при кликджекинге злоумышленник обманывает посетителя, скрывая то, на что тот на самом деле нажимает. Эта атака основывается на графическом отображении браузера и может заставить пользователя выполнять действия, которые он не собирался совершать самостоятельно. Например, кнопка «Подробнее» может скрывать под собой кнопку «Купить сейчас».

Уровень защиты от подобных атак в значительной степени обеспечивается новыми версиями популярных браузеров — таких как Firefox, Safari и Chrome, которые автоматически блокируют множество распространенных атак. Однако следует помнить, что не всегда возможно предугадать все варианты атак, поэтому регулярные обновления браузеров являются важной мерой безопасности. Некоторые браузеры также предоставляют дополнительные инструменты для защиты от клиентских атак — например, NoScript для Firefox, который блокирует большинство сценариев веб-страниц и требует разрешения на их выполнение. Внимательное использование таких инструментов может существенно снизить риск клиентских атак.

Разработка модуля Auth

Говоря об аутентификации и авторизации в модуле микросервисной архитектуры, следует рассмотреть вопрос о том, где должен находиться этот модуль. В проекте Account, который сейчас у нас есть, проверять доступ следует как минимум на изменение данных — редактирование и удаление профиля пользователя. Получается, что если вынести в отдельный микросервис авторизацию и аутентификацию, нам

все равно понадобится делать запрос к сервису User, чтобы проверить корректность ввода пароля. Однако, если мы будем нагружать систему каждый раз дополнительными запросами к сервису User, это обязательно скажется на производительности.

Поэтому есть ещё один вариант — разнести данные: Account будет хранить только «бизнесовые» данные о пользователе, а Auth — отвечать только за то, что касается аутентификации: пароли, хеши, токены.

Аутентификация в большинстве случаев реализуется посредством JWT (JSON Web Token) [2] — можно даже сказать, что это практически уже стандарт в разработке. При авторизации посредством JWT возникает дискуссионный вопрос: нужно ли на каждый запрос получать информацию о пользователе из базы данных, чтобы убедиться в установленном ему статусе блокировки. С одной стороны, это безопасность, с другой — лишняя нагрузка в виде дополнительных запросов. Поэтому чаще прибегают к другому способу гарантии предотвращения несанкционированного доступа к данным — сокращают время жизни токена до 1–2 часов.

Процесс создания скелета сервиса Auth аналогичен тому, как мы это делали для Account, с той лишь разницей, что тут будет не столь простая бизнес-логика. Если необходимо, вы можете вернуться к предыдущей главе и повторить все те же самые подготовительные этапы при настройке окружения.

Начнем с написания контракта и генерации gRPC-кода для него. Для этого нужно создать каталог `auth` в пакете контрактов и отредактировать `Makefile`, чтобы и для `auth` генерировались файлы `.pb.go` (листинг 2.1).

Листинг 2.1. Файл `Makefile`

```
PROTO_ROOT=.
DOCKER_IMAGE=proto-builder

.PHONY: docker-build gen clean

docker-build:
    docker build -t $(DOCKER_IMAGE) .

gen: docker-build
    docker run --rm -v $(abspath $(PROTO_ROOT)): /app $(DOCKER_IMAGE) \
    bash -c '\
    set -e; \
    for dir in account pagination auth; do \
    echo ">> Processing $$dir"; \
    mkdir -p /app/$$dir/go; \
    cd /app/$$dir; \
    for file in *.proto; do \
    echo "    Generating $$dir/$$file"; \
    protoc \
    -I . \
    -I /app \
```

```

-I /usr/local/include/googleapis \
--go_out=go \
--go_opt=paths=source_relative \
--go-grpc_out=go \
--go-grpc_opt=paths=source_relative \
$$file; \
done; \
done \
,

```

clean:

```
find account pagination auth -type d -name go -exec rm -rf {} +
```

Изменения тут потребовались на самом деле минимальные — указать еще и `auth` в команде `clean` и в перечислении папок (`for dir`) в команде `gen`.

Дальше опишем контракт взаимодействия с сервисом аутентификации, состоящим из запросов для регистрации, авторизации, обновления токена, валидации токена и завершения сессии (листинг 2.2).

Листинг 2.2. Файл `contracts/auth/auth_service.proto`

```

syntax = "proto3";

package auth;

option go_package = "github.com/yuliappopova/auth/pkg/auth/go;auth";

import "google/protobuf/empty.proto";

service Auth {
  rpc Register(RegisterRequest) returns (google.protobuf.Empty);
  rpc Login(LoginRequest) returns (TokenPair);
  rpc Refresh(RefreshRequest) returns (TokenPair);
  rpc Validate(ValidateRequest) returns (ValidateResponse);
  rpc Logout(RefreshRequest) returns (google.protobuf.Empty);
}

message RegisterRequest {
  string login = 1;
  string email = 2;
  string password = 3;
}

message LoginRequest {
  string login_or_email = 1;
  string password = 2;
}

```

```
message RefreshRequest {
    string refresh_token = 1;
}

message ValidateRequest {
    string access_token = 1;
}

message ValidateResponse {
    uint64 user_id = 1;
}

message TokenPair {
    string access_token = 1;
    string refresh_token = 2;
}
```

Регистрация новых пользователей выполняется здесь методом `Register`, который принимает набор данных из запроса `RegisterRequest` и возвращает пустой ответ. Авторизация осуществляется через метод `Login`, принимающий запрос `LoginRequest`, в котором пользователь может указать либо логин, либо email вместе с паролем. При успешном входе сервис возвращает структуру `TokenPair`, содержащую пару токенов: короткоживущий `access_token` и более продолжительный `refresh_token`.

Для продления сессии используется метод `Refresh`. Он принимает `RefreshRequest`, где указывается уже выданный обновляющий токен, и снова возвращает пару новых токенов. Проверить корректность и актуальность токена можно методом `Validate`, принимающим структуру `ValidateRequest` с полем `access_token` и возвращающим ответ `ValidateResponse`, включающий идентификатор пользователя при успешной проверке. Завершает картину метод `Logout`, который получает в запросе `refresh_token` и при успешном выполнении завершает связанную с ним сессию, возвращая пустой результат.

Сгенерируем теперь контракты и выпустим новую версию:

```
make gen
```

После этого нужно обязательно отправить изменения в удаленный репозиторий, в нашем случае это `github`, и выпустить затем новую версию для пакета:

```
git tag v1.1.0
git push origin v1.1.0
```

Теперь можно приступать к самому сервису `Auth`. Повторим структуру проекта `Account` и установим зависимости, указав ссылку на свой репозиторий (рис. 2.1):

```
go mod init github.com/yuliaapopova/auth
go mod tidy
```

Все изменения, которые необходимо внести в сервис, будем рассматривать относительно сервиса `Account`, чтобы не повторяться в инфраструктурных моментах: логгеры, настройки подключения к базе данных и разбиение структуры на слои.

В конфигурацию сервиса (листинг 2.3) должны добавиться несколько полей, важных для создания и проверки токенов: `JwtSecret`, `AccessTokenTTLMinutes` и `RefreshTokenTTLDays`.

Листинг 2.3. Файл `internal/config`

```
package config

import (
    "log"

    "github.com/caarlos0/env/v10"
    "github.com/joho/godotenv"
)

// Config содержит конфигурацию приложения
type Config struct {
    ServiceName string `env:"SERVICE_NAME" json:"service_name" required:"true"
                                                default:"auth-service"`
    AppEnv      string `env:"APP_ENV" json:"app_environment" required:"true"
                                                default:"development"`
    Host        string `env:"GRPC_HOST" json:"host" required:"true" default:"localhost"`
    Port        int    `env:"GRPC_PORT" json:"port" required:"true" default:"50052"`
    LogLevel    string `env:"LOG_LEVEL" json:"log_level" required:"true" default:"info"`
    DbDsn       string `env:"DB_DSN" json:"db_dsn" required:"true"`

    JwtSecret      string `env:"JWT_SECRET" json:"jwt_secret" required:"true"`
    AccessTokenTTLMinutes int    `env:"ACCESS_TOKEN_TTL_MINUTES" json:"access_ttl_min"
                                                required:"true" default:"60"`
    RefreshTokenTTLDays int    `env:"REFRESH_TOKEN_TTL_DAYS" json:"refresh_ttl_days"
                                                required:"true" default:"30"`
}

// Load загружает конфигурацию из переменных окружения
func Load() (*Config, error) {
    // Загружаем .env файл
    if err := godotenv.Load(); err != nil {
        log.Println("Warning: .env file not found, using system environment variables")
    }

    cfg := &Config{}
    err := env.Parse(cfg)
    if err != nil {
        return nil, err
    }

    return cfg, nil
}
```

Так как мы будем хранить и некоторую информацию о пользователях (логин, почта, пароль) в этом сервисе, и ещё токены, создадим для этого две таблицы и две модели (листинг 2.4).

Листинг 2.4. Файл `internal/repository/model/model.go`

```
package model

import "time"

type User struct {
    ID            uint64    `gorm:"column:id"`
    Login         string    `gorm:"column:login"`
    Email         string    `gorm:"column:email"`
    PasswordHash string    `gorm:"column:password_hash"`
    CreatedAt     time.Time `gorm:"column:created_at"`
    UpdatedAt     time.Time `gorm:"column:updated_at"`
}

func (User) TableName() string {
    return "users"
}

type RefreshToken struct {
    ID            uint64    `gorm:"column:id"`
    UserID        uint64    `gorm:"column:user_id"`
    Token         string    `gorm:"column:token"`
    ExpiresAt     time.Time `gorm:"column:expires_at"`
    RevokedAt     *time.Time `gorm:"column:revoked_at"`
    CreatedAt     time.Time `gorm:"column:created_at"`
}

func (RefreshToken) TableName() string {
    return "refresh_tokens"
}
```

А миграция и возможность ее «откатить» будут выглядеть так (листинг 2.5).

Листинг 2.5. Файл `internal/migrations/20250825102441_initdb.go`

```
package migrations

import (
    "context"
    "database/sql"

    "github.com/pressly/goose/v3"
)
```

```
func init() {
    goose.AddNamedMigrationContext("20250825102441_initdb.go", upInitdb, downInitdb)
}

func upInitdb(ctx context.Context, tx *sql.Tx) error {
    query := `CREATE TABLE IF NOT EXISTS users (
        id BIGSERIAL PRIMARY KEY,
        login TEXT NOT NULL UNIQUE,
        email TEXT NOT NULL UNIQUE,
        password_hash TEXT NOT NULL,
        created_at TIMESTAMP NOT NULL DEFAULT NOW(),
        updated_at TIMESTAMP NOT NULL DEFAULT NOW()
    );`

    _, err := tx.Exec(query)
    if err != nil {
        return err
    }

    query = `CREATE TABLE IF NOT EXISTS refresh_tokens (
        id BIGSERIAL PRIMARY KEY,
        user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
        token TEXT NOT NULL UNIQUE,
        expires_at TIMESTAMP NOT NULL,
        revoked_at TIMESTAMP NULL,
        created_at TIMESTAMP NOT NULL DEFAULT NOW()
    );`

    _, err = tx.Exec(query)
    if err != nil {
        return err
    }

    return nil
}

func downInitdb(ctx context.Context, tx *sql.Tx) error {
    query := `DROP TABLE IF EXISTS refresh_tokens;`
    _, err := tx.Exec(query)
    if err != nil {
        return err
    }

    query = `DROP TABLE IF EXISTS refresh_tokens; DROP TABLE IF EXISTS users;`
    _, err = tx.Exec(query)
    if err != nil {
        return err
    }
}
```



```
    return nil
}
```

Эта миграция, по сути, такая же, как и в Account, но с небольшим дополнением. Тут в рамках одной транзакции мы создадим сразу две таблицы и также две удалим, если что-то пойдет не так. Созданные таблицы и будут обеспечивать хранение минимальной информации о пользователях и их токенах обновления.

Работа с данными также будет вестись через слой repository (листинг 2.6).

Листинг 2.6. Файл internal/repository/repository.go

```
package repository

import (
    "context"
    "errors"
    "fmt"

    "github.com/rs/zerolog"
    "github.com/yuliaapopova/auth/internal/model"
    "github.com/yuliaapopova/auth/internal/repository/mapper"
    repomodel "github.com/yuliaapopova/auth/internal/repository/model"
    "gorm.io/gorm"
    "gorm.io/gorm/clause"
)

type Repository struct {
    db      *gorm.DB
    logger  *zerolog.Logger
}

func NewRepository(db *gorm.DB, logger *zerolog.Logger) *Repository {
    return &Repository{
        db:      db,
        logger:  logger,
    }
}

func (r *Repository) CreateUser(ctx context.Context, user model.User) error {
    userRepo := mapper.UserToRepoUser(user)
    res := r.db.WithContext(ctx).
        Clauses(clause.OnConflict{UpdateAll: true}).
        Create(&userRepo)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to save user")
        return fmt.Errorf("failed to save user: %w", res.Error)
    }
}
```

```
    return nil
}

func (r *Repository) GetUserByID(ctx context.Context, userID uint64) (model.User, error) {
    var user repomodel.User
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("id = ?", userID).
        First(&user)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return model.User{}, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user id")
        return model.User{}, res.Error
    }
    return mapper.RepoUserToUser(user), nil
}

func (r *Repository) GetUserByLoginOrEmail(ctx context.Context, loginOrEmail string)
(model.User, error) {
    var user repomodel.User
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("login = ? OR email = ?", loginOrEmail, loginOrEmail).
        First(&user)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return model.User{}, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user by login/email")
        return model.User{}, res.Error
    }
    return mapper.RepoUserToUser(user), nil
}

func (r *Repository) SaveRefreshToken(ctx context.Context, token model.RefreshToken) error {
    m := mapper.RefreshTokenToRepoRefresh(token)
    res := r.db.WithContext(ctx).Create(&m)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to save refresh token")
        return fmt.Errorf("failed to save refresh token: %w", res.Error)
    }
    return nil
}

func (r *Repository) GetRefreshToken(ctx context.Context, token string) (model.RefreshToken,
error) {
    var refreshToken repomodel.RefreshToken
```

```

res := r.db.WithContext(ctx).
    Model(&repomodel.RefreshToken{}).
    Where("token = ?", token).
    First(&refreshToken)
if errors.Is(res.Error, gorm.ErrRecordNotFound) {
    return model.RefreshToken{}, fmt.Errorf("refresh token not found")
} else if res.Error != nil {
    r.logger.Err(res.Error).Msg("failed to get refresh token")
    return model.RefreshToken{}, res.Error
}
return mapper.RepoRefreshTokenToRefresh(refreshToken), nil
}

func (r *Repository) RevokeRefreshToken(ctx context.Context, token string) error {
    res := r.db.WithContext(ctx).
        Model(&repomodel.RefreshToken{}).
        Where("token = ?", token).
        Update("revoked_at", gorm.Expr("NOW()"))
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to revoke refresh token")
        return fmt.Errorf("failed to revoke refresh token")
    }
    return nil
}

```

Тут можно заметить, что где-то указывается `Model` как схема базы данных, а в каких-то запросах нет. Это связано с тем, что в запросах на создание записей мы схему неявно передаем вместе с данными. Тогда `gorm` считает, что именно этот тип и есть модель, и автоматически строит SQL-операцию вида `INSERT INTO users (...) VALUES (...)`. То есть объект самой модели уже явно передан в метод, и поэтому `gorm` понимает, какую таблицу и поля использовать. Когда же мы выполняем поиск, `gorm` должен знать, к какой таблице привязать условия. Только если модель определяется по приемнику (`First(&user)`), модель указывать не обязательно, но желательно — это придает ясности коду.

Мапперы рассматривать отдельно не станем — там всё предельно банально, а вот логика сервиса интересна (листинг 2.7).

Листинг 2.7. Файл `internal/service/service.go`

```

package service

import {
    "context"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "time"

```

```
"github.com/golang-jwt/jwt/v5"
"github.com/rs/zerolog"
"github.com/yuliaapopova/auth/internal/config"
"github.com/yuliaapopova/auth/internal/model"
"golang.org/x/crypto/bcrypt"
)

type AuthService struct {
    repo    Repository
    cfg     *config.Config
    logger  *zerolog.Logger
}

type Repository interface {
    CreateUser(context.Context, model.User) error
    GetUserByID(context.Context, uint64) (model.User, error)
    GetUserByLoginOrEmail(context.Context, string) (model.User, error)
    SaveRefreshToken(context.Context, model.RefreshToken) error
    GetRefreshToken(context.Context, string) (model.RefreshToken, error)
    RevokeRefreshToken(context.Context, string) error
}

func New(repo Repository, cfg *config.Config, logger *zerolog.Logger) *AuthService {
    return &AuthService{repo: repo, cfg: cfg, logger: logger}
}

func (s *AuthService) Register(ctx context.Context, req model.Register) error {
    hash, err := bcrypt.GenerateFromPassword([]byte(req.Password), bcrypt.DefaultCost)
    if err != nil {
        return fmt.Errorf("failed to hash password: %w", err)
    }
    user := model.User{
        Login:      req.Login,
        Email:      req.Email,
        PasswordHash: string(hash),
        CreatedAt:  time.Now(),
        UpdatedAt:  time.Now(),
    }
    return s.repo.CreateUser(ctx, user)
}

func (s *AuthService) Login(ctx context.Context, req model.Login) (model.TokenPair, error) {
    user, err := s.repo.GetUserByLoginOrEmail(ctx, req.LoginOrEmail)
    if err != nil {
        return model.TokenPair{}, fmt.Errorf("invalid credentials")
    }
}
```

```

    if err := bcrypt.CompareHashAndPassword([]byte(user.PasswordHash), []byte(req.Password)); err != nil {
        return model.TokenPair(), fmt.Errorf("invalid credentials")
    }
    return s.issueTokens(ctx, user.ID)
}

func (s *AuthService) Refresh(ctx context.Context, refreshToken string) (model.TokenPair, error) {
    rt, err := s.repo.GetRefreshToken(ctx, refreshToken)
    if err != nil {
        return model.TokenPair(), fmt.Errorf("invalid refresh token")
    }
    if rt.RevokedAt != nil || time.Now().After(rt.ExpiresAt) {
        return model.TokenPair(), fmt.Errorf("refresh token expired or revoked")
    }
    return s.issueTokens(ctx, rt.UserID)
}

func (s *AuthService) Validate(ctx context.Context, accessToken string) (uint64, error) {
    claims := jwt.MapClaims{}
    _, err := jwt.ParseWithClaims(accessToken, claims, func(token *jwt.Token) (interface{}, error) {
        return []byte(s.cfg.JwtSecret), nil
    })
    if err != nil {
        return 0, fmt.Errorf("invalid token: %w", err)
    }
    uidFloat, ok := claims["sub"].(float64)
    if !ok {
        return 0, fmt.Errorf("invalid subject")
    }
    return uint64(uidFloat), nil
}

func (s *AuthService) Logout(ctx context.Context, refreshToken string) error {
    return s.repo.RevokeRefreshToken(ctx, refreshToken)
}

func (s *AuthService) issueTokens(ctx context.Context, userID uint64) (model.TokenPair, error) {
    now := time.Now()
    accessExp := now.Add(time.Duration(s.cfg.AccessTokenTTLMinutes) * time.Minute)
    claims := jwt.MapClaims{
        "sub": userID,
        "exp": accessExp.Unix(),
    }
}

```

```
access := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
accessStr, err := access.SignedString([]byte(s.cfg.JwtSecret))
if err != nil {
    return model.TokenPair{}, fmt.Errorf("failed to sign access token: %w", err)
}

refreshRaw := fmt.Sprintf("%d:%d:%s", userID, now.UnixNano(), s.cfg.JwtSecret)
h := sha256.Sum256([]byte(refreshRaw))
refreshStr := hex.EncodeToString(h[:])
refresh := model.RefreshToken{
    UserID:    userID,
    Token:     refreshStr,
    ExpiresAt: now.Add(time.Duration(s.cfg.RefreshTokenTTLDays) * 24 * time.Hour),
    CreatedAt: now,
}
if err := s.repo.SaveRefreshToken(ctx, refresh); err != nil {
    return model.TokenPair{}, err
}
return model.TokenPair{AccessToken: accessStr, RefreshToken: refreshStr}, nil
}
```

Этот код реализует сервис аутентификации `AuthService`, который отвечает за регистрацию пользователей, вход в систему, выпуск и обновление токенов доступа, а также их валидацию и отзыв. Сервис опирается на абстракцию `Repository` — интерфейс к базе данных, через который выполняются основные операции (создание пользователя, поиск по логину или email, сохранение и отзыв refresh-токена). Это делает бизнес-логику независимой от конкретной реализации хранилища.

При регистрации пользователя пароль предварительно хешируется алгоритмом `bcrypt`, что обеспечивает его безопасное хранение. В случае входа проверяется совпадение хеша, после чего вызывается метод `issueTokens`, генерирующий пару токенов:

- `access-token` — короткоживущий JWT с идентификатором пользователя и временем истечения;
- `refresh-token` — уникальная строка, полученная на основе хеширования случайных данных через SHA-256.

Refresh-токен сохраняется в базе и может быть отозван, что позволяет управлять сессиями и завершать их досрочно.

Метод `Refresh` используется для получения новой пары токенов по действующему refresh-токену, метод `Validate` проверяет подпись и срок годности JWT и возвращает идентификатор пользователя, а метод `Logout` отзывает refresh-токен, прерывая связанную сессию.

Таким образом, сервис инкапсулирует всю логику, связанную с безопасной аутентификацией: от хранения пользователей и проверки паролей до управления токенами и их «жизненным циклом». Это позволяет использовать его как центральный

компонент системы авторизации, изолированный от деталей базы данных и инфраструктуры.

Тут есть нюанс, на который хотелось бы обратить внимание, — это работа с паролем. Учитывая требования безопасности, недопустимо хранить его в открытом виде, — любая утечка базы данных, и вся конфиденциальная информация пользователей под угрозой! Поэтому здесь используется пакет `bcrypt`, предоставляющий криптографическую функцию для хеширования паролей. Хеши `bcrypt` подмешивает в пароль «соль» — произвольную последовательность байтов, а это гарантирует, что одинаковые пароли будут иметь разные хеши. Передается также `bcrypt.DefaultCost` — стандартное значение «стоимости», равное 10. Оно задает достаточно медленное вычисление хеша, значительно усложняющее атакующей стороне перебор паролей, но при этом не тормозящее работу приложения. Такое замедление делает брутфорс практически невозможным.

Список литературы и источников

1. Джейсон Андресс. Защита данных. От авторизации до аудита. 2021 (<https://goo.su/7cBj7>).
2. <https://golang-jwt.github.io/jwt/>.

ГЛАВА 3



Способы взаимодействия между микросервисами

Взаимодействие между микросервисами может быть организовано двумя различными методами: синхронными и асинхронными. *Синхронные* методы предполагают, что пользователь должен получить ответ сразу же, а асинхронные — что, возможно, когда-нибудь потом. Примером асинхронного взаимодействия являются уведомления: вы делаете запрос на какую-либо операцию — допустим, вывод средств, а уведомление о том, что вывод средств прошел успешно или неуспешно, вы можете получить спустя некоторое время. К синхронным методам взаимодействия (из рассматриваемых нами) относятся протоколы HTTP и gRPC, а к асинхронным — Kafka и RabbitMQ. Существуют и другие инструменты взаимодействия, но, как правило, они менее популярны.

HTTP-протокол

HTTP — самый распространенный протокол передачи данных. Изначально он разрабатывался для передачи гипертекстовых документов, о чем и говорит аббревиатура HTTP — HyperText Transfer Protocol. Чтобы лучше понять, как устроен этот протокол, нужно сначала разобраться со спецификацией OSI.

Модель OSI

OSI (Open System Interconnection) — это набор стандартов, описывающих процесс взаимодействия устройств по сети, первая стандартная модель в области сетевых коммуникаций. В модели OSI процесс передачи данных осуществляется от одного уровня к другому. Все взаимодействия происходят между двумя устройствами, которые выступают в роли отправителя и получателя, а главная задача — доставить данные. Такой обмен информацией и описывается *моделью OSI* [1].

Модель OSI состоит из семи уровней:

1. Физический уровень.
2. Канальный уровень.
3. Сетевой уровень.

4. Транспортный уровень.
5. Сеансовый уровень.
6. Уровень представления.
7. Прикладной уровень.

Физический уровень

На физическом уровне производится передача данных в одной физической среде с помощью аппаратуры передачи данных. Каналы передачи данных квалифицируются в зависимости от характера носителя сигналов на:

- ☐ оптические;
- ☐ проводные;
- ☐ радиоканалы.

Как можно понять из названия уровня, он имеет дело с передачей битов по физическим каналам связи. На этом уровне описываются характеристики физических средств передачи данных — такие как полоса пропускания и волновое сопротивление, а также определяются характеристики электрических сигналов, уровень напряжения сигнала и тип кодирования. Кроме того, здесь стандартизируются типы разъемов и назначение каждого их контакта. Функции физического уровня реализуются во всех устройствах, подключенных к сети. Со стороны компьютера выполнение функций физического уровня обеспечивается сетевым адаптером или последовательным портом.

Канальный уровень

На физическом уровне просто пересылаются биты. При этом не учитывается, что в некоторых сетях, где линии связи используются (разделяются) попеременно несколькими парами взаимодействующих компьютеров, физическая среда передачи может быть занята. Поэтому одной из задач канального уровня становится проверка доступности среды передачи. Другой задачей канального уровня является реализация механизмов обнаружения и коррекции ошибок. Для этого на канальном уровне биты группируются в наборы, называемые *кадрами*. Канальный уровень обеспечивает корректность передачи кадров, помещая специальную последовательность битов в начало и конец каждого кадра для его выделения, а также вычисляет контрольную сумму, обрабатывая все байты кадра определенным способом и добавляя к кадру их контрольную сумму. Когда кадр приходит по сети, получатель снова вычисляет контрольную сумму полученных данных и сравнивает результат с контрольной суммой из кадра. Если они совпадают, кадр считается правильным и принимается. Если контрольные суммы не совпадают, то фиксируется ошибка. Канальный уровень может не только обнаруживать ошибки, но и исправлять их за счет повторной передачи поврежденных кадров, хотя функция исправления ошибок и не является обязательной для канального уровня.

Сетевой уровень

Протоколом сетевого уровня является IP (*англ.* Internet Protocol, межсетевой протокол) — маршрутизируемый сетевой протокол.

При передаче данных по протоколу IP доставка данных не является гарантированной. Данные разделяются на так называемые *пакеты*, передаваемые от одного узла сети к другому. При этом на уровне протокола IP не дается гарантий надежной доставки пакета до адресата. Пакеты могут поступать не в том порядке, в котором были отправлены, а также могут дублироваться (когда приходят две копии одного пакета, что на самом деле бывает крайне редко) и быть поврежденными (чаще всего поврежденные пакеты удаляются) или не прийти вовсе.

Транспортный уровень

За транспортный уровень в сетевой модели OSI отвечают несколько протоколов — чаще всего упоминаются TCP и UDP, хотя есть и другие, менее распространенные. Они предназначены в первую очередь для управления передачей данных в сетях и подсетях.

- ❑ TCP (*англ.* Transmission Control Protocol, протокол управления передачей) — это транспортный механизм, предоставляющий поток данных, где соединение устанавливается заранее, благодаря чему гарантируется достоверность полученных данных. Если данные в процессе передачи были потеряны, осуществляется повторный запрос данных и устраняется дублирование при получении двух копий одного пакета.
- ❑ UDP (*англ.* User Datagram Protocol, протокол пользовательских датаграмм) — транспортный механизм, в котором, в отличие от TCP, *датаграммы* (а не пакеты) могут прийти не по порядку, дублироваться или вовсе исчезнуть без следа, но если они придут, то гарантируется, что в целостном состоянии. Это обеспечивается за счет использования простой модели передачи — без предварительной установки соединения. Протокол UDP задействуется, когда критично время доставки датаграмм. Это может быть актуально для загрузки трафика мультимедиа, где потери не так критичны, как возможные задержки.

Сеансовый уровень

Далее уже идут уровни, которые работают непосредственно с данными, а не с пакетами.

Как можно понять из названия, сеансовый уровень обеспечивает поддержание сеанса связи или сессии, благодаря чему приложения могут общаться между собой длительное время. На этом уровне происходит координация коммуникации между приложениями, и здесь можно управлять установлением, завершением и поддержанием связи в моменты неактивности приложения, синхронизацией задач и обменом информацией. Примером использования сеансового уровня являются групповые звонки или прямые трансляции. Во время сессии поддерживается синхронизированная передача аудио- и видеоданных для всех участников, а также сама связь.

Есть множество протоколов, обеспечивающих соединение на этом уровне, — например: RPC, RTCP, SMPP и H.245.

Уровень представления

На уровне представления обеспечивается преобразование протоколов и преобразование данных. При отправке данные преобразуются сначала в формат для передачи по сети, а при получении преобразуются в формат приложений. На этом уровне с данными могут происходить разные манипуляции: сжатие и распаковка, кодирование и декодирование, шифрование и дешифрование.

Прикладной уровень

Это верхний уровень модели OSI, который вы ежедневно используете. Он обеспечивает взаимодействие приложений с сетью, а с помощью его протоколов данные отображаются конечному пользователю в понятном ему формате. Сюда как раз и относятся протоколы HTTP, SMTP, POP3 и другие.

При этом понятия определения протокола прикладного уровня и уровня представления очень размыты, и принадлежность протокола может зависеть от сервиса. Например, если протокол HTTPS используется для просмотра сайтов, он расценивается как протокол прикладного уровня. А если HTTPS применяется для передачи важной зашифрованной информации — например, по протоколу ISO 8583, то тогда его можно расценивать как протокол уровня представления, а уже ISO 8583 будет протоколом приложения.

* * *

Функции всех уровней модели OSI могут быть отнесены к одной из двух групп: к функциям, зависящим от конкретной технической реализации сети, или к функциям, нацеленным на работу с приложениями.

Физический, канальный и сетевой уровни являются сетезависимыми, т. е. они часто тесно связаны с технической реализацией сети и используемым коммуникационным оборудованием. В то же время прикладной, представительный и сеансовый уровни ориентированы на приложения и слабо зависят от технических особенностей построения сети. Транспортный же уровень является промежуточным — он скрывает детали функционирования нижних уровней от верхних.

Устройство с сетевой операционной системой взаимодействует с другими устройствами по протоколам всех семи уровней. Это взаимодействие происходит через коммутаторы, маршрутизаторы, модемы и подобные им средства. В зависимости от типа коммуникационное устройство может работать либо на физическом уровне, либо на физическом и канальном, либо на физическом, канальном и сетевом, иногда захватывая транспортный уровень.

Устройство HTTP-протокола

Как мы уже выяснили, HTTP является протоколом прикладного уровня, предназначенным для передачи данных по сети Интернет с использованием стека протоколов

ТСР/IP. Это основной протокол обмена данными в сети. HTTP опирается на клиент-серверную архитектуру — т. е. инициатором взаимодействия обычно выступает клиент, который отправляет запрос к серверу, например, с помощью веб-браузера. Хотя иногда инициатором обмена может быть и сервер, классический пример этого — push-уведомления. Ответ от сервера может состоять из различных поддокументов, которые являются частью итогового документа, — например, отдельно может приходить структура документа, а отдельно текст, изображения, скрипты и многое другое.

Клиенты обычно — это устройства пользователей, т. е., когда вы открываете приложение в своем смартфоне или браузере ПК, ваше устройство выступает в роли клиента, и чтобы загрузить информацию на ваше устройство, сначала с него отправляется соответствующий HTTP-запрос на сервер.

Сервер — это некоторая виртуальная машина, на которой выполняется код и которая выполняет запросы пользователя. В роли сервера также может выступать группа виртуальных машин, объединенных балансировщиком, распределяющим нагрузку между ними. При этом сервер не обязательно расположен на одной машине, точно так же как и на одной виртуальной машине может быть организовано несколько серверов, и у них даже может числиться один хост на всех.

Участниками обмена, кроме клиента и сервера, также могут являться посредники — так называемые *прокси-серверы*. Они могут использоваться в качестве шлюза или хранителей кеша. Кеш — это некоторые ответы сервера, которые хранятся на нем какое-то время, чтобы переиспользовать эти ответы, а не нагружать систему одними и теми же запросами.

Особенно прокси-серверы актуальны в микросервисной архитектуре. Самым распространенным прокси-сервером сейчас считается *nginx*. Обычно на каждый запрос создается один поток выполнения, однако *nginx* разделяет этот поток на меньшие однотипные структуры, которые называются *рабочими соединениями*. Каждое такое соединение обрабатывается отдельным рабочим процессом, а после выполнения они формируются в единый блок и возвращают результат в основной процесс обработки данных. Одно рабочее соединение может обрабатывать до 1024 запросов одного вида одновременно.

Структура HTTP-запроса

Каждый HTTP-запрос состоит из трех частей, которые передаются в строгом порядке:

□ HTTP-метод:

- GET — запрашивает данные, никак не изменяя их при этом;
- HEAD — запрашивает ресурс как и GET, но без тела ответа;
- POST — служит для отправки сущностей к ресурсу, чаще всего используется для создания записей;
- PUT — обновление записи. Метод заменяет текущие данные конкретной записи на переданные, а если какие-то значения переданы не были, то затирает существующие;

- DELETE — удаление записи;
- PATCH — то же, что и PUT, но с той лишь разницей, что обновляет данные только частично. При этом поля, которые переданы не были, остаются с прежними значениями;
- CONNECT — устанавливает «туннель» к серверу, определенному по ресурсу;
- OPTIONS — используется для описания параметров соединения с ресурсом;
- TRACE — выполняет проверку связи по пути к ресурсу, чаще всего используется для отладки;

- путь к ресурсу (строка URL);
- версия HTTP-протокола;
- заголовки (не обязательно) — используются для передачи серверу дополнительной информации. Например, в заголовках часто передается токен авторизации;
- тело запроса — для таких методов, как POST и PUT.

Таким образом, пример запроса к нашему сервису User будет выглядеть следующим образом:

```
GET /api/v1/user
Host: localhost:9000
```

Структура HTTP-ответа

В HTTP-ответе содержатся следующие элементы:

- версия HTTP-протокола;
- код состояния HTTP, показывающий, успешно ли был выполнен запрос. Коды разделяют на пять групп:
 - 1xx — информационные;
 - 2xx — успешные;
 - 3xx — перенаправленные;
 - 4xx — клиентские ошибки (например, ресурс не найден);
 - 5xx — серверные ошибки;
- сообщение состояния — краткое описание, например: «Record not found» или «ОК»;
- HTTP-заголовки, подобные заголовкам в запросах;
- тело ответа, но оно может и отсутствовать.

gRPC

gRPC — это протокол, который использует HTTP/2 и Protobuf (Protocol Buffers, буферы протоколов) для обеспечения коммуникации между сервисами в распределенной системе. Он поддерживается большинством языков программирования —

такими как Node.js, Java, Go, Kotlin, C++, PHP, Python, Ruby, C# и т. п., благодаря чему можно легко интегрировать между собой микросервисы, написанные на разных языках [2].

Буферы протоколов были разработаны компанией Google в качестве эффективной бинарной альтернативы текстовому формату XML как нечто более компактное, простое и быстрое за счет передачи бинарных данных, оптимизированных под минимальный размер сообщения. Для сериализации данных необходимы отдельные файлы в формате .proto, в которых описывается формат сообщения. Если бы мы использовали такой формат сообщений, наш класс `User` в Protobuf выглядел бы следующим образом:

```
message User {  
    string userId = 1;  
    string login = 2;  
    string password = 3;  
    string phone = 4;  
    string firstName = 5;  
    string lastName = 6;  
}
```

То есть в Protobuf описывается `message` для структурирования данных — в других языках мы бы описывали это как класс или структуру, в которой содержатся наборы пар вида «ключ-значение» (такая пара называется *полем*). Каждое поле индексируется, поэтому если названия полей в разных сервисах будут различаться, а индекс и тип совпадать, то это может запутать разработчиков. Важно соблюдать структуру полей одинаковыми для всех проектов, поэтому часто создают отдельный репозиторий с общими для всех контрактами. Есть у Protobuf и собственные генераторы для каждого языка, в результате чего создается исходный файл с расширением, свойственным конкретному языку.

Protobuf — по спецификации gRPC — также поддерживает определение интерфейсов и поставляется с дополнительным плагином для генерации клиентского и серверного кода.

Обратимся сейчас к нашему сервису `Account` и представим, как бы выглядел запрос списка пользователей:

```
syntax = "proto3"  
option user = "user"  
  
service UserService {  
    rpc GetUsers(UsersRequest) returns (UsersResponse);  
}  
  
message UsersRequest {  
    repeated string client_ids = 1;  
    repeated string phones = 2;  
    uint32 take = 3;  
    uint32 skip = 4;  
}
```

```

message UsersResponse {
    string user_id = 1;
    string login = 2;
    string phone = 3;
    string first_name = 4;
    string last_name = 5;
    string middle_name = 6;
    string email = 7;
}

```

В этом коде мы определили gRPC-сервис под названием `UserService`, который предоставляет метод `GetUsers` и описывает входные и выходные параметры: `UsersRequest` и `UsersResponse` соответственно. Заметьте, что массив описывается через ключевое слово `repeated`, а целочисленные типы данных — как `uint32` или `uint64`. Присутствуют здесь и другие типы числовых переменных — например, для значений с плавающей точкой используются `float32` или `float64`.

Обратите внимание, что здесь нет никаких HTTP-методов, и вообще — запросы друг от друга ничем не отличаются независимо от их назначения: на создание они, на удаление или только на получение. Это разделение очень условное, и за него больше отвечают сами разработчики через именование методов. Поэтому тут можно встретить такие методы, как `CreateUser`, `GetUserById` — т. е. именно так вы сами могли бы назвать свои методы бизнес-логики.

При этом здесь можно описать еще и HTTP-взаимодействие, с учетом чего сервис мог бы работать сразу по двум протоколам. Тогда бы наш код выглядел следующим образом:

```

syntax = "proto3"
option user = "user"

import "google/api/annotations.proto";

service UserService {
    rpc GetUsers(UsersRequest) returns (UsersResponse) {
        option (google.api.http) = {
            get: "/v1/users"
        };
    };
}

message UsersRequest {
    repeated string client_ids = 1;
    repeated string phones = 2;
    uint32 take = 3;
    uint32 skip = 4;
}

message UsersResponse {
    string user_id = 1;

```

```
string login = 2;  
string phone = 3;  
string first_name = 4;  
string last_name = 5;  
string middle_name = 6;  
string email = 7;  
}
```

Это становится возможным, благодаря импорту библиотеки `google/api/annotations.proto`, в которой содержатся необходимые методы. Также прямо на уровне `proto`-файла можно сделать валидацию запросов, и тогда будет генерироваться еще и файл для валидации на используемом языке программирования.

Однако по части валидации тут есть некоторые ограничения, связанные с устройством Protobuf. Поскольку общение между сервисами происходит бинарными данными, а Protobuf занимается сериализацией данных, получается, что на входе мы не можем понять, передан ли нам массив значений или только одно значение, и не в массиве. Protobuf это всё подстроит под ожидаемый формат, и только с такими данными мы дальше уже и сможем работать. Интересная история происходит также и с перечисляемыми типами данных: `enum` обычно в Protobuf описывает возможные значения, начиная с нуля, и когда приходят данные, где ожидается `enum`, мы не можем проверить, передано ли нам нулевое значение или Protobuf просто так подставил. В результате с этой проблемой борются обходными путями — например, делают первым значением `UNDEFINED` и уже отдельно потом проверяют, пришло нам это значение или введенное пользователем действительно ожидаемое.

У gRPC есть несколько паттернов коммуникации между клиентом и сервером (в ранее рассмотренном примере использовался унарный метод, но есть и другие способы взаимодействия [3]):

- ❑ унарные RPC — клиент отправляет один запрос и получает один ответ, как при стандартном вызове функции;
- ❑ серверные потоковые RPC — клиент отправляет один запрос, а в ответ получает поток последовательности сообщений;
- ❑ клиентские потоковые RPC — клиент записывает последовательность сообщений и отправляет их на сервер. После завершения записи клиент ожидает, пока сервер прочитает все эти сообщения и вернет ему ответ;
- ❑ RPC с двунаправленной потоковой передачей — обе стороны обмениваются последовательными сообщениями через потоки чтения и записи. Эти потоки независимы.

Отдельно стоит отметить, что процесс тестирования сервисов на gRPC несколько отличается от тестирования стандартного REST API. Чтобы «триггерить» методы gRPC, необходимо либо иметь доступ к `proto`-файлам, либо составлять их самостоятельно. Сейчас реализована поддержка gRPC и в Postman, но существуют и специальные инструменты — такие как BloomRPC или Enevs. Последний, однако, используется только через консоль. Из этого также вытекают и другие сложности:

с реализацией health-check (проверки доступности сервиса), усложненной обработкой ошибок и др. А в качестве положительных моментов стоит отметить скорость выполнения запросов — gRPC и правда очень быстрый, предоставляет контракты «из коробки» в виде proto-файлов, а также имеет хорошую поддержку в интегрированных средах разработки.

RabbitMQ

Перейдем теперь к рассмотрению асинхронного взаимодействия сервисов. Один из инструментов такого взаимодействия — RabbitMQ, *брокер сообщений*, основанный на стандарте AMQP.

AMQP (Advanced Message Queuing Protocol) — это открытый протокол обмена сообщениями между различными микросервисами приложения с низкой задержкой и на высокой скорости¹. До AMQP существовали уже брокеры и приложения передачи данных, однако у них был существенный недостаток — они не могли взаимодействовать между собой. Для организации взаимодействия между сервисами приходилось реализовывать мост обмена сообщениями, т. е. дополнительный уровень преобразования. AMQP же предложил стандартизированный подход для обработки всех очередей сообщений и взаимодействия брокерских приложений [4].

AMQP работает на основе клиент-серверной модели, где микросервисы-получатели являются серверами, а отправляющий микросервис — клиентом. Каждый микросервис, желающий получать сообщения, подписывается на определенную очередь, а когда сообщение поступает в очередь, AMQP отправляет его всем зарегистрированным подписчикам (микросервисам), которые могут обработать это сообщение.

Таким образом, AMQP обеспечивает асинхронную и распределенную коммуникацию между микросервисами. Это позволяет им обмениваться данными без необходимости прямого взаимодействия.

На самом нижнем уровне протокола AMQP определяется формат кодирования данных в бинарный вид для передачи по TCP-соединению. Можно выделить несколько ключевых понятий протокола:

- ❑ *сообщение (message)* — единица передаваемых данных. Основная его часть (содержание) никак не интерпретируется сервером, но к сообщению могут быть прицеплены структурированные заголовки;
- ❑ *точка обмена (exchange)* — в нее отправляются сообщения. Точка обмена распределяет сообщения в одну или несколько очередей. При этом в точке обмена сообщения не хранятся. Точки обмена бывают трех типов:
 - Fanout — сообщение передается во все прицепленные к ней очереди;
 - Direct — сообщение передается в очередь с именем, совпадающим с ключом маршрутизации (routing key), который указывается при отправке сообщения;

¹ См. <https://ru.wikipedia.org/wiki/AMQP>.

- Topic — нечто среднее между fanout и direct. Сообщение передается в очереди, для которых совпадает маска на ключ маршрутизации. Например, при маске `app.notification.sms.*` в очередь будут доставлены все сообщения, отправленные с ключами, начинающимися с `app.notification.sms`;
- очередь (queue) — здесь хранятся сообщения до тех пор, пока они не будут забраны клиентом. Клиент всегда забирает сообщения из одной или нескольких очередей.

Теперь определимся с участниками процесса:

- брокер сообщений (message broker) — это сервер, который принимает, хранит и маршрутизирует сообщения между отправителями и получателями. Он является посредником во всем процессе обмена сообщениями;
- подписчик (Consumer) — микросервис, получающий сообщения из очереди для обработки;
- публикатор (Publisher) — микросервис, отправляющий сообщения в очередь для передачи другим подписчикам.

Процесс передачи данных при этом выглядит следующим образом:

1. Подключение к брокеру.

Первоочередной задачей для микросервисов (публикаторов и подписчиков) является установление соединения с брокером сообщений.

2. Создание очереди.

Подписчик может создать свою собственную очередь на брокере или использовать существующую.

3. Публикация сообщения.

Публикатор отправляет сообщение в определенную очередь на брокере. Сообщение может содержать данные, указывающие на определенное действие, или запрос на выполнение операции.

4. Маршрутизация сообщений.

Брокер сообщений берет на себя задачу маршрутизации сообщений. В зависимости от правил, которые настроены в брокере (например, на основе ключей сообщений), он решает, в какую очередь отправить сообщение или каким подписчикам его отправить.

5. Хранение в очереди.

Сообщение сохраняется в соответствующей очереди, пока один из подписчиков не будет готов его обработать.

6. Получение сообщений.

Когда подписчик готов обработать сообщение, он запрашивает брокера на предмет наличия новых сообщений в очереди.

7. Доставка и обработка.

Брокер доставляет сообщение подписчику, который затем обрабатывает его по необходимости.

8. Подтверждение обработки.

После успешной обработки подписчик отправляет брокеру подтверждение (acknowledgment), что сообщение было успешно получено и обработано. Брокер удаляет сообщение из очереди.

Ко всему прочему протокол AMQP еще и обеспечивает различные уровни гарантий доставки сообщений, которые позволяют разработчикам выбирать наиболее подходящий уровень надежности в зависимости от требований системы:

- ❑ на первом уровне: *At Most Once* (максимум один раз) — AMQP не предпринимает дополнительных гарантий по доставке сообщений. Это означает, что сообщение может быть потеряно при передаче и получатель его не получит. При этом AMQP не предпримет дополнительных попыток доставки, и если сообщение не будет доставлено, значит, оно будет потеряно;
- ❑ второй уровень: *At Least Once* (по крайней мере один раз) — гарантирует, что сообщение будет доставлено получателю как минимум один раз. Протокол AMQP попытается гарантированно доставить сообщение, повторяя доставку, если получатель не отправил подтверждение о получении сообщения. Таким образом, сообщение — чтобы гарантировать его обработку — может быть доставлено получателю несколько раз;
- ❑ на уровне *Exactle Once* (ровно один раз) — гарантируется, что сообщение будет доставлено ровно один раз, без потерь и дублирования. Этот уровень является самым строгим и трудным для реализации, т. к. требуются особые механизмы для обеспечения уникальности и идемпотентности сообщений. В большинстве случаев точная доставка рассматривается на уровне приложения или с помощью дополнительных механизмов — таких как уникальные идентификаторы сообщений и повторное использование идемпотентных операций.

Теперь, разобравшись с протоколом, на котором основан RabbitMQ, мы можем вернуться к рассмотрению этого инструмента. RabbitMQ реализует и дополняет протокол AMQP, он был разработан для решения сложных проблем в области обработки и доставки сообщений в распределенных системах. RabbitMQ предоставляет механизмы для надежной и эффективной передачи сообщений между приложениями, что делает его важным инструментом для масштабирования и упрощения обмена данными в современных приложениях.

В основе практически всех взаимодействий с ядром RabbitMQ лежит процесс RPC (Remote Procedure Call), мы рассматривали его последователя — gRPC — в предыдущем разделе. В спецификации AMQP также предусмотрено, что и клиент, и сервер могут вызывать команды. Это означает, что клиент ожидает взаимодействия с сервером. Команды — это классы и методы. Например, `Connection.Start` — вызов метода `Start` класса `Connection`.

В рамках конкретного подключения для обмена информацией между клиентом и сервером используются каналы. Каждый такой канал изолирован от других каналов. Если есть необходимость отправлять команды параллельно, следует создать несколько каналов, при этом в каждом канале будет создан отдельный процесс.

Одно подключение может иметь множество каналов. Однако каждый канал занимает некоторые структуры и объекты в памяти, поэтому чем больше каналов будет создано в рамках одного соединения, тем больше памяти понадобится RabbitMQ для управления этим соединением.

С учетом сказанного открывать новое соединение для каждой операции всё же не рекомендуется, потому что это приводит к большим затратам ресурсов. Впрочем, некоторые ошибки протокола могут приводить к закрытию канала, поэтому срок работы канала может быть меньше, чем у соединения.

Помимо использования RabbitMQ в качестве брокера сообщений для взаимодействия между микросервисами, его также часто применяют для фоновой обработки данных. Как правило, к этому прибегают, когда необходимо сгенерировать какой-то большой документ, процесс генерации которого занимает продолжительное время. При этом очевидно, что HTTP-соединение точно не дожидается генерации и упадет по тайм-ауту, так что получившийся файл по готовности кладут в RabbitMQ и ставят в очередь на отправку получателю, либо сообщают ему ссылку на его скачивание по почте, либо множеством других способов оповещают получателя о его готовности. В таких ситуациях RabbitMQ служит шиной события и позволяет веб-серверам заниматься основными запросами, а не выполнять трудоемкие задачи в моменте.

Apache Kafka

Apache Kafka — это еще один брокер сообщений, такой же как RabbitMQ, и предназначение у обоих глобально одинаковое, однако есть между ними и существенные различия. Первое различие, с которого хотелось бы начать, — у Kafka и RabbitMQ разные модели запросов. Как известно, существуют два типа моделей запросов: pull и push [5]:

- согласно pull-модели подписчики (consumer) сами отправляют запрос один раз в несколько секунд на сервер для получения новых сообщений. Плюсом такого подхода является возможность группировать сообщения при получении в батчи и достигать таким образом большей пропускной способности. Однако при этом обработка данных будет происходить с некоторой задержкой, и потенциально может возникнуть разбалансировка между разными подписчиками;
- согласно push-модели сервер делает запрос к клиенту, отправляя ему новые сообщения. При этом снимается время обработки данных, что позволяет контролировать сбалансированное распределение сообщений между подписчиками. А для того чтобы у подписчиков не возникла перегрузка, приходится выставлять лимиты, используя функциональность QS¹.

В Kafka реализуется pull-модель, тогда как RabbitMQ использует push-модель.

Помимо моделей запросов, эти инструменты также различаются механизмами хранения и потребления сообщений на брокере. В отличие от некоторых других сис-

¹ См. <https://www.rabbitmq.com/maxlength.html>.

тем, в Kafka сообщения сохраняются даже после обработки подписчиками, и они могут там пребывать до неопределенного момента. Такой подход позволяет обрабатывать одно и то же сообщение множеством подписчиков в различных контекстах. Несмотря на это, разработчики Kafka предусмотрели механизмы, предотвращающие проблемы с повторным чтением одних и тех же сообщений одним и тем же подписчиком.

Для того чтобы разобраться в этом подробнее, рассмотрим, как Kafka устроен.

Сообщения, притекающие от «производителей» (producers) — т. е. от других процессов, — хранятся в Kafka в формате «ключ-значение». Кроме того, данные могут быть разделены на *разделы* (partitions) внутри различных *тем* (topics). Эти сообщения строго упорядочены внутри каждого раздела на основе их *смещения* (offset) — позиции внутри раздела, и также индексируются и сохраняются вместе со временем создания. Это и позволяет не читать одни и те же сообщения множество раз.

Клиентские приложения, известные как «потребители» (consumers), способны осуществлять чтение сообщений из этих разделов. Потокосовое Streams API в Kafka облегчает разработку приложений, которые могут не только получать данные из Kafka, но и записывать их обратно. Этот интегрированный инструмент также отлично взаимодействует с различными внешними системами обработки потоков — такими как Apache Apex, Apache Beam, Apache Flink, Apache Spark, Apache Storm и Apache NiFi.

Kafka функционирует в распределенном кластере, состоящем из одного или нескольких узлов-брокеров, на которых размещены разделы всех тем. Для обеспечения надежности и отказоустойчивости разделы реплицируются на нескольких брокерах. Начиная с версии 0.11.0.0, система предоставляет возможность задействовать транзакционную модель, подобную той, которая используется в базах данных. Это позволяет обрабатывать поток данных с помощью Streams API только один раз.

Концепция тем в Kafka включает два их типа: обычные и компактные:

- *обычные темы* можно настроить с определенным сроком хранения или ограничениями по занимаемому пространству. Если лимит пространства превышен, система автоматически удаляет самые старые записи. Записи, чей срок хранения истек, также будут удалены, независимо от лимитов памяти;
- в свою очередь, *компактные темы* не обладают сроком хранения или ограничениями по памяти. Вместо этого Kafka сохраняет и обрабатывает только последние значения для каждого ключа и гарантирует, что ни одно последнее сообщение для каждого ключа никогда не будет удалено. Пользователи могут также вручную удалять сообщения, отправляя их с пустым значением для удаления записи по ключу.

Kafka Connect, также известный как Connect API, представляет собой мощный фреймворк для безупречного импорта данных из внешних систем и безукоризненного экспортирования данных в другие системы. Этот ценный инструмент впервые был представлен в версии Kafka 0.9.0.0. В рамках фреймворка Connect создаются так называемые *коннекторы*, которые берут на себя всю логику чтения и записи данных во внешние системы [6].

Чтобы обеспечить гибкость и разнообразие, Connect API определяет программный интерфейс, который может быть реализован для различных языков программирования. Поэтому существуют уже готовые реализации API для большинства популярных языков. Это позволяет разработчикам работать на предпочитаемом языке без необходимости заботиться о реализации самого API.

Важно отметить, что компания Apache Kafka не участвует напрямую в разработке конкретных библиотек для различных языков программирования. Однако благодаря открытой структуре Connect API сообщество разработчиков активно создает и поддерживает реализации API для разнообразных языков, обогащая возможности Kafka Connect и обеспечивая широкий спектр интеграций с другими системами.

Redis

Redis представляет собой организованное в оперативной памяти устройства хранилище типа «ключ-значение». Весьма часто этот инструмент задействуется в дополнение к уже рассмотренным нами ранее. Дело в том, что в базах данных вроде PostgreSQL, MySQL и MongoDB, используемых как долговременные хранилища, данные изменяются достаточно редко. Redis же как раз закрывает потребность в хранении где-то данных, к которым необходимо постоянно обращаться и которые часто могут изменяться. Это помогает повысить скорость работы системы, поскольку обращение за данными в расположенный в оперативной памяти Redis происходит значительно быстрее, чем если пытаться получить эту информацию из PostgreSQL. Однако хранение данных в оперативной памяти обходится значительно дороже, чем использование в этих целях жесткого диска, поэтому Redis не может заменить классические базы данных, а является только дополнением к ним [7].

Сначала его использовали, как и Memcached, для кеширования данных в оперативной памяти, к которой может обращаться множество доступных серверов. Позже, по мере развития Redis, ему нашли применение и во многих других ситуациях — например, при работе с очередями или в потоковой обработке данных.

Сейчас Redis поддерживает множество структур данных:

- ☐ строки;
- ☐ хеш-таблицы;
- ☐ битовые массивы;
- ☐ списки;
- ☐ множества;
- ☐ битовые поля;
- ☐ упорядоченные множества;
- ☐ геопространственные данные;
- ☐ структуры HyperLogLog;
- ☐ потоки.

По большей части Redis применяют для хранения некоторых данных, которые можно разделить на следующие группы:

- ❑ редко меняющиеся данные, к которым часто обращаются (например, хедеры, требующие проверки при запросах, различные `api-key`);
- ❑ часто меняющиеся данные, не относящиеся к критически важным (постоянно запрашиваемые сгенерированные отчеты, которые каждый раз генерировать заново слишком накладно).

Для наглядности, чтобы разобраться в том, как устроен Redis, настроим его в сервисе Auth (этой настройке и посвящен следующий раздел).

Разработка сервиса Gateway

Когда дело доходит до сложной распределенной архитектуры, например набора микросервисов, сразу возникает вопрос: кто и как будет управлять внешними запросами. Если мы допустим, что клиент напрямую обращается к каждому внутреннему сервису, то получим хаос — пользователю придется знать о множестве адресов. Каждый вызов будет требовать отдельной авторизации, а изменения в топологии заставят менять код на стороне клиента. Кроме того, откроется целый ряд технических рисков: от необходимости дублировать повторяющуюся инфраструктурную логику до трудностей в обеспечении безопасности и мониторинга.

Здесь и становится необходим паттерн `API Gateway`¹ — самостоятельный сервис, который становится единственной точкой входа во внутреннюю систему. С точки зрения клиента, существует ровно один адрес, ровно одна точка коммуникации. Все запросы попадают сначала в Gateway, и уже он решает, куда их направить дальше. Таким образом, внутренние сервисы остаются недоступными для внешнего мира и могут жить своей независимой жизнью, не раскрывая детали своей реализации.

Gateway работает как фасад. Он принимает запрос, проверяет авторизацию и аутентификацию. Может добавить или изменить заголовки, преобразовать входной формат в нужный внутренним сервисам вид, а затем пересылает вызов дальше. Ответ, полученный от конкретного микросервиса, также приходит через Gateway и может быть дополнительно обработан: например, переведен в другой формат, объединен с данными из нескольких сервисов или дополнен заголовками.

Такой сервис — это удобное место для централизованного решения общих задач. В нем размещают проверку токенов доступа, ограничение частоты запросов, кеширование, маршрутизацию по версиям API, а также агрегирование данных из разных внутренних сервисов. Классический пример: мобильное приложение запрашивает «профиль пользователя», а Gateway «под капотом» собирает информацию сразу из трех мест: основного сервиса профилей, фотохранилища и сервиса статистики, после чего возвращает единый ответ, — вместо того чтобы усложнять клиент, мы переносим всю интеграционную логику внутрь центрального шлюза.

¹ См. <https://microservices.io/patterns/apigateway.html>.

API Gateway часто называют фасадом для внешнего мира. Это сервис, который в архитектуре играет роль таможни и дирижера: он защищает внутреннюю границу, проверяет права доступа у каждого запроса и направляет поток точно туда, куда нужно. Благодаря такому подходу удастся добиться более высокой безопасности, удобства эволюции системы и явного разграничения обязанностей. Для внешнего клиента всё выглядит просто: есть один сервис, одно место входа, единые правила авторизации и одинаковый стиль запросов.

Структура проекта будет идентична уже тем, что мы описывали ранее. Всё та же точка входа, разбиение по модулям и конфигурирование сервиса. Начнем, как и всегда, с контракта для сервиса (листинг 3.1).

Листинг 3.1. Файл `contracts/gateway/gateway_service.proto`

```
syntax = "proto3";

package gateway;

option go_package = "github.com/yuliaapopova/contracts/gateway/go:gateway";

import "google/api/annotations.proto";
import "google/protobuf/empty.proto";
import "pagination/pagination.proto";
import "account/account_model.proto";
import "auth/auth_model.proto";

// Основной сервис Gateway, объединяющий функциональность auth и account
service Gateway {
    // Аутентификация и авторизация
    rpc Register(RegisterRequest) returns (RegisterResponse) {
        option (google.api.http) = {
            post: "/api/v1/auth/register",
            body: "*"
        };
    }

    rpc Login(LoginRequest) returns (LoginResponse) {
        option (google.api.http) = {
            post: "/api/v1/auth/login",
            body: "*"
        };
    }

    rpc Refresh(RefreshRequest) returns (RefreshResponse) {
        option (google.api.http) = {
            post: "/api/v1/auth/refresh",
            body: "*"
        };
    }
}
```



```
rpc Logout(LogoutRequest) returns (google.protobuf.Empty) {
    option (google.api.http) = {
        post: "/api/v1/auth/logout",
        body: "*"
    };
}

rpc ValidateToken(ValidateTokenRequest) returns (ValidateTokenResponse) {
    option (google.api.http) = {
        post: "/api/v1/auth/validate",
        body: "*"
    };
}

// Управление пользователями (требует аутентификации)
rpc CreateUser(CreateUserRequest) returns (google.protobuf.Empty) {
    option (google.api.http) = {
        post: "/api/v1/users",
        body: "*"
    };
}

rpc GetUser(GetUserRequest) returns (GetUserResponse) {
    option (google.api.http) = {
        get: "/api/v1/users/{user_id}"
    };
}

rpc GetCurrentUser(google.protobuf.Empty) returns (GetCurrentUserResponse) {
    option (google.api.http) = {
        get: "/api/v1/users/me"
    };
}

rpc GetUsers(GetUsersRequest) returns (GetUsersResponse) {
    option (google.api.http) = {
        get: "/api/v1/users"
    };
}

rpc UpdateUser(UpdateUserRequest) returns (google.protobuf.Empty) {
    option (google.api.http) = {
        put: "/api/v1/users/{user_id}",
        body: "*"
    };
}
```

```
rpc UpdateCurrentUser(UpdateCurrentUserRequest) returns (google.protobuf.Empty) {
    option (google.api.http) = {
        put: "/api/v1/users/me",
        body: "*"
    };
}

rpc DeleteUser(DeleteUserRequest) returns (google.protobuf.Empty) {
    option (google.api.http) = {
        delete: "/api/v1/users/{user_id}"
    };
}

rpc DeleteCurrentUser(google.protobuf.Empty) returns (google.protobuf.Empty) {
    option (google.api.http) = {
        delete: "/api/v1/users/me"
    };
}
}

// Запросы для аутентификации
message RegisterRequest {
    account.User user = 1; // профиль из account
    string password = 2; // пароль как отдельное поле
}

message RegisterResponse {
    account.User user = 1;
    auth.TokenPair tokens = 2;
}

message LoginRequest {
    string login_or_email = 1;
    string password = 2;
}

message LoginResponse {
    account.User user = 1;
    auth.TokenPair tokens = 2;
}

message RefreshRequest {
    string refresh_token = 1;
}

message LogoutRequest {
    string refresh_token = 1;
}
```

```
message ValidateTokenRequest {
    string access_token = 1;
}

message ValidateTokenResponse {
    uint64 user_id = 1;
    bool is_valid = 2;
}

// Запросы для управления пользователями
message CreateUserRequest {
    account.CreateUser user = 1;
}

message GetUserRequest {
    uint64 user_id = 1;
}

message GetUserResponse {
    account.User user = 1;
}

message GetCurrentUserResponse {
    account.User user = 1;
}

message GetUsersRequest {
    pagination.Pagination pagination = 1;
}

message GetUsersResponse {
    repeated account.User users = 1;
    pagination.Pagination pagination = 2;
}

message UpdateUserRequest {
    uint64 user_id = 1;
    account.User user = 2;
}

message UpdateCurrentUserRequest {
    account.User user = 1;
}

message DeleteUserRequest {
    uint64 user_id = 1;
}
```

```
message RefreshResponse {  
    auth.TokenPair token_pair = 1;  
}
```

Как и положено фасаду, он предоставляет все gRPC-вызовы, которые ранее уже были описаны в других сервисах. Из нового добавились методы получения пользователем своего профиля и различные манипуляции с ним: `GetCurrentUser`, `UpdateCurrentUser`, `DeleteCurrentUser`. Для этих методов идентификатор пользователя будет получаться из токена, с которым выполняется запрос. Проверять токен и получать из него информацию будем также на уровне сервиса `Gateway`, т. к. накладные расходы на передачу данных по сети в нашем случае не являются необходимостью.

Для того чтобы часть вызовов была закрыта авторизацией, существует концепция *промежуточного слоя* — *интерсептора* (*interceptor*), «перехватывающего» выполнение запроса или вызова до того момента, как управление попадет в основную бизнес-логику. То есть интерсептор — это точка подключения к процессу обработки, где можно встроить кросс-срезную логику: авторизацию, метрики, обработку ошибок и т. д.

В gRPC-сервисах термин «интерсептор» используется официально — они бывают двух видов:

- ❑ **Unary Interceptor** — перехватывает обычные, разовые вызовы RPC, где один запрос → один ответ;
- ❑ **Stream Interceptor** — перехватывает потоковые вызовы.

Напишем такой обработчик, как отдельный слой в нашем приложении (листинг 3.2), и создадим для него отдельный каталог:

```
mkdir -p internal/interceptor
```

Листинг 3.2. Файл `internal/interceptor/jwt_interceptor.go`

```
package interceptor  
  
import (  
    "context"  
    "fmt"  
    "strings"  
  
    "github.com/golang-jwt/jwt/v5"  
    "google.golang.org/grpc"  
    "google.golang.org/grpc/codes"  
    "google.golang.org/grpc/metadata"  
    "google.golang.org/grpc/status"  
)  
  
// JWTClaims - структура для JWT claims  
type JWTClaims struct {
```

```

    UserID uint64 `json:"user_id"`
    jwt.RegisteredClaims
}

// JWTInterceptor - интерсептор для локальной валидации JWT-токенов
type JWTInterceptor struct {
    jwtSecret []byte
}

// Публичные методы, которые не требуют аутентификации
var publicMethods = map[string]bool{
    "/gateway.Gateway/Register": true,
    "/gateway.Gateway/Login":    true,
    "/gateway.Gateway/Refresh":  true,
}

// NewJWTInterceptor создает новый экземпляр JWTInterceptor
func NewJWTInterceptor(jwtSecret string) *JWTInterceptor {
    return &JWTInterceptor{
        jwtSecret: []byte(jwtSecret),
    }
}

// UnaryInterceptor - интерсептор для unary вызовов
func (i *JWTInterceptor) UnaryInterceptor() grpc.UnaryServerInterceptor {
    return func(ctx context.Context, req interface{}, info *grpc.UnaryServerInfo, handler
        grpc.UnaryHandler) (interface{}, error) {
        // Проверяем, является ли метод публичным
        if publicMethods[info.FullMethod] {
            // Для публичных методов пропускаем аутентификацию
            return handler(ctx, req)
        }

        // Извлекаем user_id из JWT-токена
        userID, err := i.extractUserIDFromJWT(ctx)
        if err != nil {
            return nil, err
        }

        // Добавляем user_id в контекст
        ctxWithUserID := context.WithValue(ctx, UserIDKey, userID)

        // Вызываем следующий обработчик
        return handler(ctxWithUserID, req)
    }
}

```

```
// StreamInterceptor - интерцептор для stream вызовов
func (i *JWTInterceptor) StreamInterceptor() grpc.StreamServerInterceptor {
    return func(srv interface{}, stream grpc.ServerStream, info *grpc.StreamServerInfo,
        handler grpc.StreamHandler) error {

        // Проверяем, является ли метод публичным
        if publicMethods[info.FullMethod] {
            // Для публичных методов пропускаем аутентификацию
            return handler(srv, stream)
        }

        // Извлекаем user_id из JWT-токена
        userID, err := i.extractUserIDFromJWT(stream.Context())
        if err != nil {
            return err
        }

        // Создаем новый контекст с user_id
        ctxWithUserID := context.WithValue(stream.Context(), UserIDKey, userID)

        // Создаем обертку для stream с новым контекстом
        wrappedStream := &wrappedServerStream{
            ServerStream: stream,
            ctx:            ctxWithUserID,
        }

        // Вызываем следующий обработчик
        return handler(srv, wrappedStream)
    }
}

// extractUserIDFromJWT извлекает user_id из JWT-токена в метаданных
func (i *JWTInterceptor) extractUserIDFromJWT(ctx context.Context) (uint64, error) {
    md, ok := metadata.FromIncomingContext(ctx)
    if !ok {
        return 0, status.Errorf(codes.Unauthenticated, "metadata is not provided")
    }

    authHeaders := md.Get(AuthorizationHeader)
    if len(authHeaders) == 0 {
        return 0, status.Errorf(codes.Unauthenticated, "authorization header is not provided")
    }

    token := authHeaders[0]
    if !strings.HasPrefix(token, BearerPrefix) {
        return 0, status.Errorf(codes.Unauthenticated, "invalid token format")
    }
}
```

```

// Убираем префикс "Bearer "
accessToken := strings.TrimPrefix(token, BearerPrefix)
if accessToken == "" {
    return 0, status.Errorf(codes.Unauthenticated, "access token is empty")
}

// Парсим и валидируем JWT-токен
userID, err := i.parseAndValidateJWT(accessToken)
if err != nil {
    return 0, status.Errorf(codes.Unauthenticated, "invalid token: %v", err)
}

return userID, nil
}

// parseAndValidateJWT парсит и валидирует JWT-токен
func (i *JWTInterceptor) parseAndValidateJWT(tokenString string) (uint64, error) {
    // Парсим токен
    claimsNew := JWTClaims{}
    token, err := jwt.ParseWithClaims(tokenString, &claimsNew, func(token *jwt.Token) (interface{}, error) {
        // Проверяем алгоритм подписи
        if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
            return nil, fmt.Errorf("unexpected signing method: %v", token.Header["alg"])
        }
        return i.jwtSecret, nil
    })

    if err != nil {
        return 0, fmt.Errorf("failed to parse token: %w", err)
    }

    // Проверяем валидность токена
    if !token.Valid {
        return 0, fmt.Errorf("token is not valid")
    }

    // Извлекаем claims
    claims, ok := token.Claims.(*JWTClaims)
    if !ok {
        return 0, fmt.Errorf("failed to extract claims")
    }

    // Проверяем, что user_id присутствует
    if claims.UserID == 0 {
        return 0, fmt.Errorf("user_id not found in token")
    }
}

```

```
    return claims.UserID, nil
}

// ValidateTokenWithoutAuthService - функция для валидации токена без обращения к auth сервису
func (i *JWTInterceptor) ValidateTokenWithoutAuthService(tokenString string) (uint64, error) {
    return i.parseAndValidateJWT(tokenString)
}

// GetUserIDFromContext извлекает user_id из контекста
func GetUserIDFromContext(ctx context.Context) (uint64, error) {
    userID, ok := ctx.Value(UserIDKey).(uint64)
    if !ok {
        return 0, fmt.Errorf("user_id not found in context")
    }
    return userID, nil
}

// wrappedServerStream - обертка для ServerStream с кастомным контекстом
type wrappedServerStream struct {
    grpc.ServerStream
    ctx context.Context
}

func (w *wrappedServerStream) Context() context.Context {
    return w.ctx
}
```

Для того чтобы передавать дополнительную информацию в токене, принято использовать `JWTClaims` — структуру с пользовательскими «претензиями» (`claims`), которые включают в себя стандартные поля токена и дополнительное поле `UserID`, позволяющее хранить идентификатор прямо внутри токена. Для проверки подписи токена используется секретный ключ, он хранится в структуре `JWTInterceptor` и передается из конфигурации сервиса.

`publicMethods` — это перечень публичных методов, для которых не нужно проверять аутентификацию. Например, при регистрации или входе в систему очевидно, что у пользователя еще не может быть токена, и для этих вызовов следует пропустить запрос сразу на этап бизнес-логики.

Логика работы интерсептора устроена следующим образом:

1. Проверка того, является ли метод публичным, для которого не нужно проверять токен.
2. Извлечение токена — интерсептор достает метаданные запроса по заголовку `authorization`.
3. Валидация токена — из токена убирается приставка `Bearer`, затем он парсится с помощью секретного ключа. Если токен просрочен, подпись не совпадает или неверный формат — вернется ошибка, и запрос дальше не пройдет.

4. Извлечение идентификатора пользователя из токена.
5. Добавление идентификатора пользователя в контекст.
6. Запрос уходит в бизнес-логику.

Таким образом можно централизовать аутентификацию. При следующих вызовах сервисов, которые будут происходить из Gateway, нет необходимости проверять JWT каждый раз — достаточно подключить интерсептор к серверу, и он автоматически будет проверять все методы, кроме публичных. Это защищает API без дублирования кода, позволяет получить метаданные (в нашем случае идентификатор пользователя) внутри сервиса, а запросы выполняются быстрее за счет того, что проверка выполняется без вызова `auth`.

Подключение обработчика к серверу будет выглядеть следующим образом (листинг 3.3).

Листинг 3.3. Файл `internal/app/app.go`

```
package app

import (
    "context"
    "fmt"
    "net"

    accountpb "github.com/yuliaapopova/contracts/account/go"
    authpb    "github.com/yuliaapopova/contracts/auth/go"
    "github.com/yuliaapopova/gateway/internal/account"
    "github.com/yuliaapopova/gateway/internal/auth"
    "github.com/yuliaapopova/gateway/internal/config"
    "github.com/yuliaapopova/gateway/internal/interceptor"
    "github.com/yuliaapopova/gateway/internal/server"
    "github.com/yuliaapopova/gateway/internal/service"
    "google.golang.org/grpc/credentials/insecure"

    gatewaypb "github.com/yuliaapopova/contracts/gateway/go"
    _ "google.golang.org/genproto/googleapis/api/annotations"
    "google.golang.org/grpc"

    _ "github.com/lib/pq"
    "github.com/rs/zerolog"
)

type App struct {
    cfg      *config.Config
    logger   *zerolog.Logger

    service      *service.GatewayService
    authService  *auth.Service
    accountService *account.Service
}
```

```
server      *server.Server
grpcServer  *grpc.Server
)

func New(logger *zerolog.Logger, cfg *config.Config) *App {
    return &App{
        cfg:    cfg,
        logger: logger,
    }
}

func (a *App) Run(ctx context.Context) error {
    // Инициализируем gRPC-сервер
    gatewayServer, err := a.getGatewayServer(ctx)
    if err != nil {
        return fmt.Errorf("failed to get gateway server: %w", err)
    }

    // Создаем gRPC-сервер
    a.grpcServer = a.getGRPCServer(gatewayServer)

    listenAddr := fmt.Sprintf("%s:%d", a.cfg.Host, a.cfg.Port)
    lis, err := net.Listen("tcp", listenAddr)
    if err != nil {
        a.logger.Fatal().Err(err).Msg("failed to listen")
        return err
    }

    a.logger.Info().Msg("gRPC server listening")

    serveErrCh := make(chan error, 1)
    go func() {
        serveErrCh <- a.grpcServer.Serve(lis)
    }()

    select {
    case <-ctx.Done():
        a.grpcServer.GracefulStop()
        return ctx.Err()
    case err := <-serveErrCh:
        if err != nil {
            a.logger.Error().Err(err).Msg("failed to serve")
        }
        return err
    }
}
```

```
func (a *App) getGatewayService(ctx context.Context) (*service.GatewayService, error) {
    if a.service == nil {
        authService, err := a.getAuthService(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get auth service: %w", err)
        }
        accountService, err := a.getAccountService(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get account service: %w", err)
        }
        a.service = service.New(accountService, authService, a.logger)
    }
    return a.service, nil
}

func (a *App) getAuthService(ctx context.Context) (*auth.Service, error) {
    if a.authService == nil {
        conn, err := grpc.NewClient(
            a.cfg.AuthGrpcHost,
            grpc.WithTransportCredentials(insecure.NewCredentials()),
        )
        if err != nil {
            return nil, fmt.Errorf("failed to create auth service: %w", err)
        }
        client := authpb.NewAuthClient(conn)
        a.authService = auth.NewService(client)
    }
    return a.authService, nil
}

func (a *App) getAccountService(ctx context.Context) (*account.Service, error) {
    if a.accountService == nil {
        conn, err := grpc.NewClient(
            a.cfg.AccountGrpcHost,
            grpc.WithTransportCredentials(insecure.NewCredentials()),
        )
        if err != nil {
            return nil, fmt.Errorf("failed to create account service: %w", err)
        }
        client := accountpb.NewAccountClient(conn)
        a.accountService = account.New(client)
    }
    return a.accountService, nil
}

func (a *App) getGatewayServer(ctx context.Context) (*server.Server, error) {
    if a.server == nil {
        service, err := a.getGatewayService(ctx)
```

```
    if err != nil {
        return nil, fmt.Errorf("failed to get service: %w", err)
    }
    a.server = server.New(service, a.logger)
}
return a.server, nil
}

func (a *App) getGRPCServer(srv *server.Server) *grpc.Server {
    jwtInterceptor := interceptor.NewJWTInterceptor(a.cfg.JWTSecret)

    // Создаем gRPC-сервер с интерсептором
    grpcSrv := grpc.NewServer(
        grpc.UnaryInterceptor(jwtInterceptor.UnaryInterceptor()),
    )

    gatewaypb.RegisterGatewayServer(grpcSrv, srv)
    return grpcSrv
}

// Close корректно завершает работу приложения
func (a *App) Close() error {
    if a.grpcServer != nil {
        a.grpcServer.GracefulStop()
    }
    return nil
}
```

Обратить внимание тут стоит на метод `getGRPCServer` — именно с его помощью создается объект интерсептора, который будет проверять JWT-токены в каждом запросе, а секретный ключ передается из конфигурации приложения. А при инициализации сервера теперь передаем ему в параметрах интерсептор — в нашем случае только для обычных запросов. Если нужен интерсептор и для потоковой передачи данных, тогда следует создавать сервер так:

```
grpcSrv := grpc.NewServer(
    grpc.UnaryInterceptor(jwtInterceptor.UnaryInterceptor()),
    grpc.StreamInterceptor(jwtInterceptor.StreamInterceptor()),
)
```

Поскольку Gateway — это не только про единую точку аутентификации и получение информации из токена, а еще и про вызовы внутренних сервисов, добавим интеграцию с `auth` и `account`. Для этого нужно в первую очередь добавить адреса сервисов в переменные окружения (листинг 3.4) и их получение — в конфиг (листинг 3.5).

Листинг 3.4. Файл `env.example`

```
SERVICE_NAME=gateway
APP_ENV=dev
```

```

GRPC_PORT=50053
GRPC_HOST=localhost
LOG_LEVEL=debug

ACCOUNT_GRPC_HOST=localhost:50051
AUTH_GRPC_HOST=localhost:50052

JWT_SECRET=secret

```

Листинг 3.5. Файл internal/config/config.go

```

package config

import (
    "log"

    "github.com/caarlos0/env/v10"
    "github.com/joho/godotenv"
)

// Config содержит конфигурацию приложения
type Config struct {
    ServiceName string `env:"SERVICE_NAME" json:"service_name" required:"true"
                                     default:"account-service"`
    AppEnv      string `env:"APP_ENV" json:"app_environment" required:"true"
                                     default:"development"`
    Host        string `env:"GRPC_HOST" json:"host" required:"true" default:"localhost"`
    Port        int    `env:"GRPC_PORT" json:"port" required:"true" default:"50053"`
    LogLevel    string `env:"LOG_LEVEL" json:"log_level" required:"true" default:"info"`
    AccountGrpcHost string `env:"ACCOUNT_GRPC_HOST" json:"account_grpc_host" required:"true"`
    AuthGrpcHost  string `env:"AUTH_GRPC_HOST" json:"auth_grpc_host" required:"true"`
    JwtSecret     string `env:"JWT_SECRET" json:"jwt_secret" required:"true"`
}

// Load загружает конфигурацию из переменных окружения
func Load() (*Config, error) {
    // Загружаем .env файл
    if err := godotenv.Load(); err != nil {
        log.Println("Warning: .env file not found, using system environment variables")
    }

    cfg := &Config{}
    err := env.Parse(cfg)
    if err != nil {
        return nil, err
    }

    return cfg, nil
}

```

Подключение сервисов как модулей будет по своему принципу таким же, как и подключение других сторонних ресурсов (листинг 3.6). Тут это реализуется с помощью методов `getAuthService` и `getAccountService`, в которых создаются gRPC-клиенты и передается адрес, по которому нужно к ним обращаться.

При этом работа со сторонними сервисами будет также изолирована отдельным пакетом и маппингом из Protobuf-формата в привычные go-структуры (листинг 3.7).

Листинг 3.6. Файл `internal/app/app.go`

```
package app

import (
    "context"
    "fmt"
    "net"

    accountpb "github.com/yuliaapopova/contracts/account/go"
    authpb    "github.com/yuliaapopova/contracts/auth/go"
    "github.com/yuliaapopova/gateway/internal/account"
    "github.com/yuliaapopova/gateway/internal/auth"
    "github.com/yuliaapopova/gateway/internal/config"
    "github.com/yuliaapopova/gateway/internal/interceptor"
    "github.com/yuliaapopova/gateway/internal/server"
    "github.com/yuliaapopova/gateway/internal/service"
    "google.golang.org/grpc/credentials/insecure"

    gatewaypb "github.com/yuliaapopova/contracts/gateway/go"
    _ "google.golang.org/genproto/googleapis/api/annotations"
    "google.golang.org/grpc"

    _ "github.com/lib/pq"
    "github.com/rs/zerolog"
)

type App struct {
    cfg      *config.Config
    logger   *zerolog.Logger

    service      *service.GatewayService
    authService  *auth.Service
    accountService *account.Service

    server      *server.Server
    grpcServer *grpc.Server
}

func New(logger *zerolog.Logger, cfg *config.Config) *App {
    return &App{
```

```
    cfg:    cfg,
    logger: logger,
  }
}

func (a *App) Run(ctx context.Context) error {
  // Инициализируем gRPC-сервер
  gatewayServer, err := a.getGatewayServer(ctx)
  if err != nil {
    return fmt.Errorf("failed to get gateway server: %w", err)
  }

  // Создаем gRPC-сервер
  a.grpcServer = a.getGRPCServer(gatewayServer)

  listenAddr := fmt.Sprintf("%s:%d", a.cfg.Host, a.cfg.Port)
  lis, err := net.Listen("tcp", listenAddr)
  if err != nil {
    a.logger.Fatal().Err(err).Msg("failed to listen")
    return err
  }

  a.logger.Info().Msg("gRPC server listening")

  serveErrCh := make(chan error, 1)
  go func() {
    serveErrCh <- a.grpcServer.Serve(lis)
  }()

  select {
  case <-ctx.Done():
    a.grpcServer.GracefulStop()
    return ctx.Err()
  case err := <-serveErrCh:
    if err != nil {
      a.logger.Error().Err(err).Msg("failed to serve")
    }
    return err
  }
}

func (a *App) getGatewayService(ctx context.Context) (*service.GatewayService, error) {
  if a.service == nil {
    authService, err := a.getAuthService(ctx)
    if err != nil {
      return nil, fmt.Errorf("failed to get auth service: %w", err)
    }
  }
}
```

```
    accountService, err := a.getAccountService(ctx)
    if err != nil {
        return nil, fmt.Errorf("failed to get account service: %w", err)
    }
    a.service = service.New(accountService, authService, a.logger)
}
return a.service, nil
}

func (a *App) getAuthService(ctx context.Context) (*auth.Service, error) {
    if a.authService == nil {
        conn, err := grpc.NewClient(
            a.cfg.AuthGrpcHost,
            grpc.WithTransportCredentials(insecure.NewCredentials()),
        )
        if err != nil {
            return nil, fmt.Errorf("failed to create auth service: %w", err)
        }
        client := authpb.NewAuthClient(conn)
        a.authService = auth.NewService(client)
    }
    return a.authService, nil
}

func (a *App) getAccountService(ctx context.Context) (*account.Service, error) {
    if a.accountService == nil {
        conn, err := grpc.NewClient(
            a.cfg.AccountGrpcHost,
            grpc.WithTransportCredentials(insecure.NewCredentials()),
        )
        if err != nil {
            return nil, fmt.Errorf("failed to create account service: %w", err)
        }
        client := accountpb.NewAccountClient(conn)
        a.accountService = account.New(client)
    }
    return a.accountService, nil
}

func (a *App) getGatewayServer(ctx context.Context) (*server.Server, error) {
    if a.server == nil {
        service, err := a.getGatewayService(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get service: %w", err)
        }
        a.server = server.New(service, a.logger)
    }
}
```



```

    return a.server, nil
}

func (a *App) getGRPCServer(srv *server.Server) *grpc.Server {
    jwtInterceptor := interceptor.NewJWTInterceptor(a.cfg.JwtSecret)

    // Создаем gRPC-сервер с интерсептором
    grpcSrv := grpc.NewServer(
        grpc.UnaryInterceptor(jwtInterceptor.UnaryInterceptor()),
    )

    gatewaypb.RegisterGatewayServer(grpcSrv, srv)
    return grpcSrv
}

// Close корректно завершает работу приложения
func (a *App) Close() error {
    if a.grpcServer != nil {
        a.grpcServer.GracefulStop()
    }
    return nil
}

```

Листинг 3.7. Файл `internal/account/account.go`

```

package account

import (
    "context"
    "fmt"

    accountpb "github.com/yuliaapopova/contracts/account/go"
    "github.com/yuliaapopova/contracts/pagination/go"
    "github.com/yuliaapopova/gateway/internal/model"
)

type Service struct {
    client accountpb.AccountClient
}

func New(client accountpb.AccountClient) *Service {
    return &Service{
        client: client,
    }
}

func (s *Service) GetUser(ctx context.Context, userID uint64) (model.User, error) {
    user, err := s.client.GetUser(ctx, &accountpb.GetUserRequest{UserId: userID})

```

```
    if err != nil {
        return model.User{}, fmt.Errorf("failed get user: %w", err)
    }
    return PbToUser(user.User), nil
}

func (s *Service) GetUsers(ctx context.Context, limit uint32, offset uint32) ([]model.User, error) {
    users, err := s.client.GetUsers(ctx, &accountpb.GetUsersRequest{
        Pagination: &pagination.Pagination{
            Limit:    limit,
            Offset:   offset,
        },
    })
    if err != nil {
        return nil, fmt.Errorf("failed get users: %w", err)
    }
    return PbsToUsers(users.Users), nil
}

func (s *Service) DeleteUser(ctx context.Context, userID uint64) error {
    _, err := s.client.DeleteUser(ctx, &accountpb.DeleteUserRequest{
        UserId: userID,
    })
    if err != nil {
        return fmt.Errorf("failed delete user: %w", err)
    }
    return nil
}

func (s *Service) CreateUser(ctx context.Context, user model.User) (model.User, error) {
    res, err := s.client.CreateUser(ctx, &accountpb.CreateUserRequest{
        User: UserCreateToPb(user),
    })
    if err != nil {
        return model.User{}, fmt.Errorf("failed create user: %w", err)
    }
    return PbToUser(res.User), nil
}

func (s *Service) UpdateUser(ctx context.Context, userID uint64, user model.UpdateUser) error {
    _, err := s.client.UpdateUser(ctx, &accountpb.UpdateUserRequest{
        UserId: userID,
        User:   UserUpdateToPb(user),
    })
    if err != nil {
        return fmt.Errorf("failed update user: %w", err)
    }
    return nil
}
```

```
func PbsToUsers(pbs []*accountpb.User) []model.User {
    users := make([]model.User, len(pbs))
    for i, pb := range pbs {
        users[i] = PbToUser(pb)
    }
    return users
}

func PbToUser(userpb *accountpb.User) model.User {
    return model.User{
        ID:          userpb.Id,
        Login:       userpb.Login,
        Email:       userpb.Email,
        Phone:       userpb.Phone,
        FirstName:   userpb.FirstName,
        LastName:    userpb.LastName,
        MiddleName:  userpb.MiddleName,
        Age:         userpb.Age,
        CreatedAt:   userpb.CreatedAt.AsTime(),
        UpdatedAt:   userpb.UpdatedAt.AsTime(),
    }
}

func UserCreateToPb(user model.User) *accountpb.CreateUser {
    return &accountpb.CreateUser{
        Login:       user.Login,
        Email:       user.Email,
        Phone:       user.Phone,
        FirstName:   user.FirstName,
        LastName:    user.LastName,
        MiddleName:  user.MiddleName,
        Age:         user.Age,
    }
}

func UserUpdateToPb(user model.UpdateUser) *accountpb.User {
    return &accountpb.User{
        Email:       user.Email,
        Phone:       user.Phone,
        FirstName:   user.FirstName,
        LastName:    user.LastName,
        MiddleName:  user.MiddleName,
        Age:         user.Age,
    }
}
```

Смысл в этом такой же, как и в изолировании всех других ресурсов, — если что-то поменяется, например перепишется сервис, к которому требуется обращаться, дос-

точно будет сделать изменения в одном файле и поддержать формат, привычный для Gateway (в нашем случае), а не искать все места в проекте (листинг 3.8).

Листинг 3.8. Файл `internal/auth/auth.go`

```
package auth

import (
    "context"
    "fmt"

    authpb "github.com/yuliaapopova/contracts/auth/go"
    "github.com/yuliaapopova/gateway/internal/model"
)

type Service struct {
    client authpb.AuthClient
}

func NewService(client authpb.AuthClient) *Service {
    return &Service{client: client}
}

func (s *Service) Login(ctx context.Context, loginOrEmail, password string) (model.TokenPair, error) {
    res, err := s.client.Login(ctx, &authpb.LoginRequest{
        LoginOrEmail: loginOrEmail,
        Password:     password,
    })
    if err != nil {
        return model.TokenPair{}, fmt.Errorf("failed login: %w", err)
    }
    return PbToTokenPair(res.TokenPair), nil
}

func (s *Service) Refresh(ctx context.Context, refreshToken string) (model.TokenPair, error) {
    res, err := s.client.Refresh(ctx, &authpb.RefreshRequest{
        RefreshToken: refreshToken,
    })
    if err != nil {
        return model.TokenPair{}, fmt.Errorf("failed refresh token: %w", err)
    }
    return PbToTokenPair(res.TokenPair), nil
}

func (s *Service) Verify(ctx context.Context, accessToken string) (uint64, error) {
    res, err := s.client.Validate(ctx, &authpb.ValidateRequest{
        AccessToken: accessToken,
    })
}
```

```
    if err != nil {
        return 0, fmt.Errorf("failed verify access token: %w", err)
    }
    return res.UserId, nil
}

func (s *Service) Logout(ctx context.Context, refreshToken string) error {
    _, err := s.client.Logout(ctx, &authpb.LogoutRequest{
        RefreshToken: refreshToken,
    })
    if err != nil {
        return fmt.Errorf("failed logout: %w", err)
    }
    return nil
}

func (s *Service) Register(ctx context.Context, userID uint64, login string, email string, password string) error {
    _, err := s.client.Register(ctx, &authpb.RegisterRequest{
        Id:      userID,
        Login:   login,
        Email:   email,
        Password: password,
    })
    if err != nil {
        return fmt.Errorf("failed to register user: %w", err)
    }
    return nil
}

func (s *Service) DeleteUser(ctx context.Context, userID uint64) error {
    _, err := s.client.DeleteUser(ctx, &authpb.DeleteRequest{
        Id: userID,
    })
    if err != nil {
        return fmt.Errorf("failed to delete user: %w", err)
    }
    return nil
}

func PbToTokenPair(tokenPair *authpb.TokenPair) model.TokenPair {
    return model.TokenPair{
        AccessToken: tokenPair.AccessToken,
        RefreshToken: tokenPair.RefreshToken,
    }
}
```

А используется это всё в бизнес-логике теперь очень просто — к самим пакетам никаких обращений при этом нет, вся реализация организуется только через интерфейсы (листинг 3.9).

Листинг 3.9. Файл `internal/service/service.go`

```
package service

import (
    "context"
    "fmt"

    "github.com/rs/zerolog"
    "github.com/yuliaapopova/gateway/internal/mapper"
    "github.com/yuliaapopova/gateway/internal/model"
)

type GatewayService struct {
    logger *zerolog.Logger

    accountService AccountService
    authService     AuthService
}

func New(accountService AccountService, authService AuthService, logger *zerolog.Logger)
*GatewayService {
    return &GatewayService{
        accountService: accountService,
        authService:    authService,
        logger:         logger,
    }
}

type AccountService interface {
    CreateUser(context.Context, model.User) (model.User, error)
    GetUser(context.Context, uint64) (model.User, error)
    GetUsers(context.Context, uint32, uint32) ([]model.User, error)
    DeleteUser(context.Context, uint64) error
    UpdateUser(context.Context, uint64, model.UpdateUser) error
}

type AuthService interface {
    Login(context.Context, string, string) (model.TokenPair, error)
    Logout(context.Context, string) error
    Register(context.Context, uint64, string, string, string) error
    Verify(ctx context.Context, accessToken string) (uint64, error)
    Refresh(ctx context.Context, refreshToken string) (model.TokenPair, error)
    DeleteUser(context.Context, uint64) error
}
```

// Методы для аутентификации

```
func (s *GatewayService) Register(ctx context.Context, newUser model.User, password string)
(model.User, model.TokenPair, error) {
    user, err := s.accountService.CreateUser(ctx, newUser)
    if err != nil {
        return model.User{}, model.TokenPair{}, fmt.Errorf("failed to create user: %w", err)
    }

    if err := s.authService.Register(ctx, user.ID, newUser.Login, newUser.Email, password);
        err != nil {
        return model.User{}, model.TokenPair{}, fmt.Errorf("failed to register user: %w", err)
    }

    token, err := s.authService.Login(ctx, newUser.Login, password)
    if err != nil {
        return model.User{}, model.TokenPair{}, fmt.Errorf("failed to login: %w", err)
    }

    return user, token, nil
}

func (s *GatewayService) Login(ctx context.Context, loginOrEmail, password string)
(model.User, model.TokenPair, error) {
    tokens, err := s.authService.Login(ctx, loginOrEmail, password)
    if err != nil {
        return model.User{}, model.TokenPair{}, fmt.Errorf("user not found: %w", err)
    }
    user := model.User{}

    return user, tokens, nil
}

func (s *GatewayService) Logout(ctx context.Context, token string) error {
    if err := s.authService.Logout(ctx, token); err != nil {
        return fmt.Errorf("failed to logout: %w", err)
    }
    return nil
}

func (s *GatewayService) Refresh(ctx context.Context, refreshToken string) (model.TokenPair,
error) {
    return s.authService.Refresh(ctx, refreshToken)
}

func (s *GatewayService) ValidateToken(ctx context.Context, accessToken string) (uint64,
bool, error) {
    userID, err := s.authService.Verify(ctx, accessToken)
```

```
    if err != nil {
        return 0, false, err
    }
    return userID, true, nil
}

// Методы для управления пользователями
func (s *GatewayService) CreateUser(ctx context.Context, newUser model.CreateUser) error {
    user := mapper.CreateUserToUser(newUser)

    _, err := s.accountService.CreateUser(ctx, user)
    if err != nil {
        return fmt.Errorf("failed create user: %w", err)
    }
    return nil
}

func (s *GatewayService) GetUsers(ctx context.Context, limit int, offset int) ([]model.User, error) {
    return s.accountService.GetUsers(ctx, uint32(limit), uint32(offset))
}

func (s *GatewayService) GetUser(ctx context.Context, userID uint64) (model.User, error) {
    return s.accountService.GetUser(ctx, userID)
}

func (s *GatewayService) DeleteUser(ctx context.Context, userID uint64) error {
    err := s.authService.DeleteUser(ctx, userID)
    if err != nil {
        return fmt.Errorf("failed to delete user: %w", err)
    }
    return s.accountService.DeleteUser(ctx, userID)
}

func (s *GatewayService) UpdateUser(ctx context.Context, userID uint64, user model.UpdateUser) error {
    return s.accountService.UpdateUser(ctx, userID, user)
}
```

Из интересного тут — метод регистрации, потому что мы делаем сразу несколько вызовов `auth` и `account`. Сначала создаем пользователя (пароль в `account` не передаем! — он должен храниться в зашифрованном виде только в `auth`) и потом уже регистрируем его в `auth` и авторизуемся, чтобы вернуть токены для доступа к ресурсам сервиса.

Остается теперь разобраться, как из токена получить идентификатор пользователя, потому что в бизнес-логике никакой работы с токенами нет, есть лишь передача

их из сервиса `auth` пользователю. Эта часть находится в серверном слое (листинг 3.10).

Листинг 3.10. Файл `internal/server/server.go`

```
package server

import (
    "context"

    "github.com/yuliaapopova/gateway/internal/interceptor"
    "github.com/yuliaapopova/gateway/internal/mapper"
    "github.com/yuliaapopova/gateway/internal/model"

    gatewaypb "github.com/yuliaapopova/contracts/gateway/go"
    "google.golang.org/protobuf/types/known/emptypb"

    "github.com/rs/zerolog"
)

type Server struct {
    gatewaypb.UnimplementedGatewayServer

    gatewayService GatewayService
    logger          *zerolog.Logger
}

func New(gatewayService GatewayService, logger *zerolog.Logger) *Server {
    return &Server{gatewayService: gatewayService, logger: logger}
}

type GatewayService interface {
    Register(context.Context, model.User, string) (model.User, model.TokenPair, error)
    Login(context.Context, string, string) (model.User, model.TokenPair, error)
    Logout(context.Context, string) error
    Refresh(context.Context, string) (model.TokenPair, error)
    ValidateToken(context.Context, string) (uint64, bool, error)
    CreateUser(context.Context, model.CreateUser) error
    GetUser(ctx context.Context, userID uint64) (model.User, error)
    GetUsers(ctx context.Context, limit int, offset int) ([]model.User, error)
    DeleteUser(context.Context, uint64) error
    UpdateUser(context.Context, uint64, model.UpdateUser) error
}

// Методы аутентификации
func (s *Server) Register(ctx context.Context, req *gatewaypb.RegisterRequest)
(*gatewaypb.RegisterResponse, error) {
    user := mapper.PbToUser(req.GetUser())
```

```
    createdUser, tokens, err := s.gatewayService.Register(ctx, user, req.Password)
    if err != nil {
        return nil, err
    }

    return &gatewaypb.RegisterResponse{
        User:    mapper.UserToPb(createdUser),
        Tokens:  mapper.TokenPairToPb(tokens),
    }, nil
}

func (s *Server) Login(ctx context.Context, req *gatewaypb.LoginRequest)
(*gatewaypb.LoginResponse, error) {
    user, tokens, err := s.gatewayService.Login(ctx, req.GetLoginOrEmail(), req.GetPassword())
    if err != nil {
        return nil, err
    }

    return &gatewaypb.LoginResponse{
        User:    mapper.UserToPb(user),
        Tokens:  mapper.TokenPairToPb(tokens),
    }, nil
}

func (s *Server) Refresh(ctx context.Context, req *gatewaypb.RefreshRequest)
(*gatewaypb.RefreshResponse, error) {
    tokens, err := s.gatewayService.Refresh(ctx, req.GetRefreshToken())
    if err != nil {
        return nil, err
    }

    return &gatewaypb.RefreshResponse{
        TokenPair: mapper.TokenPairToPb(tokens),
    }, nil
}

func (s *Server) Logout(ctx context.Context, req *gatewaypb.LogoutRequest) (*emptypb.Empty,
error) {
    err := s.gatewayService.Logout(ctx, req.GetRefreshToken())
    if err != nil {
        return nil, err
    }

    return &emptypb.Empty(), nil
}

func (s *Server) ValidateToken(ctx context.Context, req *gatewaypb.ValidateTokenRequest)
(*gatewaypb.ValidateTokenResponse, error) {
    userID, isValid, err := s.gatewayService.ValidateToken(ctx, req.GetAccessToken())
```

```
if err != nil {
    return nil, err
}

return &gatewaypb.ValidateTokenResponse{
    UserID: userID,
    IsValid: isValid,
}, nil
}

// Методы управления пользователями
func (s *Server) CreateUser(ctx context.Context, req *gatewaypb.CreateUserRequest)
(*emptypb.Empty, error) {
    user := mapper.PbToUserCreate(req.GetUser())
    if err := s.gatewayService.CreateUser(ctx, user); err != nil {
        return nil, err
    }
    return &emptypb.Empty(), nil
}

func (s *Server) GetUser(ctx context.Context, req *gatewaypb.GetUserRequest)
(*gatewaypb.GetUserResponse, error) {
    res, err := s.gatewayService.GetUser(ctx, req.GetUserId())
    if err != nil {
        return nil, err
    }
    return &gatewaypb.GetUserResponse{
        User: mapper.UserToPb(res),
    }, nil
}

func (s *Server) GetCurrentUser(ctx context.Context, req *emptypb.Empty)
(*gatewaypb.GetCurrentUserResponse, error) {
    // Получаем userID из контекста (добавлен интерсептором)
    userID, err := interceptor.GetUserIDFromContext(ctx)
    if err != nil {
        return nil, err
    }

    res, err := s.gatewayService.GetUser(ctx, userID)
    if err != nil {
        return nil, err
    }
    return &gatewaypb.GetCurrentUserResponse{
        User: mapper.UserToPb(res),
    }, nil
}
```

```
func (s *Server) GetUsers(ctx context.Context, req *gatewaypb.GetUsersRequest)
(*gatewaypb.GetUsersResponse, error) {
    pagination := req.GetPagination()
    limit := int(pagination.GetLimit())
    offset := int(pagination.GetOffset())

    res, err := s.gatewayService.GetUsers(ctx, limit, offset)
    if err != nil {
        return nil, err
    }
    return &gatewaypb.GetUsersResponse{
        Users:      mapper.UsersToPbs(res),
        Pagination: pagination,
    }, nil
}

func (s *Server) UpdateUser(ctx context.Context, req *gatewaypb.UpdateUserRequest)
(*emptypb.Empty, error) {
    user := mapper.PbToUserUpdate(req.GetUser())
    if err := s.gatewayService.UpdateUser(ctx, req.GetUserId(), user); err != nil {
        return nil, err
    }
    return &emptypb.Empty(), nil
}

func (s *Server) UpdateCurrentUser(ctx context.Context, req
*gatewaypb.UpdateCurrentUserRequest) (*emptypb.Empty, error) {
    // Получаем userID из контекста (добавлен интерсептором)
    userID, err := interceptor.GetUserIDFromContext(ctx)
    if err != nil {
        return nil, err
    }

    user := mapper.PbToUserUpdate(req.GetUser())
    if err := s.gatewayService.UpdateUser(ctx, userID, user); err != nil {
        return nil, err
    }
    return &emptypb.Empty(), nil
}

func (s *Server) DeleteUser(ctx context.Context, req *gatewaypb.DeleteUserRequest)
(*emptypb.Empty, error) {
    if err := s.gatewayService.DeleteUser(ctx, req.GetUserId()); err != nil {
        return nil, err
    }
    return &emptypb.Empty(), nil
}
```

```
func (s *Server) DeleteCurrentUser(ctx context.Context, req *emptypb.Empty) (*emptypb.Empty, error) {  
    // Получаем userID из контекста (добавлен интерсептором)  
    userID, err := interceptor.GetUserIDFromContext(ctx)  
    if err != nil {  
        return nil, err  
    }  
  
    if err := s.gatewayService.DeleteUser(ctx, userID); err != nil {  
        return nil, err  
    }  
    return &emptypb.Empty{}, nil  
}
```

В методах `GetCurrentUser`, `UpdateCurrentUser` и `DeleteCurrentUser` есть вызов метода из пакета интерсептора:

```
userID, err := interceptor.GetUserIDFromContext(ctx)
```

В нем из контекста и достается идентификатор, а промежуточный обработчик запросов как раз туда и складывает нужные нам данные. Так же можно было поступить и с именем или логином пользователя, если бы они нам были нужны.

Список литературы и источников

1. Лекции по дисциплине «Сети» БГТУ им. В. Г. Шухова.
2. <https://habr.com/ru/companies/piter/articles/596621/>.
3. <https://ru.wikipedia.org/wiki/GRPC>.
4. <https://habr.com/ru/articles/488654/>.
5. <https://habr.com/ru/companies/southbridge/articles/550934/>.
6. https://ru.wikipedia.org/wiki/Apache_Kafka.
7. <https://habr.com/ru/companies/wunderfund/articles/685894/>.

ГЛАВА 4



Разработка модуля Transaction

Проектирование базы данных

В главе 1 мы уже начинали проектировать базу данных и рассмотрели три нормальные формы базы данных, но есть и другие нормальные формы, разговор о которых мы здесь и продолжим [1].

Частная форма третьей нормальной формы: нормальная форма Бойса — Кодда (НФБК)

Определение 3НФ не совсем подходит для следующих отношений:

- ☐ отношение имеет два или более потенциальных ключа;
- ☐ два и более потенциальных ключа являются составными;
- ☐ они пересекаются, т. е. имеют хотя бы один общий атрибут.

Для отношений с одним потенциальным ключом, который служит первичным, НФБК соответствует 3НФ. Отношение считается находящимся в НФБК, если каждая нетривиальная и неприводимая функциональная зависимость слева содержит потенциальный ключ в качестве своего детерминанта. Примером может служить отношение, представляющее данные о бронировании стоянки на день (табл. 4.1).

Таблица 4.1. Стоянки

Номер стоянки	Время начала	Время окончания	Тариф
1	09:30	10:30	Бережливый
1	11:00	12:00	Бережливый
1	14:00	15:30	Стандарт
2	10:00	12:00	Премиум-В
2	12:00	14:00	Премиум-В
2	15:00	18:00	Премиум-А

Атрибут «Тариф» имеет здесь уникальное название и зависит от выбранной стоянки и наличия льгот, в частности:

- «Бережливый»: стоянка 1 для льготников;
- «Стандарт»: стоянка 1 для не льготников;
- «Премиум-А»: стоянка 2 для льготников;
- «Премиум-В»: стоянка 2 для не льготников.

Таким образом, возможны следующие составные первичные ключи: {Номер стоянки, Время начала}, {Номер стоянки, Время окончания}, {Тариф, Время начала}, {Тариф, Время окончания}.

Это отношение вроде бы демонстрирует соответствие третьей нормальной форме (3НФ). Вторая нормальная форма (2НФ) тоже соблюдается, т. к. все атрибуты являются частью какого-либо потенциального ключа, и нет атрибутов, не входящих в ключ. Также отсутствуют транзитивные зависимости, что соответствует требованиям 3НФ.

Тем не менее следует отметить функциональную зависимость Тариф $\rightarrow$ Номер стоянки, в которой левая часть (детерминант) не является потенциальным ключом отношения, что делает это отношение несоответствующим нормальной форме Бойса — Кодда (НФБК).

Недостаток такой структуры заключается в возможности случайно присвоить тариф «Бережливый» к бронированию второй стоянки, хотя он должен относиться только к первой стоянке. Это может привести к ошибкам и нежелательным последствиям в управлении данными.

Можно улучшить структуру с помощью декомпозиции отношения на два: «Тарифы» (табл. 4.2) и «Бронирование» (табл. 4.3) — и добавления атрибута «Имеет льготы», получив отношения, удовлетворяющие НФБК.

Таблица 4.2. Тарифы

Тариф	Номер стоянки	Имеет льготы
Бережливый	1	Да
Стандарт	1	Нет
Премиум-А	2	Да
Премиум-В	2	Нет

Таблица 4.3. Бронирование

Тариф	Время начала	Время окончания
Бережливый	09:30	10:30
Бережливый	11:00	12:00
Стандарт	14:00	15:30

Таблица 4.3 (окончание)

Тариф	Время начала	Время окончания
Премиум-В	10:00	12:00
Премиум-В	12:00	14:00
Премиум-А	15:00	18:00

Четвертая нормальная форма

Отношение успешно достигает четвертой нормальной формы (4НФ), если оно находится в нормальной форме Бойса — Кодда (НФБК), и все многозначные зависимости на самом деле представляют собой функциональные зависимости от их потенциальных ключей.

Для иллюстрации рассмотрим отношение $R(A, B, C)$. Если существует многозначная зависимость $R.A \twoheadrightarrow R.B$, то это означает, что множество значений B , соответствующее каждой паре значений A и C , зависит только от A и не зависит от C .

Допустим, у нас есть данные о ресторанах, производящих различные виды пищи, и о службах доставки, которые работают только в определенных районах города. Переменная отношения имеет составной первичный ключ, включающий три атрибута: {Ресторан, Вид пищи, Район доставки}.

Однако такая переменная отношения не удовлетворяет 4НФ, т. к. имеется следующая многозначная зависимость:

- Ресторан $\twoheadrightarrow$ Вид пищи;
- Ресторан $\twoheadrightarrow$ Район доставки.

Это означает, что при добавлении нового вида пищи необходимо будет добавить по одной новой записи для каждого района доставки. Такая ситуация может привести к логическим аномалиям, когда определенному виду пищи будут соответствовать лишь некоторые районы доставки из обслуживаемых рестораном районов.

Чтобы предотвратить отмеченную аномалию, необходимо декомпонировать отношение, разместив независимые факты в разных отношениях. В этом примере следует выполнить декомпозицию на Ресторан $\rightarrow$ Вид пищи и Ресторан $\rightarrow$ Район доставки.

Однако если к исходной переменной отношения добавить атрибут, функционально зависящий от потенциального ключа, — например, цену с учетом стоимости доставки: {Ресторан, Вид пищи, Район доставки} $\rightarrow$ Цена, то полученное отношение будет удовлетворять 4НФ, и его уже нельзя будет декомпонировать без потерь.

Пятая нормальная форма

Отношения, удовлетворяющие пятой нормальной форме (5НФ), достигают этого статуса только после выполнения требований четвертой нормальной формы (4НФ) и отсутствия сложных зависимых соединений между атрибутами.

Суть сложных зависимых соединений заключается в том, что если «Атрибут_1» зависит от «Атрибута_2», а в свою очередь «Атрибут_2» зависит от «Атрибута_3» и «Атрибут_3» зависит от «Атрибута_1», то все три атрибута обязательно должны быть объединены в один кортеж.

Такое требование представляет собой очень строгое ограничение, и на практике редко можно встретить примеры реализации этого требования в чистом виде.

Допустим, у нас есть таблица с атрибутами «Поставщик», «Товар» и «Покупатель». «Покупатель_1» покупает несколько товаров у «Поставщика_1». «Покупатель_1» также покупает новый товар у «Поставщика_2». В соответствии с требованием 5НФ, «Поставщик_1» обязан поставлять «Покупателю_1» тот же самый новый товар, а «Поставщик_2» должен предоставить «Покупателю_1», помимо нового товара, всю номенклатуру товаров «Поставщика_1». Однако в реальной жизни покупатель свободен в своем выборе товаров. В связи с этим для преодоления указанной сложности все три атрибута разделяются по разным отношениям (таблицам).

После выделения трех новых отношений: «Поставщик», «Товар» и «Покупатель» — при извлечении информации (например, о покупателях и товарах) необходимо в запросе соединить все три отношения. Важно не применять комбинацию соединения двух отношений, т. к. это может привести к извлечению некорректной информации.

Зависимости между четырьмя, пятью или более атрибутами, соответствующие пятой нормальной форме, на практике встречаются крайне редко, и показать такие примеры практически невозможно.

Доменно-ключевая нормальная форма

Переменная отношения достигает доменно-ключевой нормальной формы (ДКНФ), если каждое наложенное на нее ограничение является логическим следствием ограничений доменов и ограничений ключей, примененных к этой переменной отношения.

Ограничение домена предписывает использовать для определенного атрибута только значения из заданного домена — перечня допустимых значений того или иного типа с указанием, что соответствующий атрибут должен иметь определенный тип данных.

Ограничение ключа утверждает, что некоторый атрибут или комбинация атрибутов являются потенциальным ключом.

Любая переменная отношения, удовлетворяющая ДКНФ, автоматически соответствует и пятой нормальной форме (5НФ). Тем не менее не все переменные отношения можно привести к ДКНФ, т. к. это требование представляет собой жесткое ограничение, и на практике не всегда возможно его выполнение.

Шестая нормальная форма

Когда переменная отношения находится в шестой нормальной форме (6НФ), она удовлетворяет всем нетривиальным зависимостям соединения. Таким образом,

переменная является неприводимой и не может быть декомпозирована без потерь. Каждая переменная отношения, удовлетворяющая 6НФ, автоматически соответствует и пятой нормальной форме (5НФ).

В прошлом идея «декомпозиции до конца» рассматривалась в контексте хронологических данных, однако она не получила широкой поддержки. Тем не менее в хронологических базах данных максимально возможная декомпозиция помогает бороться с избыточностью и упрощает поддержание целостности базы данных.

Для хронологических баз данных определены специальные U -операторы, которые распаковывают отношения по указанным атрибутам, выполняют соответствующую операцию и упаковывают полученный результат. В следующем примере (табл. 4.4 «Работники») соединение проекций отношения должно производиться при помощи оператора U_JOIN .

Таблица 4.4. Работники

Таб. №	Время	Должность	Домашний адрес
6575	01-01-2000:10-02-2003	слесарь	ул. Ленина, 10
6575	11-02-2003:15-06-2006	слесарь	ул. Советская, 22
6575	16-06-2006:05-03-2009	бригадир	ул. Советская, 22

Переменная отношения «Работники» не удовлетворяет 6НФ и может быть разделена на переменные отношения «Должности работников» и «Домашние адреса работников» (см. соответствующие табл. 4.5 и 4.6).

Таблица 4.5. Должности работников

Таб. №	Время	Должность
6575	01-01-2000:10-02-2003	слесарь
6575	16-06-2006:05-03-2009	бригадир

Таблица 4.6. Домашние адреса работников

Таб. №	Время	Домашний адрес
6575	01-01-2000:10-02-2003	ул. Ленина, 10
6575	11-02-2003:15-06-2006	ул. Советская, 22

Понятия миграций и транзакций в контексте базы данных PostgreSQL

Миграции

С миграциями мы уже сталкивались в главе 1. Там мы просто сделали всё необходимое, чтобы их можно было сгенерировать, и подробно не рассмотрели, что же там происходит и зачем они нужны.

Как мы уже выяснили, база данных состоит из таблиц, связанных отношениями между собой. На первом шаге таблицы только создаются: определяется собственно таблица, описываются ее столбцы (их название и тип), указывается, какие значения ее данные могут принимать и какие связи она имеет с другими таблицами.

То есть, когда мы запускаем первую миграцию, в нашем приложении создается всё необходимое для начала работы с базой данных. Все те же самые манипуляции можно было провести и вручную: создать таблицу через IDE для баз данных и заполнить всю требуемую информацию, но когда нам понадобилось бы перенести приложение вместе с базой данных на внешний сервис, опять пришлось бы делать всё это самостоятельно.

А если нужно еще и заполнять некоторые таблицы фиксированными значениями — например, если бы у нас была таблица со списком городов? При каждом сбое или переносе инфраструктуры пришлось бы снова вручную выполнять все действия. Это крайне неудобно и несет в себе определенные риски, связанные с человеческим фактором. Например, не поменяли раскладку клавиатуры и написали в слове «city» символ «с» кириллицей — такая, казалось бы, незначительная опечатка может сломать все приложение: код просто не запустится и будет выдавать ошибку, что такой таблицы не существует. Так вот, чтобы избежать подобных казусов и не прибегать каждый раз к ручному труду, придумали миграции, которые позволяют всё это автоматизировать. Миграции дают возможность автоматически создавать таблицы, редактировать их, менять типы данных, удалять или добавлять связи с другими таблицами, причем можно настроить автоматический запуск миграций при старте приложения.

Отдельно стоит обратить внимание на то, как редактируется таблица в реляционных базах данных. Если в таблице нет никаких данных, то и проблемы нет, а когда таблицы становятся наполненными уже какой-то информацией, всё становится несколько сложнее. Предположим, что мы редактируем столбец `status`, тогда система поступает следующим образом:

1. Переименовывает текущий столбец в `status_old`.
2. Создает новый столбец `status` с нужным типом.
3. Преобразовывает данные столбца.
4. Удаляет столбец `status_old`.

Если же понадобится поменять значения в столбце — например, вы решили, что вместо статуса `OK` лучше использовать статус `COMPLETED`, то для таких случаев миграцию придется писать самостоятельно: организовать проход по всему столбцу, сделать проверку на равенство статуса со значением `OK` и, если совпадет, записать значение `COMPLETED`. Однако такой порядок применим, если у вас тип столбца `string`, — при этом никаких конфликтов не возникнет, а вот если тип столбца `enum`, порядок действий будет немного отличаться: сначала нужно будет добавить в `enum` тип `COMPLETED`, выполнить эту миграцию, затем написать миграцию по замене статуса, выполнить и ее и только потом удалить значение `OK` из `enum`. На самом деле задачи подобного рода встречаются очень часто, и такие обходные пути ограничений

нужно иметь в виду. И это также означает, что обойтись без знания SQL будет очень сложно, если предстоит работать с реляционными базами данных.

Индексы

Любая нагруженная система, работающая с большим количеством информации в базе данных, вынуждена использовать *индексы*. Это специальные объекты базы данных, предназначенные для увеличения производительности, — с их помощью можно находить нужную информацию гораздо быстрее, чем без них. Однако индексы также создают дополнительную нагрузку на СУБД в целом, поэтому применять их следует с осторожностью.

Сейчас наша база данных в сервисе Account организована таким образом, что для поиска каких-либо данных будет осуществляться проход по всем строкам таблицы, — т. е. если у нас окажется зарегистрировано десять тысяч человек, и искомый пользователь будет предпоследним в базе, то, чтобы его найти, СУБД придется перед этим перебрать 9999 записей. А если таких запросов станет много и на каждый запрос придется перебирать таблицу целиком? Это огромная нагрузка на систему. Чтобы такие банальные операции, как поиск и чтение, не сопровождались высокой нагрузкой, разработчики баз данных и придумали индексы.

Проще всего объяснить работу индексов на примере с предметным указателем книги: в технической литературе в нем обычно приводится список терминов и указывается номер страницы, где находится пояснение к этому термину. Просмотреть такой предметный указатель значительно проще, чем листать всю книгу в поисках нужного материала. Точно так же и с индексами, только вместо терминов там строки таблиц, а вместо номеров страниц — ссылки. Однако если выносить в предметный указатель всю информацию книги, ваш поиск значительно замедлится. Ведь чем больше предметный указатель, тем больше времени на это понадобится. Поэтому важно предугадать и собрать такой список терминов, который чаще всего может быть нужен читателю. Перед программистами стоит аналогичная задача: проиндексировать те поля таблицы, по которым чаще всего производится выборка.

Попробуем добавить индексы в наш сервис Account. Начнем с одного поля и проиндексируем `login`, потому что при авторизации пользователя мы ищем его только по его логину. Реализуется это с помощью миграции и SQL-кода (листинг 4.1).

Листинг 4.1. Файл `internal/migrations/20250904203817_add_account_index.go`

```
package migrations

import (
    "context"
    "database/sql"
)

func upAddAccountIndex(ctx context.Context, tx *sql.Tx) error {
    query := `CREATE INDEX ix_user_login`
```

```

    ON users (login);`
    _, err := tx.Exec(query)
    return err
}

func downAddAccountIndex(ctx context.Context, tx *sql.Tx) error {
    query := `DROP INDEX ix_account_login ON users;`
    _, err := tx.Exec(query)
    return err
}

```

Здесь мы видим уже знакомые нам методы `upAddAccountIndex` и `downAddAccountIndex`, которые позволяют соответственно создать и удалить индекс. Название для индекса можно задать любое, на свое усмотрение, однако рекомендуется, чтобы по названию было очевидно, о чем речь. В нашем случае индекс называется:

```
ix_account_login
```

Чтобы изменения вступили в силу, необходимо добавить новую миграцию в функцию `init()` в «главном» файле с миграциями (листинг 4.2).

Листинг 4.2. Файл `internal/migrations/20250825102441_initdb.go`

```

package migrations

import (
    "context"
    "database/sql"

    "github.com/pressly/goose/v3"
)

func init() {
    goose.AddNamedMigrationContext("20250825102441_initdb.go", upCreateUsersTable,
                                                                           downCreateUsersTable)
    goose.AddNamedMigrationContext("20250904203817_add_account_index.go", upAddAccountIndex,
                                                                           downAddAccountIndex)
}

func upCreateUsersTable(ctx context.Context, tx *sql.Tx) error {
    _, err := tx.Exec(`
        CREATE TABLE IF NOT EXISTS users (
            id BIGSERIAL PRIMARY KEY,
            login TEXT NOT NULL UNIQUE,
            email TEXT NOT NULL UNIQUE,
            phone TEXT,
            first_name TEXT,
            last_name TEXT,

```

```
        middle_name TEXT,
        age INT,
        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );
}

return err
}

func downCreateUsersTable(ctx context.Context, tx *sql.Tx) error {
    _, err := tx.Exec(`DROP TABLE IF EXISTS users;`)
    return err
}
```

Теперь мы можем запустить сервер, который перед стартом проверит, есть ли невыполненные миграции, и применит их:

```
make run
```

и посмотреть, что получится, — в терминале будет выведена информация о ходе выполнения миграции.

Для работы с базой данных вы можете открыть ее через IDE, а можете установить специальный плагин для PostgreSQL¹. После установки перейдите в настройки подключения и введите все данные от вашей базы данных (если не помните, можно посмотреть их в файле `.env`). Если вы всё сделали правильно, то отобразится подключение к базе данных. Дополнительные плагины такого рода отображаются в панели инструментов слева (там же, где и Git, и структура проекта).

Если индекс создан, от разработчика больше не требуется никаких манипуляций для его обновления при изменении количества записей. Каждый раз, когда вы добавляете, удаляете или видоизменяете столбцы, которые были ранее проиндексированы, индекс выстраивается заново. Таким образом, индексация положительно влияет на запросы чтения при условии, что выборка происходит по индексированным полям, но отрицательно сказывается на операциях добавления, изменения или удаления данных.

При этом индексы бывают разных типов: B-дерево, хеш, GiST, SP-GiST, GIN, BRIN и расширение bloom. Разные типы индексов реализуются разными алгоритмами, но по умолчанию создается индекс типа B-дерево, считающийся эффективным в большинстве случаев [2].

Если же нам часто приходится выполнять запросы по нескольким полям таблицы, то можно создавать составные индексы. Например, в нашем сервисе Account отдельно реализован поиск по `login` и по `phones`. При этом поиск может быть осуществлен как по отдельности, так и по обоим полям сразу. Составной индекс с этими полями с использованием тех же инструментов реализуется точно так же.

¹ См. <https://marketplace.visualstudio.com/items?itemName=cweijan.vscode-database-client2>.

Транзакции

Транзакция — это группа последовательных действий с базой данных, представляющая собой логическую единицу работы с данными. Транзакция может быть выполнена либо целиком и успешно, с соблюдением целостности данных и независимо от параллельно идущих других транзакций, либо не выполнена вообще, и тогда она не должна произвести никакого эффекта [3].

Самый банальный пример, когда следует использовать транзакции, — это работа с переводами денежных средств. Необходимо гарантировать, что средства у одного пользователя либо спишутся с баланса, и ими пополнится баланс другого пользователя, либо не спишутся, и никакого пополнения не будет. Потому что, если собой произойдет где-то посередине, может получиться, что деньги с баланса одного пользователя списались, а баланс другого не пополнился, а это грубая ошибка, и такого допускать нельзя.

Изначально функциональность транзакций реализована на уровне СУБД с помощью SQL, если мы говорим про реляционные базы данных. Существуют также библиотеки, которые сами реализуют работу с транзакциями и предоставляют свои инструменты для более удобного использования баз данных в ваших проектах без необходимости писать явный SQL-код.

ACID

Аналогично набору рекомендаций в виде нескольких нормальных форм для проектирования баз данных разработаны и требования к транзакционной системе, которые позволяют обеспечить наиболее надежную и предсказуемую ее работу.

Такой набор требований к транзакциям [4] принято представлять в виде аббревиатуры ACID:

□ Atomicity (Атомарность).

Атомарность как раз и гарантирует, что транзакция должна быть либо выполнена полностью, либо не выполнена совсем. Если бы всё работало стабильно и без ошибок, это требование было бы ни к чему, однако так не бывает, и нам регулярно приходится сталкиваться с различными сбоями: пропало соединение, база данных вышла из строя, да и просто случайно в переменную попал недопустимый тип, и это вызвало ошибку при сохранении. Поэтому всякий раз, когда мы сталкиваемся с ошибками, нам и нужна атомарность: чтобы сразу же все уже внесенные изменения были отменены, если только что-то пошло нештатно. Ранее рассмотренный пример с переводами как раз сюда и относится.

□ Consistency (Согласованность).

Это свойство тесно связано с предыдущим. Можно легко попасть в ситуацию, когда база данных оказывается неконсистентной, если вы использовали транзакцию, а набор действий, который должен быть атомарным, выполнен лишь частично. Таким образом, транзакция позволяет переводить базу данных из одного согласованного состояния в другое так, чтобы база оставалась консистентной и до выполнения транзакции, и после, независимо от результата выполнения.

❑ Isolation (Изолированность).

Во время выполнения транзакции параллельно запущенные транзакции или просто операции, работающие с базой данных, не должны оказывать влияния на результат. Из этого требования при распараллеливании запросов (а если у нас многопользовательская система, иначе и быть не может) могут возникать различные «феномены», которые решаются через уровни изоляции для транзакций и некоторыми другими способами.

❑ Durability (Надежность).

Если мы отправили подтверждение от системы, что операция выполнена успешно, то никакие непредвиденные обстоятельства и внешние факторы не должны этого изменить.

Параллельные транзакции

Есть несколько эффектов, к которым могут приводить параллельные транзакции, — в первую очередь это связано с *состоянием гонки* (race condition).

Потерянная запись (lost update)

Когда несколько транзакций обращаются к одной и той же записи, может возникнуть ситуация, что одна транзакция после фиксации изменений перезаписала данные, обновленные и зафиксированные другой транзакцией.

Например, один и тот же условно общий семейный счет пополняют два клиента. На балансе до начала транзакций находится 5000 рублей. Петя пополняет счет на 500 рублей, а Саша — на 2000. Если окажется, что пополнение Пети происходит в момент, когда транзакция Саши еще не закрыта, т. е. на счету находится все еще 5000 рублей (а мы помним, что результат транзакции фиксируется только тогда, когда она выполнена полностью), то его транзакция перезапишет данные пополнения Саши. В результате на счету окажется 5500 рублей, а не 7500, как ожидалось. Получается, мы потеряли пополнение Саши: кто последний успел, того транзакция и засчиталась, — а так быть не должно.

Грязное чтение (dirty read)

Принцип тот же, что и с потерянной записью. Петя оплатил покупку с общего счета, а в момент, когда проходила транзакция, Саша — перед тем как сделать заказ в ресторане — решила проверить их общий баланс. В итоге Саша была уверена, что на счету у них 3000 рублей, а на самом деле оставалось уже только 500. Позже, когда понадобится оплатить счет в ресторане, у нее, вероятно, могут возникнуть проблемы.

Неповторяющееся чтение (non-repeatable read)

При повторном чтении одних и тех же данных в рамках одной транзакции оказывается, что другая транзакция уже успела изменить и зафиксировать эти данные. Таким образом, один и тот же запрос выдает разные результаты.

Фантомное чтение (phantom read)

Саша решила составить отчет о месячных тратах за месяц по общему счету с Петей. Пока выполнялась транзакция Саши на составление отчета, Петя несколько раз успешно расплатился с этого счета в магазине. В результате, если Саша запросит отчет заново, у нее получится другой результат и другое количество операций. Собственно, разница тут между грязным чтением и неповторяющимся чтением состоит в том, что там данные изменяются, а тут добавляются/удаляются, т. е. изменяется еще и их количество.

Аномалия сериализации (serialization anomaly)

Результат успешной фиксации группы транзакций, выполняющихся параллельно, не совпадает с результатом ни одного из возможных вариантов упорядочения этих транзакций, если бы они выполнялись последовательно.

Уровни изоляции транзакций в SQL

На разных уровнях изоляции решаются различные проблемы, связанные с параллельным выполнением транзакций [5].

☐ Read Uncommitted (чтение незафиксированных данных).

Самый низкий уровень изоляции транзакций. По стандарту SQL на этом уровне допускается чтение незафиксированных данных.

☐ Read Committed (чтение фиксированных данных).

На этом уровне решается проблема грязного чтения путем чтения только фиксированных данных. Если первая транзакция в процессе выполнения изменила некоторый набор данных, то вторая транзакция, которой необходимо обратиться к этим же данным, должна дождаться завершения первой транзакции.

☐ Repeatable Read (повторяемое чтение).

Решение проблемы потерянных изменений заключается в том, что данные, прочитанные одной транзакцией, запрещено менять до ее завершения. Остальные транзакции должны дождаться завершения первой, перед тем как получить доступ к данным.

☐ Serializable (упорядоченное чтение).

Этот уровень предотвращает эффект фантомного чтения. Транзакция выполняется строго последовательно, и запрещается обновлять, добавлять и удалять записи, попадающие под условия запроса. Если в запросе необходимы данные всей таблицы, то таблица целиком становится недоступна для других транзакций до завершения первой.

Отдельно стоит отметить, что в PostgreSQL стандарты немного отличаются от общепринятых. Например, на уровне Read Uncommitted не допускается чтение незафиксированных данных. Отличаются и требования к уровню Repeatable Read — там не допускается фантомное чтение.

Разработка модуля Transaction

Микросервис Transaction будет содержать информацию о проведенных пользователем операциях. В рамках этого мы реализуем транзакции на уровне СУБД, а для примера возьмем ситуацию, когда надо выполнить перевод средств. Как правило, в такой операции происходят сразу две операции: по списанию средств и по их пополнению. Соответственно нужно, чтобы эти два действия либо строго выполнились, либо строго не выполнились.

Итак, создадим репозиторий Transaction по образу и подобию сервиса Account — также с подключением PostgreSQL.

Первое, с чего мы начнем, — это отредактируем Docker-Compose под новые требования. Теперь у нас будут три базы данных, а контейнер для них один, чтобы было удобнее при локальной разработке (листинг 4.3).

Листинг 4.3. Файл Docker-compose.yaml

```
services:
  account:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: account
    ports:
      - "5432:5432"
    volumes:
      - postgres_account_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  auth:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: auth
    ports:
      - "5433:5432"
    volumes:
      - postgres_auth_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
```

```
    interval: 5s
    timeout: 5s
    retries: 5
networks:
  - app-network

transaction:
  image: postgres:15-alpine
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: transaction
  ports:
    - "5434:5432"
  volumes:
    - postgres_transaction_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - app-network

volumes:
  postgres_account_data:
  postgres_auth_data:
  postgres_transaction_data:

networks:
  app-network:
    driver: bridge
```

Здесь мы добавили сервис для базы данных `transaction` — аналогично тому, как уже добавляли для `auth`. Важная деталь: порты для сервисов, по которому к ним можно подключаться, должны различаться, поэтому у `auth` порт 5432, у `account` — 5433, а у `transaction` — 5434.

Теперь перейдем непосредственно к разработке сервиса `Transaction`. Этот сервис будет отвечать только за создание, хранение и выдачу информации о транзакциях. В основу контрактов возьмем два ключевых объекта, первый — сама транзакция, она отражает факт операции, содержит уникальный идентификатор, тип (перевод, пополнение или списание), текущее состояние и временные метки. Второй объект — движение по счету, или `entry`. Оно фиксирует конкретное изменение на указанном счете, указывает направление (дебет или кредит), размер суммы и связь

с транзакцией. Такое разделение дает нам возможность моделировать и простые операции вроде пополнений и списаний, и более сложные, например переводы, которые включают одновременно две операции: уменьшение средств отправителя и увеличение баланса получателя.

В результате такого подхода получится универсальная схема: каждое действие фиксируется транзакцией, и каждая транзакция состоит из одной или нескольких записей движений, а все взаимодействие с транзакциями будет собрано в сервис Transaction.

В первую очередь опишем контракты, по которым будет работать сервис, а также создадим новый каталог transaction для этого (листинги 4.4 и 4.5).

Листинг 4.4. Contracts: transaction/transaction_service.proto

```
syntax = "proto3";

package transaction;

option go_package = "github.com/yuliaapopova/contracts/transaction/go;transaction";

import "google/protobuf/timestamp.proto";
import "google/protobuf/empty.proto";
import "transaction_model.proto";
import "pagination/pagination.proto";

service TransactionService {
  rpc Deposit(DepositRequest) returns (google.protobuf.Empty);
  rpc Withdraw(WithdrawRequest) returns (google.protobuf.Empty);
  rpc Transfer(TransferRequest) returns (google.protobuf.Empty);
  rpc GetTransactions(GetTransactionsRequest) returns (GetTransactionsResponse);
}

message DepositRequest {
  uint64 user_id = 1;
  uint64 amount = 2;
}

message WithdrawRequest {
  uint64 user_id = 1;
  uint64 amount = 2;
}

message TransferRequest {
  uint64 user_id = 1;
  uint64 amount = 2;
  uint64 recipient = 3;
}
```

```
message GetTransactionsRequest {
    optional uint64 user_id = 1;
    optional string type = 2;
    optional string status = 3;
    optional google.protobuf.Timestamp date_from = 4; // фильтр по дате от
    optional google.protobuf.Timestamp date_to = 5;   // фильтр по дате до
    optional pagination.Pagination pagination = 6;
}

message GetTransactionsResponse {
    repeated TransactionDetails transactions = 1;
}
```

Листинг 4.5. Contracts: transaction/transaction_model.proto

```
syntax = "proto3";

package transaction;

option go_package = "github.com/yuliaapopova/contracts/transaction/go;transaction";

import "google/protobuf/timestamp.proto";

message Transaction {
    string id = 1;
    string type = 2; // TRANSFER | DEPOSIT | WITHDRAW
    string status = 3; // INITIATED | DONE | FAILED
    google.protobuf.Timestamp created_at = 4;
    google.protobuf.Timestamp updated_at = 5;
}

message TransactionEntry {
    string id = 1;
    string transaction_id = 2;
    string account_id = 3;
    string direction = 4; // DEBIT | CREDIT
    double amount = 5;
}

message TransactionDetails {
    Transaction transaction = 1;
    repeated TransactionEntry entries = 2;
}
```

Тут мы реализуем минимально необходимый набор вызовов: пополнение, списание, перевод и получение списка транзакций с фильтрацией. Теперь можем перейти к разработке самого сервиса.

Так как мы определились, что будем использовать, так называемую «бухгалтерскую» двойную запись с направлением, опишем структуры данных в Go (листинг 4.6).

Листинг 4.6. Transaction: internal/model/transaction.go

```
package model

import "time"

// TransactionStatus представляет статус транзакции
type TransactionStatus string

const (
    TransactionStatusPending TransactionStatus = "pending"
    TransactionStatusCompleted TransactionStatus = "completed"
    TransactionStatusFailed TransactionStatus = "failed"
    TransactionStatusCancelled TransactionStatus = "cancelled"
)

type TransactionType string

const (
    TransactionTypeDeposit TransactionType = "deposit"
    TransactionTypeWithdraw TransactionType = "withdraw"
    TransactionTypeTransfer TransactionType = "transfer"
)

type Transaction struct {
    ID          uint64
    UserID      uint64
    Amount      int64 // сумма в копейках
    Status      TransactionStatus
    Type        TransactionType
    CreatedAt   time.Time
    UpdatedAt   time.Time
}

type UpdateTransaction struct {
    Status TransactionStatus
}

// TransactionEntryDirection представляет направление записи транзакции
type TransactionEntryDirection string

const (
    TransactionEntryDirectionDebit TransactionEntryDirection = "DEBIT"
    TransactionEntryDirectionCredit TransactionEntryDirection = "CREDIT"
)
```

```
// TransactionEntry представляет запись транзакции (дебет/кредит)
type TransactionEntry struct {
    ID            uint64
    TransactionID  uint64
    AccountID     uint64
    Direction     TransactionEntryDirection
    Amount        float64
    CreatedAt     time.Time
    UpdatedAt     time.Time
}

// CreateTransactionEntry представляет данные для создания записи транзакции
type CreateTransactionEntry struct {
    TransactionID uint64
    AccountID     uint64
    Direction     TransactionEntryDirection
    Amount        float64
}

// TransactionDetails представляет детали транзакции с записями
type TransactionDetails struct {
    Transaction Transaction
    Entries      []TransactionEntry
}

// GetTransactionsParams представляет параметры для получения транзакций
type GetTransactionsParams struct {
    UserID  *uint64
    Type    *string
    Status  *string
    DateFrom *time.Time
    DateTo   *time.Time
    Limit    int
    Offset   int
}

// DepositParams представляет параметры для операции депозита
type DepositParams struct {
    UserID uint64
    Amount float64
}

// WithdrawParams представляет параметры для операции снятия средств
type WithdrawParams struct {
    AccountID uint64
    Amount    float64
}
```

```
// TransferParams представляет параметры для операции перевода средств
type TransferParams struct {
    UserID      uint64
    Recipient   uint64
    Amount      float64
}
```

Помимо основных моделей, тут также добавлены структуры для передачи параметров в репозиторий: `DepositParams`, `WithdrawParams`, `TransferParams`, `GetTransactionsParams`. Это частая практика — создавать структуры под параметры, когда они сложнее, чем просто передать одно поле (листинг 4.7).

Листинг 4.7. Transaction: `internal/migrations/20250825102441_initdb.go`

```
package migrations

import (
    "context"
    "database/sql"

    "github.com/pressly/goose/v3"
)

func init() {
    goose.AddNamedMigrationContext("20250825102441_initdb.go", upCreateTransactionsTable,
        downCreateTransactionsTable)
}

func upCreateTransactionsTable(ctx context.Context, tx *sql.Tx) error {
    _, err := tx.Exec(`
        CREATE TABLE IF NOT EXISTS transactions (
            id BIGSERIAL PRIMARY KEY,
            user_id BIGINT NOT NULL,
            amount BIGINT NOT NULL,
            type TEXT NOT NULL CHECK (type IN {'transfer', 'deposit', 'withdraw'}),
            status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN {'pending', 'completed',
                'failed', 'cancelled'}),
            created_at TIMESTAMP NOT NULL DEFAULT NOW(),
            updated_at TIMESTAMP NOT NULL DEFAULT NOW()
        );
    `)
    if err != nil {
        return err
    }

    _, err = tx.Exec(`
        CREATE TABLE IF NOT EXISTS transaction_entries (
            id BIGSERIAL PRIMARY KEY,
```



```

        transaction_id BIGINT NOT NULL,
        account_id BIGINT NOT NULL,
        direction TEXT NOT NULL CHECK (direction IN ('DEBIT', 'CREDIT')),
        amount DECIMAL(20,2) NOT NULL,
        created_at TIMESTAMP NOT NULL DEFAULT NOW(),
        updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
        FOREIGN KEY (transaction_id) REFERENCES transactions(id) ON DELETE CASCADE
    );
}

if err != nil {
    return err
}

_, err = tx.Exec(`
    CREATE INDEX IF NOT EXISTS idx_transaction_entries_transaction_id ON
                                                transaction_entries(transaction_id);
    CREATE INDEX IF NOT EXISTS idx_transaction_entries_account_id ON
                                                transaction_entries(account_id);
    CREATE INDEX IF NOT EXISTS idx_transaction_entries_direction ON
                                                transaction_entries(direction);
`)
if err != nil {
    return err
}

_, err = tx.Exec(`
    CREATE INDEX IF NOT EXISTS idx_transactions_user_id ON transactions(user_id);
    CREATE INDEX IF NOT EXISTS idx_transactions_status ON transactions(status);
    CREATE INDEX IF NOT EXISTS idx_transactions_created_at ON transactions(created_at);
`)
return err
}

func downCreateTransactionsTable(ctx context.Context, tx *sql.Tx) error {
    queries := []string{
        `DROP INDEX idx_transaction_entries_transaction_id ON transaction_entries;`,
        `DROP INDEX idx_transaction_entries_account_id ON transaction_entries;`,
        `DROP INDEX idx_transaction_entries_direction ON transaction_entries;`,
        `DROP INDEX idx_transactions_user_id ON transactions;`,
        `DROP INDEX idx_transactions_status ON transactions;`,
        `DROP INDEX idx_transactions_created_at ON transactions;`,
        `ALTER TABLE transaction_entries DROP PRIMARY KEY;`,
        `ALTER TABLE transactions DROP PRIMARY KEY;`,
    }

    for _, q := range queries {
        _, err := tx.Exec(q)
    }
}

```

```
        if err != nil {
            return err
        }
    }

    queries = []string{
        `DROP TABLE IF EXISTS transactions;`,
        `DROP TABLE IF EXISTS transaction_entries;`,
    }

    for _, q := range queries {
        _, err := tx.Exec(q)
        if err != nil {
            return err
        }
    }
    return nil
}
```

Такая миграция описывает схему таблиц для хранения транзакций и записей, а также создает набор индексов для ускорения выборов по ключевым полям (`user_id`, `status`, `created_at`). Важно отметить порядок действий при откате: сначала должны удаляться все индексы и первичные ключи, чтобы разорвать их зависимость от таблиц, и только потом сами таблицы. Такой подход гарантирует корректное выполнение миграции и предотвращает ошибки, возникающие при попытке удалить таблицу, к которой всё еще привязаны индексы.

Чтобы дальше было проще понять код репозитория, посмотрим еще и на `go`-структуру, через которую будем взаимодействовать с базой данных (листинг 4.8).

Листинг 4.8. Transaction: `internal/repository/model/model.go`

```
package model

import "time"

type Transaction struct {
    ID          uint64    `gorm:"column:id"`
    UserID      uint64    `gorm:"column:user_id"`
    Amount      int64     `gorm:"column:amount"`
    Status      string    `gorm:"column:status"`
    Type        string    `gorm:"column:type"`
    CreatedAt   time.Time `gorm:"column:created_at"`
    UpdatedAt   time.Time `gorm:"column:updated_at"`
}

func (Transaction) TableName() string {
    return "transactions"
}
```

```

type TransactionEntry struct {
    ID            uint64    `gorm:"column:id"`
    TransactionID uint64    `gorm:"column:transaction_id"`
    AccountID     uint64    `gorm:"column:account_id"`
    Direction     string    `gorm:"column:direction"`
    Amount        float64   `gorm:"column:amount"`
    CreatedAt     time.Time `gorm:"column:created_at"`
    UpdatedAt     time.Time `gorm:"column:updated_at"`
}

func (TransactionEntry) TableName() string {
    return "transaction_entries"
}

```

Теперь всё готово, чтобы перейти к коду репозитория. Здесь будет вся логика работы транзакций: не только чтение и фильтрация данных, но и операции снятия, пополнения и перевода средств. Это можно было бы сделать и в сервисном слое, но нам важна атомарность этих операций, поэтому принципиально оборачивать такие действия в механизм транзакции базы данных, чтобы гарантировать целостность и согласованность данных (листинг 4.9).

Листинг 4.9. Transaction: internal/repository/repository.go

```

package repository

import (
    "context"
    "errors"
    "fmt"

    "transaction/internal/model"
    "transaction/internal/repository/mapper"
    repomodel "transaction/internal/repository/model"

    "github.com/rs/zerolog"
    "gorm.io/gorm"
    "gorm.io/gorm/clause"
)

type Repository struct {
    db      *gorm.DB
    logger  *zerolog.Logger
}

func NewRepository(db *gorm.DB, logger *zerolog.Logger) *Repository {
    return &Repository{
        db:      db,
        logger:  logger,
    }
}

```

```
func (r *Repository) GetTransactions(ctx context.Context, params model.GetTransactionsParams)
([]model.Transaction, error) {
    var transactions []repomodel.Transaction

    query := r.db.WithContext(ctx).Model(&repomodel.Transaction{})

    // Применяем фильтры
    if params.UserID != nil {
        query = query.Where("user_id = ?", *params.UserID)
    }
    if params.Type != nil {
        query = query.Where("type = ?", *params.Type)
    }
    if params.Status != nil {
        query = query.Where("status = ?", *params.Status)
    }
    if params.DateFrom != nil {
        query = query.Where("created_at >= ?", *params.DateFrom)
    }
    if params.DateTo != nil {
        query = query.Where("created_at <= ?", *params.DateTo)
    }

    // Применяем пагинацию и сортировку
    res := query.
        Offset(params.Offset).
        Limit(params.Limit).
        Order("created_at DESC").
        Find(&transactions)

    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get transactions")
        return nil, fmt.Errorf("failed to get transactions: %w", res.Error)
    }
    return mapper.RepoTransactionsToTransactions(transactions), nil
}

func (r *Repository) GetTransactionDetails(ctx context.Context, transactionID uint64)
(model.TransactionDetails, error) {
    // Получаем транзакцию
    var transaction repomodel.Transaction
    res := r.db.WithContext(ctx).
        Model(&repomodel.Transaction{}).
        Where("id = ?", transactionID).
        First(&transaction)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return model.TransactionDetails{}, fmt.Errorf("transaction not found")
    }
}
```

```

    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get transaction")
        return model.TransactionDetails{}, fmt.Errorf("failed to get transaction: %w", res.Error)
    }

    // Получаем записи транзакции
    var entries []repo.TransactionEntry
    res = r.db.WithContext(ctx).
        Model(&repo.TransactionEntry{}).
        Where("transaction_id = ?", transactionID).
        Order("created_at ASC").
        Find(&entries)

    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get transaction entries")
        return model.TransactionDetails{}, fmt.Errorf("failed to get transaction entries: %w", res.Error)
    }

    return model.TransactionDetails{
        Transaction: mapper.RepoTransactionToTransaction(transaction),
        Entries:     mapper.RepoTransactionEntriesToTransactionEntries(entries),
    }, nil
}

// Deposit выполняет операцию депозита в рамках одной транзакции БД
func (r *Repository) Deposit(ctx context.Context, params model.DepositParams) (model.TransactionDetails, error) {
    var result model.TransactionDetails

    err := r.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        // Создаем транзакцию
        transaction := model.Transaction{
            UserID: params.UserID,
            Amount: int64(params.Amount * 100), // конвертируем в копейки
            Status: model.TransactionStatusPending,
            Type:   model.TransactionTypeDeposit,
        }

        transactionRepo := mapper.TransactionToRepoTransaction(transaction)
        res := tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&transactionRepo)
        if res.Error != nil {
            r.logger.Err(res.Error).Msg("failed to create transaction in deposit")
            return fmt.Errorf("failed to create transaction: %w", res.Error)
        }
    })
}

```

```

// Создаем запись дебета (увеличение баланса)
entry := model.TransactionEntry{
    TransactionID: transactionRepo.ID,
    AccountID:     params.UserID,
    Direction:     model.TransactionEntryDirectionDebit,
    Amount:        params.Amount,
}

entryRepo := mapper.TransactionEntryToRepoTransactionEntry(entry)
res = tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&entryRepo)
if res.Error != nil {
    r.logger.Err(res.Error).Msg("failed to create transaction entry in deposit")
    return fmt.Errorf("failed to create transaction entry: %w", res.Error)
}

// Обновляем статус транзакции на completed
updateData := mapper.UpdateTransactionToRepoTransaction(model.UpdateTransaction{
    Status: model.TransactionStatusCompleted,
})
res = tx.Where("id = ?", transactionRepo.ID).Updates(&updateData)
if res.Error != nil {
    r.logger.Err(res.Error).Msg("failed to update transaction status in deposit")
    return fmt.Errorf("failed to update transaction status: %w", res.Error)
}

// Получаем детали созданной транзакции
transactionRepo.Status = string(model.TransactionStatusCompleted)
result = model.TransactionDetails{
    Transaction: mapper.RepoTransactionToTransaction(transactionRepo),
    Entries:
        []model.TransactionEntry{mapper.RepoTransactionEntryToTransactionEntry(entryRepo)},
}

return nil
})

if err != nil {
    return model.TransactionDetails{}, err
}

return result, nil
}

// Withdraw выполняет операцию снятия средств в рамках одной транзакции БД
func (r *Repository) Withdraw(ctx context.Context, params model.WithdrawParams)
(model.TransactionDetails, error) {
    var result model.TransactionDetails

```

```

err := r.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
    // Создаем транзакцию
    transaction := model.Transaction{
        UserID: params.AccountID,          // UserID теперь - это accountID
        Amount: int64(params.Amount * 100), // конвертируем в копейки
        Status: model.TransactionStatusPending,
        Type:   model.TransactionTypeWithdraw,
    }

    transactionRepo := mapper.TransactionToRepoTransaction(transaction)
    res := tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&transactionRepo)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to create transaction in withdraw")
        return fmt.Errorf("failed to create transaction: %w", res.Error)
    }

    // Создаем запись кредита (уменьшение баланса)
    entry := model.TransactionEntry{
        TransactionID: transactionRepo.ID,
        AccountID:     params.AccountID,
        Direction:     model.TransactionEntryDirectionCredit,
        Amount:        params.Amount,
    }

    entryRepo := mapper.TransactionEntryToRepoTransactionEntry(entry)
    res = tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&entryRepo)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to create transaction entry in withdraw")
        return fmt.Errorf("failed to create transaction entry: %w", res.Error)
    }

    // Обновляем статус транзакции на completed
    updateData := mapper.UpdateTransactionToRepoTransaction(model.UpdateTransaction{
        Status: model.TransactionStatusCompleted,
    })
    res = tx.Where("id = ?", transactionRepo.ID).Updates(&updateData)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to update transaction status in withdraw")
        return fmt.Errorf("failed to update transaction status: %w", res.Error)
    }

    // Получаем детали созданной транзакции
    transactionRepo.Status = string(model.TransactionStatusCompleted)
    result = model.TransactionDetails{
        Transaction: mapper.RepoTransactionToTransaction(transactionRepo),
        Entries:     []model.TransactionEntry{mapper.RepoTransactionEntryToTransactionEntry(entryRepo)},
    }
}

```

```
    return nil
  })

  if err != nil {
    return model.TransactionDetails{}, err
  }

  return result, nil
}

// Transfer выполняет операцию перевода средств в рамках одной транзакции БД
func (r *Repository) Transfer(ctx context.Context, params model.TransferParams)
(model.TransactionDetails, error) {
  var result model.TransactionDetails

  err := r.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
    // Создаем транзакцию
    transaction := model.Transaction{
      UserID: params.UserID,           // UserID - автор перевода
      Amount: int64(params.Amount * 100), // конвертируем в копейки
      Status: model.TransactionStatusPending,
      Type:   model.TransactionTypeTransfer,
    }

    transactionRepo := mapper.TransactionToRepoTransaction(transaction)
    res := tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&transactionRepo)
    if res.Error != nil {
      r.logger.Err(res.Error).Msg("failed to create transaction in transfer")
      return fmt.Errorf("failed to create transaction: %w", res.Error)
    }

    // Создаем запись кредита для счета отправителя (уменьшение баланса)
    creditEntry := model.TransactionEntry{
      TransactionID: transactionRepo.ID,
      AccountID:     params.UserID,
      Direction:     model.TransactionEntryDirectionCredit,
      Amount:        params.Amount,
    }

    creditEntryRepo := mapper.TransactionEntryToRepoTransactionEntry(creditEntry)
    res = tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&creditEntryRepo)
    if res.Error != nil {
      r.logger.Err(res.Error).Msg("failed to create credit transaction entry in transfer")
      return fmt.Errorf("failed to create credit transaction entry: %w", res.Error)
    }
  })
}
```



```

// Создаем запись дебета для счета получателя (увеличение баланса)
debitEntry := model.TransactionEntry{
    TransactionID: transactionRepo.ID,
    AccountID:     params.Recipient,
    Direction:     model.TransactionEntryDirectionDebit,
    Amount:        params.Amount,
}

debitEntryRepo := mapper.TransactionEntryToRepoTransactionEntry(debitEntry)
res = tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&debitEntryRepo)
if res.Error != nil {
    r.logger.Err(res.Error).Msg("failed to create debit transaction entry in transfer")
    return fmt.Errorf("failed to create debit transaction entry: %w", res.Error)
}

// Обновляем статус транзакции на completed
updateData := mapper.UpdateTransactionToRepoTransaction(model.UpdateTransaction{
    Status: model.TransactionStatusCompleted,
})
res = tx.Where("id = ?", transactionRepo.ID).Updates(&updateData)
if res.Error != nil {
    r.logger.Err(res.Error).Msg("failed to update transaction status in transfer")
    return fmt.Errorf("failed to update transaction status: %w", res.Error)
}

// Получаем детали созданной транзакции
transactionRepo.Status = string(model.TransactionStatusCompleted)
result = model.TransactionDetails{
    Transaction: mapper.RepoTransactionToTransaction(transactionRepo),
    Entries: []model.TransactionEntry{
        mapper.RepoTransactionEntryToTransactionEntry(creditEntryRepo),
        mapper.RepoTransactionEntryToTransactionEntry(debitEntryRepo),
    },
}

return nil
})

if err != nil {
    return model.TransactionDetails{}, err
}

return result, nil
}

```

Обратите внимание, что у нас нет методов для удаления транзакций, т. к. они нам не понадобятся, — ведь работа с финансами всегда должна быть зафиксирована,

поэтому мы реализовали в Repository только методы, которые создают, обновляют или извлекают транзакции. Согласно паттерну YAGNI (*англ.* You ain't gonna need it, что переводится как «Вам это не понадобится») следует реализовывать только ту функциональность, которая нужна прямо сейчас. Создавать функции, которые не факт что когда-либо понадобятся, считается плохой практикой, потому что это захламляет код, а в дальнейшем может быть уже трудно определить, нужны они, или их можно удалить.

Во всех операциях связанных с добавлением/изменением данных, логика построена по одному принципу: сначала создается запись о самой транзакции в статусе pending, затем добавляются связанные движения по счетам (дебит или кредит — в зависимости от операции). После успешного выполнения всех действий статус обновляется на completed, а наружу возвращаются уже готовые детали транзакции. Такой подход обеспечивает консистентность данных и позволяет безопасно работать с финансовыми операциями, минимизируя риск рассинхронизации между основной транзакцией и ее проводками (листинг 4.10).

Листинг 4.10. Transaction: internal/service/service.go

```
package service

import (
    "context"

    "transaction/internal/model"

    "github.com/rs/zerolog"
)

type TransactionService struct {
    repo    Repository
    logger  *zerolog.Logger
}

func New(repo Repository, logger *zerolog.Logger) *TransactionService {
    return &TransactionService{repo: repo, logger: logger}
}

type Repository interface {
    // Transaction methods
    GetTransactions(context.Context, model.GetTransactionsParams) ([]model.Transaction, error)

    // TransactionEntry methods
    GetTransactionDetails(context.Context, uint64) (model.TransactionDetails, error)

    // Business operations
    Deposit(context.Context, model.DepositParams) (model.TransactionDetails, error)
```

```

    Withdraw(context.Context, model.WithdrawParams) (model.TransactionDetails, error)
    Transfer(context.Context, model.TransferParams) (model.TransactionDetails, error)
}

func (s *TransactionService) GetTransactionsWithDetails(ctx context.Context, params
model.GetTransactionsParams) ([]model.TransactionDetails, error) {
    // Получаем список транзакций с фильтрацией
    transactions, err := s.repo.GetTransactions(ctx, params)
    if err != nil {
        s.logger.Error().Err(err).Msg("failed to get transactions")
        return nil, err
    }

    // Для каждой транзакции получаем детали
    var transactionDetails []model.TransactionDetails
    for _, transaction := range transactions {
        details, err := s.repo.GetTransactionDetails(ctx, transaction.ID)
        if err != nil {
            s.logger.Error().Err(err).Uint64("transaction_id", transaction.ID).Msg("failed to
            get transaction details")
            continue
        }
        transactionDetails = append(transactionDetails, details)
    }

    return transactionDetails, nil
}

// Deposit Business logic methods
func (s *TransactionService) Deposit(ctx context.Context, userID uint64, amount float64)
(model.TransactionDetails, error) {
    params := model.DepositParams{
        UserID: userID,
        Amount: amount,
    }

    return s.repo.Deposit(ctx, params)
}

func (s *TransactionService) Withdraw(ctx context.Context, accountID uint64, amount float64)
(model.TransactionDetails, error) {
    params := model.WithdrawParams{
        AccountID: accountID,
        Amount:    amount,
    }

    return s.repo.Withdraw(ctx, params)
}

```

```
func (s *TransactionService) Transfer(ctx context.Context, userID, recipient uint64, amount
float64) (model.TransactionDetails, error) {
    params := model.TransferParams{
        UserID:    userID,
        Recipient: recipient,
        Amount:    amount,
    }

    return s.repo.Transfer(ctx, params)
}
```

При таком подходе к репозиторию сервисный слой у нас получится теперь совсем простым.

Интеграция Transaction и Account

Мы реализовали сервис с транзакциями, однако содержащийся в нем список операций сейчас ни на что не влияет, а должен быть напрямую связан с балансом. В больших системах принято разделять логику с учетными записями о пользователях и их баланс, но у нас цель немного другая, и мы позволим себе объединить информацию об учетной записи пользователя с его «кошельком». Поэтому добавим в таблицу User в сервисе Account информацию о балансе. И соответственно отредактируем все связанные с этим методы (листинг 4.11).

Листинг 4.11. Contracts: account/account_model.proto

```
syntax = "proto3";

package account;
option go_package = "github.com/yuliaapopova/contracts/account/go;account";

import "google/protobuf/timestamp.proto";

message CreateUser {
    string login = 1;
    string phone = 2;
    string first_name = 3;
    string last_name = 4;
    string middle_name = 5;
    string email = 6;
    uint32 age = 7;
    float balance = 8;
}

message User {
    uint64 id = 1;
    string login = 2;
```

```

    string phone = 3;
    string first_name = 4;
    string last_name = 5;
    string middle_name = 6;
    string email = 7;
    uint32 age = 8;
    google.protobuf.Timestamp created_at = 9;
    google.protobuf.Timestamp updated_at = 10;
    float balance = 11;
    bool is_deleted = 12;
}

```

Помимо баланса, мы добавили сюда также поле `isDeleted`, по которому будем отличать активных пользователей от удаленных. Это нужно для того, чтобы в какой-то момент у нас не оказались транзакции с `userId`, которых не существует. Поэтому теперь мы будем не удалять полностью пользователей, а менять значение флага `isDeleted` на `true`. По умолчанию значение `isDeleted` должно быть `false`, а баланс равен 0.

Сгенерируем и выполним миграцию для дальнейших работ (листинг 4.12).

Листинг 4.12. Account: internal/migrations/ 20250906123744_add_balance_to_users.go

```

package migrations

import (
    "context"
    "database/sql"
)

func upAddBalanceToUsers(ctx context.Context, tx *sql.Tx) error {
    _, err := tx.Exec(`
        ALTER TABLE users
        ADD COLUMN balance DECIMAL(15,2) NOT NULL DEFAULT 0.00,
        ADD COLUMN is_deleted BOOLEAN NOT NULL DEFAULT FALSE;
    `)
    return err
}

func downAddBalanceToUsers(ctx context.Context, tx *sql.Tx) error {
    _, err := tx.Exec(`
        ALTER TABLE users
        DROP COLUMN balance,
        DROP COLUMN is_deleted;
    `)
    return err
}

```

Подробнее останавливаться на созданной миграции нет смысла — в ней ничего интересного нет. А нам теперь надо отредактировать `UserRepository` и учесть, что удаленных пользователей возвращать мы не должны (листинг 4.13).

Листинг 4.13. Account: internal/repository/repository.go

```
package repository

import (
    "context"
    "errors"
    "fmt"

    "account/internal/model"
    "account/internal/repository/mapper"
    repomodel "account/internal/repository/model"

    "github.com/rs/zerolog"
    "gorm.io/gorm"
    "gorm.io/gorm/clause"
)

type Repository struct {
    db      *gorm.DB
    logger  *zerolog.Logger
}

func NewRepository(db *gorm.DB, logger *zerolog.Logger) *Repository {
    return &Repository{
        db:      db,
        logger:  logger,
    }
}

func (r *Repository) CreateUser(ctx context.Context, user model.User) (model.User, error) {
    userRepo := mapper.UserToRepoUser(user)
    fmt.Println(userRepo)
    res := r.db.WithContext(ctx).
        Clauses(clause.OnConflict{UpdateAll: true}).
        //Clauses(clause.Returning{}).
        Create(&userRepo)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to save user")
        return model.User{}, fmt.Errorf("failed to save user: %w", res.Error)
    }
    fmt.Println(userRepo)
    return mapper.RepoUserToUser(userRepo), nil
}
```

```

func (r *Repository) GetUser(ctx context.Context, userID uint64) (model.User, error) {
    var user repomodel.User
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("id = ? AND is_deleted = ?", userID, false).
        First(&user)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return model.User{}, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user id")
    }
    return mapper.RepoUserToUser(user), nil
}

func (r *Repository) GetUsers(ctx context.Context, limit int, offset int) ([]model.User,
error) {
    var users []repomodel.User
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("is_deleted = ?", false).
        Offset(offset).
        Limit(limit).
        Find(&users)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return nil, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user id")
    }
    return mapper.RepoUsersToUsers(users), nil
}

func (r *Repository) DeleteUser(ctx context.Context, userID uint64) error {
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("id = ?", userID).
        Update("is_deleted", true)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to delete user")
        return fmt.Errorf("failed to delete user")
    }
    return nil
}

func (r *Repository) UpdateUser(ctx context.Context, userID uint64, user model.UpdateUser)
error {
    repoUser := mapper.UpdateUserToRepoUser(user)

```

```

res := r.db.WithContext(ctx).
    Model(&repomodel.User{}).
    Where("id = ? AND is_deleted = ?", userID, false).
    Updates(repoUser)
if res.Error != nil {
    r.logger.Err(res.Error).Msg("failed to update user")
    return fmt.Errorf("failed to update user")
}
return nil
}

```

Сам сервис и контроллер при этом редактировать не требуется — сделанного достаточно, чтобы логика работала корректно. Остается только отредактировать мапперы — чтобы в ответе возвращались еще и новые поля (листинги 4.14 и 4.15).

Листинг 4.14. Account: internal/mapper/mapper.go

```

package mapper

import (
    accountpb "github.com/yuliaapopova/contracts/account/go"

    "google.golang.org/protobuf/types/known/timestamppb"

    "account/internal/model"
)

func PbToUser(userpb *accountpb.User) model.User {
    return model.User{
        ID:          userpb.Id,
        Login:       userpb.Login,
        Email:       userpb.Email,
        Phone:       userpb.Phone,
        FirstName:   userpb.FirstName,
        LastName:    userpb.LastName,
        MiddleName:  userpb.MiddleName,
        Age:         userpb.Age,
        Balance:     float32(userpb.Balance),
        IsDeleted:   userpb.IsDeleted,
        CreatedAt:   userpb.CreatedAt.AsTime(),
        UpdatedAt:   userpb.UpdatedAt.AsTime(),
    }
}

func UserToPb(user model.User) *accountpb.User {
    return &accountpb.User{
        Id:          user.ID,
        Login:       user.Login,

```



```

    Email:      user.Email,
    Phone:      user.Phone,
    FirstName:  user.FirstName,
    LastName:   user.LastName,
    MiddleName: user.MiddleName,
    Age:        user.Age,
    Balance:    float32(user.Balance),
    IsDeleted:  user.IsDeleted,
    CreatedAt:  timestampb.New(user.CreatedAt),
    UpdatedAt:  timestampb.New(user.UpdatedAt),
}
}

func UsersToPbs(users []model.User) []*accountpb.User {
    pbs := make([]*accountpb.User, len(users))
    for i, user := range users {
        pbs[i] = UserToPb(user)
    }
    return pbs
}

func PbsToUsers(pbs []*accountpb.User) []model.User {
    users := make([]model.User, len(pbs))
    for i, pb := range pbs {
        users[i] = PbToUser(pb)
    }
    return users
}

func PbToUserCreate(accountpbUser *accountpb.CreateUser) model.CreateUser {
    return model.CreateUser{
        Login:      accountpbUser.Login,
        Email:      accountpbUser.Email,
        Phone:      accountpbUser.Phone,
        FirstName:  accountpbUser.FirstName,
        LastName:   accountpbUser.LastName,
        MiddleName: accountpbUser.MiddleName,
        Age:        accountpbUser.Age,
        Balance:    float32(accountpbUser.Balance),
    }
}

func PbToUserUpdate(userpb *accountpb.User) model.UpdateUser {
    return model.UpdateUser{
        Email:      userpb.Email,
        Phone:      userpb.Phone,
        FirstName:  userpb.FirstName,

```

```
        LastName: userpb.LastName,  
        MiddleName: userpb.MiddleName,  
        Age: userpb.Age,  
        Balance: float32(userpb.Balance),  
    }  
}
```

Листинг 4.15. Account: internal/repository/mapper/mapper.go

```
package mapper  
  
import (  
    "account/internal/model"  
    repomodel "account/internal/repository/model"  
)  
  
func UserToRepoUser(user model.User) repomodel.User {  
    return repomodel.User{  
        ID: user.ID,  
        Login: user.Login,  
        Email: user.Email,  
        Phone: user.Phone,  
        FirstName: user.FirstName,  
        LastName: user.LastName,  
        MiddleName: user.MiddleName,  
        Age: user.Age,  
        Balance: user.Balance,  
        IsDeleted: user.IsDeleted,  
        CreatedAt: user.CreatedAt,  
        UpdatedAt: user.UpdatedAt,  
    }  
}  
  
func RepoUserToUser(user repomodel.User) model.User {  
    return model.User{  
        ID: user.ID,  
        Login: user.Login,  
        Email: user.Email,  
        Phone: user.Phone,  
        FirstName: user.FirstName,  
        LastName: user.LastName,  
        MiddleName: user.MiddleName,  
        Age: user.Age,  
        Balance: user.Balance,  
        IsDeleted: user.IsDeleted,  
        CreatedAt: user.CreatedAt,  
        UpdatedAt: user.UpdatedAt,  
    }  
}
```

```

func RepoUsersToUsers(users []repmode1.User) []model.User {
    res := make([]model.User, len(users))
    for i, user := range users {
        res[i] = RepoUserToUser(user)
    }
    return res
}

func UpdateUserToRepoUser(user model.UpdateUser) repmode1.User {
    return repmode1.User{
        Email:      user.Email,
        Phone:      user.Phone,
        FirstName:  user.FirstName,
        LastName:   user.LastName,
        MiddleName: user.MiddleName,
        Age:        user.Age,
        Balance:    user.Balance,
    }
}

func CreateUserToRepoUser(user model.CreateUser) repmode1.User {
    return repmode1.User{
        Login:      user.Login,
        Email:      user.Email,
        Phone:      user.Phone,
        FirstName:  user.FirstName,
        LastName:   user.LastName,
        MiddleName: user.MiddleName,
        Age:        user.Age,
        Balance:    user.Balance,
    }
}

```

Помимо добавления баланса, в схемы нужно также добавить методы для изменения и получения баланса (листинг 4.16).

Листинг 4.16. Contracts: account/account_service.proto

```

syntax = "proto3";

package account;

option go_package = "github.com/yuliaapopova/contracts/account/go;account";

import "google/api/annotations.proto";
import "google/protobuf/empty.proto";
import "account_model.proto";
import "pagination/pagination.proto";

```

```
service Account {  
    rpc CreateUser(CreateUserRequest) returns (CreateUserResponse);  
    rpc GetUser(GetUserRequest) returns (GetUserResponse);  
    rpc GetUsers(GetUsersRequest) returns (GetUsersResponse);  
    rpc DeleteUser(DeleteUserRequest) returns (google.protobuf.Empty);  
    rpc UpdateUser(UpdateUserRequest) returns (google.protobuf.Empty);  
  
    // управление балансом  
    rpc Deposit(DepositRequest) returns (DepositResponse);  
    rpc Withdraw(WithdrawRequest) returns (WithdrawResponse);  
    rpc Transfer(TransferRequest) returns (TransferResponse);  
    rpc GetBalance(GetBalanceRequest) returns (GetBalanceResponse);  
}  
  
message CreateUserRequest {  
    account.CreateUser user = 1;  
}  
  
message CreateUserResponse {  
    account.User user = 1;  
}  
  
message GetUserRequest {  
    uint64 user_id = 1;  
}  
  
message GetUserResponse {  
    User user = 1;  
}  
  
message GetUsersRequest {  
    pagination.Pagination pagination = 1;  
}  
  
message GetUsersResponse {  
    repeated account.User users = 1;  
    pagination.Pagination pagination = 2;  
}  
  
message DeleteUserRequest {  
    uint64 user_id = 1;  
}  
  
message UpdateUserRequest {  
    uint64 user_id = 1;  
    User user = 2;  
}
```

```
// Пополнение
message DepositRequest {
    uint64 user_id = 1;
    int64 amount = 2;
    uint64 operation_id = 3;
}

message DepositResponse {
    string status = 1; // "completed" / "failed"
    int64 balance = 2;
}

// Списание
message WithdrawRequest {
    uint64 user_id = 1;
    int64 amount = 2;
    string operation_id = 3;
}

message WithdrawResponse {
    string status = 1;
    int64 balance = 2;
}

// Перевод
message TransferRequest {
    uint64 user_id = 1;
    uint64 recipient_id = 2;
    int64 amount = 3;
    string operation_id = 4;
}

message TransferResponse {
    string status = 1;
    int64 user_balance = 2;
    int64 recipient_balance = 3;
}

// Получение баланса
message GetBalanceRequest {
    uint64 user_id = 1;
}

message GetBalanceResponse {
    int64 balance = 1;
}
```

Не забывайте также после каждого изменения контрактов обновлять их версию и подтягивать ее в сервис.

Теперь можно приступить к логике, связанной с балансом, а именно — с пополнением, списанием и переводом средств. В `repository` добавим метод `updateBalance`, который будет отвечать за изменение баланса, а для перевода — отдельный метод `transferBalance` (листинг 4.17).

Листинг 4.17. Account: internal/repository/repository.go

```
package repository

import (
    "context"
    "errors"
    "fmt"

    "account/internal/model"
    "account/internal/repository/mapper"
    repomodel "account/internal/repository/model"

    "github.com/rs/zerolog"
    "gorm.io/gorm"
    "gorm.io/gorm/clause"
)

type Repository struct {
    db      *gorm.DB
    logger  *zerolog.Logger
}

func NewRepository(db *gorm.DB, logger *zerolog.Logger) *Repository {
    return &Repository{
        db:      db,
        logger:  logger,
    }
}

func (r *Repository) CreateUser(ctx context.Context, user model.User) (model.User, error) {
    userRepo := mapper.UserToRepoUser(user)
    fmt.Println(userRepo)
    res := r.db.WithContext(ctx).
        Clauses(clause.OnConflict{UpdateAll: true}).
        //Clauses(clause.Returning{}).
        Create(&userRepo)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to save user")
        return model.User{}, fmt.Errorf("failed to save user: %w", res.Error)
    }
}
```

```
fmt.Println(userRepo)
return mapper.RepoUserToUser(userRepo), nil
}

func (r *Repository) GetUser(ctx context.Context, userID uint64) (model.User, error) {
    var user repomodel.User
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("id = ? AND is_deleted = ?", userID, false).
        First(&user)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return model.User{}, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user id")
    }
    return mapper.RepoUserToUser(user), nil
}

func (r *Repository) GetUsers(ctx context.Context, limit int, offset int) ([]model.User, error) {
    var users []repomodel.User
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("is_deleted = ?", false).
        Offset(offset).
        Limit(limit).
        Find(&users)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return nil, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user id")
    }
    return mapper.RepoUsersToUsers(users), nil
}

func (r *Repository) DeleteUser(ctx context.Context, userID uint64) error {
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("id = ?", userID).
        Update("is_deleted", true)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to delete user")
        return fmt.Errorf("failed to delete user")
    }
    return nil
}
```

```

func (r *Repository) UpdateUser(ctx context.Context, userID uint64, user model.UpdateUser)
error {
    repoUser := mapper.UpdateUserToRepoUser(user)
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("id = ? AND is_deleted = ?", userID, false).
        Updates(repoUser)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to update user")
        return fmt.Errorf("failed to update user")
    }
    return nil
}

```

// GetBalance получает текущий баланс пользователя

```

func (r *Repository) GetBalance(ctx context.Context, userID uint64) (float32, error) {
    var user repomodel.User
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Select("balance").
        Where("id = ? AND is_deleted = ?", userID, false).
        First(&user)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return 0, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user balance")
        return 0, fmt.Errorf("failed to get user balance: %w", res.Error)
    }
    return user.Balance, nil
}

```

// UpdateBalance обновляет баланс пользователя

```

func (r *Repository) UpdateBalance(ctx context.Context, userID uint64, amount float32,
operationType model.BalanceOperationType) (float32, float32, error) {
    var user repomodel.User

    // Сначала получаем текущий баланс
    res := r.db.WithContext(ctx).
        Model(&repomodel.User{}).
        Where("id = ? AND is_deleted = ?", userID, false).
        First(&user)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return 0, 0, fmt.Errorf("user not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get user for balance update")
    }
}

```



```

    return 0, 0, fmt.Errorf("failed to get user for balance update: %w", res.Error)
}

oldBalance := user.Balance
var newBalance float32

// Вычисляем новый баланс в зависимости от типа операции
switch operationType {
case model.BalanceOperationDeposit:
    newBalance = oldBalance + amount
case model.BalanceOperationCredit:
    newBalance = oldBalance - amount
    if newBalance < 0 {
        return 0, 0, fmt.Errorf("insufficient balance: current balance %.2f, requested
                                amount %.2f", oldBalance, amount)
    }
default:
    return 0, 0, fmt.Errorf("invalid balance operation type: %s", operationType)
}

// Обновляем баланс в базе данных
res = r.db.WithContext(ctx).
    Model(&repomodel.User{}).
    Where("id = ? AND is_deleted = ?", userID, false).
    Update("balance", newBalance)

if res.Error != nil {
    r.logger.Err(res.Error).Msg("failed to update user balance")
    return 0, 0, fmt.Errorf("failed to update user balance: %w", res.Error)
}

return oldBalance, newBalance, nil
}

// TransferBalance переводит средства между пользователями
func (r *Repository) TransferBalance(ctx context.Context, fromUserID, toUserID uint64, amount
float32) error {
    if amount <= 0 {
        return fmt.Errorf("transfer amount must be positive")
    }

    if fromUserID == toUserID {
        return fmt.Errorf("cannot transfer to the same user")
    }

    // Начинаем транзакцию
    return r.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {

```

```
// Проверяем существование получателя
var toUser repomodel.User
res := tx.Model(&repomodel.User{}).
    Where("id = ? AND is_deleted = ?", toUserID, false).
    First(&toUser)
if errors.Is(res.Error, gorm.ErrRecordNotFound) {
    return fmt.Errorf("recipient user not found")
} else if res.Error != nil {
    return fmt.Errorf("failed to get recipient user: %w", res.Error)
}

// Получаем текущий баланс отправителя
var fromUser repomodel.User
res = tx.Model(&repomodel.User{}).
    Where("id = ? AND is_deleted = ?", fromUserID, false).
    First(&fromUser)
if errors.Is(res.Error, gorm.ErrRecordNotFound) {
    return fmt.Errorf("sender user not found")
} else if res.Error != nil {
    return fmt.Errorf("failed to get sender user: %w", res.Error)
}

// Проверяем достаточность средств
if fromUser.Balance < amount {
    return fmt.Errorf("insufficient balance: current balance %.2f, transfer amount %.2f", fromUser.Balance, amount)
}

// Списываем средства с отправителя
res = tx.Model(&repomodel.User{}).
    Where("id = ? AND is_deleted = ?", fromUserID, false).
    Update("balance", fromUser.Balance-amount)
if res.Error != nil {
    return fmt.Errorf("failed to deduct from sender balance: %w", res.Error)
}

// Зачисляем средства получателю
res = tx.Model(&repomodel.User{}).
    Where("id = ? AND is_deleted = ?", toUserID, false).
    Update("balance", toUser.Balance+amount)
if res.Error != nil {
    return fmt.Errorf("failed to add to recipient balance: %w", res.Error)
}

return nil
})
}
```

Метод `updateBalance` инкапсулирует логику изменения средств в зависимости от типа операции — пополнение или списание, при этом сразу проверяется достаточность средств на счете, чтобы предотвратить уход в отрицательный баланс. Для перевода средств добавлен метод `TransferBalance`, который содержит более сложную логику, — в рамках одной транзакции базы данных выполняется перевод средств от одного пользователя к другому: сперва проверяется наличие обоих пользователей и корректность суммы, затем средства списываются с отправителя и зачисляются получателю. Так что операция либо проходит целиком, либо полностью откатывается. Эти методы делают работу с балансом надежной из-за таких проверок и использования транзакций.

В сервис же добавим следующую логику (листинг 4.18).

Листинг 4.18. Account: internal/service/service.go

```
package service

import (
    "context"
    "fmt"

    "account/internal/model"
    "account/internal/repository/mapper"

    "github.com/rs/zerolog"
)

type AccountService struct {
    repo Repository
    logger *zerolog.Logger
}

func New(repo Repository, logger *zerolog.Logger) *AccountService {
    return &AccountService{repo: repo, logger: logger}
}

type Repository interface {
    CreateUser(context.Context, model.User) (model.User, error)
    GetUser(context.Context, uint64) (model.User, error)
    GetUsers(context.Context, int, int) ([]model.User, error)
    DeleteUser(context.Context, uint64) error
    UpdateUser(context.Context, uint64, model.UpdateUser) error
    GetBalance(context.Context, uint64) (float32, error)
    UpdateBalance(context.Context, uint64, float32, model.BalanceOperationType) (float32,
                                                                    float32, error)
    TransferBalance(context.Context, uint64, uint64, float32) error
}
```

```
func (s *AccountService) CreateUser(ctx context.Context, newUser model.CreateUser)
(model.User, error) {
    repoUser := mapper.CreateUserToRepoUser(newUser)
    user := mapper.RepoUserToUser(repoUser)

    return s.repo.CreateUser(ctx, user)
}

func (s *AccountService) GetUsers(ctx context.Context, limit int, offset int) ([]model.User,
error) {
    return s.repo.GetUsers(ctx, limit, offset)
}

func (s *AccountService) GetUser(ctx context.Context, userID uint64) (model.User, error) {
    return s.repo.GetUser(ctx, userID)
}

func (s *AccountService) DeleteUser(ctx context.Context, userID uint64) error {
    return s.repo.DeleteUser(ctx, userID)
}

func (s *AccountService) UpdateUser(ctx context.Context, userID uint64, user
model.UpdateUser) error {
    return s.repo.UpdateUser(ctx, userID, user)
}

// GetBalance получает текущий баланс пользователя
func (s *AccountService) GetBalance(ctx context.Context, userID uint64)
(model.GetBalanceResponse, error) {
    balance, err := s.repo.GetBalance(ctx, userID)
    if err != nil {
        s.logger.Err(err).Uint64("userID", userID).Msg("failed to get user balance")
        return model.GetBalanceResponse{}, err
    }

    return model.GetBalanceResponse{
        UserID: userID,
        Balance: balance,
    }, nil
}

// UpdateBalance обновляет баланс пользователя
func (s *AccountService) UpdateBalance(ctx context.Context, req model.UpdateBalanceRequest)
(model.UpdateBalanceResponse, error) {
    // Валидация входных данных
    if req.Amount < 0 {
        return model.UpdateBalanceResponse{}, fmt.Errorf("amount cannot be negative")
    }
}
```

```

if req.Type == model.BalanceOperationCredit && req.Amount == 0 {
    return model.UpdateBalanceResponse{}, fmt.Errorf("amount must be positive for subtract
                                                    operation")
}

oldBalance, newBalance, err := s.repo.UpdateBalance(ctx, req.UserID, req.Amount, req.Type)
if err != nil {
    s.logger.Err(err).
        Uint64("userID", req.UserID).
        Float32("amount", req.Amount).
        Str("operation", string(req.Type)).
        Msg("failed to update user balance")
    return model.UpdateBalanceResponse{}, err
}

s.logger.Info().
    Uint64("userID", req.UserID).
    Float32("oldBalance", oldBalance).
    Float32("newBalance", newBalance).
    Float32("amount", req.Amount).
    Str("operation", string(req.Type)).
    Msg("user balance updated successfully")

return model.UpdateBalanceResponse{
    UserID:    req.UserID,
    OldBalance: oldBalance,
    NewBalance: newBalance,
    Amount:    req.Amount,
    Operation: req.Type,
}, nil
}

// TransferBalance переводит средства между пользователями
func (s *AccountService) TransferBalance(ctx context.Context, fromUserID, toUserID uint64,
amount float32) error {
    if amount <= 0 {
        return fmt.Errorf("transfer amount must be positive")
    }

    if fromUserID == toUserID {
        return fmt.Errorf("cannot transfer to the same user")
    }

    err := s.repo.TransferBalance(ctx, fromUserID, toUserID, amount)
    if err != nil {
        s.logger.Err(err).
            Uint64("fromUserID", fromUserID).

```

```
        Uint64("toUserID", toUserID).
        Float32("amount", amount).
        Msg("failed to transfer balance")
    return err
}

s.logger.Info().
    Uint64("fromUserID", fromUserID).
    Uint64("toUserID", toUserID).
    Float32("amount", amount).
    Msg("balance transferred successfully")

return nil
}

// Deposit пополняет баланс пользователя
func (s *AccountService) Deposit(ctx context.Context, userID uint64, amount int64)
(model.UpdateBalanceResponse, error) {
    updateReq := model.UpdateBalanceRequest{
        UserID: userID,
        Amount: float32(amount),
        Type:   model.BalanceOperationDeposit,
    }

    response, err := s.UpdateBalance(ctx, updateReq)
    if err != nil {
        s.logger.Err(err).
            Uint64("userID", userID).
            Int64("amount", amount).
            Msg("failed to deposit")
        return model.UpdateBalanceResponse{}, err
    }

    s.logger.Info().
        Uint64("userID", userID).
        Int64("amount", amount).
        Float32("newBalance", response.NewBalance).
        Msg("deposit successful")

    return response, nil
}

// Withdraw списывает средства с баланса пользователя
func (s *AccountService) Withdraw(ctx context.Context, userID uint64, amount int64)
(model.UpdateBalanceResponse, error) {
    updateReq := model.UpdateBalanceRequest{
        UserID: userID,
```

```

    Amount: float32(amount),
    Type:    model.BalanceOperationCredit,
}

response, err := s.UpdateBalance(ctx, updateReq)
if err != nil {
    s.logger.Err(err).
        Uint64("userID", userID).
        Int64("amount", amount).
        Msg("failed to withdraw")
    return model.UpdateBalanceResponse{}, err
}

s.logger.Info().
    Uint64("userID", userID).
    Int64("amount", amount).
    Float32("newBalance", response.NewBalance).
    Msg("withdraw successful")

return response, nil
}

// Transfer переводит средства между пользователями и возвращает балансы
func (s *AccountService) Transfer(ctx context.Context, fromUserID, toUserID uint64, amount
int64) (model.GetBalanceResponse, model.GetBalanceResponse, error) {
    err := s.TransferBalance(ctx, fromUserID, toUserID, float32(amount))
    if err != nil {
        s.logger.Err(err).
            Uint64("fromUserID", fromUserID).
            Uint64("toUserID", toUserID).
            Int64("amount", amount).
            Msg("failed to transfer")
        return model.GetBalanceResponse{}, model.GetBalanceResponse{}, err
    }

    // Получаем балансы обоих пользователей для ответа
    fromUserBalance, err := s.GetBalance(ctx, fromUserID)
    if err != nil {
        s.logger.Err(err).Uint64("userID", fromUserID).Msg("failed to get sender balance")
        return model.GetBalanceResponse{}, model.GetBalanceResponse{}, err
    }

    toUserBalance, err := s.GetBalance(ctx, toUserID)
    if err != nil {
        s.logger.Err(err).Uint64("userID", toUserID).Msg("failed to get recipient balance")
        return model.GetBalanceResponse{}, model.GetBalanceResponse{}, err
    }
}

```

```

s.logger.Info(),
    Uint64("fromUserID", fromUserID),
    Uint64("toUserID", toUserID),
    Int64("amount", amount),
    Msg("transfer successful")

    return fromUserBalance, toUserBalance, nil
}

```

Здесь мы уже не обрабатываем ситуацию, когда недостаточно средств на счету, — это задача репозитория, но тут мы добавили больше логирования, чтобы отслеживать спорные моменты.

И последний штрих — добавить новые методы в серверный слой (листинг 4.19).

Листинг 4.19. Account: internal/server/server.go

```

package server

import (
    "context"

    "account/internal/mapper"
    "account/internal/model"
    "github.com/rs/zerolog"

    accountpb "github.com/yuliaapopova/contracts/account/go"
    "google.golang.org/protobuf/types/known/emptypb"
)

type Server struct {
    accountpb.UnimplementedAccountServer

    accountService AccountService
    logger          *zerolog.Logger
}

func New(accountService AccountService, logger *zerolog.Logger) *Server {
    return &Server{accountService: accountService, logger: logger}
}

type AccountService interface {
    CreateUser(context.Context, model.CreateUser) (model.User, error)
    GetUser(ctx context.Context, userID uint64) (model.User, error)
    GetUsers(ctx context.Context, limit int, offset int) ([]model.User, error)
    DeleteUser(ctx context.Context, userID uint64) error
    UpdateUser(ctx context.Context, userID uint64, user model.UpdateUser) error
    GetBalance(ctx context.Context, userID uint64) (model.GetBalanceResponse, error)
}

```



```

Deposit(ctx context.Context, userID uint64, amount int64) (model.UpdateBalanceResponse,
                                                                    error)
Withdraw(ctx context.Context, userID uint64, amount int64) (model.UpdateBalanceResponse,
                                                                    error)
Transfer(ctx context.Context, fromUserID, toUserID uint64, amount int64)
                                                                    (model.GetBalanceResponse, model.GetBalanceResponse, error)
}

func (s *Server) CreateUser(ctx context.Context, req *accountpb.CreateUserRequest)
(*accountpb.CreateUserResponse, error) {
    newUser := mapper.PbToUserCreate(req.GetUser())
    user, err := s.accountService.CreateUser(ctx, newUser)
    if err != nil {
        return nil, err
    }
    return &accountpb.CreateUserResponse{User: mapper.UserToPb(user)}, nil
}

func (s *Server) GetUser(ctx context.Context, req *accountpb.GetUserRequest)
(*accountpb.GetUserResponse, error) {
    res, err := s.accountService.GetUser(ctx, req.GetUserId())
    if err != nil {
        return nil, err
    }
    return &accountpb.GetUserResponse{
        User: mapper.UserToPb(res),
    }, nil
}

func (s *Server) GetUsers(ctx context.Context, req *accountpb.GetUsersRequest)
(*accountpb.GetUsersResponse, error) {
    res, err := s.accountService.GetUsers(ctx, int(req.GetPagination().GetLimit()),
                                                                    int(req.GetPagination().GetOffset()))
    if err != nil {
        return nil, err
    }
    return &accountpb.GetUsersResponse{
        Users: mapper.UsersToPbs(res),
    }, nil
}

func (s *Server) UpdateUser(ctx context.Context, req *accountpb.UpdateUserRequest)
(*emptypb.Empty, error) {
    user := mapper.PbToUserUpdate(req.GetUser())
    if err := s.accountService.UpdateUser(ctx, uint64(req.GetUserId()), user); err != nil {
        return nil, err
    }
    return &emptypb.Empty{}, nil
}

```

[illegible]

```

    if err != nil {
        return &accountpb.TransferResponse{
            Status:      "failed",
            UserBalance: 0,
            RecipientBalance: 0,
        }, err
    }

    return &accountpb.TransferResponse{
        Status:      "completed",
        UserBalance: int64(fromUserBalance.Balance),
        RecipientBalance: int64(toUserBalance.Balance),
    }, nil
}

// GetBalance получает баланс пользователя
func (s *Server) GetBalance(ctx context.Context, req *accountpb.GetBalanceRequest)
(*accountpb.GetBalanceResponse, error) {
    response, err := s.accountService.GetBalance(ctx, req.GetUserId())
    if err != nil {
        return &accountpb.GetBalanceResponse{
            Balance: 0,
        }, err
    }

    return &accountpb.GetBalanceResponse{
        Balance: int64(response.Balance),
    }, nil
}

```

Теперь сервис Account у нас полностью готов для интеграции с сервисом Transaction, и мы можем добавить в transaction вызов account для изменения баланса. Создадим для этого в каталоге internal каталог account (листинг 4.20).

Листинг 4.20. Transaction: internal/account/account.go

```

package account

import (
    "context"
    "fmt"
    "strconv"

    accountpb "github.com/yuliaapopova/contracts/account/go"
)

type Service struct {
    client accountpb.AccountClient
}

```

```
func New(client accountpb.AccountClient) *Service {
    return &Service{
        client: client,
    }
}

func (s *Service) GetBalance(ctx context.Context, userID uint64) (int64, error) {
    res, err := s.client.GetBalance(ctx, &accountpb.GetBalanceRequest{
        UserId: userID,
    })
    if err != nil {
        return 0, fmt.Errorf("failed to get account balance: %w", err)
    }
    return res.Balance, nil
}

func (s *Service) Deposit(ctx context.Context, userID uint64, amount int64, operationID
uint64) (int64, error) {
    res, err := s.client.Deposit(ctx, &accountpb.DepositRequest{
        UserId:      userID,
        Amount:      amount,
        OperationId: operationID,
    })
    if err != nil {
        return 0, fmt.Errorf("failed to deposit account: %w", err)
    }
    return res.Balance, nil
}

func (s *Service) Withdraw(ctx context.Context, userID uint64, amount int64, operationID
uint64) (int64, error) {
    res, err := s.client.Withdraw(ctx, &accountpb.WithdrawRequest{
        UserId:      userID,
        Amount:      amount,
        OperationId: strconv.FormatUint(operationID, 10),
    })
    if err != nil {
        return 0, fmt.Errorf("failed to withdraw account: %w", err)
    }
    return res.Balance, nil
}

func (s *Service) Transfer(ctx context.Context, userID, recipient uint64, amount int64,
operationID uint64) (string, error) {
    res, err := s.client.Transfer(ctx, &accountpb.TransferRequest{
        UserId:      userID,
        RecipientId: recipient,
```

```

    Amount:      amount,
    OperationId: strconv.FormatUint(operationID, 10),
})
return res.Status, err
}

```

Здесь сервис максимально простой — только вызов методов изменения и получения баланса.

Добавим теперь ACCOUNT_GRPC_HOST в конфигурацию сервиса (листинг 4.21).

Листинг 4.21. Transaction: internal/config/config.go

```

package config

import (
    "log"

    "github.com/caarlos0/env/v10"
    "github.com/joho/godotenv"
)

// Config содержит конфигурацию приложения
type Config struct {
    ServiceName string `env:"SERVICE_NAME" json:"service_name" required:"true"
                                                default:"transaction-service"`
    AppEnv       string `env:"APP_ENV" json:"app_environment" required:"true"
                                                default:"development"`
    Host         string `env:"GRPC_HOST" json:"host" required:"true" default:"localhost"`
    Port         int    `env:"GRPC_PORT" json:"port" required:"true" default:"9000"`
    LogLevel     string `env:"LOG_LEVEL" json:"log_level" required:"true" default:"info"`
    DbDsn        string `env:"DB_DSN" json:"db_dsn" required:"true"`
    AccountGrpcHost string `env:"ACCOUNT_GRPC_HOST" json:"account_grpc_host" required:"true"`
}

// Load загружает конфигурацию из переменных окружения
func Load() (*Config, error) {
    // Загружаем .env файл
    if err := godotenv.Load(); err != nil {
        log.Println("Warning: .env file not found, using system environment variables")
    }

    cfg := &Config{}
    err := env.Parse(cfg)
    if err != nil {
        return nil, err
    }

    return cfg, nil
}

```

И не забываем добавить `ACCOUNT_GRPC_HOST` в переменные окружения (листинг 4.22).

Листинг 4.22. Transaction: .env

```
SERVICE_NAME=transaction
APP_ENV=dev

GRPC_PORT=50054
GRPC_HOST=localhost
LOG_LEVEL=debug

DB_DSN=postgres://postgres:postgres@localhost:5434/transaction?sslmode=disable

ACCOUNT_GRPC_HOST=localhost:50051
```

Последний штрих — инициализация пакета `account` с системой (листинг 4.23).

Листинг 4.23. Transaction: internal/app/app.go

```
package app

import (
    "context"
    "database/sql"
    "fmt"
    "net"

    "transaction/internal/account"
    "transaction/internal/config"
    _ "transaction/internal/migrations"
    "transaction/internal/repository"
    "transaction/internal/server"
    "transaction/internal/service"

    "github.com/pressly/goose/v3"
    accountpb "github.com/yuliaapopova/contracts/account/go"
    transactionpb "github.com/yuliaapopova/contracts/transaction/go"
    _ "google.golang.org/genproto/googleapis/api/annotations"
    "google.golang.org/grpc"

    _ "github.com/lib/pq"
    "github.com/rs/zerolog"
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
)

type App struct {
    cfg    *config.Config
    logger *zerolog.Logger
```

```

transactionRepository *repository.Repository
transactionService    *service.TransactionService
transactionServer     *server.Server
grpcServer            *grpc.Server

accountService *account.Service
)

func New(logger *zerolog.Logger, cfg *config.Config) *App {
    return &App{
        cfg:    cfg,
        logger: logger,
    }
}

func (a *App) Run(ctx context.Context) error {
    // Инициализируем gRPC-сервер
    transactionServer, err := a.getTransactionServer(ctx)
    if err != nil {
        return fmt.Errorf("failed to get transaction server: %w", err)
    }

    // Создаем gRPC-сервер
    a.grpcServer = getGRPCServer(transactionServer)

    listenAddr := fmt.Sprintf("%s:%d", a.cfg.Host, a.cfg.Port)
    lis, err := net.Listen("tcp", listenAddr)
    if err != nil {
        a.logger.Fatal().Err(err).Msg("failed to listen")
        return err
    }

    a.logger.Info().Msg("gRPC server listening")

    serveErrCh := make(chan error, 1)
    go func() {
        serveErrCh <- a.grpcServer.Serve(lis)
    }()

    select {
    case <-ctx.Done():
        a.grpcServer.GracefulStop()
        return ctx.Err()
    case err := <-serveErrCh:
        if err != nil {
            a.logger.Error().Err(err).Msg("failed to serve")
        }
    }
}

```

```
        return err
    }
}

func (a *App) getRepository(ctx context.Context) (*repository.Repository, error) {
    if a.transactionRepository == nil {
        if err := a.runMigrations(ctx); err != nil {
            return nil, fmt.Errorf("failed to get repository: %w", err)
        }
        db, err := gorm.Open(postgres.Open(a.cfg.DbDsn), &gorm.Config{})
        if err != nil {
            return nil, fmt.Errorf("gorm init failed: %w", err)
        }
        a.transactionRepository = repository.NewRepository(db, a.logger)
    }
    return a.transactionRepository, nil
}

func (a *App) runMigrations(ctx context.Context) error {
    if err := goose.SetDialect("postgres"); err != nil {
        return fmt.Errorf("failed to select migrations dialect: %w", err)
    }

    dbGoose, err := sql.Open("postgres", a.cfg.DbDsn)
    if err != nil {
        return fmt.Errorf("failed to create sql connection: %w", err)
    }

    if err := goose.UpContext(ctx, dbGoose, "internal/migrations"); err != nil {
        return fmt.Errorf("failed to run up migrations: %w", err)
    }
    return nil
}

func (a *App) getTransactionService(ctx context.Context) (*service.TransactionService, error) {
    if a.transactionService == nil {
        repo, err := a.getRepository(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get repository: %w", err)
        }

        accountSvc, err := a.getAccountService(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get account service: %w", err)
        }

        a.transactionService = service.New(repo, accountSvc, a.logger)
    }
}
```



```
    return a.transactionService, nil
}

func (a *App) getTransactionServer(ctx context.Context) (*server.Server, error) {
    if a.transactionServer == nil {
        service, err := a.getTransactionService(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get service: %w", err)
        }
        a.transactionServer = server.New(service, a.logger)
    }
    return a.transactionServer, nil
}

func (a *App) getAccountService(ctx context.Context) (*account.Service, error) {
    if a.accountService == nil {
        // Создаем gRPC-соединение с account service
        conn, err := grpc.Dial(a.cfg.AccountGrpcHost, grpc.WithInsecure())
        if err != nil {
            return nil, fmt.Errorf("failed to connect to account service: %w", err)
        }

        client := accountpb.NewAccountClient(conn)
        a.accountService = account.New(client)
    }
    return a.accountService, nil
}

func getGRPCServer(srv *server.Server) *grpc.Server {
    grpcSrv := grpc.NewServer()
    transactionpb.RegisterTransactionServiceServer(grpcSrv, srv)
    return grpcSrv
}

// Close корректно завершает работу приложения
func (a *App) Close() error {
    if a.grpcServer != nil {
        a.grpcServer.GracefulStop()
    }
    return nil
}
```

Теперь мы можем перейти к доработке непосредственно логики создания транзакции. Тут нужно отредактировать слой репозитория, т. к. перед установкой транзакции статуса, что всё прошло успешно, должен отработать запрос изменения баланса в account. Это касается как переводов, так и остальных операций (листинг 4.24).

Листинг 4.24. Transaction: internal/repository/repository.go

```
package repository

import (
    "context"
    "errors"
    "fmt"

    "transaction/internal/model"
    "transaction/internal/repository/mapper"
    repomodel "transaction/internal/repository/model"

    "github.com/rs/zerolog"
    "gorm.io/gorm"
    "gorm.io/gorm/clause"
)

type Repository struct {
    db      *gorm.DB
    logger  *zerolog.Logger
}

func NewRepository(db *gorm.DB, logger *zerolog.Logger) *Repository {
    return &Repository{
        db:      db,
        logger:  logger,
    }
}

func (r *Repository) GetTransactions(ctx context.Context, params model.GetTransactionsParams) (
    []model.Transaction, error) {
    var transactions []repomodel.Transaction

    query := r.db.WithContext(ctx).Model(&repomodel.Transaction{})

    // Применяем фильтры
    if params.UserID != nil {
        query = query.Where("user_id = ?", *params.UserID)
    }
    if params.Type != nil {
        query = query.Where("type = ?", *params.Type)
    }
    if params.Status != nil {
        query = query.Where("status = ?", *params.Status)
    }
    if params.DateFrom != nil {
        query = query.Where("created_at >= ?", *params.DateFrom)
    }
}
```

```

if params.DateTo != nil {
    query = query.Where("created_at <= ?", *params.DateTo)
}

// Применяем пагинацию и сортировку
res := query.
    Offset(params.Offset).
    Limit(params.Limit).
    Order("created_at DESC").
    Find(&transactions)

if res.Error != nil {
    r.logger.Err(res.Error).Msg("failed to get transactions")
    return nil, fmt.Errorf("failed to get transactions: %w", res.Error)
}
return mapper.RepoTransactionsToTransactions(transactions), nil
}

func (r *Repository) GetTransactionDetails(ctx context.Context, transactionID uint64)
(model.TransactionDetails, error) {
    // Получаем транзакцию
    var transaction repomodel.Transaction
    res := r.db.WithContext(ctx).
        Model(&repomodel.Transaction{}).
        Where("id = ?", transactionID).
        First(&transaction)

    if errors.Is(res.Error, gorm.ErrRecordNotFound) {
        return model.TransactionDetails{}, fmt.Errorf("transaction not found")
    } else if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get transaction")
        return model.TransactionDetails{}, fmt.Errorf("failed to get transaction: %w",
            res.Error)
    }

    // Получаем записи транзакции
    var entries []repomodel.TransactionEntry
    res = r.db.WithContext(ctx).
        Model(&repomodel.TransactionEntry{}).
        Where("transaction_id = ?", transactionID).
        Order("created_at ASC").
        Find(&entries)

    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to get transaction entries")
        return model.TransactionDetails{}, fmt.Errorf("failed to get transaction entries: %w",
            res.Error)
    }
}

```

```

return model.TransactionDetails{
    Transaction: mapper.RepoTransactionToTransaction(transaction),
    Entries:     mapper.RepoTransactionEntriesToTransactionEntries(entries),
}, nil
}

// Deposit выполняет операцию депозита в рамках одной транзакции БД
func (r *Repository) Deposit(ctx context.Context, params model.DepositParams)
(model.TransactionDetails, error) {
    var result model.TransactionDetails

    err := r.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        // Создаем транзакцию
        transaction := model.Transaction{
            UserID: params.UserID,
            Amount: int64(params.Amount * 100), // конвертируем в копейки
            Status: model.TransactionStatusPending,
            Type:   model.TransactionTypeDeposit,
        }

        transactionRepo := mapper.TransactionToRepoTransaction(transaction)
        res := tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&transactionRepo)
        if res.Error != nil {
            r.logger.Err(res.Error).Msg("failed to create transaction in deposit")
            return fmt.Errorf("failed to create transaction: %w", res.Error)
        }

        // Создаем запись дебета (увеличение баланса)
        entry := model.TransactionEntry{
            TransactionID: transactionRepo.ID,
            AccountID:     params.UserID,
            Direction:     model.TransactionEntryDirectionDebit,
            Amount:        params.Amount,
        }

        entryRepo := mapper.TransactionEntryToRepoTransactionEntry(entry)
        res = tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&entryRepo)
        if res.Error != nil {
            r.logger.Err(res.Error).Msg("failed to create transaction entry in deposit")
            return fmt.Errorf("failed to create transaction entry: %w", res.Error)
        }

        // Получаем детали созданной транзакции (оставляем в статусе pending)
        transactionRepo.Status = string(model.TransactionStatusPending)
        result = model.TransactionDetails{
            Transaction: mapper.RepoTransactionToTransaction(transactionRepo),
            Entries:     []model.TransactionEntry{mapper.RepoTransactionEntryToTransactionEntry(entryRepo)},
        }
    })
}

```

```

    return nil
})

if err != nil {
    return model.TransactionDetails{}, err
}

return result, nil
}

// Withdraw выполняет операцию снятия средств в рамках одной транзакции БД
func (r *Repository) Withdraw(ctx context.Context, params model.WithdrawParams)
(model.TransactionDetails, error) {
    var result model.TransactionDetails

    err := r.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        // Создаем транзакцию
        transaction := model.Transaction{
            UserID: params.AccountID,           // UserID теперь - это accountID
            Amount: int64(params.Amount * 100), // конвертируем в копейки
            Status: model.TransactionStatusPending,
            Type:   model.TransactionTypeWithdraw,
        }

        transactionRepo := mapper.TransactionToRepoTransaction(transaction)
        res := tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&transactionRepo)
        if res.Error != nil {
            r.logger.Err(res.Error).Msg("failed to create transaction in withdraw")
            return fmt.Errorf("failed to create transaction: %w", res.Error)
        }

        // Создаем запись кредита (уменьшение баланса)
        entry := model.TransactionEntry{
            TransactionID: transactionRepo.ID,
            AccountID:     params.AccountID,
            Direction:     model.TransactionEntryDirectionCredit,
            Amount:        params.Amount,
        }

        entryRepo := mapper.TransactionEntryToRepoTransactionEntry(entry)
        res = tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&entryRepo)
        if res.Error != nil {
            r.logger.Err(res.Error).Msg("failed to create transaction entry in withdraw")
            return fmt.Errorf("failed to create transaction entry: %w", res.Error)
        }

        // Получаем детали созданной транзакции (оставляем в статусе pending)
        transactionRepo.Status = string(model.TransactionStatusPending)
    })
}

```

```
    result = model.TransactionDetails{
        Transaction: mapper.RepoTransactionToTransaction(transactionRepo),
        Entries:
            []model.TransactionEntry{mapper.RepoTransactionEntryToTransactionEntry(entryRepo)},
    }

    return nil
})

if err != nil {
    return model.TransactionDetails{}, err
}

return result, nil
}

// Transfer выполняет операцию перевода средств в рамках одной транзакции БД
func (r *Repository) Transfer(ctx context.Context, params model.TransferParams)
(model.TransactionDetails, error) {
    var result model.TransactionDetails

    err := r.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        // Создаем транзакцию
        transaction := model.Transaction{
            UserID: params.UserID,           // UserID - автор перевода
            Amount: int64(params.Amount * 100), // конвертируем в копейки
            Status: model.TransactionStatusPending,
            Type:    model.TransactionTypeTransfer,
        }

        transactionRepo := mapper.TransactionToRepoTransaction(transaction)
        res := tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&transactionRepo)
        if res.Error != nil {
            r.logger.Err(res.Error).Msg("failed to create transaction in transfer")
            return fmt.Errorf("failed to create transaction: %w", res.Error)
        }

        // Создаем запись кредита для счета отправителя (уменьшение баланса)
        creditEntry := model.TransactionEntry{
            TransactionID: transactionRepo.ID,
            AccountID:     params.UserID,
            Direction:     model.TransactionEntryDirectionCredit,
            Amount:        params.Amount,
        }

        creditEntryRepo := mapper.TransactionEntryToRepoTransactionEntry(creditEntry)
        res = tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&creditEntryRepo)
```

```

    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to create credit transaction entry in transfer")
        return fmt.Errorf("failed to create credit transaction entry: %w", res.Error)
    }

    // Создаем запись дебета для счета получателя (увеличение баланса)
    debitEntry := model.TransactionEntry{
        TransactionID: transactionRepo.ID,
        AccountID:     params.Recipient,
        Direction:     model.TransactionEntryDirectionDebit,
        Amount:        params.Amount,
    }

    debitEntryRepo := mapper.TransactionEntryToRepoTransactionEntry(debitEntry)
    res = tx.Clauses(clause.OnConflict{UpdateAll: true}).Create(&debitEntryRepo)
    if res.Error != nil {
        r.logger.Err(res.Error).Msg("failed to create debit transaction entry in transfer")
        return fmt.Errorf("failed to create debit transaction entry: %w", res.Error)
    }

    // Получаем детали созданной транзакции (оставляем в статусе pending)
    transactionRepo.Status = string(model.TransactionStatusPending)
    result = model.TransactionDetails{
        Transaction: mapper.RepoTransactionToTransaction(transactionRepo),
        Entries: []model.TransactionEntry{
            mapper.RepoTransactionEntryToTransactionEntry(creditEntryRepo),
            mapper.RepoTransactionEntryToTransactionEntry(debitEntryRepo),
        },
    }

    return nil
})

if err != nil {
    return model.TransactionDetails{}, err
}

return result, nil
}

// UpdateTransactionStatus обновляет статус транзакции
func (r *Repository) UpdateTransactionStatus(ctx context.Context, transactionID uint64,
status model.TransactionStatus) error {
    updateData := mapper.UpdateTransactionToRepoTransaction(model.UpdateTransaction{
        Status: status,
    })
}

```

```

res := r.db.WithContext(ctx).
    Model(&repomodel.Transaction{}).
    Where("id = ?", transactionID).
    Updates(&updateData)

if res.Error != nil {
    r.logger.Err(res.Error).Uint64("transaction_id", transactionID).Str("status",
        string(status)).Msg("failed to update transaction status")
    return fmt.Errorf("failed to update transaction status: %w", res.Error)
}

if res.RowsAffected == 0 {
    return fmt.Errorf("transaction with id %d not found", transactionID)
}

return nil
}

```

В репозитории мы убрали установку успешного статуса прямо внутри транзакции и вынесли обновление статуса в отдельный метод `UpdateTransactionStatus` (листинг 4.25).

Листинг 4.25. Transaction: internal/service/service.go

```

package service

import (
    "context"

    "transaction/internal/model"

    "github.com/rs/zerolog"
)

type TransactionService struct {
    repo Repository
    logger *zerolog.Logger

    accountService AccountService
}

func New(repo Repository, accountService AccountService, logger *zerolog.Logger)
*TransactionService {
    return &TransactionService{
        repo:      repo,
        accountService: accountService,
        logger:    logger,
    }
}

```



```

type Repository interface {
    // Transaction methods
    GetTransactions(context.Context, model.GetTransactionsParams) ([]model.Transaction, error)

    // TransactionEntry methods
    GetTransactionDetails(context.Context, uint64) (model.TransactionDetails, error)

    // Business operations
    Deposit(context.Context, model.DepositParams) (model.TransactionDetails, error)
    Withdraw(context.Context, model.WithdrawParams) (model.TransactionDetails, error)
    Transfer(context.Context, model.TransferParams) (model.TransactionDetails, error)

    // Update transaction status
    UpdateTransactionStatus(context.Context, uint64, model.TransactionStatus) error
}

type AccountService interface {
    GetBalance(ctx context.Context, userID uint64) (int64, error)
    Deposit(ctx context.Context, userID uint64, amount int64, operationID uint64) (int64, error)
    Withdraw(ctx context.Context, userID uint64, amount int64, operationID uint64) (int64, error)
    Transfer(ctx context.Context, fromAccountID, toAccountID uint64, amount int64, operationID
                                                    uint64) (string, error)
}

func (s *TransactionService) GetTransactionsWithDetails(ctx context.Context, params
model.GetTransactionsParams) ([]model.TransactionDetails, error) {
    // Получаем список транзакций с фильтрацией
    transactions, err := s.repo.GetTransactions(ctx, params)
    if err != nil {
        s.logger.Error().Err(err).Msg("failed to get transactions")
        return nil, err
    }

    // Для каждой транзакции получаем детали
    var transactionDetails []model.TransactionDetails
    for _, transaction := range transactions {
        details, err := s.repo.GetTransactionDetails(ctx, transaction.ID)
        if err != nil {
            s.logger.Error().Err(err).Uint64("transaction_id", transaction.ID).Msg("failed
                                                    to get transaction details")
            continue
        }
        transactionDetails = append(transactionDetails, details)
    }

    return transactionDetails, nil
}

```

```
// Deposit Business logic methods
func (s *TransactionService) Deposit(ctx context.Context, userID uint64, amount float64)
(model.TransactionDetails, error) {
    params := model.DepositParams{
        UserID: userID,
        Amount: amount,
    }

    // Создаем транзакцию в pending статусе
    res, err := s.repo.Deposit(ctx, params)
    if err != nil {
        s.logger.Error().Err(err).Uint64("user_id", userID).Msg("failed to create deposit
                                                                    transaction")

        return model.TransactionDetails{}, err
    }

    // Выполняем операцию с балансом через account service
    _, err = s.accountService.Deposit(ctx, userID, int64(amount*100), res.Transaction.ID)
                                                                    // конвертируем в копейки
    if err != nil {
        s.logger.Error().Err(err).Uint64("user_id", userID).Uint64("transaction_id",
                                                                    res.Transaction.ID).Msg("failed to deposit to account")
        // Обновляем статус транзакции на failed
        updateErr := s.repo.UpdateTransactionStatus(ctx, res.Transaction.ID,
                                                                    model.TransactionStatusFailed)
        if updateErr != nil {
            s.logger.Error().Err(updateErr).Uint64("transaction_id",
                                                                    res.Transaction.ID).Msg("failed to update transaction status to failed")
        }
        return model.TransactionDetails{}, err
    }

    // Обновляем статус транзакции на completed
    err = s.repo.UpdateTransactionStatus(ctx, res.Transaction.ID, model.TransactionStatusCompleted)
    if err != nil {
        s.logger.Error().Err(err).Uint64("transaction_id", res.Transaction.ID).Msg("failed to
                                                                    update transaction status to completed")

        return model.TransactionDetails{}, err
    }

    // Обновляем статус в возвращаемом результате
    res.Transaction.Status = model.TransactionStatusCompleted
    return res, nil
}

func (s *TransactionService) Withdraw(ctx context.Context, accountID uint64, amount float64)
(model.TransactionDetails, error) {
    params := model.WithdrawParams{
        AccountID: accountID,

```

```

    Amount:    amount,
}

// Создаем транзакцию в pending статусе
res, err := s.repo.Withdraw(ctx, params)
if err != nil {
    s.logger.Error().Err(err).Uint64("account_id", accountID).Msg("failed to create
                                                                    withdraw transaction")
    return model.TransactionDetails{}, err
}

// Выполняем операцию с балансом через account service
_, err = s.accountService.Withdraw(ctx, accountID, int64(amount*100), res.Transaction.ID)
                                                                    // конвертируем в копейки
if err != nil {
    s.logger.Error().Err(err).Uint64("account_id", accountID).Uint64("transaction_id",
                                                                    res.Transaction.ID).Msg("failed to withdraw from account")
    // Обновляем статус транзакции на failed
    updateErr := s.repo.UpdateTransactionStatus(ctx, res.Transaction.ID,
                                                                    model.TransactionStatusFailed)
    if updateErr != nil {
        s.logger.Error().Err(updateErr).Uint64("transaction_id",
                                                                    res.Transaction.ID).Msg("failed to update transaction status to failed")
    }
    return model.TransactionDetails{}, err
}

// Обновляем статус транзакции на completed
err = s.repo.UpdateTransactionStatus(ctx, res.Transaction.ID,
                                                                    model.TransactionStatusCompleted)
if err != nil {
    s.logger.Error().Err(err).Uint64("transaction_id", res.Transaction.ID).Msg("failed to
                                                                    update transaction status to completed")
    return model.TransactionDetails{}, err
}

// Обновляем статус в возвращаемом результате
res.Transaction.Status = model.TransactionStatusCompleted
return res, nil
}

func (s *TransactionService) Transfer(ctx context.Context, userID, recipient uint64, amount
float64) (model.TransactionDetails, error) {
    params := model.TransferParams{
        UserID:    userID,
        Recipient: recipient,
        Amount:    amount,
    }
}

```

```
// Создаем транзакцию в pending statuе
res, err := s.repo.Transfer(ctx, params)
if err != nil {
    s.logger.Error().Err(err).Uint64("user_id", userID).Uint64("recipient",
        recipient).Msg("failed to create transfer transaction")
    return model.TransactionDetails{}, err
}

// Выполняем операцию с балансом через account service
_, err = s.accountService.Transfer(ctx, userID, recipient, int64(amount*100),
    res.Transaction.ID) // конвертируем в копейки
if err != nil {
    s.logger.Error().Err(err).Uint64("user_id", userID).Uint64("recipient",
        recipient).Uint64("transaction_id", res.Transaction.ID).Msg("failed to transfer
        between accounts")

    // Обновляем статус транзакции на failed
    updateErr := s.repo.UpdateTransactionStatus(ctx, res.Transaction.ID,
        model.TransactionStatusFailed)
    if updateErr != nil {
        s.logger.Error().Err(updateErr).Uint64("transaction_id",
            res.Transaction.ID).Msg("failed to update transaction status to failed")
    }
    return model.TransactionDetails{}, err
}

// Обновляем статус транзакции на completed
err = s.repo.UpdateTransactionStatus(ctx, res.Transaction.ID,
    model.TransactionStatusCompleted)
if err != nil {
    s.logger.Error().Err(err).Uint64("transaction_id", res.Transaction.ID).Msg("failed to
        update transaction status to completed")
    return model.TransactionDetails{}, err
}

// Обновляем статус в возвращаемом результате
res.Transaction.Status = model.TransactionStatusCompleted
return res, nil
}
```

А в самой логике переделали суть так, что теперь сначала создается транзакция, потом изменяется баланс у пользователя, и только после этого транзакция меняет статус на `completed`.

Проблема распределенных транзакций

Сейчас наша транзакция реализована таким образом, что в ней производятся запросы в другой сервис — в Account. И если в случае ошибки транзакции откатятся, то изменения в Account останутся актуальными. Можно — в случае ошибки — реали-

зовать обратное списание и пополнение средств, только вот если ошибка возникнет как раз в запросе изменения баланса, есть шанс, что мы в попытке откатить изменения только всё усугубим. Для пояснения предположим, что у первого пользователя запрос на изменение баланса не выполнен, — тогда ему на счет будут зачислены средства, а у пользователя, которому перевод предназначался, произойдет списание. То есть окажется, что запрос создавался на перевод от Саши к Пете, а при откате изменений произошел перевод от Пети к Саше. Что абсолютно неправильно, и такого допускать нельзя.

Можно попытаться получать статус ответа запроса, проверять, на каком этапе транзакция сломалась, и в зависимости от этого выполнять определенные действия по откату. Но как быть, если с сервисом просто что-то случится — банально выключат электричество? Мы не сможем достучаться до Account, и в ответе нам будет приходиться сообщение об ошибке, связанной с тем, что время запроса истекло. Транзакция таким образом может зависнуть на неопределенное время, а если таких транзакций окажется еще и много, весьма вероятно, что это повлияет на работоспособность всей системы.

Такая транзакция, которая распространяется на несколько баз данных, во множестве систем называется *распределенной*. Главная проблема, которая возникает при этом, — несогласованность данных, т. е. нарушение принципов ACID. Есть два способа решения проблем, связанных с распределенными транзакциями (они описаны далее), однако совет номер один: постараться полностью их избежать. К сожалению, в больших системах это практически невозможно.

Двухфазная фиксация

Как можно понять из названия, двухфазная фиксация состоит из двух фаз: подготовки данных и их фиксации. При этом должен существовать координатор транзакций, который отвечает за их жизненный цикл.

На первом этапе все микросервисы, участвующие в транзакции, выполняют необходимые для фиксации подготовительные работы и уведомляют координатора о том, что готовы завершить транзакцию. На втором этапе либо происходит фиксация, либо координатор выдает команду всем микросервисам произвести откат [6].

Если применить этот подход к разрабатываемой нами системе, то получится, что нам понадобится отдельный координатор транзакций, который при получении запроса от пользователя на проведение операции направлял бы сервису Account команду зарезервировать средства на балансе, а сервису Transaction — команду подготовить данные для сохранения транзакции. Когда сервисы Account и Transaction будут готовы выполнить запрос, они заблокируют записи в своих таблицах от дальнейших изменений и уведомят об этом координатора транзакций. Как только координатор транзакций подтвердит, что Account и Transaction готовы внести свои изменения, он даст им команду сохранить их, запросив фиксацию транзакции. В этот момент все записи, участвующие в транзакции, будут разблокированы. Если же вдруг какой-либо из сервисов, Transaction или Account, не успеет приготовиться к фиксации, координатор транзакций отменит транзакцию и начнет сценарий отка-

та. Когда все данные будут приведены в консистентное состояние — с успешным выполнением транзакции или нет, — координатор транзакций запросит разблокировку объектов баз данных.

Такой подход обеспечивает атомарность операции — она выполнится в любом случае: либо когда все участники транзакции внесут свои изменения, либо если они не внесут никаких изменений. Обеспечивает он также и изолированность транзакций, поскольку у других транзакций к изменениям не будет доступа, пока координатор эти изменения не зафиксирует. Причем весь процесс выполняется синхронно — т. е. сразу после его завершения можно будет уведомить пользователя о результате выполнения его запроса.

Отрицательные моменты этого подхода заключаются в том, что описанное взаимодействие способно значительно снизить скорость работы системы, поскольку осуществляется гораздо медленнее, чем если бы операции выполнялись в одном микросервисе. К тому же все микросервисы здесь сильно зависят от координатора транзакций, что также способно неблагоприятно сказаться на скорости работы системы в период высокой нагрузки. Да и блокировка записей в базах данных может создать узкое место, тоже влияющее на производительность, причем не исключено возникновение взаимных блокировок, когда две транзакции мешают друг другу выполняться.

Saga

Saga — это еще один паттерн, предложенный для решения проблемы распределенных транзакций, и к тому же самый распространенный. Определение Saga [7]:

- ☐ каждый сервис публикует событие всякий раз, когда обновляет свои данные;
- ☐ другие сервисы подписываются на события;
- ☐ при получении события сервис обновляет свои данные.

Если попытаться адаптировать этот паттерн под нашу систему, то получится, что сервисы Transaction и Account должны будут выполнять свои операции отдельно и асинхронно и обмениваться результатами через шину событий. В качестве шины событий чаще всего выступают брокеры сообщений, которые мы уже рассматривали ранее.

Разберемся с этим подробнее. Первое, с чего начинается наша транзакция, — сохранение записи в базу данных transaction-db, после чего идет вызов в сервис Account на списание либо пополнение баланса. На этом этапе, если действовать согласно Saga, мы должны отправить в брокер сообщений событие, что транзакция сохранена. А со стороны Account должны подписаться на это событие, и когда получим сигнал о том, что на стороне сервиса Transaction всё выполнено, приступим к изменению баланса. Однако здесь появляется промежуток времени, когда транзакция будет видна, а баланс еще не изменен. Это можно смягчить, добавив для транзакций статусы. Например, при первой итерации в сервисе Transaction установить статус «created» или «in processed» — т. е. транзакция создана. Когда Account изменит баланс у себя, то Transaction будет ожидать соответствующего события, при получении которого изменит статус на «completed». При этом для пользователя

транзакция будет отображаться в списке операций, но не сможет его смутить. То, что он увидит транзакцию, которая находится еще в процессе, и изменение баланса, вызовет у него меньшее замешательство, чем простое сообщение о транзакции без информации о ее состоянии и изменении баланса.

И по такому же принципу будет работать кейс с ошибочной транзакцией. Если в сервисе Account по каким-то причинам не удалось пополнить или списать средства, то об этом публикуется событие, и сервис Transaction выставляет для транзакции статус «failed». На любом из этапов нужно иметь в виду и обратное событие, которое может откатить изменения для сохранения консистентной базы данных. Пример схемы взаимодействия системы для нашего случая получится простым (рис. 4.1), а для более сложных систем, конечно же, взаимодействий может быть значительно больше.

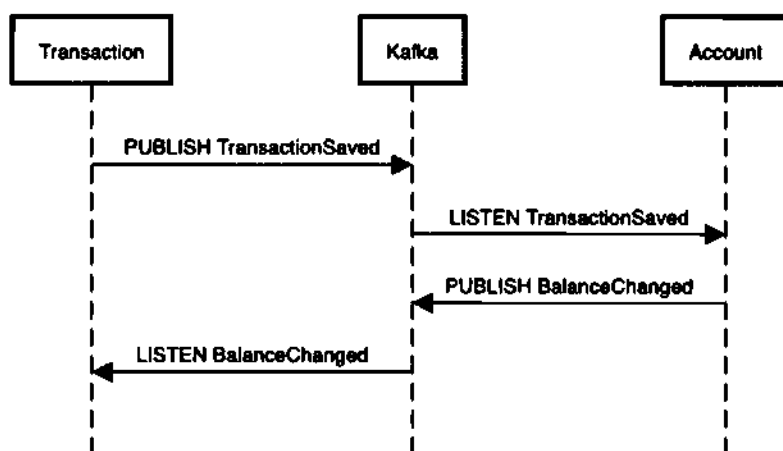


Рис. 4.1. Схема взаимодействия системы

Главное преимущество такого подхода заключается в том, что каждый микросервис сосредоточен на выполнении только своей конкретной части атомарной транзакции, а распределенные транзакции как таковые не используются вовсе. И сами микросервисы при этом слабо связаны между собой, потому что общение происходит лишь на основе событий, и они ничего не знают друг о друге. В результате работа микросервисов не блокируется, если выполнение операции в одном из сервисов занимает слишком много времени. Благодаря тому, что паттерн Saga основан на асинхронном взаимодействии на основе событий, система способна справляться с большими нагрузками и позволяет обеспечить необходимую масштабируемость в будущем.

Однако этот подход является довольно сложным, поскольку разработчику системы необходимо создать не только успешный сценарий, но и компенсирующие транзакции, которые явно отменяют изменения, сделанные ранее. Кроме того, когда разработчик только погружается в систему, построенную подобным образом, у него могут возникнуть сложности с прослеживанием полной цепочки взаимодействий между сервисами и пониманием того, как работает цепочка событий.

Главным же недостатком такого подхода является нарушение изолированности. А конкретно — в нем не обеспечивается изоляция при чтении. В нашем случае при рассмотрении применения паттерна мы частично решили эту проблему установкой статуса транзакции «in processed» — другие транзакции, видя этот статус, будут знать, что операция еще не завершена. Есть также для предотвращения возникновения аномалий и другие контрмеры, которые хотя бы пытаются минимизировать последствия для бизнеса [6].

Реализация паттерна Saga

В качестве брокера сообщений для транзакций выберем Kafka: во-первых, чтобы на примере посмотреть, как с ним работать, а во-вторых, в случае с такими чувствительными данными, как платежи, хотелось бы, чтобы информация о них сохранялась и после использования.

Начнем с подключения к Kafka. Разворачивать Kafka будем так же, как и остальные инструменты, — через Docker-Compose. Удалить существующий Docker-Compose полностью можно с помощью команды:

```
docker-compose down -v
```

Помимо самого брокера Kafka (рис. 4.2), добавим еще и интерфейс Kafka UI (kafka-ui), чтобы через него можно было бы просматривать отправленные в брокер сообщений события (листинг 4.26).

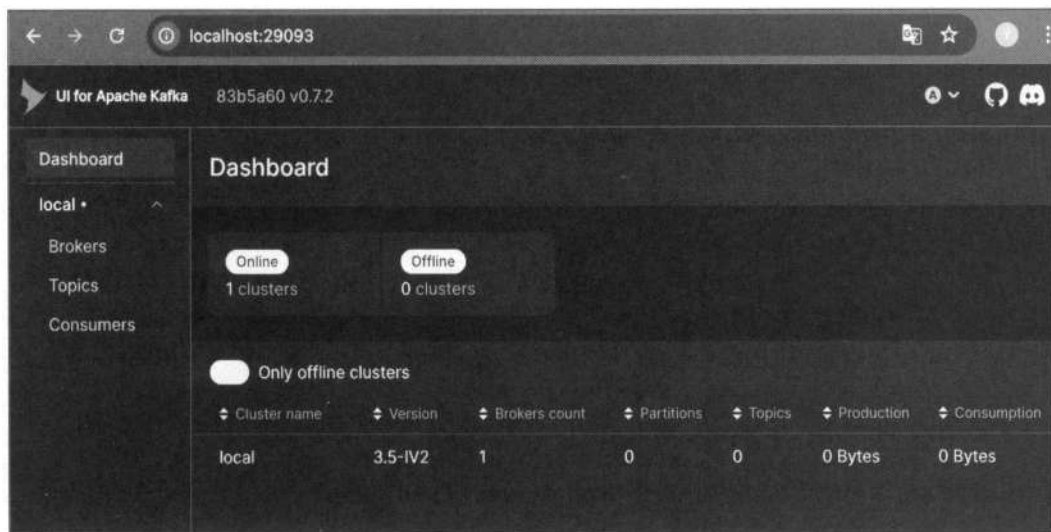


Рис. 4.2. Окно Kafka UI

Листинг 4.26. Файл docker/docker-compose.yml

```
services:
  account:
    image: postgres:15-alpine
```



```
environment:
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: postgres
  POSTGRES_DB: account
ports:
  - "5432:5432"
volumes:
  - postgres_account_data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 5s
  timeout: 5s
  retries: 5
networks:
  - app-network

auth:
  image: postgres:15-alpine
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: auth
  ports:
    - "5433:5432"
  volumes:
    - postgres_auth_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - app-network

transaction:
  image: postgres:15-alpine
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: transaction
  ports:
    - "5434:5432"
  volumes:
    - postgres_transaction_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
```

```
    timeout: 5s
    retries: 5
networks:
  - app-network

kafka:
  image: confluentinc/cp-kafka:7.5.0
  container_name: book-kafka
  platform: linux/x86_64
  environment:
    CLUSTER_ID: 5DG6k02sQY6c8l0fs8nG5A
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER

    # объявляем все виды listeners, в том числе PLAINTEXT_HOST
    KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,PLAINTEXT_HOST://:29092

    # объявляем external/host endpoints
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092

    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
      PLAINTEXT:PLAINTEXT, PLAINTEXT_HOST:PLAINTEXT, CONTROLLER:PLAINTEXT
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093

    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
    KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
    KAFKA_LOG_DIRS: /var/lib/kafka/data
  ports:
    - "29092:29092" # внешний порт
    - "9092:9092"   # внутренняя сеть docker
  restart: unless-stopped

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  container_name: book-kafka-ui
  platform: linux/x86_64
  environment:
    - KAFKA_CLUSTERS_0_NAME=local
    - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:9092
  depends_on:
    - kafka
  ports:
    - "29093:8080"
  restart: unless-stopped
```

```
volumes:
  postgres_account_data:
  postgres_auth_data:
  postgres_transaction_data:

networks:
  app-network:
    driver: bridge
```

Подключение к Kafka происходит через github.com/segmentio/kafka-go — это самая популярная Go-библиотека, которая к тому же написана полностью на Go.

Настройку начнем с микросервиса Transaction: вынесем взаимодействие с Kafka в отдельный пакет и начнем с конфигурации, т. к. опций много, и для нашей задачи они все не нужны (листинг 4.27).

Листинг 4.27. Transaction: internal/kafka/config.go

```
package kafka

import (
    "time"

    "github.com/segmentio/kafka-go"
)

// Config содержит конфигурацию для Kafka
type Config struct {
    Brokers []string
    GroupID string
    Topics []string
}

// ProducerConfig содержит конфигурацию для Kafka Producer
type ProducerConfig struct {
    Brokers      []string
    BatchSize    int
    BatchTimeout time.Duration
    RequiredAcks kafka.RequiredAcks
}

// ConsumerConfig содержит конфигурацию для Kafka Consumer
type ConsumerConfig struct {
    Brokers      []string
    GroupID      string
    Topics       []string
    MinBytes     int
    MaxBytes     int
}
```

```

MaxWait          time.Duration
ReadBatchTimeout time.Duration
HeartbeatInterval time.Duration
CommitInterval   time.Duration
}

// DefaultProducerConfig возвращает конфигурацию по умолчанию для Producer
func DefaultProducerConfig(brokers []string) ProducerConfig {
    return ProducerConfig{
        Brokers:      brokers,
        BatchSize:     100,
        BatchTimeout: 10 * time.Millisecond,
        RequiredAcks: kafka.RequireOne,
    }
}

// DefaultConsumerConfig возвращает конфигурацию по умолчанию для Consumer
func DefaultConsumerConfig(brokers []string, groupID string, topics []string) ConsumerConfig {
    return ConsumerConfig{
        Brokers:      brokers,
        GroupID:      groupID,
        Topics:      topics,
        MinBytes:     10e3, // 10KB
        MaxBytes:     10e6, // 10MB
        MaxWait:      1 * time.Second,
        ReadBatchTimeout: 1 * time.Second,
        HeartbeatInterval: 1 * time.Second,
        CommitInterval: time.Second,
    }
}

```

Тут мы отдельно определили конфигурации для продюсера и консюмера, в которых задали некоторые значения по умолчанию. Это позволяет централизованно определять параметры подключения к брокерам, особенности работы с сообщениями (размер пачки, тайм-ауты и подтверждения), а также управлять настройками групп потребителей. А параметры — такие как идентификатор группы, топика и адреса подключений — задаем чем переменные окружения сервиса (листинг 4.28).

Листинг 4.28. Transaction: internal/config/config.go

```

package config

import (
    "log"

    "github.com/caarlos0/env/v10"
    "github.com/joho/godotenv"
)

```

```
// Config содержит конфигурацию приложения
type Config struct {
    ServiceName    string `env:"SERVICE_NAME" json:"service_name" required:"true"
                                                default:"transaction-service"`
    AppEnv         string `env:"APP_ENV" json:"app_environment" required:"true"
                                                default:"development"`
    Host           string `env:"GRPC_HOST" json:"host" required:"true" default:"localhost"`
    Port           int    `env:"GRPC_PORT" json:"port" required:"true" default:"9000"`
    LogLevel       string `env:"LOG_LEVEL" json:"log_level" required:"true" default:"info"`
    DbDsn          string `env:"DB_DSN" json:"db_dsn" required:"true"`
    AccountGrpcHost string `env:"ACCOUNT_GRPC_HOST" json:"account_grpc_host" required:"true"`

    KafkaBrokers    []string `env:"KAFKA_BROKER_HOST" envSeparator:", "
                                                json:"kafka_brokers" required:"true"`
    KafkaGroupID     string   `env:"KAFKA_CONSUMER_GROUP" json:"kafka_consumer_group"
                                                required:"true"`
    KafkaTransactionTopic string `env:"KAFKA_TRANSACTION_TOPIC"
                                                json:"kafka_transaction_topic" required:"true"`
}

// Load загружает конфигурацию из переменных окружения
func Load() (*Config, error) {
    // Загружаем .env файл
    if err := godotenv.Load(); err != nil {
        log.Println("Warning: .env file not found, using system environment variables")
    }

    cfg := &Config{}
    err := env.Parse(cfg)
    if err != nil {
        return nil, err
    }

    return cfg, nil
}
```

Обновляем переменные окружения в соответствии с ожиданиями конфига (листинг 4.29).

Листинг 4.29. Transaction: .env

```
SERVICE_NAME=transaction
APP_ENV=dev

GRPC_PORT=50054
GRPC_HOST=localhost
LOG_LEVEL=debug
```

```
DB_DSN=postgres://postgres:postgres@localhost:5434/transaction?sslmode=disable
```

```
ACCOUNT_GRPC_HOST=localhost:50051
```

```
KAFKA_BROKER_HOST='localhost:29092'
```

```
KAFKA_CONSUMER_GROUP=consumer
```

```
KAFKA_TRANSACTION_TOPIC=transaction_response
```

Помимо отдельной реализации конфигурации, также отдельно вынесем работу с producer, consumer и методами для основного использования (листинг 4.30).

Листинг 4.30. Transaction: internal/kafka/producer.go

```
package kafka

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "github.com/rs/zerolog"
    "github.com/segmentio/kafka-go"
)

// Producer представляет Kafka Producer
type Producer struct {
    writer *kafka.Writer
    logger *zerolog.Logger
}

// Message представляет сообщение для отправки в Kafka
type Message struct {
    Topic    string
    Key      []byte
    Value    []byte
    Headers  map[string]string
    Partition int
}

// NewProducer создает новый Kafka Producer
func NewProducer(cfg ProducerConfig, logger *zerolog.Logger) *Producer {
    writer := &kafka.Writer{
        Addr:      kafka.TCP(cfg.Brokers...),
        Balancer:  &kafka.LeastBytes{},
        BatchSize: cfg.BatchSize,
        BatchTimeout: cfg.BatchTimeout,
```

```

    RequiredAcks: cfg.RequiredAcks,
    Async:       false,
}

return &Producer{
    writer: writer,
    logger: logger,
}
}

// SendMessage отправляет сообщение в Kafka
func (p *Producer) SendMessage(ctx context.Context, msg Message) error {
    kafkaMsg := kafka.Message{
        Topic:    msg.Topic,
        Key:      msg.Key,
        Value:    msg.Value,
        Partition: msg.Partition,
        Time:     time.Now(),
    }

    // Добавляем заголовки
    for key, value := range msg.Headers {
        kafkaMsg.Headers = append(kafkaMsg.Headers, kafka.Header{
            Key:    key,
            Value: []byte(value),
        })
    }

    err := p.writer.WriteMessages(ctx, kafkaMsg)
    if err != nil {
        p.logger.Error().
            Err(err).
            Str("topic", msg.Topic).
            Str("key", string(msg.Key)).
            Msg("failed to send message to kafka")
        return fmt.Errorf("failed to send message to kafka: %w", err)
    }

    p.logger.Debug().
        Str("topic", msg.Topic).
        Str("key", string(msg.Key)).
        Msg("message sent to kafka successfully")

    return nil
}

// SendJSONMessage отправляет JSON-сообщение в Kafka
func (p *Producer) SendJSONMessage(ctx context.Context, topic string, key string, data
interface{}) error {
    value, err := json.Marshal(data)

```

```
if err != nil {
    return fmt.Errorf("failed to marshal message to JSON: %w", err)
}

msg := Message{
    Topic: topic,
    Key:   []byte(key),
    Value: value,
    Headers: map[string]string{
        "content-type": "application/json",
    },
}

return p.SendMessage(ctx, msg)
}

// Close закрывает Producer
func (p *Producer) Close() error {
    if p.writer != nil {
        return p.writer.Close()
    }
    return nil
}
```

Обертка над Kafka Producer нужна для того, чтобы инкапсулировать объект `Writer`, который позволяет создавать продюсер с нужной конфигурацией, отправлять в Kafka «сырые» или готовые JSON-объекты, а также безопасно закрывать соединение методом `Close`. Мы также добавили тут подробное логирование, чтобы можно было отследить данные на каждом этапе обработки (листинг 4.31).

Листинг 4.31. Transaction: internal/kafka/consumer.go

```
package kafka

import (
    "context"
    "time"

    "github.com/rs/zerolog"
    "github.com/segmentio/kafka-go"
)

// Consumer представляет Kafka Consumer
type Consumer struct {
    reader *kafka.Reader
    logger *zerolog.Logger
}
```



```
// GetConfig возвращает конфигурацию reader'a
func (c *Consumer) GetConfig() kafka.ReaderConfig {
    return c.reader.Config()
}

// ConsumerMessageHandler представляет функцию-обработчик сообщений для consumer
type ConsumerMessageHandler func(ctx context.Context, msg kafka.Message) error

// NewConsumer создает новый Kafka Consumer
func NewConsumer(cfg ConsumerConfig, logger *zerolog.Logger) *Consumer {
    reader := kafka.NewReader(kafka.ReaderConfig{
        Brokers:      cfg.Brokers,
        GroupID:      cfg.GroupID,
        Topic:        cfg.Topics[0], // Используем первый топик
        MinBytes:     cfg.MinBytes,
        MaxBytes:     cfg.MaxBytes,
        MaxWait:      cfg.MaxWait,
        ReadBatchTimeout: cfg.ReadBatchTimeout,
        HeartbeatInterval: cfg.HeartbeatInterval,
        CommitInterval:  cfg.CommitInterval,
    })

    return &Consumer{
        reader: reader,
        logger: logger,
    }
}

// Start начинает потребление сообщений
func (c *Consumer) Start(ctx context.Context, handler ConsumerMessageHandler) error {
    c.logger.Info().
        Str("topic", c.reader.Config().Topic).
        Str("group_id", c.reader.Config().GroupID).
        Msg("starting kafka consumer")

    for {
        select {
        case <-ctx.Done():
            c.logger.Info().Msg("stopping kafka consumer")
            return ctx.Err()
        default:
            msg, err := c.reader.ReadMessage(ctx)
            if err != nil {
                c.logger.Error().Err(err).Msg("failed to read message from kafka")
                time.Sleep(time.Second)
                continue
            }
        }
    }
}
```

```

// Обрабатываем сообщение
if err := handler(ctx, msg); err != nil {
    c.logger.Error().
        Err(err).
        Str("topic", msg.Topic).
        Int("partition", msg.Partition).
        Int64("offset", msg.Offset).
        Msg("failed to process message")
} else {
    c.logger.Debug().
        Str("topic", msg.Topic).
        Int("partition", msg.Partition).
        Int64("offset", msg.Offset).
        Msg("message processed successfully")
}
}
}

// Close закрывает Consumer
func (c *Consumer) Close() error {
    if c.reader != nil {
        return c.reader.Close()
    }
    return nil
}

```

Также поступим и с Kafka Consumer — написали для него свою обертку. Тут создается клиент для чтения сообщений из Kafka согласно установленной конфигурации и запускается бесконечный цикл чтения сообщений (Start), который передает их во внешний обработчик-функцию с логированием успешной обработки или ошибок, корректно завершается при отмене контекста и предоставляет метод Close для закрытия reader'a (листинг 4.32).

Листинг 4.32. Transaction: internal/kafka/kafka.go

```

package kafka

import (
    "context"
    "fmt"

    "github.com/rs/zerolog"
    "github.com/segmentio/kafka-go"
)

// Client представляет универсальный интерфейс для работы с Kafka
type Client interface {

```

```

// Publish отправляет сообщение в указанный топик
Publish(ctx context.Context, topic string, key string, data interface{}) error

// Subscribe подписывается на топик и обрабатывает сообщения
Subscribe(ctx context.Context, topic string, handler MessageHandler) error

// Close закрывает все соединения
Close() error
}

// MessageHandler представляет функцию-обработчик сообщений
type MessageHandler func(ctx context.Context, topic string, key string, data []byte) error

// Kafka представляет основной клиент для работы с Kafka
type Kafka struct {
    producer *Producer
    brokers  []string
    groupID  string
    logger   *zerolog.Logger
}

// New создает новый экземпляр Kafka
func New(producer *Producer, brokers []string, groupID string, logger *zerolog.Logger) *Kafka {
    return &Kafka{
        producer: producer,
        brokers:  brokers,
        groupID:  groupID,
        logger:   logger,
    }
}

// Publish отправляет сообщение в указанный топик
func (k *Kafka) Publish(ctx context.Context, topic string, key string, data interface{}) error {
    return k.producer.SendJSONMessage(ctx, topic, key, data)
}

// Subscribe подписывается на топик и обрабатывает сообщения
func (k *Kafka) Subscribe(ctx context.Context, topic string, handler MessageHandler) error {
    // Создаем уникальный GroupID для каждого топика
    uniqueGroupID := k.groupID + "_" + topic
    cfg := DefaultConsumerConfig(k.brokers, uniqueGroupID, []string{topic})
    consumer := NewConsumer(cfg, k.logger)

    // Запускаем consumer в отдельной горутине
    go func() {
        k.logger.Info().

```

```

    Str("topic", topic).
    Str("group_id", uniqueGroupID).
    Msg("subscribing to topic")
    messageHandler := ConsumerMessageHandler(func(ctx context.Context, msg kafka.Message)
                                                error {
            return handler(ctx, msg.Topic, string(msg.Key), msg.Value)
        })
    consumer.Start(ctx, messageHandler)
}()

return nil
}

// Close закрывает все соединения с Kafka
func (k *Kafka) Close() error {
    if k.producer != nil {
        if err := k.producer.Close(); err != nil {
            return fmt.Errorf("failed to close producer: %w", err)
        }
    }

    return nil
}

```

Теперь объединим логику продюсера и консюмера в единый клиент Kafka. Он определяет интерфейс `Client` с методами `Publish`, `Subscribe` и `Close`, а структура `Kafka` реализует его, используя ранее описанный `Producer` и динамически создавая `Consumer` под каждую подписку. Метод `Publish` упрощает отправку сообщений, автоматически сериализуя данные в JSON, а метод `Subscribe` создает для каждого топика отдельный `Consumer` с уникальным `group_id`, запуская его в горутине и передавая сообщения в пользовательский обработчик. Таким образом, этот слой обеспечивает простой и единообразный API для сервисов, которым нужно публиковать сообщения в Kafka, и подписываться на них.

На этом подготовку для работы с Kafka можно считать завершенной, остается только подключить ее и передать в сервисный слой (листинг 4.33).

Листинг 4.33. Transaction: internal/app/app.go

```

package app

import (
    "context"
    "database/sql"
    "fmt"
    "net"

    "transaction/internal/account"
    "transaction/internal/config"

```

```

"transaction/internal/kafka"
_ "transaction/internal/migrations"
"transaction/internal/repository"
"transaction/internal/server"
"transaction/internal/service"

"github.com/pressly/goose/v3"
accountpb "github.com/yuliaapopova/contracts/account/go"
transactionpb "github.com/yuliaapopova/contracts/transaction/go"
_ "google.golang.org/genproto/googleapis/api/annotations"
"google.golang.org/grpc"

_ "github.com/lib/pq"
"github.com/rs/zerolog"
"gorm.io/driver/postgres"
"gorm.io/gorm"
}

type App struct {
    cfg      *config.Config
    logger   *zerolog.Logger

    transactionRepository *repository.Repository
    transactionService    *service.TransactionService
    transactionServer     *server.Server
    grpcServer            *grpc.Server

    accountService *account.Service
    kafkaClient    *kafka.Kafka
}

func New(logger *zerolog.Logger, cfg *config.Config) *App {
    return &App{
        cfg:    cfg,
        logger: logger,
    }
}

func (a *App) Run(ctx context.Context) error {
    // Инициализируем gRPC сервер
    transactionServer, err := a.getTransactionServer(ctx)
    if err != nil {
        return fmt.Errorf("failed to get transaction server: %w", err)
    }

    // Создаем gRPC-сервер
    a.grpcServer = getGRPCServer(transactionServer)

```

```
listenAddr := fmt.Sprintf("%s:%d", a.cfg.Host, a.cfg.Port)
lis, err := net.Listen("tcp", listenAddr)
if err != nil {
    a.logger.Fatal().Err(err).Msg("failed to listen")
    return err
}

a.logger.Info().Msg("gRPC server listening")

serveErrCh := make(chan error, 1)
go func() {
    serveErrCh <- a.grpcServer.Serve(lis)
}()

select {
case <-ctx.Done():
    a.grpcServer.GracefulStop()
    return ctx.Err()
case err := <-serveErrCh:
    if err != nil {
        a.logger.Error().Err(err).Msg("failed to serve")
    }
    return err
}
}

func (a *App) getRepository(ctx context.Context) (*repository.Repository, error) {
    if a.transactionRepository == nil {
        if err := a.runMigrations(ctx); err != nil {
            return nil, fmt.Errorf("failed to get repository: %w", err)
        }
        db, err := gorm.Open(postgres.Open(a.cfg.DbDsn), &gorm.Config{})
        if err != nil {
            return nil, fmt.Errorf("gorm init failed: %w", err)
        }
        a.transactionRepository = repository.NewRepository(db, a.logger)
    }
    return a.transactionRepository, nil
}

func (a *App) runMigrations(ctx context.Context) error {
    if err := goose.SetDialect("postgres"); err != nil {
        return fmt.Errorf("failed to select migrations dialect: %w", err)
    }

    dbGoose, err := sql.Open("postgres", a.cfg.DbDsn)
    if err != nil {
        return fmt.Errorf("failed to create sql connection: %w", err)
    }
}
```

```

    if err := goose.UpContext(ctx, dbGoose, "internal/migrations"); err != nil {
        return fmt.Errorf("failed to run up migrations: %w", err)
    }
    return nil
}

func (a *App) getTransactionService(ctx context.Context) (*service.TransactionService, error) {
    if a.transactionService == nil {
        repo, err := a.getRepository(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get repository: %w", err)
        }

        // Получаем gRPC-сервис
        grpcSvc, err := a.getAccountService(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get account service: %w", err)
        }

        // Теперь получаем Kafka-клиент и подписываемся
        kafkaClient, err := a.getKafkaClient(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get kafka client: %w", err)
        }

        // Обновляем сервис с Kafka-клиентом
        a.transactionService = service.New(repo, grpcSvc, kafkaClient, a.logger)

        a.kafkaClient.Subscribe(ctx, "transaction_response",
            a.transactionService.HandleAccountResponse)
    }
    return a.transactionService, nil
}

func (a *App) getKafkaClient(ctx context.Context) (*kafka.Kafka, error) {
    if a.kafkaClient == nil {
        // Создаем producer
        producerCfg := kafka.DefaultProducerConfig(a.cfg.KafkaBrokers)
        producer := kafka.NewProducer(producerCfg, a.logger)

        // Создаем Kafka-клиент
        kafkaClient := kafka.New(producer, a.cfg.KafkaBrokers, a.cfg.KafkaGroupID, a.logger)

        a.kafkaClient = kafkaClient

        a.logger.Info().Msg("kafka client created")
    }
}

```

```
    return a.kafkaClient, nil
}

func (a *App) getTransactionServer(ctx context.Context) (*server.Server, error) {
    if a.transactionServer == nil {
        service, err := a.getTransactionService(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get service: %w", err)
        }
        a.transactionServer = server.New(service, a.logger)
    }
    return a.transactionServer, nil
}

func (a *App) getAccountService(ctx context.Context) (*account.Service, error) {
    if a.accountService == nil {
        // Создаем gRPC-соединение с account service
        conn, err := grpc.Dial(a.cfg.AccountGrpcHost, grpc.WithInsecure())
        if err != nil {
            return nil, fmt.Errorf("failed to connect to account service: %w", err)
        }

        client := accountpb.NewAccountClient(conn)
        a.accountService = account.New(client)
    }
    return a.accountService, nil
}

func getGRPCServer(srv *server.Server) *grpc.Server {
    grpcSrv := grpc.NewServer()
    transactionpb.RegisterTransactionServiceServer(grpcSrv, srv)
    return grpcSrv
}

// Close корректно завершает работу приложения
func (a *App) Close() error {
    var errs []error

    if a.grpcServer != nil {
        a.grpcServer.GracefulStop()
    }

    if a.kafkaClient != nil {
        if err := a.kafkaClient.Close(); err != nil {
            errs = append(errs, fmt.Errorf("failed to close kafka client: %w", err))
        }
    }
}
```



```

    if len(errs) > 0 {
        return fmt.Errorf("errors closing app: %v", errs)
    }

    return nil
}

```

Тут в структуру App добавился `KafkaClient` и метод для инициализации `getKafkaClient`, который будет использоваться внутри сервиса транзакций. При инициализации `TransactionService` клиент Kafka передается в сервис и сразу же используется для подписки на топик `transaction_response`, чтобы в дальнейшем обрабатывать ответы от микросервиса Account.

Остается последний момент для сервиса транзакций — заменить gRPC-вызовы Account на отправку сообщений в Kafka, а также написать сам обработчик для получения данных из брокера сообщений (листинг 4.34).

Листинг 4.34. Transaction: internal/service/service.go

```

package service

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "transaction/internal/model"

    "github.com/rs/zerolog"
)

type TransactionService struct {
    repo Repository
    logger *zerolog.Logger

    accountService AccountService

    // Kafka support
    kafkaPublisher KafkaPublisher
}

func New(repo Repository, accountService AccountService, kafkaPublisher KafkaPublisher,
    logger *zerolog.Logger) *TransactionService {
    return &TransactionService{
        repo:          repo,
        accountService: accountService,
        kafkaPublisher: kafkaPublisher,
        logger:        logger,
    }
}

```

```
// AccountResponse представляет структуру ответа от account service
type AccountResponse struct {
    RequestType string `json:"request_type"`
    UserID      uint64 `json:"user_id"`
    OperationID uint64 `json:"operation_id"`
    Result      bool   `json:"result"`
}

type Repository interface {
    // Transaction methods
    GetTransactions(context.Context, model.GetTransactionsParams) ([]model.Transaction, error)

    // TransactionEntry methods
    GetTransactionDetails(context.Context, uint64) (model.TransactionDetails, error)

    // Business operations
    Deposit(context.Context, model.DepositParams) (model.TransactionDetails, error)
    Withdraw(context.Context, model.WithdrawParams) (model.TransactionDetails, error)
    Transfer(context.Context, model.TransferParams) (model.TransactionDetails, error)

    // Update transaction status
    UpdateTransactionStatus(context.Context, uint64, model.TransactionStatus) error
}

type AccountService interface {
    GetBalance(ctx context.Context, userID uint64) (int64, error)
    Deposit(ctx context.Context, userID uint64, amount int64, operationID uint64) (int64, error)
    Withdraw(ctx context.Context, userID uint64, amount int64, operationID uint64) (int64, error)
    Transfer(ctx context.Context, fromAccountID, toAccountID uint64, amount int64,
                                                    operationID uint64) (string, error)
}

// KafkaPublisher интерфейс для публикации сообщений в Kafka
type KafkaPublisher interface {
    Publish(ctx context.Context, topic string, key string, data interface{}) error
}

func (s *TransactionService) GetTransactionsWithDetails(ctx context.Context, params
model.GetTransactionsParams) ([]model.TransactionDetails, error) {
    // Получаем список транзакций с фильтрацией
    transactions, err := s.repo.GetTransactions(ctx, params)
    if err != nil {
        s.logger.Error().Err(err).Msg("failed to get transactions")
        return nil, err
    }

    // Для каждой транзакции получаем детали
    var transactionDetails []model.TransactionDetails

```

```

for _, transaction := range transactions {
    details, err := s.repo.GetTransactionDetails(ctx, transaction.ID)
    if err != nil {
        s.logger.Error().Err(err).Uint64("transaction_id", transaction.ID).Msg("failed to
                                get transaction details")
        continue
    }
    transactionDetails = append(transactionDetails, details)
}

return transactionDetails, nil
}

// Deposit Business logic methods
func (s *TransactionService) Deposit(ctx context.Context, userID uint64, amount float64)
(model.TransactionDetails, error) {
    params := model.DepositParams{
        UserID: userID,
        Amount: amount,
    }

    // Создаем транзакцию в pending-статусе
    res, err := s.repo.Deposit(ctx, params)
    if err != nil {
        s.logger.Error().Err(err).Uint64("user_id", userID).Msg("failed to create deposit
                                transaction")

        return model.TransactionDetails{}, err
    }

    // Отправляем запрос через Kafka
    request := map[string]interface{}{
        "request_type": "deposit",
        "user_id":     userID,
        "amount":      int64(amount * 100), // конвертируем в копейки
        "operation_id": res.Transaction.ID,
        "timestamp":    time.Now().UTC(),
    }

    err = s.kafkaPublisher.Publish(ctx, "transaction_data", fmt.Sprintf("%d", userID), request)
    if err != nil {
        s.logger.Error().Err(err).Uint64("user_id", userID).Uint64("transaction_id",
                                res.Transaction.ID).Msg("failed to send deposit request via kafka")
        // Обновляем статус транзакции на failed
        updateErr := s.repo.UpdateTransactionStatus(ctx, res.Transaction.ID,
                                model.TransactionStatusFailed)

        if updateErr != nil {
            s.logger.Error().Err(updateErr).Uint64("transaction_id",
                                res.Transaction.ID).Msg("failed to update transaction status to failed")
        }
    }
}

```

```
    return model.TransactionDetails{}, err
}

s.logger.Info().Uint64("user_id", userID).Uint64("transaction_id",
    res.Transaction.ID).Msg("deposit request sent via kafka")
// Транзакция остается в pending-статусе до получения ответа
return res, nil
}

func (s *TransactionService) Withdraw(ctx context.Context, accountID uint64, amount float64)
(model.TransactionDetails, error) {
    params := model.WithdrawParams{
        AccountID: accountID,
        Amount:    amount,
    }

    // Создаем транзакцию в pending-статусе
    res, err := s.repo.Withdraw(ctx, params)
    if err != nil {
        s.logger.Error().Err(err).Uint64("account_id", accountID).Msg("failed to create
            withdraw transaction")

        return model.TransactionDetails{}, err
    }

    // Отправляем запрос через Kafka
    request := map[string]interface{}{
        "request_type": "withdraw",
        "user_id":     accountID,
        "amount":      int64(amount * 100), // конвертируем в копейки
        "operation_id": res.Transaction.ID,
        "timestamp":   time.Now().UTC(),
    }

    err = s.kafkaPublisher.Publish(ctx, "transaction_data", fmt.Sprintf("%d", accountID),
        request)

    if err != nil {
        s.logger.Error().Err(err).Uint64("account_id", accountID).Uint64("transaction_id",
            res.Transaction.ID).Msg("failed to send withdraw request via kafka")
        // Обновляем статус транзакции на failed
        updateErr := s.repo.UpdateTransactionStatus(ctx, res.Transaction.ID,
            model.TransactionStatusFailed)

        if updateErr != nil {
            s.logger.Error().Err(updateErr).Uint64("transaction_id",
                res.Transaction.ID).Msg("failed to update transaction status to failed")
        }

        return model.TransactionDetails{}, err
    }
}
```

```

s.logger.Info().Uint64("account_id", accountID).Uint64("transaction_id",
    res.Transaction.ID).Msg("withdraw request sent via kafka")
// Транзакция остается в pending-статусе до получения ответа
return res, nil
)

func (s *TransactionService) Transfer(ctx context.Context, userID, recipient uint64, amount
float64) (model.TransactionDetails, error) {
    params := model.TransferParams{
        UserID:    userID,
        Recipient: recipient,
        Amount:    amount,
    }

    // Создаем транзакцию в pending- статусе
    res, err := s.repo.Transfer(ctx, params)
    if err != nil {
        s.logger.Error().Err(err).Uint64("user_id", userID).Uint64("recipient",
            recipient).Msg("failed to create transfer transaction")
        return model.TransactionDetails{}, err
    }

    // Отправляем запрос через Kafka
    request := map[string]interface{}{
        "request_type": "transfer",
        "user_id":      userID,
        "amount":       int64(amount * 100), // конвертируем в копейки
        "operation_id": res.Transaction.ID,
        "recipient_id": recipient,
        "timestamp":    time.Now().UTC(),
    }

    err = s.kafkaPublisher.Publish(ctx, "transaction_data", fmt.Sprintf("%d", userID), request)
    if err != nil {
        s.logger.Error().Err(err).Uint64("user_id", userID).Uint64("recipient",
            recipient).Uint64("transaction_id", res.Transaction.ID).Msg("failed to send transfer
            request via kafka")

        // Обновляем статус транзакции на failed
        updateErr := s.repo.UpdateTransactionStatus(ctx, res.Transaction.ID,
            model.TransactionStatusFailed)

        if updateErr != nil {
            s.logger.Error().Err(updateErr).Uint64("transaction_id",
                res.Transaction.ID).Msg("failed to update transaction status to failed")
        }
        return model.TransactionDetails{}, err
    }

    s.logger.Info().Uint64("user_id", userID).Uint64("recipient",
        recipient).Uint64("transaction_id", res.Transaction.ID).Msg("transfer request sent
        via kafka")
}

```

```
// Транзакция остается в pending-статусе до получения ответа
return res, nil
}

// HandleAccountResponse обрабатывает ответы от account service через Kafka
func (s *TransactionService) HandleAccountResponse(ctx context.Context, topic string, key
string, data []byte) error {
    var response AccountResponse
    if err := json.Unmarshal(data, &response); err != nil {
        return fmt.Errorf("failed to unmarshal account response: %w", err)
    }

    if response.RequestType == "" || response.UserID == 0 || response.OperationID == 0 {
        return fmt.Errorf("missing or invalid")
    }

    s.logger.Info().
        Str("request_type", response.RequestType).
        Uint64("user_id", response.UserID).
        Uint64("operation_id", response.OperationID).
        Interface("result", response.Result).
        Msg("received account response via kafka")

    // Обновляем статус транзакции
    var status model.TransactionStatus
    var logMsg string
    if response.Result {
        status = model.TransactionStatusCompleted
        logMsg = "transaction completed successfully"
    } else {
        status = model.TransactionStatusFailed
        logMsg = "transaction failed"
    }

    err := s.repo.UpdateTransactionStatus(ctx, response.OperationID, status)
    if err != nil {
        s.logger.Error().Err(err).Uint64("operation_id", response.OperationID).Msgf("failed to
            update transaction status to %s", status)
        return fmt.Errorf("failed to update transaction status to %s: %w", status, err)
    }

    s.logger.Info().Uint64("operation_id", response.OperationID).Msg(logMsg)
    return nil
}
```

Обработка событий тут заключается только в том, чтобы по сообщению определить статус изменения баланса и в соответствии с ним установить актуальный статус транзакции в базе данных.

Теперь можно перейти к настройке сервиса Account для интеграции через Kafka. Подключение и конфиг будут выглядеть одинаково — буквально переносим пакет kafka из transaction в account. На самом деле такие пакеты, как Kafka, принято выносить в отдельный репозиторий и переиспользовать в сервисах, по примеру работы с контрактами, но мы позволим себе такое упрощение и просто скопируем его в соседний сервис, тем более что в других местах kafka задействован не будет.

А вот переменные окружения в account будут другими (листинг 4.35).

Листинг 4.35. Account: .env

```
SERVICE_NAME=account
APP_ENV=dev

GRPC_PORT=50051
GRPC_HOST=localhost
LOG_LEVEL=debug

DB_DSN=postgres://postgres:postgres@localhost:5432/account?sslmode=disable

KAFKA_BROKER_HOST='localhost:29092'
KAFKA_CONSUMER_GROUP=consumer-account
KAFKA_TRANSACTION_TOPIC=transaction_request
```

В сервисе Account отредактировать надо не так много: добавить подписку на событие и отправку события после изменения баланса со статусом «успешно» или нет. Мы также получаем здесь `operationId` — чтобы сервис Transaction знал, для какой именно операции нужно обновить статус. Так что добавим недостающие структуры, как делали и для Transaction в сервисном слое (листинг 4.36).

Листинг 4.36. Account: internal/service/service.go

```
package service

import (
    "context"
    "encoding/json"
    "fmt"
    "strconv"

    "account/internal/model"
    "account/internal/repository/mapper"

    "github.com/rs/zerolog"
)

type AccountService struct {
    repo Repository
    logger *zerolog.Logger
}
```

```

    kafkaPublisher KafkaPublisher
}

func New(repo Repository, kafkaPublisher KafkaPublisher, logger *zerolog.Logger)
*AccountService {
    return &AccountService{
        repo:          repo,
        kafkaPublisher: kafkaPublisher,
        logger:         logger,
    }
}

type TransactionRequest struct {
    RequestType string `json:"request_type"`
    UserID      uint64 `json:"user_id"`
    Amount      int64  `json:"amount"`
    OperationID uint64 `json:"operation_id"`
    RecipientID uint64 `json:"recipient_id"`
}

type TransactionResponse struct {
    RequestType string          `json:"request_type"`
    UserID      uint64          `json:"user_id"`
    OperationID uint64          `json:"operation_id"`
    Result      map[string]interface{} `json:"result"`
}

type Repository interface {
    CreateUser(context.Context, model.User) (model.User, error)
    GetUser(context.Context, uint64) (model.User, error)
    GetUsers(context.Context, int, int) ([]model.User, error)
    DeleteUser(context.Context, uint64) error
    UpdateUser(context.Context, uint64, model.UpdateUser) error
    GetBalance(context.Context, uint64) (float32, error)
    UpdateBalance(context.Context, uint64, float32, model.BalanceOperationType) (float32,
                                                                                   float32, error)
    TransferBalance(context.Context, uint64, uint64, float32) error
}

// KafkaPublisher интерфейс для публикации сообщений в Kafka
type KafkaPublisher interface {
    Publish(ctx context.Context, topic string, key string, data interface{}) error
}

func (s *AccountService) CreateUser(ctx context.Context, newUser model.CreateUser)
(model.User, error) {
    repoUser := mapper.CreateUserToRepoUser(newUser)
    user := mapper.RepoUserToUser(repoUser)

```



```
    return s.repo.CreateUser(ctx, user)
}

func (s *AccountService) GetUsers(ctx context.Context, limit int, offset int) ([]model.User,
error) {
    return s.repo.GetUsers(ctx, limit, offset)
}

func (s *AccountService) GetUser(ctx context.Context, userID uint64) (model.User, error) {
    return s.repo.GetUser(ctx, userID)
}

func (s *AccountService) DeleteUser(ctx context.Context, userID uint64) error {
    return s.repo.DeleteUser(ctx, userID)
}

func (s *AccountService) UpdateUser(ctx context.Context, userID uint64, user
model.UpdateUser) error {
    return s.repo.UpdateUser(ctx, userID, user)
}

// GetBalance получает текущий баланс пользователя
func (s *AccountService) GetBalance(ctx context.Context, userID uint64)
(model.GetBalanceResponse, error) {
    balance, err := s.repo.GetBalance(ctx, userID)
    if err != nil {
        s.logger.Err(err).Uint64("userID", userID).Msg("failed to get user balance")
        return model.GetBalanceResponse{}, err
    }

    return model.GetBalanceResponse{
        UserID:  userID,
        Balance: balance,
    }, nil
}

// UpdateBalance обновляет баланс пользователя
func (s *AccountService) UpdateBalance(ctx context.Context, req model.UpdateBalanceRequest)
(model.UpdateBalanceResponse, error) {
    // Валидация входных данных
    if req.Amount < 0 {
        return model.UpdateBalanceResponse{}, fmt.Errorf("amount cannot be negative")
    }

    if req.Type == model.BalanceOperationCredit && req.Amount == 0 {
        return model.UpdateBalanceResponse{}, fmt.Errorf("amount must be positive for subtract
operation")
    }
}
```

```
oldBalance, newBalance, err := s.repo.UpdateBalance(ctx, req.UserID, req.Amount, req.Type)
if err != nil {
    s.logger.Err(err).
        Uint64("userID", req.UserID).
        Float32("amount", req.Amount).
        Str("operation", string(req.Type)).
        Msg("failed to update user balance")
    return model.UpdateBalanceResponse{}, err
}

s.logger.Info().
    Uint64("userID", req.UserID).
    Float32("oldBalance", oldBalance).
    Float32("newBalance", newBalance).
    Float32("amount", req.Amount).
    Str("operation", string(req.Type)).
    Msg("user balance updated successfully")

return model.UpdateBalanceResponse{
    UserID:    req.UserID,
    OldBalance: oldBalance,
    NewBalance: newBalance,
    Amount:    req.Amount,
    Operation: req.Type,
}, nil
}

// Deposit пополняет баланс пользователя
func (s *AccountService) Deposit(ctx context.Context, userID uint64, amount int64)
(model.UpdateBalanceResponse, error) {
    updateReq := model.UpdateBalanceRequest{
        UserID: userID,
        Amount: float32(amount),
        Type:   model.BalanceOperationDeposit,
    }

    response, err := s.UpdateBalance(ctx, updateReq)
    if err != nil {
        s.logger.Err(err).
            Uint64("userID", userID).
            Int64("amount", amount).
            Msg("failed to deposit")
        return model.UpdateBalanceResponse{}, err
    }

    s.logger.Info().
        Uint64("userID", userID).
```

```

    Int64("amount", amount).
    Float32("newBalance", response.NewBalance).
    Msg("deposit successful")

    return response, nil
}

// Withdraw списывает средства с баланса пользователя
func (s *AccountService) Withdraw(ctx context.Context, userID uint64, amount int64)
(model.UpdateBalanceResponse, error) {
    updateReq := model.UpdateBalanceRequest{
        UserID: userID,
        Amount: float32(amount),
        Type:   model.BalanceOperationCredit,
    }

    response, err := s.UpdateBalance(ctx, updateReq)
    if err != nil {
        s.logger.Err(err).
            Uint64("userID", userID).
            Int64("amount", amount).
            Msg("failed to withdraw")
        return model.UpdateBalanceResponse{}, err
    }

    s.logger.Info().
        Uint64("userID", userID).
        Int64("amount", amount).
        Float32("newBalance", response.NewBalance).
        Msg("withdraw successful")

    return response, nil
}

// Transfer переводит средства между пользователями и возвращает балансы
func (s *AccountService) Transfer(ctx context.Context, fromUserID, toUserID uint64, amount
int64) (model.GetBalanceResponse, model.GetBalanceResponse, error) {
    if amount <= 0 {
        return model.GetBalanceResponse{}, model.GetBalanceResponse{}, fmt.Errorf("amount must
be positive")
    }

    if fromUserID == toUserID {
        return model.GetBalanceResponse{}, model.GetBalanceResponse{}, fmt.Errorf("cannot
transfer to the same user")
    }
}

```

```

err := s.repo.TransferBalance(ctx, fromUserID, toUserID, float32(amount))
if err != nil {
    s.logger.Err(err).
        Uint64("fromUserID", fromUserID).
        Uint64("toUserID", toUserID).
        Int64("amount", amount).
        Msg("failed to transfer balance")
    return model.GetBalanceResponse{}, model.GetBalanceResponse{}, err
}

s.logger.Info().
    Uint64("fromUserID", fromUserID).
    Uint64("toUserID", toUserID).
    Int64("amount", amount).
    Msg("balance transferred successfully")

// Получаем балансы обоих пользователей для ответа
fromUserBalance, err := s.GetBalance(ctx, fromUserID)
if err != nil {
    s.logger.Err(err).Uint64("userID", fromUserID).Msg("failed to get sender balance")
    return model.GetBalanceResponse{}, model.GetBalanceResponse{}, err
}

toUserBalance, err := s.GetBalance(ctx, toUserID)
if err != nil {
    s.logger.Err(err).Uint64("userID", toUserID).Msg("failed to get recipient balance")
    return model.GetBalanceResponse{}, model.GetBalanceResponse{}, err
}

s.logger.Info().
    Uint64("fromUserID", fromUserID).
    Uint64("toUserID", toUserID).
    Int64("amount", amount).
    Msg("transfer successful")

return fromUserBalance, toUserBalance, nil
}

// HandleTransaction обрабатывает сообщения из Kafka от transaction service
func (s *AccountService) HandleTransaction(ctx context.Context, topic string, key string,
data []byte) error {
    s.logger.Info().Msg("handling transaction request")
    var res TransactionRequest
    var err error
    if err = json.Unmarshal(data, &res); err != nil {
        return fmt.Errorf("failed to unmarshal account response: %w", err)
    }
}

```

```

result := map[string]interface{}{
    "request_type": res.RequestType,
    "user_id":      res.UserID,
    "operation_id": res.OperationID,
    "result":       false,
}

switch res.RequestType {
case "withdraw":
    _, err = s.Withdraw(ctx, res.UserID, res.Amount)
case "deposit":
    _, err = s.Deposit(ctx, res.UserID, res.Amount)
case "transfer":
    _, _, err = s.Transfer(ctx, res.UserID, res.RecipientID, res.Amount)
default:
    err = fmt.Errorf("unknown request type: %s", res.RequestType)
}

if err != nil {
    s.logger.Err(err).
        Uint64("userID", res.UserID).
        Uint64("operationID", res.OperationID).
        Msg("failed to handle transaction request")
    s.kafkaPublisher.Publish(ctx, "transaction_response", "key", result)
    return fmt.Errorf("failed to handle transaction request: %w", err)
}

result["result"] = true
s.kafkaPublisher.Publish(ctx, "transaction_response", strconv.FormatUint(res.UserID, 10),
                                                                    result)

return nil
}

```

Здесь мы выполняем необходимые для транзакции проверки, определяем по `request_type` тип запроса (пополнение, списание или перевод), вызываем соответствующий метод логики и отправляем результат, если успешно: `result = true`, иначе `result = false`.

И остается только добавить подписку на событие (листинг 4.37).

Листинг 4.37. Account: internal/app/app.go

```

package app

import (
    "context"
    "database/sql"
    "fmt"
    "net"

```

```
"account/internal/config"
"account/internal/kafka"
_ "account/internal/migrations"
"account/internal/repository"
"account/internal/server"
"account/internal/service"

"github.com/pressly/goose/v3"
accountpb "github.com/yuliaapopova/contracts/account/go"
_ "google.golang.org/genproto/googleapis/api/annotations"
"google.golang.org/grpc"

_ "github.com/lib/pq"
"github.com/rs/zerolog"
"gorm.io/driver/postgres"
"gorm.io/gorm"
}

type App struct {
    cfg      *config.Config
    logger   *zerolog.Logger

    accountRepository *repository.Repository
    accountService    *service.AccountService
    accountServer     *server.Server
    grpcServer        *grpc.Server

    kafkaClient *kafka.Kafka
}

func New(logger *zerolog.Logger, cfg *config.Config) *App {
    return &App{
        cfg:      cfg,
        logger:   logger,
    }
}

func (a *App) Run(ctx context.Context) error {
    // Инициализируем gRPC-сервер
    accountServer, err := a.getAccountServer(ctx)
    if err != nil {
        return fmt.Errorf("failed to get account server: %w", err)
    }

    // Создаем gRPC-сервер
    a.grpcServer = getGRPCServer(accountServer)
```

```

listenAddr := fmt.Sprintf("%s:%d", a.cfg.Host, a.cfg.Port)
lis, err := net.Listen("tcp", listenAddr)
if err != nil {
    a.logger.Fatal().Err(err).Msg("failed to listen")
    return err
}

a.logger.Info().Msg("gRPC server listening")

serveErrCh := make(chan error, 1)
go func() {
    serveErrCh <- a.grpcServer.Serve(lis)
}()

select {
case <-ctx.Done():
    a.grpcServer.GracefulStop()
    return ctx.Err()
case err := <-serveErrCh:
    if err != nil {
        a.logger.Error().Err(err).Msg("failed to serve")
    }
    return err
}
}

func (a *App) getRepository(ctx context.Context) (*repository.Repository, error) {
    if a.accountRepository == nil {
        if err := a.runMigrations(ctx); err != nil {
            return nil, fmt.Errorf("failed to get repository: %w", err)
        }
        db, err := gorm.Open(postgres.Open(a.cfg.DbDsn), &gorm.Config{})
        if err != nil {
            return nil, fmt.Errorf("gorm init failed: %w", err)
        }
        a.accountRepository = repository.NewRepository(db, a.logger)
    }
    return a.accountRepository, nil
}

func (a *App) runMigrations(ctx context.Context) error {
    if err := goose.SetDialect("postgres"); err != nil {
        return fmt.Errorf("failed to select migrations dialect: %w", err)
    }

    dbGoose, err := sql.Open("postgres", a.cfg.DbDsn)
    if err != nil {

```

```
    return fmt.Errorf("failed to create sql connection: %w", err)
}

if err := goose.UpContext(ctx, dbGoose, "internal/migrations"); err != nil {
    return fmt.Errorf("failed to run up migrations: %w", err)
}

return nil
}

func (a *App) getAccountService(ctx context.Context) (*service.AccountService, error) {
    if a.accountService == nil {
        repo, err := a.getRepository(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get repository: %w", err)
        }

        kafkaClient, err := a.getKafkaClient(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get kafka client: %w", err)
        }

        a.accountService = service.New(repo, kafkaClient, a.logger)

        //Подписываемся на ответы от account service после создания сервиса
        a.kafkaClient.Subscribe(ctx, a.cfg.KafkaTransactionTopic,
            a.accountService.HandleTransaction)

        a.logger.Info().Msg("kafka client attached to transaction service")
    }
    return a.accountService, nil
}

func (a *App) getKafkaClient(ctx context.Context) (*kafka.Kafka, error) {
    if a.kafkaClient == nil {
        // Создаем producer
        producerCfg := kafka.DefaultProducerConfig(a.cfg.KafkaBrokers)
        producer := kafka.NewProducer(producerCfg, a.logger)

        // Создаем Kafka-клиент
        kafkaClient := kafka.New(producer, a.cfg.KafkaBrokers, a.cfg.KafkaGroupID, a.logger)

        a.kafkaClient = kafkaClient

        a.logger.Info().Msg("kafka client created")
    }
    return a.kafkaClient, nil
}
```



```

func (a *App) getAccountServer(ctx context.Context) (*server.Server, error) {
    if a.accountServer == nil {
        service, err := a.getAccountService(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get service: %w", err)
        }
        a.accountServer = server.New(service, a.logger)
    }
    return a.accountServer, nil
}

func getGRPCServer(srv *server.Server) *grpc.Server {
    grpcSrv := grpc.NewServer()
    accountpb.RegisterAccountServer(grpcSrv, srv)
    return grpcSrv
}

// Close корректно завершает работу приложения
func (a *App) Close() error {
    var errs []error

    if a.grpcServer != nil {
        a.grpcServer.GracefulStop()
    }

    if a.kafkaClient != nil {
        if err := a.kafkaClient.Close(); err != nil {
            errs = append(errs, fmt.Errorf("failed to close kafka client: %w", err))
        }
    }

    if len(errs) > 0 {
        return fmt.Errorf("errors closing app: %v", errs)
    }

    return nil
}

```

В результате мы реализовали взаимодействие между сервисами Account и Transaction через брокер сообщений Kafka по паттерну Saga.

Для полноценной картины микросервисного взаимодействия нашей системы не хватает только добавить методы работы с транзакциями в Gateway, но приводить код этого тут мы не будем — там все подобно тому, как мы уже делали для Gateway с другими сервисами, подробности вы можете рассмотреть сами в архиве с кодом.

Список литературы и источников

1. Лекции по дисциплине «Базы данных» БГТУ им. В. Г. Шухова.
2. Документация к PostgreSQL 15.3
(<http://repo.postgrespro.ru/doc/pgsql/15.3/ru/postgres-A4.pdf>).
3. https://ru.wikipedia.org/wiki/Транзакция_%28информатика%29.
4. <https://habr.com/ru/articles/317884/>.
5. <https://habr.com/ru/articles/446662/>.
6. Крис Ричардсон. Микросервисы. Паттерны разработки и рефакторинга
(<https://goo.su/ThpJ1B>).
7. <https://microservices.io/patterns/data/saga.html>.



Развертывание микросервисов

Считается, что самыми основными и популярными способами развертывания микросервисов являются следующие пять [1]. Выбор наиболее приемлемого в каждом конкретном случае зависит от потребностей системы. Итак:

1. Один сервер, несколько процессов.
2. Несколько серверов, несколько процессов.

Когда мощностей одного сервера уже не хватает, можно просто добавить дополнительные серверы и распределить нагрузку между ними.

3. Контейнеры.

Упаковка микросервисов в контейнер упрощает их развертывание и запуск вместе с другими сервисами. Это является первым шагом к оркестровке на основе Kubernetes.

4. Оркестратор.

Такие оркестраторы, как Kubernetes или Nomad, представляют собой полноценные платформы, предназначенные для одновременного запуска большого количества контейнеров.

5. Бессерверный.

Этот способ позволяет не заморачиваться процессами, контейнерами и серверами, а выполнять код непосредственно в облаке.

Таким образом, у нас есть два варианта пути развития: первый — от процессов к контейнерам и в итоге к Kubernetes. И второй — бессерверный.

К первому способу (один сервер, несколько процессов) для наглядности можно отнести ситуацию, когда вы на своей локальной машине разворачиваете сервер для разработки. При этом вы можете запустить микросервисное приложение в виде нескольких процессов на разных портах. Собственно, так мы и делали до сих пор ранее. Однако разница с производственной средой и локальной разработкой заключается в том, что в первом случае практически всегда используются несколько инстансов (можно сказать, что это уже стандарт), которые следуют за балансировщиком нагрузки. Балансировщик нагрузки контролирует, чтобы ресурсы запущенных

приложений нагружались равномерно и не было перекаса, когда один инстанс загружен на 80%, а другой, к примеру, только на 20%.

Если вы хотите запустить свое приложение на домашнем сервере, вам понадобится получить публичный IP-адрес и открыть соответствующие порты, на которых и будет запущен ваш проект.

В качестве положительного момента такого способа можно отметить, что запустить на виртуальной машине (ВМ) один инстанс проекта, моделирующий процессы, выполняющиеся на сервере, не составляет особой проблемы. Для этого также не потребуется специально ничего изучать — достаточно подключиться к ВМ по SSH, перенести в нее проект, установить необходимые зависимости и запустить на выполнение. Подробно рассматривать способ реализации этого подхода на практике мы не станем, поскольку он буквально 1:1 повторяет то, как мы устанавливали и настраивали всё это локально, с той лишь разницей, что тут можно работать только через терминал, а если понадобится отредактировать какой-либо текстовый файл, то делать это придется через встроенные в терминал редакторы `vim` или `nano`.

Однако когда появляется необходимость масштабировать подобное решение, то начинаются сложности, поскольку добавить в него ресурсы сервера бывает затруднительно, если это решение не облачное. Об отказоустойчивости системы в таком случае можно и не говорить — если ВМ отключить от сети, то приложение перестанет быть доступно. Так что развертывание при каких-либо сбоях может потребоваться делать каждый раз заново собственноручно.

При втором способе (несколько серверов, несколько процессов) происходит всё то же самое, что и при первом, с той лишь разницей, что для балансировки и создания общего контекста для инстансов необходимо использовать веб-серверы `nginx` (HTTP-сервер, который чаще всего выступает в роли прокси-сервера и/или балансировщика нагрузки) или `Apache`.

Чтобы установить `nginx` на ВМ с предустановленной системой `Ubuntu 24.04`, необходимо выполнить команду:

```
sudo apt install nginx
```

Проверить, что `nginx` установился и запустился, можно командой:

```
sudo systemctl status nginx
```

Если он запущен, то будет выведен следующий результат:

```
• nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-09-10 12:23:59 UTC; 15s ago
     Docs: man:nginx(8)
   Process: 1459 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on;
                                                    (code=exited, status=0/SUCCESS)
   Process: 1462 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited,
                                                    status=0/SUCCESS)
   Main PID: 1493 (nginx)
     Tasks: 3 (limit: 2316)
```

Memory: 2.2M (peak: 4.9M)

CPU: 22ms

CGroup: /system.slice/nginx.service

└─1493 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"

└─1495 "nginx: worker process"

└─1496 "nginx: worker process"

Sep 10 12:23:59 compute-vm-2-2-10-ssd-1757506897549 systemd[1]: Starting nginx.service

- A high performance web server and a reverse proxy server...

Sep 10 12:23:59 compute-vm-2-2-10-ssd-1757506897549 systemd[1]: Started nginx.service

- A high performance web server and a reverse proxy server.

Теперь, если перейти по публичному IP-адресу вашей ВМ, вы увидите, что nginx запущен (рис. 5.1).



Рис. 5.1. Nginx запущен

При штатной установке nginx будет запущен со своими стандартными настройками. А чтобы запустить его в режиме прокси-сервера, необходимо отредактировать конфигурацию: открыть настройки nginx через консольный редактор (рис. 5.2):

```
sudo nano /etc/nginx/sites-available/default
```

и подправить их следующим образом:

```
location / {  
    proxy_pass http://localhost:50053;  
}
```

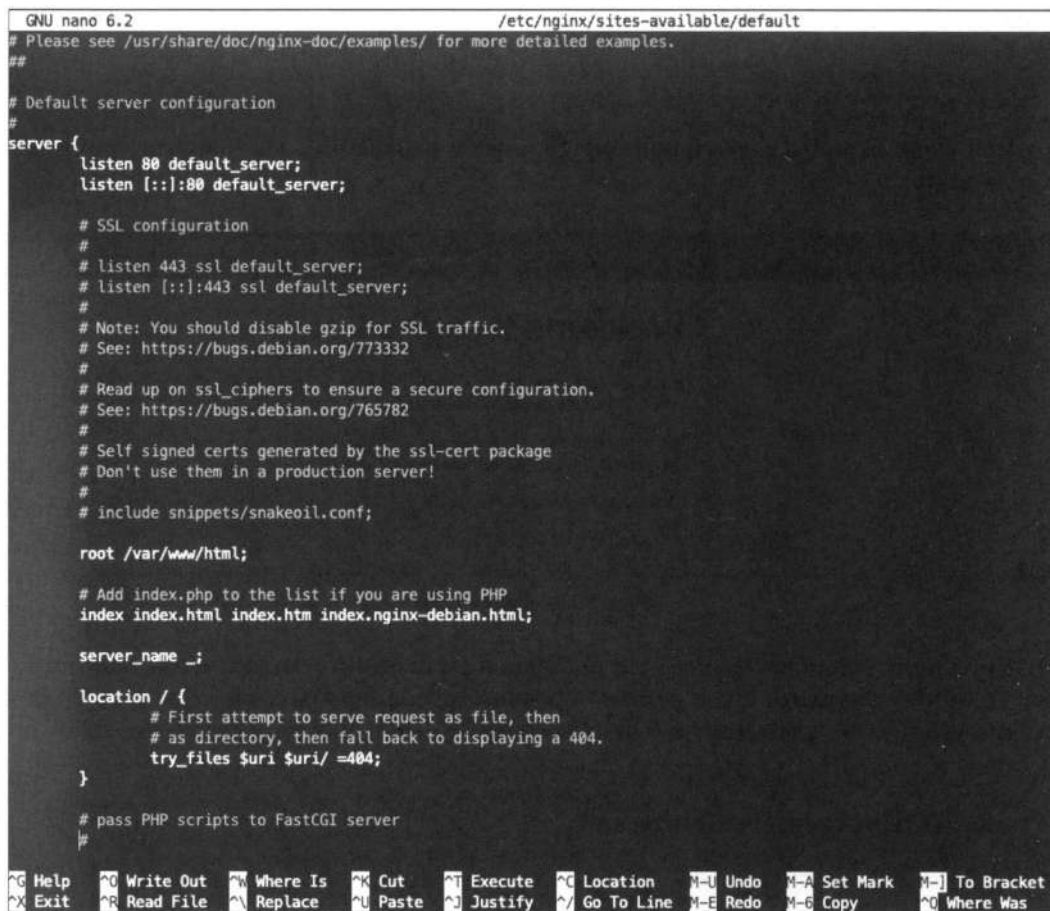
Здесь указывается, куда перенаправлять запрос, который поступает на 80-й порт (по умолчанию открытым под nginx чаще всего оказывается порт 8080 или 80).

Однако такой метод настройки развертывания очень неудобный. Чтобы проект запустился, мы должны клонировать весь его код с GitHub, установить и развернуть Docker-Compose для баз данных, установить и запустить каждый сервер по отдельности, а затем еще настроить nginx. К тому же если вы отключитесь от удаленной машины или закроете соединение, где поднимаются сервисы, то сервисы завершат свою работу. Чтобы этого избежать, можно настроить запуск через службы.

В общем, что в первом способе, что во втором оказывается слишком много ручной работы. Причем для второго способа, когда задействованы несколько ВМ, настрой-

ка nginx будет еще сложнее, поскольку там его ключевая задача — именно балансировка нагрузки.

Перед тем как перейти к рассмотрению оставшихся способов развертывания микросервисов, хотелось бы немного поговорить о CI/CD — механизме непрерывной интеграции / непрерывной доставки (Continuous Integration/Continuous Delivery).



```
GNU nano 6.2 /etc/nginx/sites-available/default
# Please see /usr/share/doc/nginx-doc/examples/ for more detailed examples.
##

# Default server configuration
#
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    # SSL configuration
    #
    # listen 443 ssl default_server;
    # listen [::]:443 ssl default_server;
    #
    # Note: You should disable gzip for SSL traffic.
    # See: https://bugs.debian.org/773332
    #
    # Read up on ssl_ciphers to ensure a secure configuration.
    # See: https://bugs.debian.org/765782
    #
    # Self signed certs generated by the ssl-cert package
    # Don't use them in a production server!
    #
    # include snippets/snakeoil.conf;

    root /var/www/html;

    # Add index.php to the list if you are using PHP
    index index.html index.htm index.nginx-debian.html;

    server_name _;

    location / {
        # First attempt to serve request as file, then
        # as directory, then fall back to displaying a 404.
        try_files $uri $uri/ =404;
    }

    # pass PHP scripts to FastCGI server
    #
}
```

Рис. 5.2. Редактирование конфигурации nginx

Механизм CI/CD используется для обеспечения автоматической доставки кода на серверы, где он должен выполняться, для предварительной проверки качества кода, а также и для других, не менее важных процессов, которые необходимо автоматизировать.

Для коммерческих проектов на основе механизма CI/CD обычно применяют GitLab и внедренный в него GitLab Runner. Впрочем, у GitHub есть и своя альтернатива в виде GitHub Actions. Эти методологии разработки больше относятся уже к практике DevOps и позволяют автоматизировать различные процессы — проверку того, что все тесты выполняются, что код соответствует всем принятым соглашениям, а

само приложение успешно собирается. Также — через настройку CI/CD — производится и выкатка микросервиса на стенды.

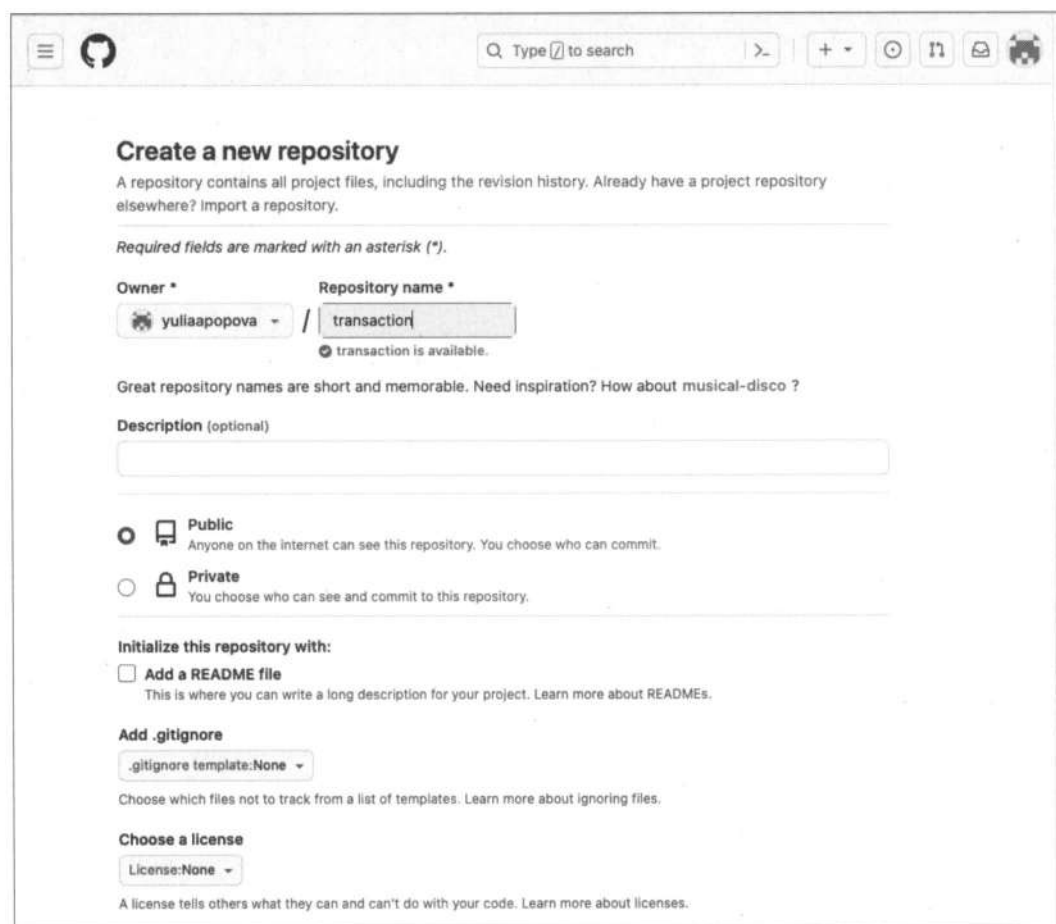
Мы рассмотрим, как всё это устроено, на примере GitHub Actions — бесплатного плана для наших целей более чем достаточно, и дополнительные затраты не потребуются.

Свои эксперименты мы будем производить над проектом Transaction — для этого понадобится залить его на GitHub. При этом мы подразумеваем, что все необходимые действия с Git уже проделаны и подключение по SSH активно¹.

Для заливки проекта на GitHub выполните специальную команду:

```
git init
```

В открывшемся окне создайте в GitHub репозиторий, назвав его transaction (рис. 5.3).



The screenshot shows the GitHub 'Create a new repository' page. At the top, there's a search bar and navigation icons. The main heading is 'Create a new repository'. Below it, a subtext explains that a repository contains all project files and revision history. There's a link to 'Import a repository' if one already exists elsewhere. A note states that required fields are marked with an asterisk (*). The 'Owner' field is set to 'yuliaapopova' and the 'Repository name' field is set to 'transaction'. A message below the name field says 'transaction is available.' There's a tip about great repository names being short and memorable, with a suggestion for 'musical-disco'. The 'Description (optional)' field is empty. Under 'Visibility', the 'Public' radio button is selected, with a note that anyone on the internet can see the repository. The 'Private' option is also available. The 'Initialize this repository with:' section has three options: 'Add a README file' (checked), 'Add .gitignore' (with a dropdown set to '.gitignore template: None'), and 'Choose a license' (with a dropdown set to 'License: None'). Each option has a brief description and a link to learn more.

Рис. 5.3. Создание репозитория в GitHub

¹ В любом случае сделать это можно по следующей инструкции:
<https://docs.github.com/ru/authentication/connecting-to-github-with-ssh>.

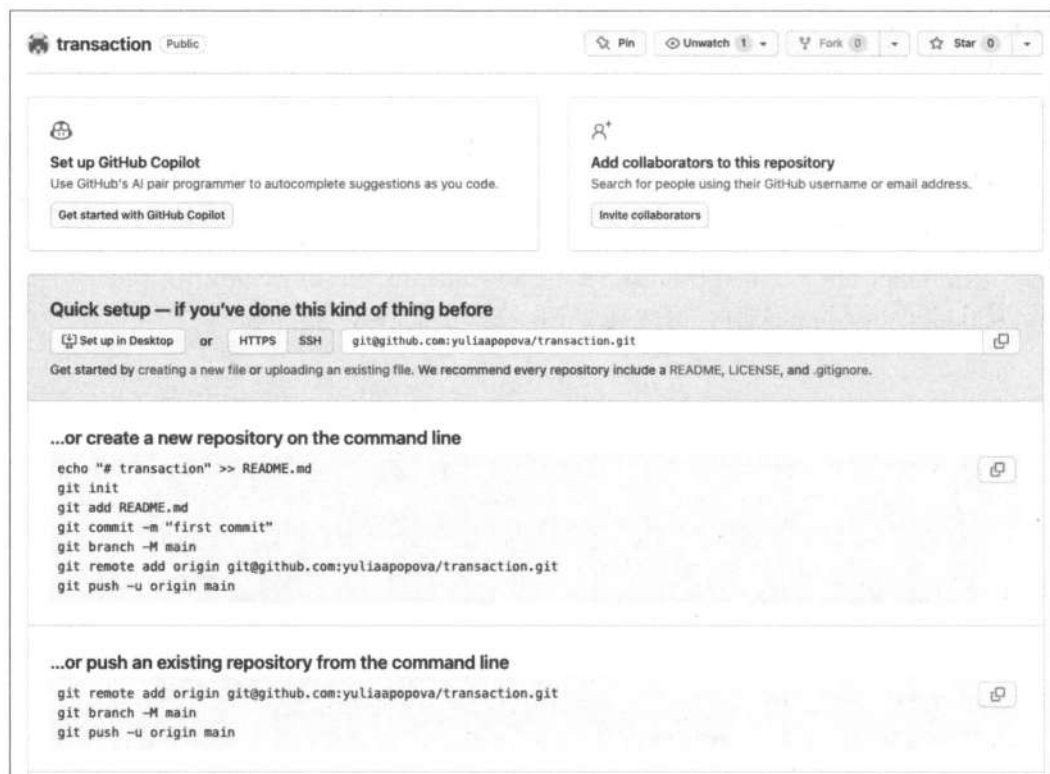


Рис. 5.4. Инструкция по подключению к удаленному репозиторию

Нажмите кнопку **Create repository** и пролистайте открывшуюся страницу до самого низа — вы увидите инструкцию по работе с новым репозиторием (рис. 5.4).

Нас здесь интересует вторая часть инструкции, которая прописана в разделе **...or push an existing repository from command line** (...или нажмите существующий репозиторий из командной строки). Именно это мы и выполним, предварительно закоммитив наши изменения, так что вместо ветки `main` у нас будет ветка `master`. В результате весь код окажется на GitHub в созданном только что репозитории.

Теперь можно переключиться непосредственно в GitHub Actions. Для этого на странице репозитория откройте вкладку **Actions**. Там приведены различные готовые варианты сценариев непрерывной интеграции, но чтобы лучше в этом разобраться, создадим свой сценарий с нуля сами. Для этого надо перейти по ссылке: **set up a workflow yourself** (рис. 5.5).

GitHub Actions — это платформа непрерывной интеграции и непрерывной доставки (CI/CD), позволяющая автоматизировать многие процессы, связанные с репозиторием и кодом, который в нем хранится.

GitHub Actions использует **Workflow** — определенный набор инструкций в формате YAML, описывающий последовательность выполнения задач и работающий на основе событий. Например, можно настроить запуск тестов и билд-проекта на каждый коммит, поскольку коммит — это событие. Или настроить его на определен-

ные ветки — например, на `develop`. Или настроить его таким образом, чтобы для веток `develop` или `master` в порядке непрерывной интеграции выполнялись одни действия, а для новых веток — другие.

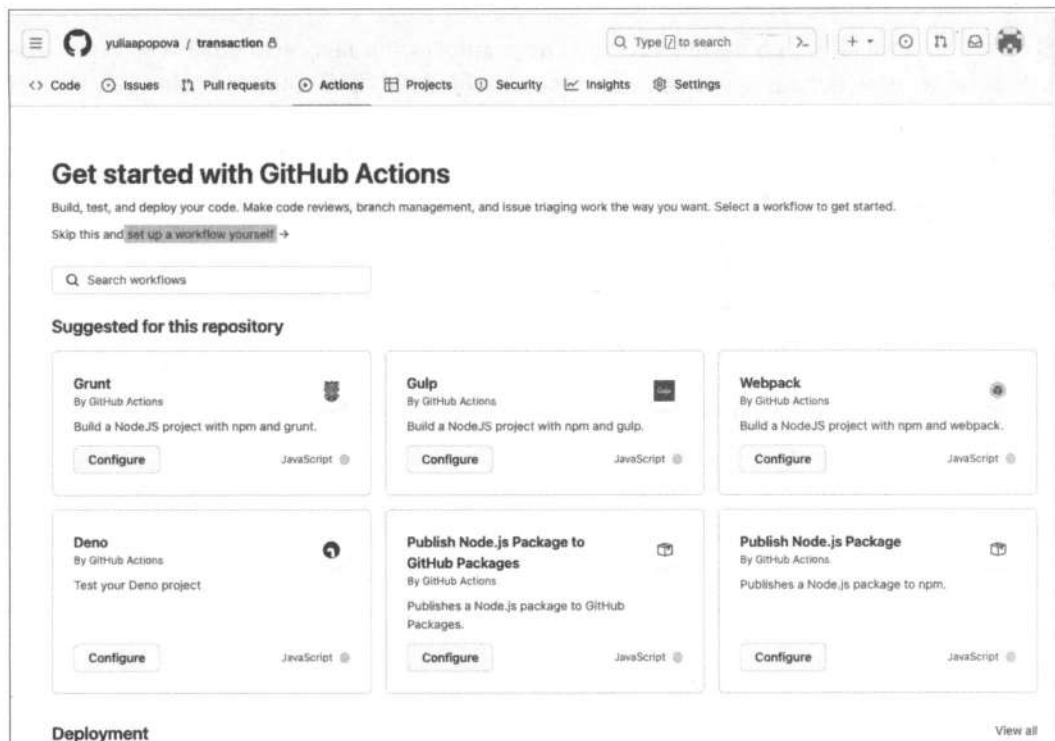


Рис. 5.5. GitHub Actions: переходим по ссылке **set up a workflow yourself**

Благодаря **WorkFlow** мы можем тестировать и автоматизировать сборку проекта или заливать его, куда нужно. **GitHub Actions** также позволяет полностью интегрировать **CI/CD** при работе с репозиторием.

WorkFlow можно создавать на каждый репозиторий. Кроме того, может быть создано также несколько **WorkFlow** для одного проекта.

Начнем мы с простого варианта и создадим **WorkFlow**, выполняющий просто печать в консоль (листинг 5.1).

Листинг 5.1. Файл `.github/workflows/master.yml`

```
name: Transaction
on: workflow_dispatch
jobs:
  stage_dev:
    runs-on: ubuntu-latest
    steps:
      - name: lint
        run: echo Hello
```

Рассмотрим подробнее имеющиеся здесь параметры:

- ❑ `name` — имя `WorkFlow` задается произвольно. В нашем случае мы выбрали его по названию сервиса, но часто его также называют по стенду, для которого выполняются команды;
- ❑ `on` — подписка на событие. Их довольно много для выбора: можно задать одно событие или перечислить их массивом событий, при триггере которых будут выполняться описанные процессы. Событие `workflow_dispatch` означает, что триггерить выполнение их мы будем вручную, пока без автоматизации;
- ❑ `jobs` — некоторые работы, которые предстоит выполнить;
- ❑ `stage_dev` — произвольное имя для первого из `jobs`;
- ❑ `runs-on` — здесь указывается операционная система, на которой необходимо выполнять все действия. Поддерживаются Linux, macOS, Windows и множество версий для них;
- ❑ `steps` — шаги, которые выполняются в рамках `jobs`. Шагов может быть сколько угодно;
- ❑ `- name` — название шага, задается произвольно;
- ❑ `run` — команда, которую необходимо выполнить для заданного шага в указанном `jobs`. Сейчас мы ожидаем, что будет выведено **Hello**, а ключевое слово `echo` обозначает печать в консоль (так в Linux, которую мы указали как систему. Если вы укажете другую систему, то команды могут отличаться).

Теперь мы можем сохранить изменения, сделав коммит. После чего перейти в **Actions/Transaction** (название, которое мы задали для `WorkFlow`) и запустить наши инструкции (рис. 5.6).

Как можно здесь видеть, сначала были выполнены все подготовительные работы, затем запущен образ, соответствующий выбранной нами системе, и в консоли напечатано **Hello**. После завершения выполнения процесс был очищен.

Теперь попробуем задать команды посерьезнее — например, выполнить проверку на соответствие кода линтеру. Для начала воспользуемся специальным Marketplace от GitHub. Там содержится большое количество дополнительной функциональности, которая способна упростить как разработку, так и работу с Actions, — нас же сейчас интересует как раз последнее. Marketplace позволяет не описывать всё прямо с нуля, а воспользоваться уже готовыми наборами. Применим набор `Actions Checkout` от GitHub¹ — он извлекает исходный код из репозитория, чтобы CI мог получить к нему доступ (листинг 5.2).

Листинг 5.2. Transaction: `.github/workflows/master.yaml`

```
name: Transaction
on: workflow_dispatch
```

¹ См. <https://github.com/marketplace/actions/checkout>.

Transaction #3

Summary

Jobs

- stage_dev**
succeeded now in 8s
- stage_dev**
- Run details
- Usage
- Workflow file

Re-run all jobs ...

Search logs

8s

Set up job

```

1 Current runner version: '2.328.0'
2 ▶ Runner Image Provisioner
7 ▶ Operating System
11 ▶ Runner Image
16 ▶ GITMIB_TOKEN Permissions
20 Secret sources: Actions
21 Prepare workflow directory
22 Prepare all required actions
23 Getting action download info
24 Download action repository 'actions/checkout@v3.5.3' (SHA1:c85c95e3d7251135ab7dc9cc3241c5835cc395a9)
25 Complete job name: stage_dev

```

get code

```

1 ▶ Run actions/checkout@v3.5.3
13 Syncing repository: yuliaanopova/transaction
14 ▶ Getting Git version info
18 Temporarily overriding HOME='/home/runner/work/_temp/7c483ccc-56f0-44f2-8a2-8a66939346a9' before making global git config changes
19 Adding repository directory to the temporary git global config as a safe directory
20 /usr/bin/git config --global --add safe.directory /home/runner/work/transaction/transaction
21 Deleting the contents of '/home/runner/work/transaction/transaction'
22 ▶ Initializing the repository
38 ▶ Disabling automatic garbage collection
40 ▶ Setting up auth
46 ▶ Fetching the repository
131 ▶ Determining the checkout info
132 ▶ Checking out the ref
136 /usr/bin/git log -1 --format '%H'
137 '6408990e48e4e4f2cd97a95f9acc447d1c89'

```

Рис. 5.6. Выполнение Workflow Transaction

```
jobs:
  stage_dev:
    runs-on: ubuntu-latest
    steps:
      - name: get code
        uses: actions/checkout@v3.5.3
      - name: install deps
        run: go mod download
```

Здесь мы задаем распаковку проекта и установку зависимостей.

Далее выполняется проверка линтером и запускаются тесты. Но чтобы запустить тесты, предварительно следует их написать — хотя бы один — для демонстрации работы команды:

```
go test -v -race -coverprofile=coverage.out ./...
```

Так что напомним тесты на создание транзакции.

Перед этим необходимо доустановить недостающие пакеты для написания тестов:

```
go get go.uber.org/mock/gomock github.com/stretchr/testify/assert
```

И добавить (листинг 5.3) автоматическую генерацию моков для юнит-тестов в Makefile (команда `generate-mocks`).

Листинг 5.3. Transaction: Makefile

```
# Go application Makefile
APP_NAME ?= transaction
BINARY_NAME ?= bin/$(APP_NAME)
MAIN_PATH ?= ./cmd
VERSION ?= $(shell git describe --tags --always --dirty 2>/dev/null || echo "dev")
BUILD_TIME ?= $(shell date -u '+%Y-%m-%d_%H:%M:%S')
LDFLAGS ?= -ldflags "-X main.Version=$(VERSION) -X main.BuildTime=$(BUILD_TIME)"

# Go files to format and lint
GOFILES ?= $(shell find . -name "*.go" -type f -not -path "./vendor/*" -not -path "./.git/*")

# Default target
.DEFAULT_GOAL := build

# Build the application
build:
    @echo "Building $(APP_NAME)..."
    @mkdir -p bin
    go build $(LDLAGS) -o $(BINARY_NAME) $(MAIN_PATH)

# Run the application
run: clean build
    @echo "Running $(APP_NAME)..."
    ./${BINARY_NAME}
```

```
# Run without building (if binary exists)
run-only:
    @echo "Running ${APP_NAME}..."
    ./$(BINARY_NAME)

# Install dependencies
deps:
    @echo "Installing dependencies..."
    go mod download
    go mod tidy

# Update dependencies
deps-update:
    @echo "Updating dependencies..."
    go get -u ./...
    go mod tidy

# Format code
fmt:
    @echo "Formatting code..."
    gofmt -s -w $(GOFILES)
    gofumpt -l -w -s $(GOFILES)
    goimports -l -w $(GOFILES)

# Run linter
lint:
    @echo "Running linter..."
    golangci-lint run --timeout=10m -v ./...

# Run tests
test:
    @echo "Running tests..."
    go test -v ./...

# Run tests with coverage
test-coverage:
    @echo "Running tests with coverage..."
    go test -v -coverprofile=coverage.out ./...
    go tool cover -html=coverage.out -o coverage.html
    @echo "Coverage report generated: coverage.html"

# Run benchmarks
bench:
    @echo "Running benchmarks..."
    go test -bench=. ./...
```

```
# Clean build artifacts
clean:
    @echo "Cleaning build artifacts..."
    rm -rf bin/
    rm -f coverage.out coverage.html

# Install the application
install: build
    @echo "Installing $(APP_NAME)..."
    go install $(LD_FLAGS) $(MAIN_PATH)

generate-mocks:
    @echo "Generating mocks..."
    @go install go.uber.org/mock/mockgen@latest
    @mockgen -source=internal/service/service.go -destination=internal/service/mocks/mocks.go
                                                    -package=mocks

# Generate documentation
docs:
    @echo "Generating documentation..."
    godoc -http=:6060

# Show help
help:
    @echo "Available targets:"
    @echo "  build      - Build the application"
    @echo "  run        - Build and run the application"
    @echo "  run-only   - Run the application (without building)"
    @echo "  deps       - Install dependencies"
    @echo "  deps-update - Update dependencies"
    @echo "  fmt        - Format code"
    @echo "  lint       - Run linter"
    @echo "  test       - Run tests"
    @echo "  test-coverage - Run tests with coverage report"
    @echo "  bench      - Run benchmarks"
    @echo "  clean      - Clean build artifacts"
    @echo "  install    - Install the application"
    @echo "  docs       - Generate documentation"
    @echo "  help       - Show this help"

# Development workflow
dev: fmt lint test build

# CI/CD workflow
ci: deps fmt lint test build

.PHONY: build run run-only deps deps-update fmt lint test test-coverage bench clean install
docs help dev ci
```

Сгенерировав моки, мы теперь можем написать тесты на методы транзакций (листинг 5.4). Все файлы с тестами имеют в названии фрагмент *_test.go.

Листинг 5.4. Transaction: internal/service/service_test.go

```
package service

import (
    "context"
    "encoding/json"
    "errors"
    "testing"
    "time"

    "transaction/internal/model"
    "transaction/internal/service/mocks"

    "github.com/rs/zerolog"
    "github.com/stretchr/testify/assert"
    "go.uber.org/mock/gomock"
)

func TestTransactionService_Deposit(t *testing.T) {
    tests := []struct {
        name          string
        userID        uint64
        amount        float64
        setupMocks    func(*mocks.MockRepository, *mocks.MockKafkaPublisher)
        expectedError bool
        expectedResult model.TransactionDetails
    }{
        {
            name: "successful deposit",
            userID: 1,
            amount: 100.50,
            setupMocks: func(mockRepo *mocks.MockRepository, mockKafka *mocks.MockKafkaPublisher) {
                expectedParams := model.DepositParams{
                    UserID: 1,
                    Amount: 100.50,
                }
            },
            expectedResult := model.TransactionDetails{
                Transaction: model.Transaction{
                    ID:      1,
                    UserID:  1,
                    Amount:  10050, // 100.50 * 100
                    Status:  model.TransactionStatusPending,
                },
            },
        },
    }

    for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
            mockRepo := mocks.NewMockRepository(t)
            mockKafka := mocks.NewMockKafkaPublisher(t)
            test.setupMocks(mockRepo, mockKafka)

            ctx := context.Background()
            result, err := Deposit(ctx, test.userID, test.amount)

            if test.expectedError {
                assert.Error(t, err)
            } else {
                assert.NoError(t, err)
                assert.Equal(t, test.expectedResult, result)
            }
        })
    }
}
```



```

        Type:      model.TransactionTypeDeposit,
        CreatedAt: time.Now(),
        UpdatedAt: time.Now(),
    },
    Entries: []model.TransactionEntry{},
}

mockRepo.EXPECT().
    Deposit(gomock.Any(), expectedParams).
    Return(expectedResult, nil)

mockKafka.EXPECT().
    Publish(gomock.Any(), "transaction_data", "1", gomock.Any()).
    Return(nil)
},
expectedError: false,
expectedResult: model.TransactionDetails{
    Transaction: model.Transaction{
        ID:          1,
        UserID:      1,
        Amount:      10050,
        Status:      model.TransactionStatusPending,
        Type:        model.TransactionTypeDeposit,
        CreatedAt:   time.Now(),
        UpdatedAt:   time.Now(),
    },
    Entries: []model.TransactionEntry{},
},
},
{
    name: "repository error",
    userID: 1,
    amount: 100.50,
    setupMocks: func(mockRepo *mocks.MockRepository, mockKafka
                                                                    *mocks.MockKafkaPublisher) {
        expectedParams := model.DepositParams{
            UserID: 1,
            Amount: 100.50,
        }

        mockRepo.EXPECT().
            Deposit(gomock.Any(), expectedParams).
            Return(model.TransactionDetails{}, errors.New("database error"))
    },
    expectedError: true,
    expectedResult: model.TransactionDetails{},
},

```

```

{
    name: "kafka publish error",
    userID: 1,
    amount: 100.50,
    setupMocks: func(mockRepo *mocks.MockRepository, mockKafka
                                                                    *mocks.MockKafkaPublisher) {

        expectedParams := model.DepositParams{
            UserID: 1,
            Amount: 100.50,
        }

        expectedResult := model.TransactionDetails{
            Transaction: model.Transaction{
                ID:          1,
                UserID:      1,
                Amount:       10050,
                Status:      model.TransactionStatusPending,
                Type:         model.TransactionTypeDeposit,
                CreatedAt:    time.Now(),
                UpdatedAt:    time.Now(),
            },
            Entries: []model.TransactionEntry{},
        }

        mockRepo.EXPECT().
            Deposit(gomock.Any(), expectedParams).
            Return(expectedResult, nil)

        mockKafka.EXPECT().
            Publish(gomock.Any(), "transaction_data", "1", gomock.Any()).
            Return(errors.New("kafka error"))

        mockRepo.EXPECT().
            UpdateTransactionStatus(gomock.Any(), uint64(1), model.TransactionStatusFailed).
            Return(nil)
    },
    expectedError: true,
    expectedResult: model.TransactionDetails{},
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        ctrl := gomock.NewController(t)
        defer ctrl.Finish()

        mockRepo := mocks.NewMockRepository(ctrl)
        mockAccountService := mocks.NewMockAccountService(ctrl)
    })
}

```

```

mockKafka := mocks.NewMockKafkaPublisher(ctrl)
logger := zerolog.Nop()

tt.setupMocks(mockRepo, mockKafka)

service := New(mockRepo, mockAccountService, mockKafka, &logger)

result, err := service.Deposit(context.Background(), tt.userID, tt.amount)

if tt.expectedError {
    assert.Error(t, err)
} else {
    assert.NoError(t, err)
    assert.Equal(t, tt.expectedResult.Transaction.ID, result.Transaction.ID)
    assert.Equal(t, tt.expectedResult.Transaction.UserID, result.Transaction.UserID)
    assert.Equal(t, tt.expectedResult.Transaction.Amount, result.Transaction.Amount)
    assert.Equal(t, tt.expectedResult.Transaction.Status, result.Transaction.Status)
    assert.Equal(t, tt.expectedResult.Transaction.Type, result.Transaction.Type)
}
})
}
}

func TestTransactionService_Withdraw(t *testing.T) {
    tests := []struct {
        name           string
        accountID      uint64
        amount         float64
        setupMocks     func(*mocks.MockRepository, *mocks.MockKafkaPublisher)
        expectedError  bool
        expectedResult model.TransactionDetails
    }{
        {
            name:           "successful withdraw",
            accountID: 1,
            amount:        50.25,
            setupMocks: func(mockRepo *mocks.MockRepository, mockKafka
                                *mocks.MockKafkaPublisher) {
                expectedParams := model.WithdrawParams{
                    AccountID: 1,
                    Amount:    50.25,
                }

                expectedResult := model.TransactionDetails{
                    Transaction: model.Transaction{
                        ID:        2,
                        UserID:    1,

```

```

        Amount:    5025, // 50.25 * 100
        Status:    model.TransactionStatusPending,
        Type:      model.TransactionTypeWithdraw,
        CreatedAt: time.Now(),
        UpdatedAt: time.Now(),
    },
    Entries: []model.TransactionEntry{},
}

mockRepo.EXPECT().
    Withdraw(gomock.Any(), expectedParams).
    Return(expectedResult, nil)

mockKafka.EXPECT().
    Publish(gomock.Any(), "transaction_data", "1", gomock.Any()).
    Return(nil)
},
expectedError: false,
expectedResult: model.TransactionDetails{
    Transaction: model.Transaction{
        ID:        2,
        UserID:    1,
        Amount:    5025,
        Status:    model.TransactionStatusPending,
        Type:      model.TransactionTypeWithdraw,
        CreatedAt: time.Now(),
        UpdatedAt: time.Now(),
    },
    Entries: []model.TransactionEntry{},
},
},
{
    name:        "repository error",
    accountID: 1,
    amount:      50.25,
    setupMocks: func(mockRepo *mocks.MockRepository, mockKafka
                                                *mocks.MockKafkaPublisher) {
        expectedParams := model.WithdrawParams{
            AccountID: 1,
            Amount:    50.25,
        }

        mockRepo.EXPECT().
            Withdraw(gomock.Any(), expectedParams).
            Return(model.TransactionDetails{}, errors.New("database error"))
    },

```

```

        expectedError: true,
        expectedResult: model.TransactionDetails(),
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        ctrl := gomock.NewController(t)
        defer ctrl.Finish()

        mockRepo := mocks.NewMockRepository(ctrl)
        mockAccountService := mocks.NewMockAccountService(ctrl)
        mockKafka := mocks.NewMockKafkaPublisher(ctrl)
        logger := zerolog.Nop()

        tt.setupMocks(mockRepo, mockKafka)

        service := New(mockRepo, mockAccountService, mockKafka, &logger)

        result, err := service.Withdraw(context.Background(), tt.accountID, tt.amount)

        if tt.expectedError {
            assert.Error(t, err)
        } else {
            assert.NoError(t, err)
            assert.Equal(t, tt.expectedResult.Transaction.ID, result.Transaction.ID)
            assert.Equal(t, tt.expectedResult.Transaction.UserID, result.Transaction.UserID)
            assert.Equal(t, tt.expectedResult.Transaction.Amount, result.Transaction.Amount)
            assert.Equal(t, tt.expectedResult.Transaction.Status, result.Transaction.Status)
            assert.Equal(t, tt.expectedResult.Transaction.Type, result.Transaction.Type)
        }
    })
}

func TestTransactionService_Transfer(t *testing.T) {
    tests := []struct {
        name           string
        userID         uint64
        recipient       uint64
        amount         float64
        setupMocks     func(*mocks.MockRepository, *mocks.MockKafkaPublisher)
        expectedError  bool
        expectedResult model.TransactionDetails
    }{
        {
            name:         "successful transfer",
            userID:       1,

```

```

recipient: 2,
amount:    75.00,
setupMocks: func(mockRepo *mocks.MockRepository, mockKafka
                                                         *mocks.MockKafkaPublisher) {

    expectedParams := model.TransferParams{
        UserID:    1,
        Recipient: 2,
        Amount:    75.00,
    }

    expectedResult := model.TransactionDetails{
        Transaction: model.Transaction{
            ID:        3,
            UserID:    1,
            Amount:    7500, // 75.00 * 100
            Status:    model.TransactionStatusPending,
            Type:      model.TransactionTypeTransfer,
            CreatedAt: time.Now(),
            UpdatedAt: time.Now(),
        },
        Entries: []model.TransactionEntry{},
    }

    mockRepo.EXPECT().
        Transfer(gomock.Any(), expectedParams).
        Return(expectedResult, nil)

    mockKafka.EXPECT().
        Publish(gomock.Any(), "transaction_data", "1", gomock.Any()).
        Return(nil)
},
expectedError: false,
expectedResult: model.TransactionDetails{
    Transaction: model.Transaction{
        ID:        3,
        UserID:    1,
        Amount:    7500,
        Status:    model.TransactionStatusPending,
        Type:      model.TransactionTypeTransfer,
        CreatedAt: time.Now(),
        UpdatedAt: time.Now(),
    },
    Entries: []model.TransactionEntry{},
},
},
{
    name:      "repository error",
    userID:    1,

```

```

recipient: 2,
amount:    75.00,
setupMocks: func(mockRepo *mocks.MockRepository, mockKafka
                                                         *mocks.MockKafkaPublisher) {
    expectedParams := model.TransferParams{
        UserID:    1,
        Recipient: 2,
        Amount:    75.00,
    }

    mockRepo.EXPECT().
        Transfer(gomock.Any(), expectedParams).
        Return(model.TransactionDetails{}, errors.New("database error"))
},
expectedError: true,
expectedResult: model.TransactionDetails{},
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        ctrl := gomock.NewController(t)
        defer ctrl.Finish()

        mockRepo := mocks.NewMockRepository(ctrl)
        mockAccountService := mocks.NewMockAccountService(ctrl)
        mockKafka := mocks.NewMockKafkaPublisher(ctrl)
        logger := zerolog.Nop()

        tt.setupMocks(mockRepo, mockKafka)

        service := New(mockRepo, mockAccountService, mockKafka, &logger)

        result, err := service.Transfer(context.Background(), tt.userID, tt.recipient,
                                                         tt.amount)

        if tt.expectedError {
            assert.Error(t, err)
        } else {
            assert.NoError(t, err)
            assert.Equal(t, tt.expectedResult.Transaction.ID, result.Transaction.ID)
            assert.Equal(t, tt.expectedResult.Transaction.UserID, result.Transaction.UserID)
            assert.Equal(t, tt.expectedResult.Transaction.Amount, result.Transaction.Amount)
            assert.Equal(t, tt.expectedResult.Transaction.Status, result.Transaction.Status)
            assert.Equal(t, tt.expectedResult.Transaction.Type, result.Transaction.Type)
        }
    })
}
}

```

```
func TestTransactionService_HandleAccountResponse(t *testing.T) {
    tests := []struct {
        name          string
        response       AccountResponse
        setupMocks     func(*mocks.MockRepository)
        expectedError bool
    }{
        {
            name: "successful response - transaction completed",
            response: AccountResponse{
                RequestType: "deposit",
                UserID:      1,
                OperationID: 1,
                Result:      true,
            },
            setupMocks: func(mockRepo *mocks.MockRepository) {
                mockRepo.EXPECT().
                    UpdateTransactionStatus(gomock.Any(), uint64(1),
                                                                model.TransactionStatusCompleted).
                    Return(nil)
            },
            expectedError: false,
        },
        {
            name: "failed response - transaction failed",
            response: AccountResponse{
                RequestType: "withdraw",
                UserID:      1,
                OperationID: 2,
                Result:      false,
            },
            setupMocks: func(mockRepo *mocks.MockRepository) {
                mockRepo.EXPECT().
                    UpdateTransactionStatus(gomock.Any(), uint64(2), model.TransactionStatusFailed).
                    Return(nil)
            },
            expectedError: false,
        },
        {
            name: "repository error",
            response: AccountResponse{
                RequestType: "deposit",
                UserID:      1,
                OperationID: 1,
                Result:      true,
            },
        },
    }
```



```

    setupMocks: func(mockRepo *mocks.MockRepository) {
        mockRepo.EXPECT().
            UpdateTransactionStatus(gomock.Any(), uint64(1),
                                   model.TransactionStatusCompleted).
            Return(errors.New("database error"))
    },
    expectedError: true,
},
{
    name: "invalid response - missing request type",
    response: AccountResponse{
        RequestType: "",
        UserID:      1,
        OperationID: 1,
        Result:      true,
    },
    setupMocks: func(mockRepo *mocks.MockRepository) {
        // No repository calls expected for invalid response
    },
    expectedError: true,
},
{
    name: "invalid response - missing user ID",
    response: AccountResponse{
        RequestType: "deposit",
        UserID:      0,
        OperationID: 1,
        Result:      true,
    },
    setupMocks: func(mockRepo *mocks.MockRepository) {
        // No repository calls expected for invalid response
    },
    expectedError: true,
},
{
    name: "invalid response - missing operation ID",
    response: AccountResponse{
        RequestType: "deposit",
        UserID:      1,
        OperationID: 0,
        Result:      true,
    },
    setupMocks: func(mockRepo *mocks.MockRepository) {
        // No repository calls expected for invalid response
    },
    expectedError: true,
},
}
}

```

```

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        ctrl := gomock.NewController(t)
        defer ctrl.Finish()

        mockRepo := mocks.NewMockRepository(ctrl)
        mockAccountService := mocks.NewMockAccountService(ctrl)
        mockKafka := mocks.NewMockKafkaPublisher(ctrl)
        logger := zerolog.Nop()

        tt.setupMocks(mockRepo)

        service := New(mockRepo, mockAccountService, mockKafka, &logger)

        // Convert response to JSON bytes
        responseBytes, err := json.Marshal(tt.response)
        assert.NoError(t, err)

        err = service.HandleAccountResponse(context.Background(), "test_topic", "test_key",
            responseBytes)

        if tt.expectedError {
            assert.Error(t, err)
        } else {
            assert.NoError(t, err)
        }
    })
}

func TestTransactionService_GetTransactionsWithDetails(t *testing.T) {
    tests := []struct {
        name      string
        params     model.GetTransactionsParams
        setupMocks func(*mocks.MockRepository)
        expectedError bool
        expectedCount int
    }{
        {
            name: "successful get transactions",
            params: model.GetTransactionsParams{
                UserID: func() *uint64 { id := uint64(1); return &id }(),
                Limit: 10,
                Offset: 0,
            },
            setupMocks: func(mockRepo *mocks.MockRepository) {
                transactions := []model.Transaction{

```

```

        {
            ID:      1,
            UserID:  1,
            Amount:  10000,
            Status:  model.TransactionStatusCompleted,
            Type:    model.TransactionTypeDeposit,
            CreatedAt: time.Now(),
            UpdatedAt: time.Now(),
        },
        {
            ID:      2,
            UserID:  1,
            Amount:  5000,
            Status:  model.TransactionStatusCompleted,
            Type:    model.TransactionTypeWithdraw,
            CreatedAt: time.Now(),
            UpdatedAt: time.Now(),
        },
    ],

    mockRepo.EXPECT().
        GetTransactions(gomock.Any(), gomock.Any()).
        Return(transactions, nil)

    // Mock GetTransactionDetails for each transaction
    for _, tx := range transactions {
        details := model.TransactionDetails{
            Transaction: tx,
            Entries:     []model.TransactionEntry{},
        }
        mockRepo.EXPECT().
            GetTransactionDetails(gomock.Any(), tx.ID).
            Return(details, nil)
    }
},
expectedError: false,
expectedCount: 2,
},
{
    name: "repository error on get transactions",
    params: model.GetTransactionsParams{
        UserID: func() *uint64 { id := uint64(1); return &id }(),
        Limit:  10,
        Offset: 0,
    },
    setupMocks: func(mockRepo *mocks.MockRepository) {
        mockRepo.EXPECT().

```

```

        GetTransactions(gomock.Any(), gomock.Any()).
        Return(nil, errors.New("database error"))
    },
    expectedError: true,
    expectedCount: 0,
},
{
    name: "error on get transaction details",
    params: model.GetTransactionsParams{
        UserID: func() *uint64 { id := uint64(1); return &id }(),
        Limit: 10,
        Offset: 0,
    },
    setupMocks: func(mockRepo *mocks.MockRepository) {
        transactions := []model.Transaction{
            {
                ID: 1,
                UserID: 1,
                Amount: 10000,
                Status: model.TransactionStatusCompleted,
                Type: model.TransactionTypeDeposit,
                CreatedAt: time.Now(),
                UpdatedAt: time.Now(),
            },
        },
        mockRepo.EXPECT().
            GetTransactions(gomock.Any(), gomock.Any()).
            Return(transactions, nil)

        mockRepo.EXPECT().
            GetTransactionDetails(gomock.Any(), uint64(1)).
            Return(model.TransactionDetails(), errors.New("details error"))
    },
    expectedError: false, // Method continues despite individual detail errors
    expectedCount: 0,     // But returns empty slice
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        ctrl := gomock.NewController(t)
        defer ctrl.Finish()

        mockRepo := mocks.NewMockRepository(ctrl)
        mockAccountService := mocks.NewMockAccountService(ctrl)
        mockKafka := mocks.NewMockKafkaPublisher(ctrl)
        logger := zerolog.Nop()
    })
}

```

```

    tt.setupMocks(mockRepo)

    service := New(mockRepo, mockAccountService, mockKafka, &logger)

    result, err := service.GetTransactionsWithDetails(context.Background(), tt.params)

    if tt.expectedError {
        assert.Error(t, err)
    } else {
        assert.NoError(t, err)
        assert.Len(t, result, tt.expectedCount)
    }
}
}
}

```

Эти unit-тесты на самом деле дают довольно неплохое покрытие — coverage составляет 84,4%. Если говорить о покрытии кода тестами в общем, то для unit-тестирования желательно иметь кейсы на все ветвления в коде и проверять, как поведет себя программа, если в ответе из вызываемых функций получит неожиданный результат. То есть проверять надо не только успешные кейсы, но также важно выполнять проверку и на возврат ошибок.

Здесь в тестах проверяются все бизнес-операции и обработка ответов от сервиса Account. Для изоляции логики используются моки, созданные при помощи пакета GoMock, — они имитируют работу репозитория и Kafka, чтобы тесты проверяли исключительно поведение сервисного слоя без обращения к реальной базе данных или очереди сообщений. Каждый тест включает несколько сценариев, моделирующих как успешное выполнение операции, так и различные виды ошибок (например, сбой репозитория или неудачную публикации события), а пакет Testify применяется для удобного сравнения результатов.

Итак, теперь с этим тестом запустим в Git изменения и запустим обновленный Actions (рис. 5.7). Как мы можем здесь видеть, тесты выполнены успешно.

Теперь — для добавления автоматизации — нужно сделать так, чтобы триггером был не только `workflow_dispatch`, но и, например, еще и `push` (листинг 5.5).

Листинг 5.5. Transaction: `.github/workflows/master.yaml`

```

name: Transaction
on: [workflow_dispatch, push]
jobs:
  stage_dev:
    runs-on: ubuntu-latest
    steps:
      - name: get code
        uses: actions/checkout@v3.5.3

```

- name: install deps
 - run: go mod download
- name: Run tests
 - run: go test -v -race -coverprofile=coverage.out ./...

stage_dev
succeeded 3 minutes ago in 1m 22s

Search logs

Run tests 1m 11s

```

29  --- PASS: TestTransactionService_Withdraw/successful_withdraw (0.00s)
30  --- PASS: TestTransactionService_Withdraw/repository_error (0.00s)
31  === RUN TestTransactionService_Transfer
32  === RUN TestTransactionService_Transfer/successful_transfer
33  === RUN TestTransactionService_Transfer/repository_error
34  --- PASS: TestTransactionService_Transfer (0.00s)
35  --- PASS: TestTransactionService_Transfer/successful_transfer (0.00s)
36  --- PASS: TestTransactionService_Transfer/repository_error (0.00s)
37  === RUN TestTransactionService_HandleAccountResponse
38  === RUN TestTransactionService_HandleAccountResponse/successful_response_-_transaction_completed
39  === RUN TestTransactionService_HandleAccountResponse/failed_response_-_transaction_failed
40  === RUN TestTransactionService_HandleAccountResponse/repository_error
41  === RUN TestTransactionService_HandleAccountResponse/invalid_response_-_missing_request_type
42  === RUN TestTransactionService_HandleAccountResponse/invalid_response_-_missing_user_ID
43  === RUN TestTransactionService_HandleAccountResponse/invalid_response_-_missing_operation_ID
44  --- PASS: TestTransactionService_HandleAccountResponse (0.00s)
45  --- PASS: TestTransactionService_HandleAccountResponse/successful_response_-_transaction_completed (0.00s)
46  --- PASS: TestTransactionService_HandleAccountResponse/failed_response_-_transaction_failed (0.00s)
47  --- PASS: TestTransactionService_HandleAccountResponse/repository_error (0.00s)
48  --- PASS: TestTransactionService_HandleAccountResponse/invalid_response_-_missing_request_type (0.00s)
49  --- PASS: TestTransactionService_HandleAccountResponse/invalid_response_-_missing_user_ID (0.00s)
50  --- PASS: TestTransactionService_HandleAccountResponse/invalid_response_-_missing_operation_ID (0.00s)
51  === RUN TestTransactionService_GetTransactionsWithDetails
52  === RUN TestTransactionService_GetTransactionsWithDetails/successful_get_transactions
53  === RUN TestTransactionService_GetTransactionsWithDetails/repository_error_on_get_transactions
54  === RUN TestTransactionService_GetTransactionsWithDetails/error_on_get_transaction_details
55  --- PASS: TestTransactionService_GetTransactionsWithDetails (0.00s)
56  --- PASS: TestTransactionService_GetTransactionsWithDetails/successful_get_transactions (0.00s)
57  --- PASS: TestTransactionService_GetTransactionsWithDetails/repository_error_on_get_transactions (0.00s)
58  --- PASS: TestTransactionService_GetTransactionsWithDetails/error_on_get_transaction_details (0.00s)
59  PASS
60  coverage: 84.4% of statements
61  ok      transaction/internal/service  1.017s  coverage: 84.4% of statements
62  transaction/internal/service/mocks  coverage: 0.0% of statements
  
```

> Post get code 0s

> Complete job 0s

Рис. 5.7. Результат выполнения обновленного Actions

В результате мы можем видеть, как по коммиту выполняется Workflow, — это уже некоторая автоматизация (рис. 5.8), поскольку код прошел проверки на то, что тесты не поломались, и он соответствует соглашениям линтера.

Следующий этап — доставка кода на хостинг. Бесплатно предоставляемые VPS (виртуальные частные серверы) подходят разве что для простых приложений, а т. к. у нас задействован довольно большой инструментарий, воспользуемся Yandex Cloud, который предоставляет стартовый грант для изучения возможностей платформы. К тому же он сейчас всё чаще применяется и в промышленной разработке.

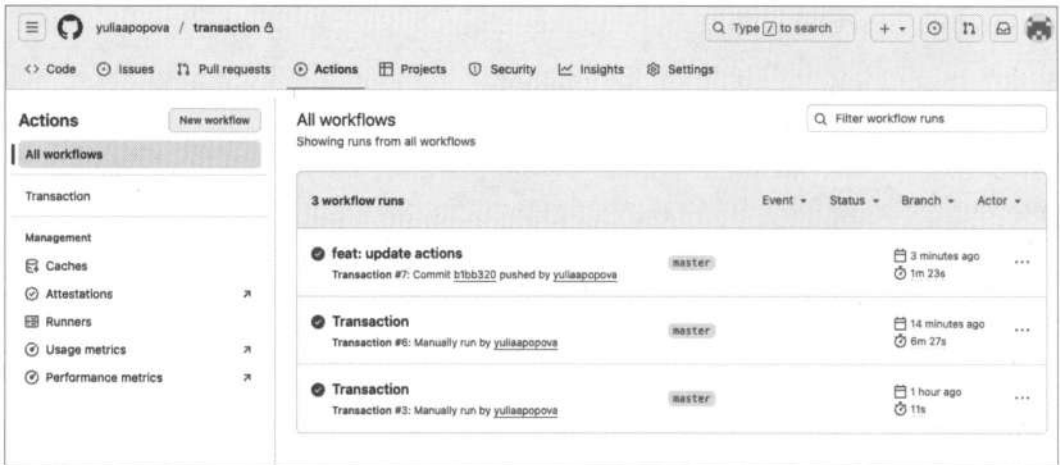


Рис. 5.8. Выполнение Actions по команде push

Обертывание микросервисов в docker-контейнеры

Для того чтобы поместить все наши микросервисы в один контейнер, необходимо в первую очередь создать в каждом сервисе файл `Dockerfile`, в котором будут прописаны инструкции для создания необходимого окружения. Начнем с сервиса Account (листинг 5.6).

Листинг 5.6. Файл Dockerfile

```
FROM golang:1.24.5-alpine AS build
WORKDIR /app

COPY go.mod go.sum ./
RUN go mod download

# копируем весь код
COPY . .

# если main.go в корне проекта
RUN go build -o app ./cmd

# --- финальный образ ---
FROM alpine:3.18
WORKDIR /app

COPY --from=build /app/app .
COPY --from=build /app/internal/migrations ./internal/migrations

CMD ["/app"]
```

Итак, что же здесь происходит?

- ❑ Команда `FROM` показывает, что будет использоваться в качестве системы. Обратите внимание, что значение может быть как `golang:latest`, так и `golang:1.24.5`. Если установить значение `golang:latest`, то могут возникнуть непредвиденные сложности с совместимостью. Во втором же случае всё будет установлено так, как нужно, однако версия с окончанием на `alpine` занимает гораздо меньше места, чем ее аналоги.
- ❑ Далее мы устанавливаем каталог `app`, который будет считаться для нашего контейнера актуальным.
- ❑ Команда `COPY go.mod go.sum ./` копирует в контейнер только файлы `go.mod` и `go.sum`. Это копирование выполняется в первую очередь отдельным шагом для оптимизации — сначала устанавливаются зависимости, а потом уже копируется «тяжелый» исходный код. Так работает Docker layer cache: если код поменялся, но зависимости нет — они не скачиваются заново.
- ❑ Скачиваем все зависимости командой `RUN go mod download`.
- ❑ Команда `COPY . .` копирует все содержимое из текущего каталога в каталог `app` в контейнере.
- ❑ Команда `RUN go build -o ./cmd` нужна, чтобы собрать весь проект в один исполняемый файл.
- ❑ Устанавливаем чистый Alpine Linux без компилятора и инструментов, т.е. минимальную сборку ~5 Мбайт.
- ❑ Внутри нового образа рабочий каталог тоже будет `/app`.
- ❑ Копируем бинарник `app`, который собрали на первом этапе (`--from=build` — значит, взять файл из стадии `build`).
- ❑ Копируем папку с миграциями из стадии `build`. Так как наш сервис при старте проверяет миграции, без этого он не запустится.
- ❑ Команда `CMD` означает, что это будет выполнено, когда контейнер станут запускать. Команды `RUN` при этом выполняются в момент создания образа.

Чтобы корректно использовать копирование в контейнер, нужно добавить также файл `.dockerignore`, — он работает по тому же принципу, что и `.gitignore` (листинг 5.7).

Листинг 5.7. Файл `.dockerignore`

```
.idea  
vendor
```

Здесь мы указываем файлы и каталоги, которые не должны попасть в контейнер, тем самым увеличивая его размер.

Эти два файла: `Dockerfile` и `.dockerignore` — будут одинаковыми для всех наших сервисов, поэтому просто скопируем их везде.

И доработаем также наш файл `docker-compose` — чтобы из созданного образа формировались все сервисы наравне с базами данных (листинг 5.8).

Листинг 5.8. Файл docker-compose.yaml

```
services:
  account_db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: account
    ports:
      - "5432:5432"
    volumes:
      - postgres_account_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  auth_db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: auth
    ports:
      - "5433:5432"
    volumes:
      - postgres_auth_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  transaction_db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: transaction
    ports:
      - "5434:5432"
```

```
volumes:
  - postgres_transaction_data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 5s
  timeout: 5s
  retries: 5
networks:
  - app-network
```

kafka:

```
image: confluentinc/cp-kafka:7.5.0
container_name: book-kafka
platform: linux/x86_64
environment:
  CLUSTER_ID: 5DG6kO2sQY6c8lUfs8nG5A
  KAFKA_NODE_ID: 1
  KAFKA_PROCESS_ROLES: broker,controller
  KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER

  # объявляем все виды listeners, в том числе PLAINTEXT_HOST
  KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,PLAINTEXT_HOST://:29092

  # объявляем external/host endpoints
  KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092

  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
    PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT,CONTROLLER:PLAINTEXT
  KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
  KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093

  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
  KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
  KAFKA_LOG_DIRS: /var/lib/kafka/data
ports:
  - "29092:29092" # внешний порт
  - "9092:9092"   # внутренняя сеть docker
restart: unless-stopped
```

kafka-ui:

```
image: provectuslabs/kafka-ui:latest
container_name: book-kafka-ui
platform: linux/x86_64
environment:
  - KAFKA_CLUSTERS_0_NAME=local
  - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:9092
```

```
depends_on:
  - kafka
ports:
  - "29093:8080"
restart: unless-stopped

transaction:
  image: transaction:latest
  build: ./transaction
  ports:
    - "50054:8080"
  environment:
    DB_DSN: postgres://postgres:postgres@transaction_db:5432/transaction?sslmode=disable
    ACCOUNT_GRPC_HOST: account:8080
    KAFKA_BROKER_HOST: kafka:9092
    KAFKA_CONSUMER_GROUP: consumer
    KAFKA_TRANSACTION_TOPIC: transaction_response
  depends_on:
    - transaction_db
    - kafka
  networks:
    - app-network

account:
  image: account:latest
  build: ./account
  ports:
    - "50051:8080"
  environment:
    DB_DSN: postgres://postgres:postgres@account_db:5432/account?sslmode=disable
    KAFKA_BROKER_HOST: kafka:9092
    KAFKA_CONSUMER_GROUP: consumer-account
    KAFKA_TRANSACTION_TOPIC: transaction_request
  depends_on:
    - account_db
    - kafka
  networks:
    - app-network

auth:
  image: auth:latest
  build: ./auth
  ports:
    - "50052:8080"
  environment:
    JWT_SECRET: secret
    ACCESS_TOKEN_TTL_MINUTES: 60
```

```
    REFRESH_TOKEN_TTL_DAYS: 30
    DB_DSN: postgres://postgres:postgres@auth_db:5432/auth?sslmode=disable
  depends_on:
    - auth_db
  networks:
    - app-network

gateway:
  image: gateway:latest
  build: ./gateway
  ports:
    - "50053:8080"
  environment:
    JWT_SECRET: secret
    ACCOUNT_GRPC_HOST: account:8080
    AUTH_GRPC_HOST: auth:8080
    TRANSACTION_GRPC_HOST: transaction:8080
  networks:
    - app-network

volumes:
  postgres_account_data:
  postgres_auth_data:
  postgres_transaction_data:

networks:
  app-network:
    driver: bridge
```

Как можно видеть, здесь добавлена сборка для `transaction`, `account`, `auth`, `gateway`. Через поле `build` указывается, где искать образ, из которого нужно собрать сервис. Через `depends_on` устанавливаются зависимости, чтобы сервис запускался после них.

Кроме того, поскольку теперь подключения из сервисов к `kafka` и `postgresql` идут изнутри контейнеров, то и хост для подключения также изменился в переменных окружения, которые теперь все заданы в `docker-compose`.

То есть впредь в качестве хоста необходимо указывать название сервиса, к которому идет подключение.

Также мы разместили этот файл `docker-compose.yaml` в уровень выше всех проектов, чтобы файл `Dockerfile`, по которому нужно собрать образ, было удобнее искать, просто указав каталог в `build`. В результате структура проекта будет выглядеть так, как показано на рис. 5.9.

Чтобы запустить контейнеры, выполним команду:

```
docker-compose up
```

В результате через `Docker Desktop` мы можем увидеть состояние всех контейнеров (рис. 5.10).

Имя	Дата изменения	Размер	Тип
> account	Сегодня, 21:09	--	Папка
> auth	Сегодня, 21:08	--	Папка
docker-compose.yaml	Сегодня, 21:17	4 КБ	YAML
> gateway	Сегодня, 21:09	--	Папка
> transaction	Сегодня, 21:26	--	Папка

Рис. 5.9. Структура проекта для контейнеризации

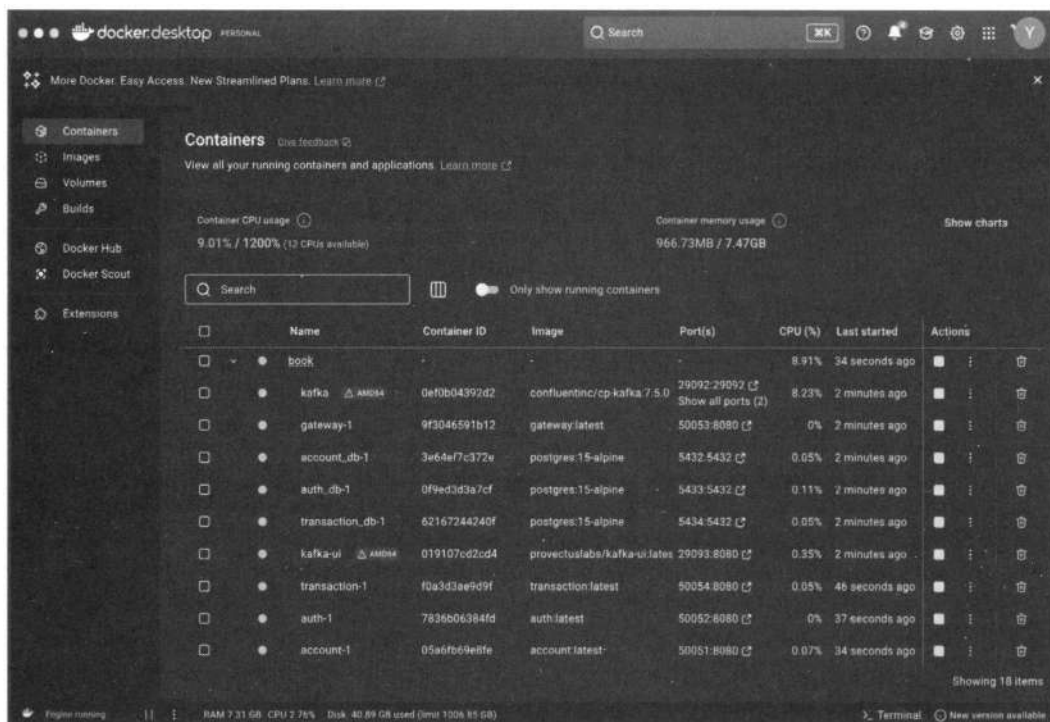


Рис. 5.10. book-all с запущенными docker-контейнерами

На следующем шаге мы можем доставить докер-образы на виртуальную машину — для этого нужно собранный образ залить в Docker Hub. Но сначала придется там зарегистрироваться — по адресу: <https://hub.docker.com/>.

Затем открыть терминал и авторизоваться в Docker:

```
docker login
```

Если всё выполнено верно, то вы увидите такой ответ:

```
Login Succeeded
```

Logging in with your password grants your terminal complete access to your account.

For better security, log in with a limited-privilege personal access token. Learn more at <https://docs.docker.com/go/access-tokens/>

Теперь нужно пометить созданные нами образы тегами и закоммитить их в Docker Hub. Но сначала посмотрим, какие образы у нас есть:

```
docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
transaction	latest	9fd2c7c61955	3 minutes ago	53.8MB
account	latest	efcc5810a079	3 minutes ago	53.6MB
auth	latest	7524c8820fc4	3 minutes ago	49.1MB
gateway	latest	2a517bf236f3	3 minutes ago	38.4MB

Здесь нас интересуют все четыре образа — их как раз и необходимо загрузить в Docker Hub. Для этого выполним команду:

```
docker tag book_all-auth yuliapopova/book_all-auth:latest
```

Здесь:

- ☐ `book_all-auth` — название образа в локальном репозитории;
- ☐ `yuliapopova` — логин пользователя;
- ☐ `book_all-auth` — имя для образа на хабе, а `latest` — версия.

Теперь образ готов к коммиту:

```
docker push yuliapopova/book_all-auth:latest
```

Повторим то же самое и для остальных образов. В результате на главной странице Docker Hub в вашей учетной записи можно будет увидеть все созданные докер-образы (рис. 5.11), причем доступ к ним сейчас возможен откуда угодно. Соответственно, мы можем запустить эти образы и на удаленном сервере.

Отредактируем файл `docker-compose.yml` так, чтобы образы теперь загружались из облака, а не собирались локально (листинг 5.9).

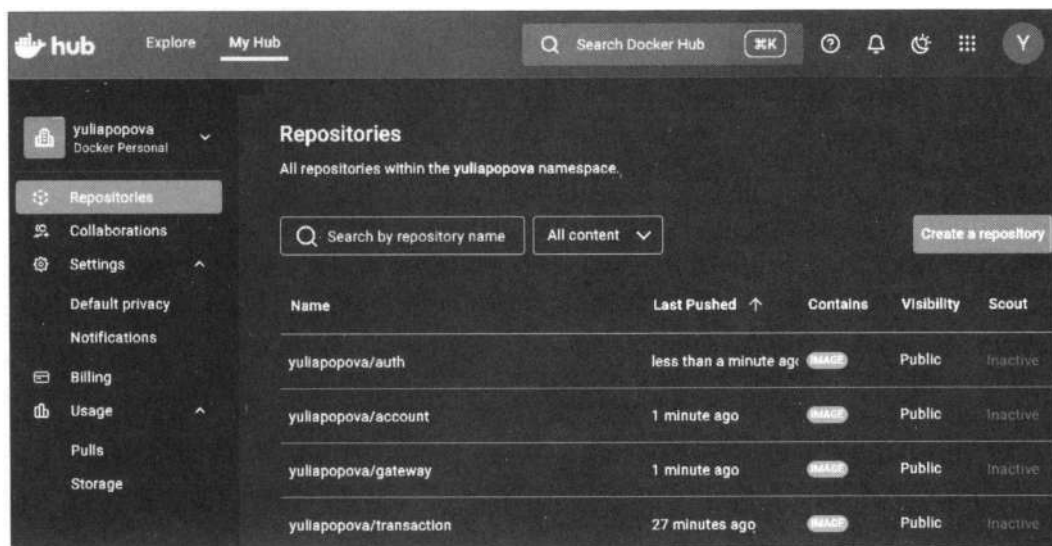


Рис. 5.11. Опубликованные docker-образы

Листинг 5.9. Файл docker-compose.yaml

```
services:
  account_db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: account
    ports:
      - "5432:5432"
    volumes:
      - postgres_account_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  auth_db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: auth
    ports:
      - "5433:5432"
    volumes:
      - postgres_auth_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  transaction_db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: transaction
    ports:
      - "5434:5432"
```

```
volumes:
  - postgres_transaction_data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 5s
  timeout: 5s
  retries: 5
networks:
  - app-network
```

kafka:

```
image: confluentinc/cp-kafka:7.5.0
container_name: book-kafka
platform: linux/x86_64
environment:
  CLUSTER_ID: 5DG6k02sQY6c8lUfs8nG5A
  KAFKA_NODE_ID: 1
  KAFKA_PROCESS_ROLES: broker,controller
  KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER

  # объявляем все виды listeners, в том числе PLAINTEXT_HOST
  KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,PLAINTEXT_HOST://:29092

  # объявляем external/host endpoints
  KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092

  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
    PLAINTEXT:PLAINTEXT, PLAINTEXT_HOST:PLAINTEXT, CONTROLLER:PLAINTEXT
  KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
  KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093

  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
  KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
  KAFKA_LOG_DIRS: /var/lib/kafka/data
ports:
  - "29092:29092" # внешний порт
  - "9092:9092"   # внутренняя сеть docker
restart: unless-stopped
networks:
  - app-network
```

kafka-ui:

```
image: provectuslabs/kafka-ui:latest
container_name: book-kafka-ui
platform: linux/x86_64
```



```
environment:
  - KAFKA_CLUSTERS_0_NAME=local
  - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:9092
depends_on:
  - kafka
ports:
  - "29093:8080"
restart: unless-stopped
```

```
transaction:
  image: yuliapopova/transaction:latest
  build: ./transaction
  ports:
    - "50054:8080"
  environment:
    DB_DSN: postgres://postgres:postgres@transaction_db:5432/transaction?sslmode=disable
    ACCOUNT_GRPC_HOST: account:8080
    KAFKA_BROKER_HOST: kafka:9092
    KAFKA_CONSUMER_GROUP: consumer
    KAFKA_TRANSACTION_TOPIC: transaction_response
    GRPC_PORT: 50054
  depends_on:
    - transaction_db
    - kafka
  networks:
    - app-network
```

```
account:
  image: yuliapopova/account:latest
  build: ./account
  ports:
    - "50051:8080"
  environment:
    DB_DSN: postgres://postgres:postgres@account_db:5432/account?sslmode=disable
    KAFKA_BROKER_HOST: kafka:9092
    KAFKA_CONSUMER_GROUP: consumer-account
    KAFKA_TRANSACTION_TOPIC: transaction_request
    GRPC_PORT: 50051
  depends_on:
    - account_db
    - kafka
  networks:
    - app-network
```

```
auth:
  image: yuliapopova/auth:latest
  build: ./auth
```

```
ports:
  - "50052:8080"
environment:
  JWT_SECRET: secret
  ACCESS_TOKEN_TTL_MINUTES: 60
  REFRESH_TOKEN_TTL_DAYS: 30
  DB_DSN: postgres://postgres:postgres@auth_db:5432/auth?sslmode=disable
  GRPC_PORT: 50052
depends_on:
  - auth_db
networks:
  - app-network

gateway:
  image: yuliapopova/gateway:latest
  build: ./gateway
  ports:
    - "50053:8080"
  environment:
    JWT_SECRET: secret
    ACCOUNT_GRPC_HOST: account:8080
    AUTH_GRPC_HOST: auth:8080
    TRANSACTION_GRPC_HOST: transaction:8080
    GRPC_PORT: 50053
  networks:
    - app-network

volumes:
  postgres_account_data:
  postgres_auth_data:
  postgres_transaction_data:

networks:
  app-network:
    driver: bridge
```

Остается только доставить Docker-Compose и переменные окружения на виртуальную машину и запустить. Сделать это можно, например, через GitHub.

В этом разделе мы рассмотрели третий способ развертывания микросервисов. В результате его применения работы по настройке приложения на сторонних серверах или компьютерах заметно сократились.

Масштабирование при помощи оркестратора Kubernetes

Kubernetes — это мощный инструмент для оркестровки контейнеризированных приложений с открытым исходным кодом, позволяющий автоматизировать процесс развертывания, масштабирования и координации сервисов в условиях кластера [2].

Kubernetes предоставляет следующие возможности [3]:

- ❑ планирование распределения рабочей нагрузки для множества хостов;
- ❑ предоставление декларативного, расширяемого API-интерфейса для взаимодействия с системой;
- ❑ утилиту командной строки `kubectl` для взаимодействия с API-сервером в ручном режиме;
- ❑ приведение объектов от текущего состояния к желаемому;
- ❑ предоставление базового служебного слоя для направления запросов к приложениям и обратно;
- ❑ несколько интерфейсов для поддержки подключаемых модулей, отвечающих за работу с сетью, хранилищем и т. д.

Есть несколько важных понятий, которые необходимо знать, перед тем как продолжить разбираться дальше.

❑ Узел (node).

Отдельная физическая или виртуальная машина, на которой развернуты и выполняются контейнеры приложений. Каждый узел содержит сервисы для запуска приложений в контейнерах и компоненты для централизованного управления узлом.

❑ Под (pod).

Базовая единица для запуска и управления приложениями. Под представляет собой один или несколько контейнеров, гарантированных для запуска на одном узле. В поде обеспечивается разделение ресурсов и межпроцессное взаимодействие, а также предоставляется уникальный IP-адрес в пределах кластера. Это позволяет приложениям использовать фиксированные номера портов без риска конфликта. Управление подами можно осуществлять через API Kubernetes или контроллер.

❑ Том (volume).

Общий ресурс хранения для совместного использования контейнерами в пределах одного пода. Том обеспечивает доступ к данным между контейнерами внутри пода.

❑ Контроллер реплик (replication controllers).

Обеспечивает масштабирование, запуская необходимое количество копий пода в кластере. Также реализует запуск новых экземпляров пода, если какие-то выходят из строя.

❑ Сервис (services).

Сервис в Kubernetes — это абстракция, которая определяет логический объединенный в группу под и политику доступа к нему.

Более подробно разбираться с тем, что такое Kubernetes, мы не будем, а перейдем лучше к практике. Первое, что необходимо сделать, — это установить Kubernetes, выполнив соответствующие инструкции¹.

Когда всё установится, создадим для простоты примера под nginx:

```
kubect1 run nginx --image=nginx:latest --port=80
```

Чтобы посмотреть информацию об этом поде, необходимо ввести в терминале:

```
kubect1 describe pods nginx
```

В ответ будет выведена следующая информация:

```
Name:          nginx
Namespace:     default
Priority:       0
Service Account: default
Node:          minikube/192.168.49.2
Start Time:    Wed, 10 Sep 2025 20:04:01 +0000
Labels:        run=nginx
Annotations:    <none>
Status:        Pending
IP:
IPs:           <none>
Containers:
  nginx:
    Container ID:
    Image:        nginx:latest
    Image ID:
    Port:         80/TCP
    Host Port:    0/TCP
    State:        Waiting
      Reason:     ContainerCreating
    Ready:        False
    Restart Count: 0
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-zmkkr (ro)
Conditions:
  Type          Status
  PodReadyToStartContainers  False
  Initialized    True
  Ready          False
```

¹ См. <https://kubernetes.io/docs/tasks/tools/install-kubect1-linux/>.

```

ContainersReady      False
PodScheduled          True
Volumes:
  kube-api-access-zmkk:
    Type:              Projected (a volume that contains injected data from multiple
                        sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:      kube-root-ca.crt
    Optional:           false
    DownwardAPI:        true
QoS Class:            BestEffort
Node-Selectors:        <none>
Tolerations:          node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                      node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason      Age    From          Message
  ----    -
  Normal  Scheduled   6s     default-scheduler Successfully assigned default/nginx to minikube
  Normal  Pulling     5s     kubelet       Pulling image "nginx:latest"

```

Но так создавать поды не рекомендуется — желательно делать это через конфигурацию, которая в контексте Kubernetes называется *манифестом* (листинг 5.10).

Листинг 5.10. Файл pod.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-nginx-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: cr.yandex/crpv7tlcpgb30qpgkij/ubuntu-nginx:latest

```

Разберемся, из чего состоит этот манифест:

- ❑ `apiVersion` — определяет, для какой версии Kubernetes он написан. В разных версиях это значение может различаться;

- ❑ `kind` — механизм использования, может принимать различные заданные значения. Чтобы развернуть приложение, указываем `Deployment`;
- ❑ `metadata` — определяет метаданные приложения. Через них можно производить различные манипуляции с объектами кластера;
- ❑ `spec` — описание объектов Kubernetes;
- ❑ `containers` — контейнеры, которые будут загружены.

Чтобы применить настройки манифеста, нужно выполнить команду:

```
kubectl apply -f pod.yaml
```

В результате появится сообщение:

```
deployment.apps/my-nginx-deployment created
```

Попробуем теперь перенести в Kubernetes наше приложение, для чего воспользуемся инструментом `kompose`, созданным специально для упрощения перехода из `Docker-Compose` в Kubernetes. В использовании `kompose` есть нюанс — он не работает с переменными окружения, если их больше одной, но у нас все переменные окружения и так уже в `docker-compose`.

Теперь нужно установить сам `kompose`. Если вы работаете под `macOS`, то достаточно выполнить команду:

```
brew install kompose
```

В противном случае перейдите по ссылке: <https://kompose.io/installation/> и установите `kompose` удобным для вас способом в зависимости от вашей операционной системы.

Теперь перейдите в каталог с файлом `docker-compose.yaml` и выполните миграцию:

```
kompose convert -f docker-compose.yaml
```

В результате для каждого сервиса будут созданы отдельные файлы-манифесты `deployment` и `service` (рис. 5.12).

Остается только перенести всю систему на нашу виртуальную машину. Снова воспользуемся для этого GitHub: запустим сгенерированные файлы-манифесты туда, а на виртуальной машине просто склонируем репозиторий. Но перед тем как запустить поды по этим манифестам, нужно удалить оттуда файл `docker-compose.yaml`, т. к. утилита `kubectl` попытается считать его манифестом и не даст выполниться остальным.

Запустить поды можно командой:

```
kubectl apply -f
```

В ответ вы увидите сообщение о том, что файлы `service` и `deployment` сконфигурированы. Убедиться в этом можно, посмотрев, какие поды существуют сейчас:

```
kubectl get pods
```

Это выведет в вашем Kubernetes все поды, а также покажет их статус (рис. 5.13).

Чтобы узнать, что происходит с конкретным подом, можно зайти в него и посмотреть последние логи:

```
kubectl logs <название пода>
```

The screenshot shows a file manager window titled 'kubernetes'. It displays a list of generated YAML files for migration from Docker Compose. The files are organized into columns: Name (Имя), Size (Размер), and Type (Тип).

Имя	Размер	Тип
account_db-service.yaml	330 Б	YAML
account-db-deployment.yaml	1 КБ	YAML
account-deployment.yaml	1 КБ	YAML
account-service.yaml	323 Б	YAML
auth_db-service.yaml	321 Б	YAML
auth-db-deployment.yaml	1 КБ	YAML
auth-deployment.yaml	1 КБ	YAML
auth-service.yaml	314 Б	YAML
docker-compose.yaml	5 КБ	YAML
gateway-deployment.yaml	1 КБ	YAML
gateway-service.yaml	323 Б	YAML
kafka-deployment.yaml	2 КБ	YAML
kafka-service.yaml	377 Б	YAML
kafka-ui-deployment.yaml	874 Б	YAML
kafka-ui-service.yaml	326 Б	YAML
postgres-account-data-persistentvolumeclaim.yaml	228 Б	YAML
postgres-auth-data-persistentvolumeclaim.yaml	222 Б	YAML
postgres-transaction-data-persistentvolumeclaim.yaml	236 Б	YAML
transaction_db-service.yaml	342 Б	YAML
transaction-db-deployment.yaml	1 КБ	YAML
transaction-deployment.yaml	1 КБ	YAML
transaction-service.yaml	335 Б	YAML

Рис. 5.12. Сгенерированные через `kompose` файлы для миграции из Docker-Compose в Kubernetes

```
yuliapopova@compute-vm-2-2-10-ssd-1757532945531:~/k$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
account-c66bfbf74-qgznw	0/1	ContainerCreating	0	15s
account-db-6487586575-lkd4v	1/1	Running	0	15s
auth-78d79b9f4c-x9p9n	0/1	ContainerCreating	0	13s
auth-db-c6d8d5649-nkb69	1/1	Running	0	14s
gateway-567df784d-9q9dm	0/1	ContainerCreating	0	12s
kafka-dc6d8c5bc-jvl7z	0/1	ContainerCreating	0	11s
kafka-ui-7b44c685bc-tgk4m	0/1	ContainerCreating	0	11s
transaction-6c5c766b65-5n7l2	0/1	ContainerCreating	0	8s
transaction-db-865dc657bc-jll45	0/1	ContainerCreating	0	9s

Рис. 5.13. Запущенные поды в Kubernetes

Заключение

Итак, мы сначала упаковали наши микросервисы в `docker-compose.yml`, а затем мигрировали их в Kubernetes. Разумеется, это сильно упрощенный путь — в реальности всё настраивается гораздо сложнее: докер-образы собираются через CI/CD, через них же публикуются в репозитории в облаке, затем доставляются в контейнеры и запускаются. Весь этот путь — от коммита в Git до развернутого сервиса на стейдже — в итоге может занимать несколько минут, но бывает, что и значительно дольше, поскольку зависит от производственных мощностей и количества параллельно выполняемых таких же операций.

Последний, пятый способ развертывания приложения — бессерверный — мы не рассмотрели. Но там все зависит от выбранной платформы и, как правило, выполняется строго по инструкции.

Список литературы и источников

1. <https://semaphoreci.com/blog/deploy-microservices>.
2. <https://ru.wikipedia.org/wiki/Kubernetes>.
3. Александр Бранд, Рич Ландер, Джош Росс, Джон Харрис. Kubernetes на практике. 2022 (<https://goo.su/9w24e>).

ПРИЛОЖЕНИЕ

Описание файлового архива

Файловый архив с финальным проектом, созданным в книге, выложен на сервер издательства «БХВ» по адресу: <https://zip.bhv.ru/9785977521208.zip>. Ссылка доступна и со страницы книги на сайте <https://bhv.ru/>.

Структура архива представлена в табл. П1.

Таблица П1. Структура файлового архива

Каталог, файл	Описание
account	Исходный код микросервиса Account. Инструкции по запуску внутри каталога в файле README.md
auth	Исходный код микросервиса Auth. Инструкции по запуску внутри каталога в файле README.md
transaction	Исходный код микросервиса Transaction. Инструкции по запуску внутри каталога в файле README.md
gateway	Исходный код микросервиса Gateway. Инструкции по запуску внутри каталога в файле README.md
contracts	Исходный пакет contracts. Инструкции по генерации контрактов в файле README.md
local-env	Содержит файлы для создания локального окружения
local-env/ docker-compose.yaml	Файл docker-compose.yaml для создания kafka, kafka-ui, postgresql в контейнерах
kubernetes	Манифесты для создания подов в Kubernetes. Были сгенерированы из файла docker-compose.yaml через kompose
kubernetes/ docker-compose	Из этого файла docker-compose.yaml были сгенерированы манифесты для kubernetes

Предметный указатель

\$

\$GOPATH 15

•

.gitignore 32

A

access-token 109

ACID 166

AMQP 120

Apache Kafka 123

API 24

API Gateway 126

B

bcrypt 110

C

CI/CD 270

cURL 24

D

Docker 22

Docker Hub 300

Docker-compose 22

Dockerfile 294

DTO 52

G

Git 19

git flow 20

git tag 57

GitHub Actions 272

GitHub Actions Marketplace
274

glide 15

Go Modules 14

godep 15

Golang 14

GoMock 292

GORM 41

govendor 16

GVM 17

H

HTTP 111

I

Insomnia 25

IntelliSense 13

interceptor 131

J

JWT 92

JWTClaims 135

K

Keycloak 93

kompose 309

Kubernetes 267, 306

Kubernetes pod 306

Kubernetes replication
controllers 306

Kubernetes services 307

Kubernetes volume 306

M

main.go 27

Makefile 28

N

Nginx 115, 268

Node Kubernetes 306

O

ORM 40

OSI 111

P

Postman 25

Protobuf 18, 116

protoc-gen-go 18

R

Read Committed 168

Read Uncommitted 168

Redis 125

refresh-token 109

Repeatable Read 168

RESTful API 43

rice condition 167

S

Saga 229

Serializable 168

spew 37

Swagger UI 25

V

Visual Studio Code 13

Y

YAGNI 185

Z

zerolog 37

W

Workflow 272

A

Авторизация 87

Аутентификация 87

Б

Балансировщик нагрузки 267

Брокер сообщений 121

Г

Графовые базы данных 48

Д

Двухфазная фиксация 228

И

Идентификация 87

Индексы 163

К

Ключ доступа 91

Колоночные базы данных
48**Л**

Логирование 36

М

Миграции 161

ННереляционные базы данных
48

Нормальная форма 48

П

Пагинация 66

Пакетные менеджеры для Go
14

Переменные окружения 32

Р

Редакторы кода 13

Реляционные базы данных 48

С

СУБД 47

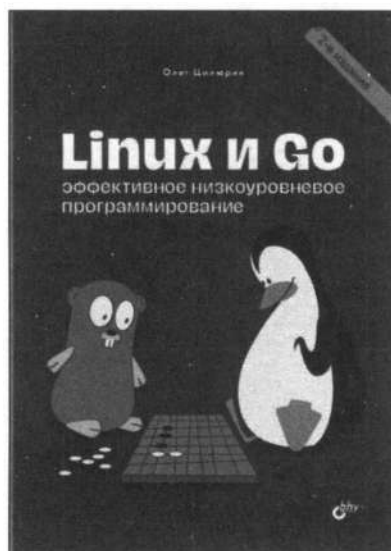
Т

Типы индексов 165

Транзакция 166

Цилюрик О.
Linux и Go.
**Эффективное низкоуровневое
программирование. 2-е издание**

Отдел оптовых поставок:
e-mail: opt@bhv.ru



Во втором издании:

- Материал обновлен в соответствии с новой версией 1.22 языка Go
- Рассмотрена миниатюрная реализация языка TinyGo
- Дополнительно приведены новые примеры использования языка

Ядро операционной системы Linux и множество модулей для различных дистрибутивов написаны на языке C. Притом что язык C продолжает развиваться и активно использоваться на практике, в область системного программирования постепенно проникают и более молодые языки, в частности Go. Перед вами — первая фундаментальная книга об использовании Go в Linux. Здесь вы познакомитесь с основами системного программирования, изучите детали взаимодействия между ядром и пользовательским пространством Linux, а также узнаете об интероперабельности между C и Go, о том, в каких аспектах и нюансах Linux язык Go может заменить и уже заменяет язык C. Особое внимание уделено конкурентности, параллелизму и стандарту POSIX. В конце книги для закрепления материала приведены реализации нескольких популярных алгоритмов.

Книга ориентирована на программистов и системных администраторов, работающих с Linux, будет интересна разработчикам ядра Linux и драйверов устройств.

Олег Иванович Цилюрик — программист-разработчик с более чем 40-летним опытом, преподаватель, автор нескольких книг по Linux и Unix, высоко оцененных профессионалами и широкой читательской аудиторией.

От монолита к микросервисам

Отдел оптовых поставок:

e-mail: opt@bhv.ru



- Идеально подходит для организаций, которые хотят перейти на микросервисы, не занимаясь перестройкой
- Помогает компаниям определяться с тем, следует ли мигрировать, когда и с чего начинать
- Решает вопросы коммуникации, интеграции и миграции унаследованных систем
- Обсуждает несколько шаблонов миграции и мест их применения
- Рассматривает примеры миграции баз данных, а также стратегии синхронизации
- Описывает декомпозицию приложений, включая несколько архитектурных шаблонов рефакторизации

Как распутать монолитную систему и мигрировать на микросервисы? Как это сделать, поддерживая работу организации в обычном режиме? В качестве дополнения к чрезвычайно популярной книге Сэма Ньюмена «Создание микросервисов» его новая книга подробно описывает проверенный метод перевода существующей монолитной системы на архитектуру микросервисов.

Это практическое руководство содержит ряд наглядных примеров и шаблонов миграции, массу практических советов по переводу монолитной системы на платформу для микросервисов, различные сценарии и стратегии успешной миграции, начиная с первичного планирования и заканчивая декомпозицией приложений и баз данных. Описанные шаблоны и методы опробованы и надежны, их можно использовать для миграции уже существующей архитектуры.

Сэм Ньюмен, принимал участие в нескольких стартапах, 12 лет проработал в ThoughtWorks и теперь является независимым консультантом. Специализируется на микросервисах и облачной архитектуре, проводит обучение и дает консультации, помогает клиентам быстрее и надежнее разрабатывать программно-информационное обеспечение, выступает на конференциях по всему миру. Автор известной книги «Создание микросервисов» издательства O'Reilly.



www.bhv.ru

Беллемар А.

Создание событийно-управляемых микросервисов

Отдел оптовых поставок:

e-mail: opt@bhv.ru



Вы узнаете:

- Принципы использования событийно-управляемых архитектур для обеспечения исключительной бизнес-ценности
- Роль микросервисов в поддержке событийно-управляемых проектов
- Архитектурные шаблоны, обеспечивающие успех как разработчиков вашей организации, так и внештатных команд
- Шаблоны приложений для создания мощных событийно-управляемых микросервисов
- Компоненты и инструментарий, необходимые для разработки экосистемы микросервисов

Книга описывает методы создания событийно-управляемых микросервисов для обработки больших объемов данных и предлагает шаблоны приложений, использующих подобную архитектуру. Рассказано о роли микросервисов в поддержке событийно-управляемых проектов, представлены примеры практических реализаций подобных архитектур как силами сотрудников организации, так и с привлечением сторонних специалистов. Подробно описаны инструменты, необходимые для разработки экосистемы микросервисов. Приведены способы решения возникающих проблем, даны рекомендации по налаживанию взаимодействия команд и отдельных сотрудников в процессе создания событийно-управляемых микросервисных систем.

Адам Беллемар, инженер по обработке данных в компании Shopify, разработчик продуктов и платформ для создания событийно-управляемых архитектур. Ранее руководил миграцией на микросервисы и продвигал распределенные асинхронные проекты. Имеет богатый опыт в архитектурном и техническом управлении распределенными вычислениями и создании методов обработки данных.



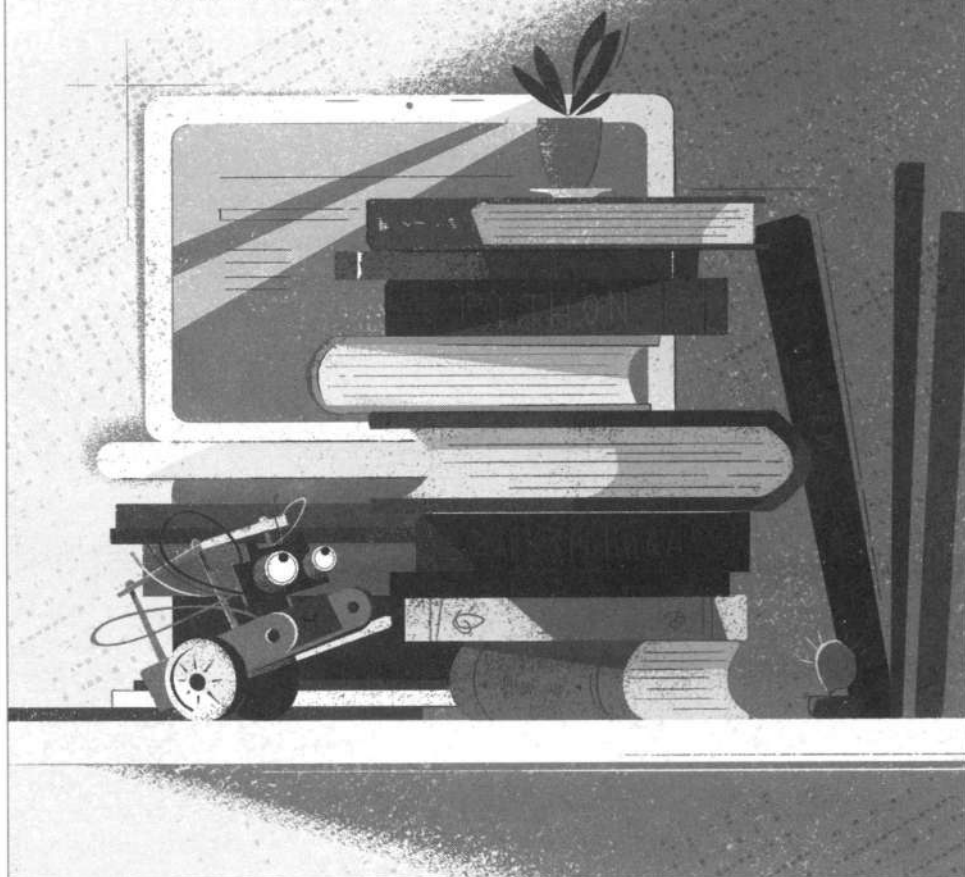
ИНТЕРНЕТ-МАГАЗИН

BHV.RU

КНИГИ, РОБОТЫ,
ЭЛЕКТРОНИКА

Интернет-магазин издательства «БХВ»

- Более 30 лет на российском рынке
- Книги и наборы по электронике и робототехнике по издательским ценам
- Электронные архивы книг и компакт-дисков
- Ответы на вопросы читателей



БХВ-Электроника bhv.ru/elements

Электронные компоненты
для мейкеров



Микросервисы на Go: осваиваем аккуратную декомпозицию

Современная практика enterprise-разработки и возникающие вызовы связаны, прежде всего, с обеспечением отказоустойчивости и расширяемости приложений. Сложно рассчитывать на реализацию таких качеств без применения микросервисной архитектуры. В книге по порядку рассматривается создание целого приложения с нуля. На материале готового продукта показано, как написать и развернуть четыре микросервиса, управлять СУБД, настроить брокер сообщений Kafka, внедрить кеш Redis и объединить эти решения в среде Docker-Compose и оркестраторе Kubernetes. Все паттерны, актуальные при проектировании микросервисов для веб-архитектуры, разобраны на практических примерах.

Книга интересна в качестве вводной по микросервисам на Golang и будет полезна как начинающим разработчикам, так и архитекторам, занятым модернизацией архитектуры с применением микросервисов.

Юлия Юрьевна Попова, практикующий инженер-программист, занимающийся разработкой коммерческих продуктов более 5 лет. Автор книги «Node.js: разработка приложений в микросервисной архитектуре с нуля».



Электронный архив с дополнительными материалами, не вошедшими в книгу, можно скачать по ссылке <https://zip.bhv.ru/9785977521208.zip>, а также со страницы книги на сайте bhv.ru



191036, Санкт-Петербург,
Гончарная ул., 20
Тел.: (812) 717-10-50,
339-54-17, 339-54-28
E-mail: mail@bhv.ru
Internet: www.bhv.ru



ISBN 978-5-9775-2120-8

